

STEFANO NOVELLANI

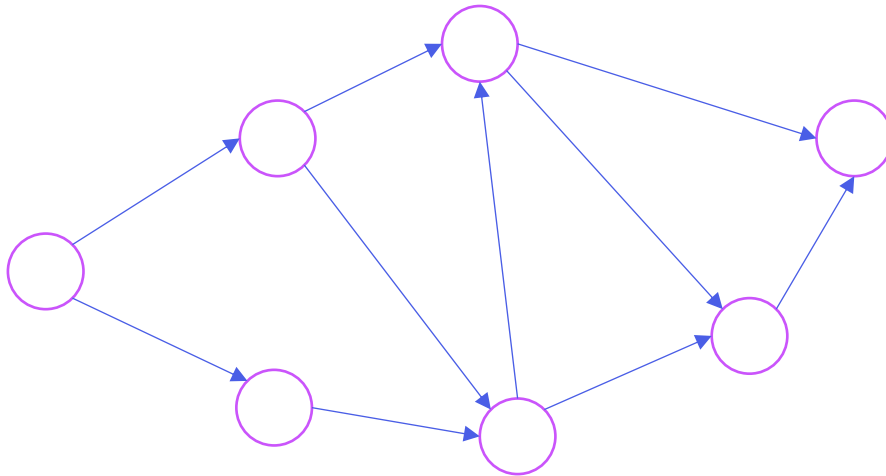
GRAFI E RETI DI FLUSSO



GRAFI E PROBLEMI DI OTTIMIZZAZIONE

Molti problemi di ottimizzazione sono caratterizzati da una struttura di **grafo**:

- in molti casi questa struttura emerge in modo naturale,
- in altri casi, nasce dal modo in cui i problemi vengono modellati.



$$G = (N, A)$$

GRAFI E PROBLEMI DI OTTIMIZZAZIONE

Esempi di strutture a grafo

- **Rete stradale o ferroviaria:**

nodi = incroci / stazioni, archi = strade / ferrovie

- **Rete di telecomunicazioni:**

nodi = router, archi = collegamenti (wireless, fibra ottica, ...)

- **Rete idrica o del gas:**

archi = condutture, nodi = raccordi (eventualmente con valvole o compressori)

- **Rete elettrica**

archi = elettrodotti, nodi = trasformatori e collegamenti

- *In altri problemi la struttura di grafo è più nascosta: per esempio la schedulazione della produzione.*



FLUSSI SU RETI

PAG 48 DISPENSE

FLUSSI SU RETI

Una **rete** è un grafo orientato pesato $G=(N,A)$

- $n=|N|$ numero di nodi
- $m=|A|$ numero di archi

Gli **archi** rappresentano canali di flusso:

- Discreti: es. numero di auto, numero di messaggi su rete di telecomunicazione
- Continui: es. quantità di petrolio in un oleodotto
- Possono essere valori assoluti o valori relativi (per unità di tempo)

DEFICIT DEI NODI

Ad ogni nodo $i \in N$ è associato un **valore reale** b_i detto **deficit**:

$b_i > 0$ **domanda del nodo**

Quantità di bene che **esce** dalla rete

Nodo detto **destinazione**, **pozzo** o **nodo di output**

$b_i < 0$ **offerta del nodo**

Quantità di bene che **entra** nella rete

Nodo detto **origine**, **sorgente** o **nodo di input**

$b_i = 0$ **nodo di trasferimento**

DEFICIT DEI NODI

Nei problemi di flusso: **domanda globale = offerta globale**

Formalmente:

$D = \{i \in N : b_i > 0\}$: nodi di **domanda**

$O = \{i \in N : b_i < 0\}$: nodi di **offerta**

$$\sum_{i \in D} b_i = - \sum_{i \in O} b_i$$

Il **vettore dei bilanci** b soddisfa:

$$\sum_{i \in N} b_i = 0$$

PESI E CAPACITÀ DEGLI ARCHI

Ad ogni arco $a_k = (i, j)$ sono associati:

Costo c_k o c_{ij} : costo per unità di bene che attraversa l'arco

Capacità inferiore l_k o l_{ij} : minimo numero di unità trasportabili

Capacità superiore u_k o u_{ij} : massimo numero di unità trasportabili

Nota: spesso $l_k = 0$ e viene considerata solo la **capacità superiore**

FLUSSO AMMISSIBILE

Un vettore $x = [x_{ij}]_{(i,j) \in A}$ rappresenta un **flusso su G** se soddisfa i *vincoli di conservazione del flusso*

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = b_i, \forall i \in N$$

$BS(i) \rightarrow$ archi **entranti** in i
 $FS(i) \rightarrow$ archi **uscenti** da i

Un **flusso ammissibile**: rispetta i vincoli di capacità:

$$l_{ij} \leq x_{ij} \leq u_{ij}, (i,j) \in A$$

COSTO DI UN FLUSSO

Ad ogni flusso x è associato un **costo totale**:

$$c(x) = \sum_{(i,j) \in A} c_{ij} x_{ij}$$

ESEMPIO: SCHEDULAZIONE DELLA PRODUZIONE

L'industria dolciaria PereCani, nota produttrice di panettoni, deve decidere come utilizzare al meglio, nel corso dell'anno, la sua linea di produzione per tali dolci. La PereCani conosce una stima $b_i, i = 1, \dots, 12$, del numero di panettoni che venderà in ogni mese dell'anno. Il costo unitario di produzione e la massima quantità di panettoni producibile in un mese variano anch'esse, a seconda di alcuni fattori quali il prezzo delle materie prime e la disponibilità di personale impegnato anche su altre linee di produzione, e anche di esse si hanno le stime, rispettivamente, c_i ed $u_i, i = 1, \dots, 12$. I panettoni prodotti al mese i possono essere venduti immediatamente, oppure immagazzinati per essere poi venduti nei mesi successivi: il magazzino ha una capacità massima di U , ed un costo unitario di immagazzinamento pari a C . All'inizio il magazzino contiene b_0 panettoni, e si desidera che alla fine dell'anno ne contenga b_{13} .

ESEMPIO: SCHEDULAZIONE DELLA PRODUZIONE

Se volessimo scrivere una formulazione matematica per il problema:

Variabili

x_i : quantità di panettoni prodotti nel mese i

s_i : quantità di panettoni immagazzinati alla fine del mese i

Funzione obiettivo

Minimizzare il **costo totale** di produzione e magazzino:

$$\min \sum_{i=1}^{12} (c_i x_i + C s_i)$$

ESEMPIO: SCHEDULAZIONE DELLA PRODUZIONE

Vincoli

Vincolo di bilancio del magazzino (conservazione dei flussi):

$$s_{i-1} + x_i - s_i = b_i, i = 1, \dots, 12$$

dove $s_0 = b_0$ e $s_{12} = b_{13}$

Vincolo di capacità di produzione:

$$0 \leq x_i \leq u_i, i = 1, \dots, 12$$

Vincolo di capacità del magazzino:

$$0 \leq s_i \leq U, i = 1, \dots, 12$$

ESEMPIO: SCHEDULAZIONE DELLA PRODUZIONE

Il problema della PereCani, noto in letteratura come problema di **Lot Sizing**, può essere formulato come un problema di flusso di costo minimo:

Possiamo definire il problema del **flusso di costo minimo (MCF)**:

$$\min c(x) \text{ s.t. } Ex = b, l \leq x \leq u$$

Dove:

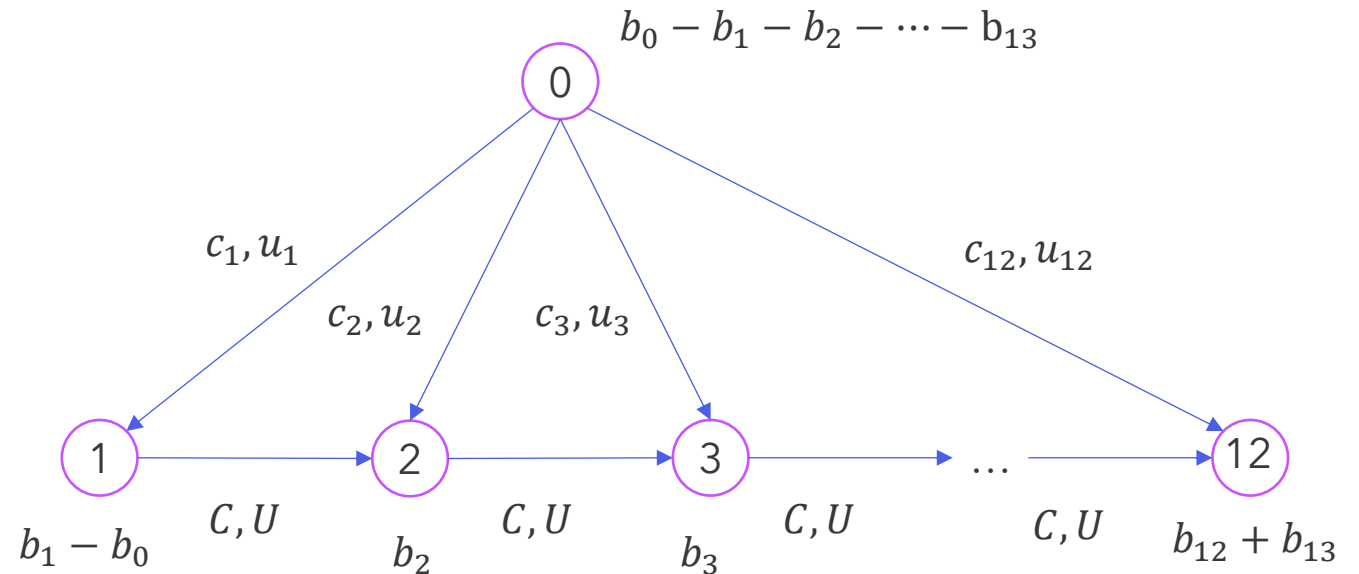
$Ex = b$: vincoli di conservazione del flusso in forma vettoriale

$l \leq x \leq u$: vincoli di capacità degli archi

ESEMPIO: SCHEDULAZIONE DELLA PRODUZIONE

Il problema della PereCani, noto in letteratura come problema di **Lot Sizing**, può essere formulato come un *problema di flusso di costo minimo*:

- Gli archi dal nodo fittizio 0 ai nodi $1, \dots, 12$ rappresentano la **produzione**
- Gli archi di tipo $(i, i + 1)$ rappresentano il **magazzino**
- I bilanci ai nodi sono scelti in modo da rappresentare la vendita di panettoni in ogni mese, al netto dei panettoni già presenti in magazzino (per il nodo 1) o di quelli che dovranno esservi lasciati (per il nodo 12)
- Il bilancio al nodo 0 è la produzione totale di panettoni durante l'anno.





TRASFORMAZIONI EQUIVALENTI

PAG 51-52 DISPENSE

TRASFORMAZIONI EQUIVALENTI

- Molti problemi reali possono essere formulati come Problemi di Flusso di Costo Minimo (MCF).
- Su tali problemi si possono fare alcune assunzioni che ne semplificano la descrizione e l'analisi.
- Se l'istanza originale non le rispetta, è possibile costruire un'istanza equivalente tramite le **trasformazioni equivalenti**:

Assunzioni	Trasformazioni equivalenti
Singola sorgente - singolo pozzo	Sorgenti e pozzi multipli
Capacità inferiori nulle	Capacità inferiori non nulle
Nessuna capacità associata ai nodi	Limiti di capacità sui nodi
Nessun eccesso di domanda o di offerta	Eccesso di domanda o offerta

SORGENTI E POZZI MULTIPLI

Se ci sono più sorgenti e/o pozzi si introduce una **rete ampliata** $G' = (N', A')$

Nodi fittizi:

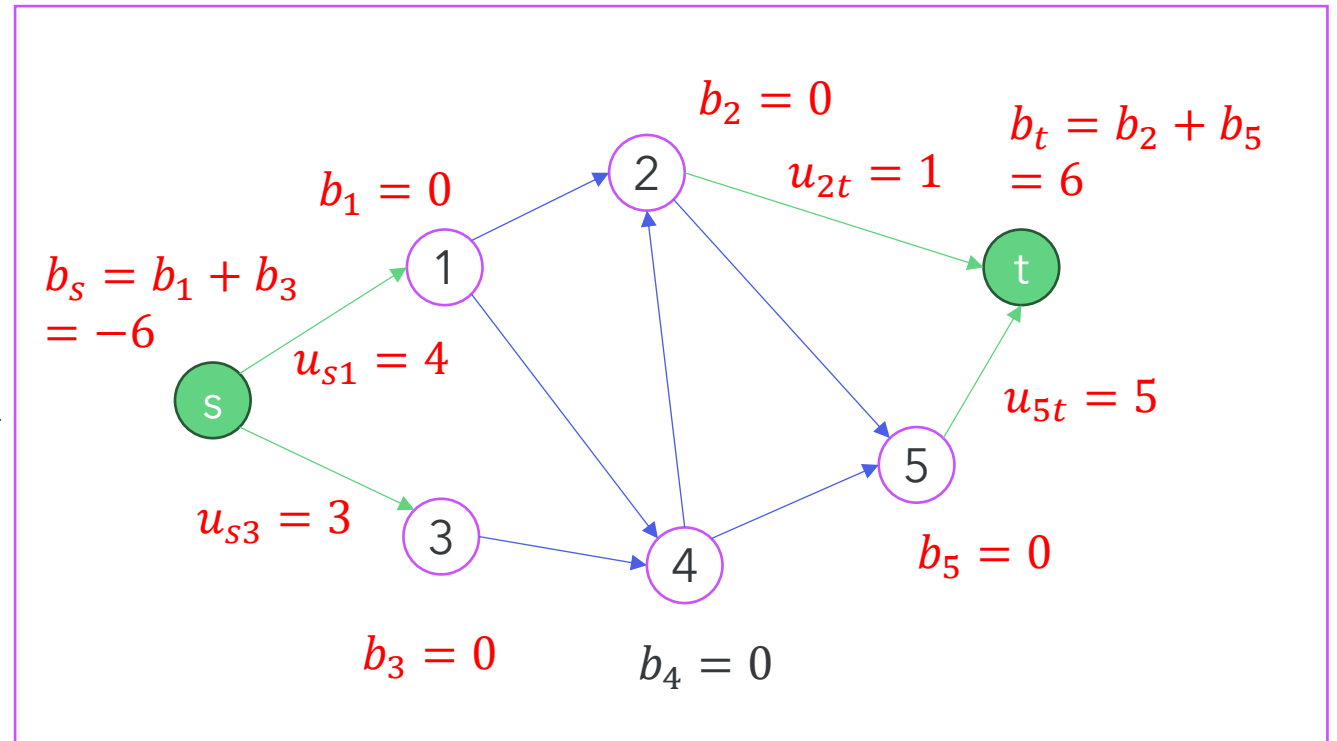
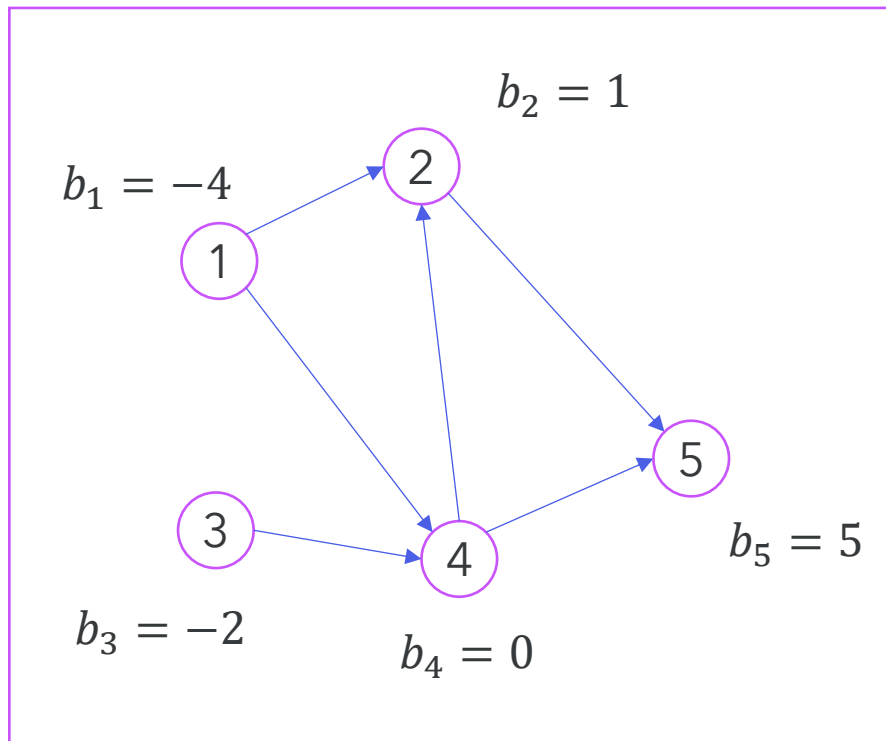
$$N' = N \cup \{s, t\}$$

- s : *super sorgente* con offerta $b_s = \sum_{j \in O} b_j$
- t : *super pozzo* con domanda $b_t = \sum_{i \in D} b_i$
- Tutti gli altri nodi: nodi di *trasferimento* $b_i = 0$

Archi fittizi:

- (s, j) per $j \in O$, con capacità $u_{sj} = -b_j$ e costo 0
- (i, t) per $i \in D$ con capacità $u_{it} = b_i$ e costo 0

SORGENTI E POZZI MULTIPLI



CAPACITÀ INFERIORI NON NULLE

Se un arco (i, j) ha **capacità minima** $l_{ij} \neq 0$: il flusso x_{ij} deve soddisfare: $x_{ij} \geq l_{ij}$

Per eliminare l_{ij} si procede con la trasformazione:

Definizione del nuovo flusso:

$$x'_{ij} = x_{ij} - l_{ij}$$

Aggiornamento dei parametri:

Bilanci:

$$b'_i = b_i + l_{ij},$$

$$b'_j = b_j - l_{ij}$$

Capacità superiore:

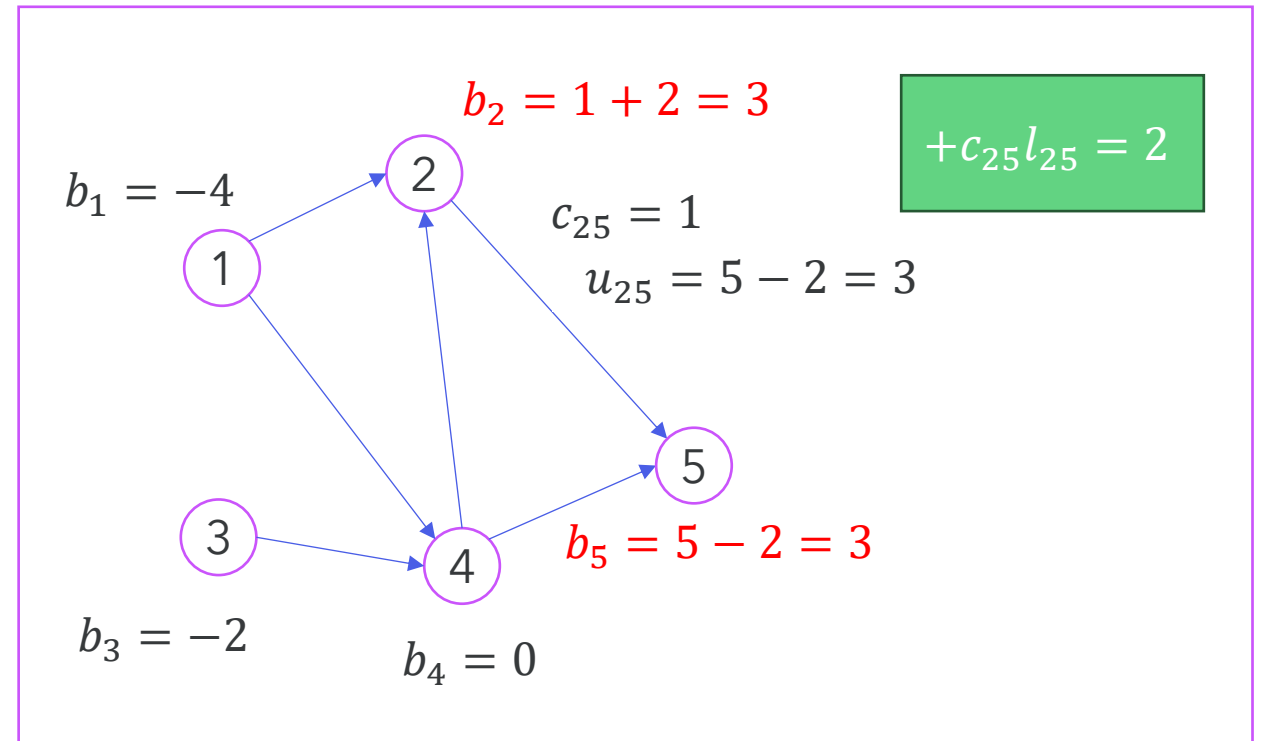
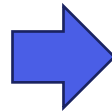
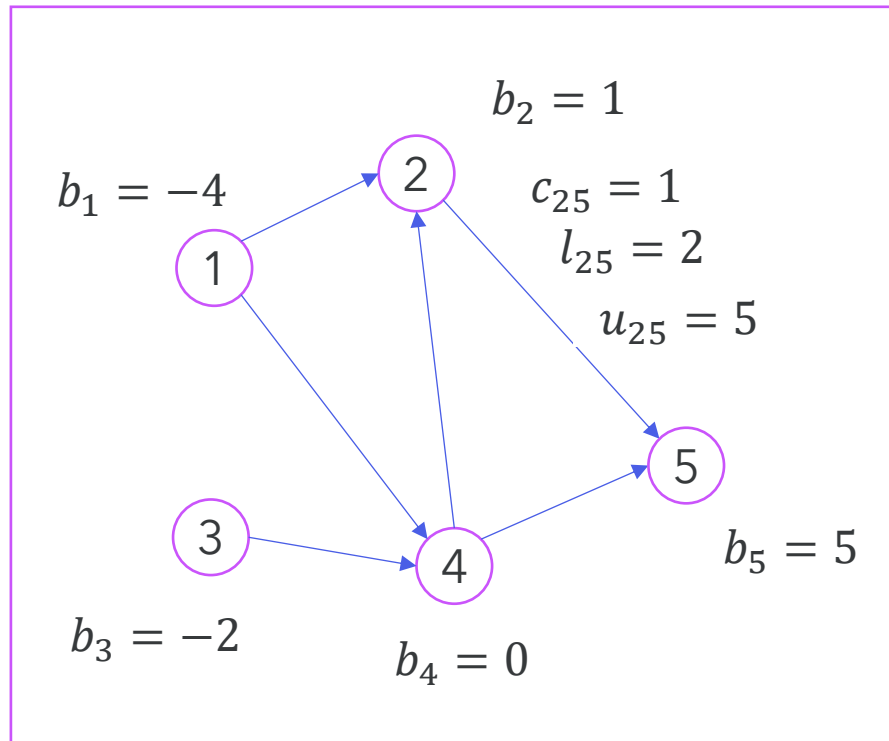
$$u'_{ij} = u_{ij} - l_{ij}$$

Costo fisso aggiunto alla funzione obiettivo:
 $+ c_{ij} l_{ij}$

La trasformazione equivale a considerare l_{ij} unità di flusso **permanentemente presenti** sull'arco (i, j) . Allora si correggono il flusso e la capacità massima togliendo quello che già passa sull'arco e aggiungendo un costo fisso alla funzione obiettivo

Per ogni arco: si aggiunge l_{ij} a b_i perché il nodo i ha già spedito l_{ij} unità, mentre tolgo l_{ij} a b_j perché il nodo j ha già ricevuto l_{ij} unità.

CAPACITÀ INFERIORI NON NULLE



LIMITI DI CAPACITÀ SUI NODI

In alcuni casi, un **nodo** i può avere:

- una **capacità massima** u_i , (cioè non può far passare più di u_i unità di flusso),
- e/o una **capacità minima** l_i .

Tali vincoli non sono gestibili direttamente nel modello standard del flusso di costo minimo.

Li possiamo eliminare introducendo **due nodi** e **un arco fittizio**.

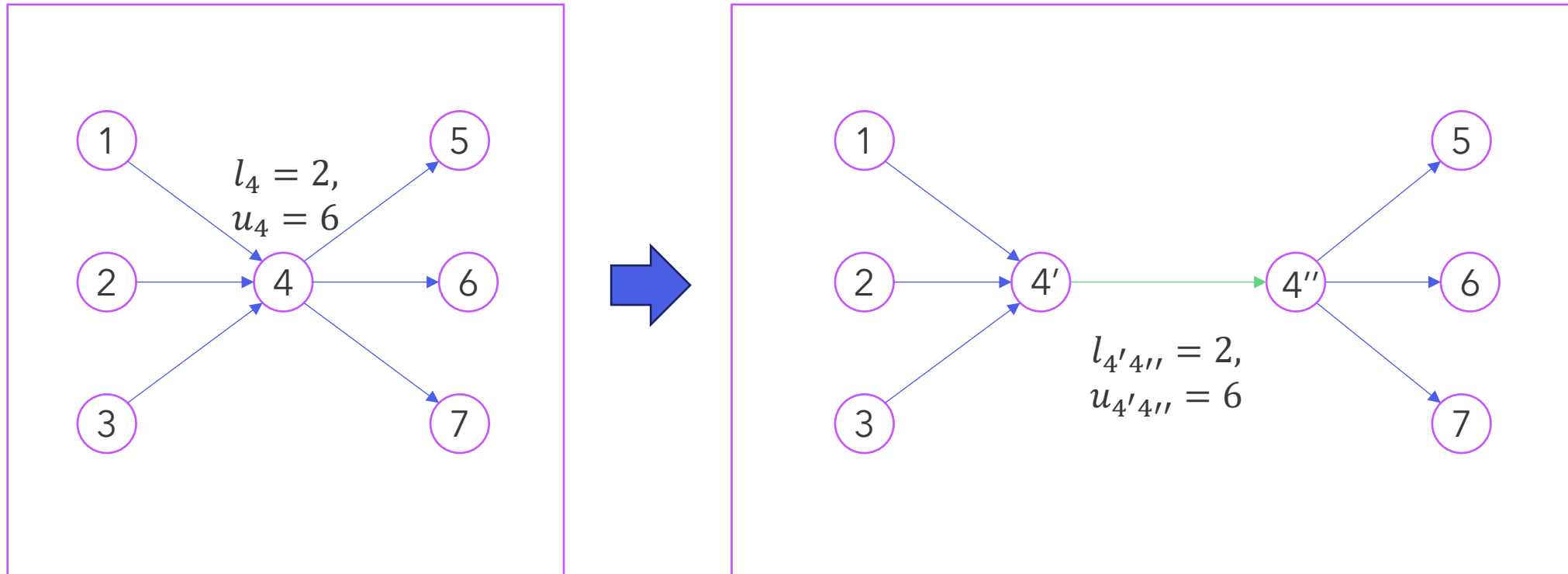
Si sostituisce il nodo i con due nodi:

i' : nodo di ingresso

i'' : nodo di uscita

- Tutti gli archi **entranti in** i diventano (j, i')
- Tutti gli archi **uscenti da** i diventano (i'', j)
- Si aggiunge un **arco fittizio** (i', i'') con:
costo $c_{i'i''} = 0$
capacità $[l_i, u_i]$

LIMITI DI CAPACITÀ SUI NODI



ECCESSO DI DOMANDA O OFFERTA

In alcuni casi, la quantità disponibile ai nodi di offerta $i \in O$ rappresenta una **massima offerta possibile**, non necessariamente da immettere tutta nella rete.

Il vincolo di conservazione del flusso diventa **disuguaglianza**:

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} \geq b_i, i \in O$$

dove $b_i < 0$ rappresenta la massima offerta del nodo i .

ECCESSO DI DOMANDA

Si costruisce una **rete ampliata** $G' = (N', A')$ aggiungendo un **nodo sorgente fittizio** s :

$$N' = N \cup \{s\}$$

Nuovi bilanci:

$$b_s = - \sum_{j \in D} b_j,$$
$$b_i = 0 \text{ per } i \in O$$

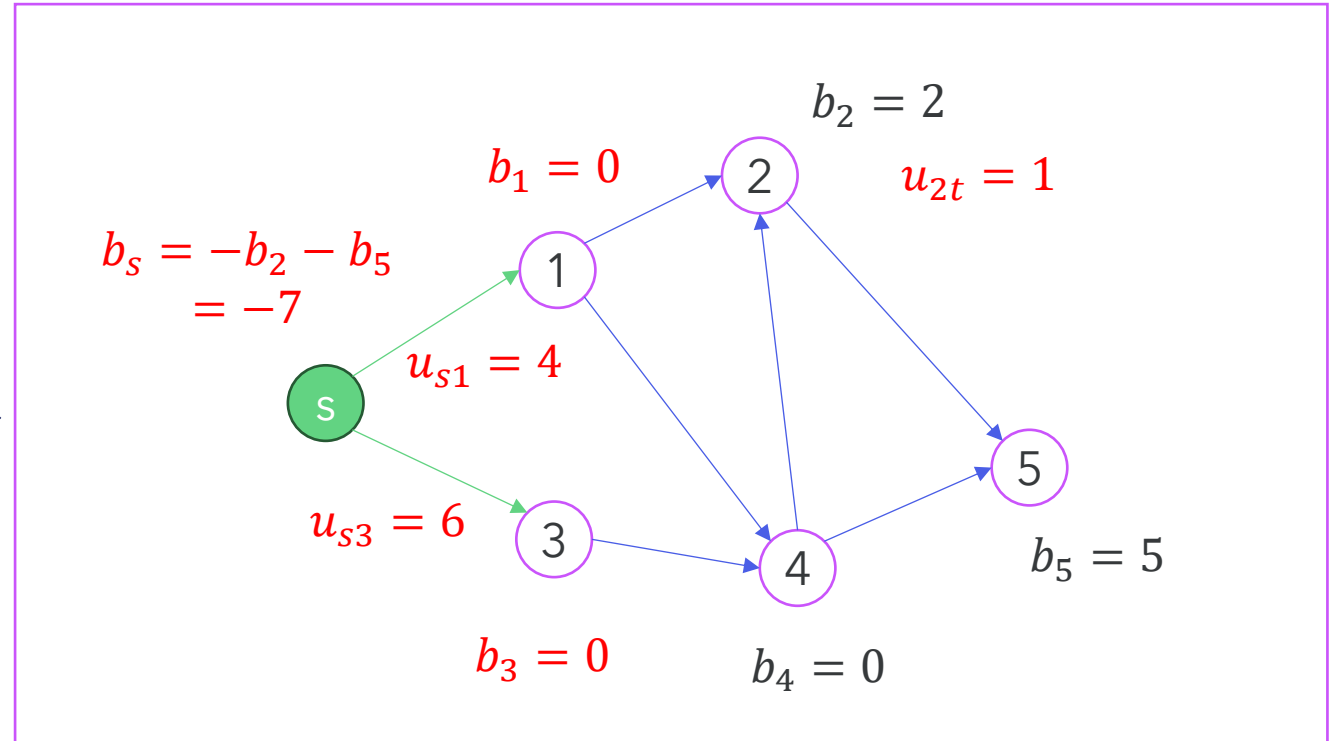
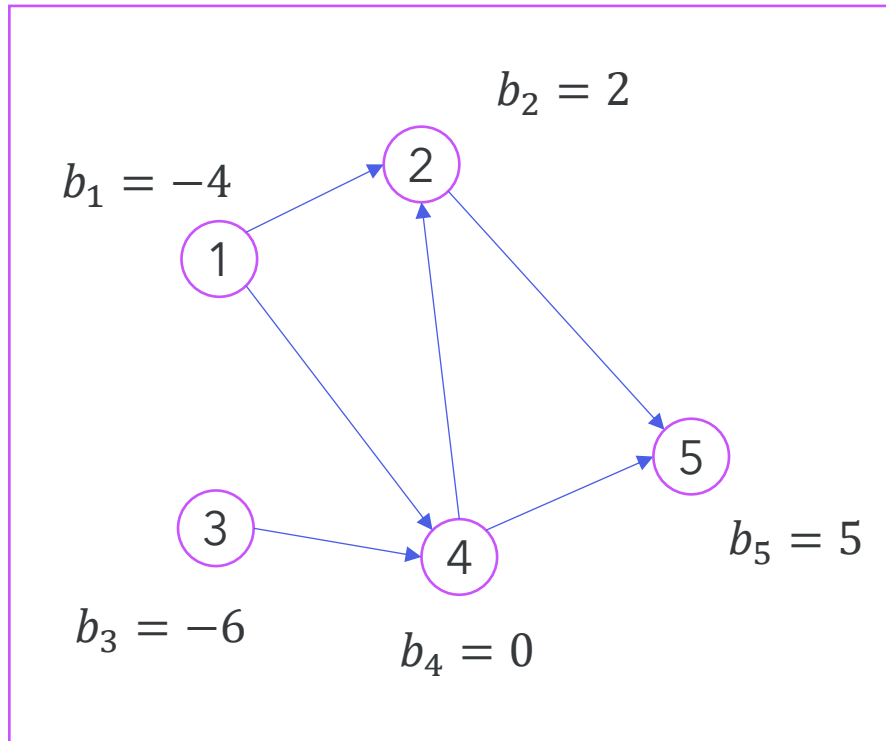
Nuovi archi:

con:

$$(s, i), i \in O$$
$$\text{costo: } c_{si} = 0,$$
$$\text{capacità: } u_{si} = -b_i$$

Analogamente per
l'eccesso di domanda
si creerà un nodo
pozzo fittizio

ECCESSO DI DOMANDA



ESEMPIO: ISPEZIONI SU UNA LINEA DI PRODUZIONE

Linea di produzione

- Una linea di produzione ha n celle di lavorazione.
- Ogni lotto ha B **pezzi** che passano in sequenza nelle celle.
- In ciascuna cella i un pezzo può diventare **difettoso** con probabilità p_i .
- Lotti con difetti non possono essere inviati ai clienti.

Ispezioni

- Ispezioni possibili **alla fine di ogni lavorazione**.
- Pezzi difettosi trovati **vengono scartati**.
- Ispezione finale obbligatoria, ma **ispezioni intermedie possono ridurre costi**.
- **Obiettivo:** evitare lavorazioni superflue su pezzi destinati a essere scartati.

Costi

- **Costo unitario di lavorazione** alla cella i : q_i
- **Costo fisso di ispezione** di un lotto all'uscita della cella j nell'ipotesi che la precedente ispezione fosse stata effettuata all'uscita della cella i ($i < j$): f_{ij}
- **Costo unitario** per pezzo ispezionato all'uscita della cella j , nell'ipotesi che la precedente ispezione fosse stata effettuata all'uscita della cella i ($i < j$): h_{ij}

Un problema di cammino minimo può formulare e risolvere anche altri problemi non tipicamente rappresentati su grafi.

ESEMPIO: ISPEZIONI SU UNA LINEA DI PRODUZIONE

Numero atteso di pezzi non difettosi alla fine della lavorazione i :

$$B_i = B \prod_{k=1}^i (1 - p_k)$$

- B_i = numero atteso di pezzi **non difettosi** alla fine della cella i (è anche il numero di pezzi su cui si effettueranno le lavorazioni nelle celle successive alla cella i , sino a quella in cui si effettuerà una nuova ispezione).
- Serve per calcolare i costi delle lavorazioni successive fino alla prossima ispezione.

ESEMPIO: ISPEZIONI SU UNA LINEA DI PRODUZIONE

Costo globale (produzione e ispezione) alla cella j dopo la cella i ($i < j$):

$$c_{ij} = f_{ij} + B_i h_{ij} + B_i \sum_{k=i+1}^j q_k$$

Include:

- costo fisso di ispezione
- costo ispezione unitario per pezzi non difettosi
- costo di lavorazione dei pezzi nel segmento $i + 1 \rightarrow j$

ESEMPIO: ISPEZIONI SU UNA LINEA DI PRODUZIONE

Formulazione come problema di cammino di costo minimo

Cella fittizia

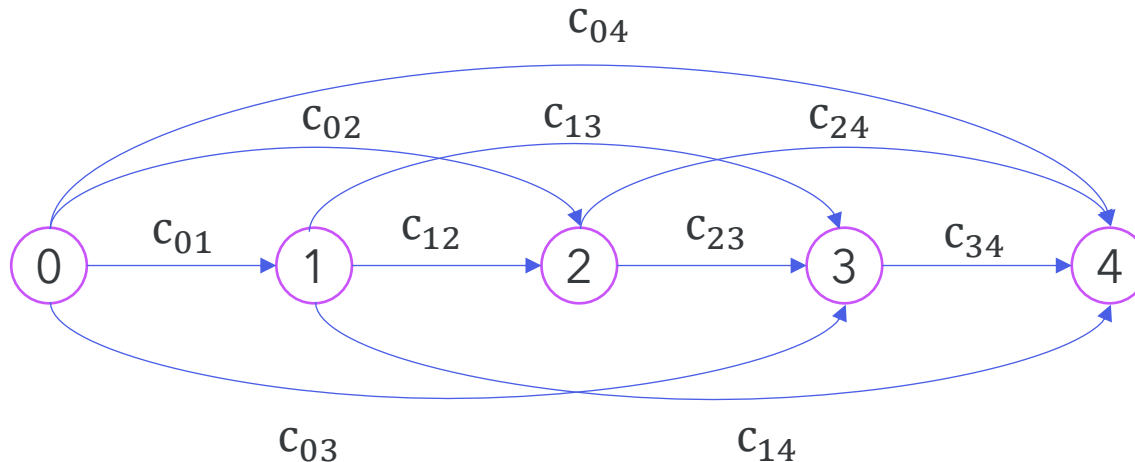
- Introduciamo **cella 0** (cella fittizia) prima dell'inizio della produzione, affinché siano definiti i valori f_{0j} , h_{0j} , c_{0j} e $B_0 = B$ relativi al caso in cui la prima ispezione sia effettuata nella cella j .

Riformulazione come grafo

- Grafo $G = (N, A)$ con
 - Nodi: $N = \{0, 1, \dots, n\}$
 - Archi: $A = \{(i, j): i \in N \setminus \{n\}, j > i\}$
 - Ad ogni arco (i, j) associato costo c_{ij}

ESEMPIO: ISPEZIONI SU UNA LINEA DI PRODUZIONE

- Ogni **cammino da 0 a n** corrisponde ad un possibile piano di ispezioni.
- Il **cammino di costo minimo dal nodo 0 al nodo n** , nel grafo $G = (N, A)$ corrisponde al piano ottimale.



Esempio con 4
celle



CAMMINI DI COSTO MINIMO

DA PAG 52 DISPENSE

IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

- Uno dei problemi **più semplici ma più importanti** su reti
- Applicazioni: GPS, Google Maps, trasporto pubblico
- Grafi con milioni di nodi/archi: tempi di calcolo rapidissimi richiesti
- Non solo reti stradali: modello generale utile anche in problemi apparentemente lontani dai grafi (vedi esempio ispezione su linea di produzione).

DEFINIZIONE DEL PROBLEMA

Definito su un **grafo orientato e pesato** $G = (N, A)$

- Nodi $N = \{1, \dots, n\}$
- Ogni **arco** $(i, j) \in A$ ha **costo** $c_{ij} \in \mathbb{R}$

Cammino P : sequenza di archi che connette due nodi

- **Costo di un cammino**: $C(P) = \sum_{(i,j) \in P} c_{ij}$

Dati r (origine) e t (destinazione), $r, t \in N$, P_{rt} è **l'insieme dei cammini** da r a t .

Problema del cammino minimo: (SP) $\min\{C(P): P \in P_{rt}\}$

FORMULAZIONE COME FLUSSO DI COSTO MINIMO

Il problema del **cammino minimo** può essere visto come un **caso particolare del problema di flusso a costo minimo (Minimum Cost Flow, MCF)**.

Ipotesi:

- Archi con **capacità infinita** ($u_{ij} = +\infty, (i, j) \in A$)
- Costi = pesi degli archi (il costo del flusso coincide con la lunghezza del cammino)

Definizione delle offerte/domande:

$$\begin{aligned} b_r &= -1 \text{ (sorgente)} \\ b_t &= +1 \text{ (pozzo)} \\ b_i &= 0 \text{ (altri nodi)} \end{aligned}$$

L'unità di flusso deve viaggiare da r a t e segue un cammino minimo: minimizzare il costo totale del flusso equivale a trovare il **cammino minimo da r a t** .

CAMMINI MINIMI MULTIPLI

Se più cammini hanno stesso costo minimo, il flusso può ripartirsi tra essi. In questo caso la soluzione ottima non è unica.

Per garantire un solo cammino dobbiamo inserire il vincolo di **integrità**:

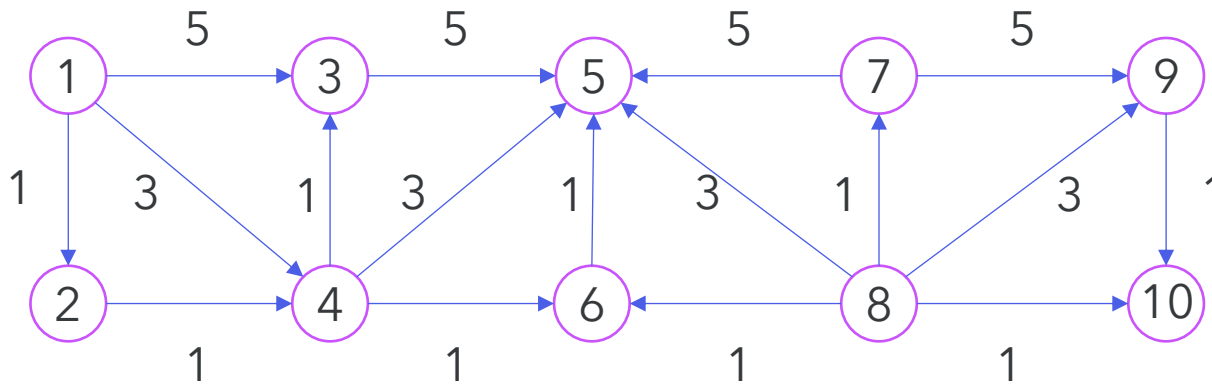
$$x \in \{0,1\}^m$$

Nel caso del cammino minimo (SP), gli algoritmi classici restituiscono comunque **soluzioni intere**.

Si veda l'esempio
riportato in
Tecniche di
Modellazione

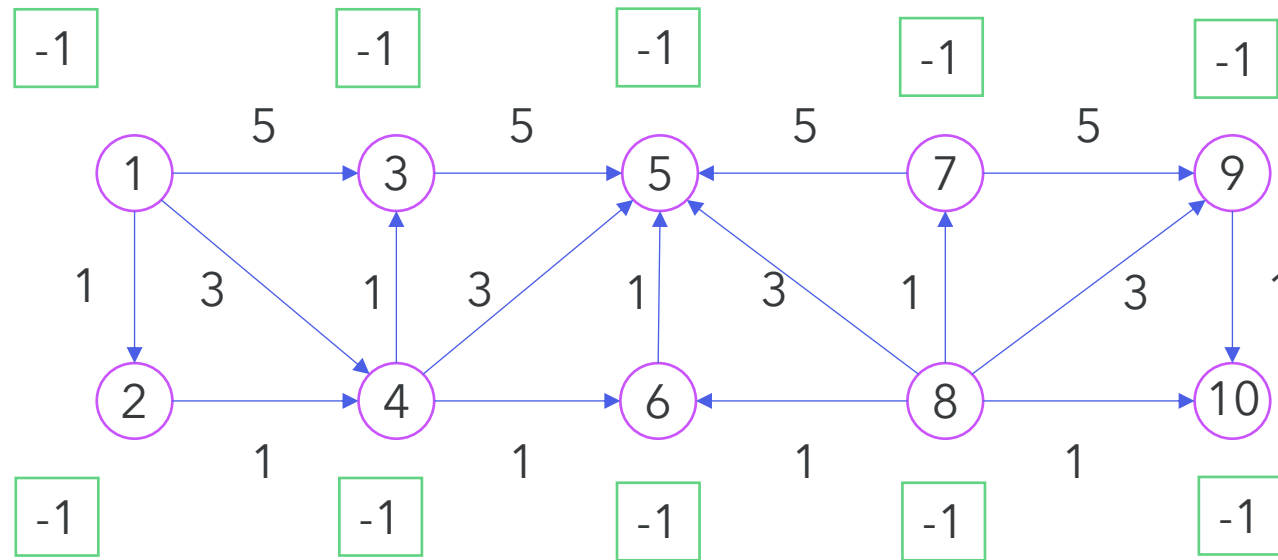
INESISTENZA

(SP) può essere **vuoto**: se non esiste alcun cammino da r a t



Non esiste un cammino da $r=1$ a $t=9$, per esempio.

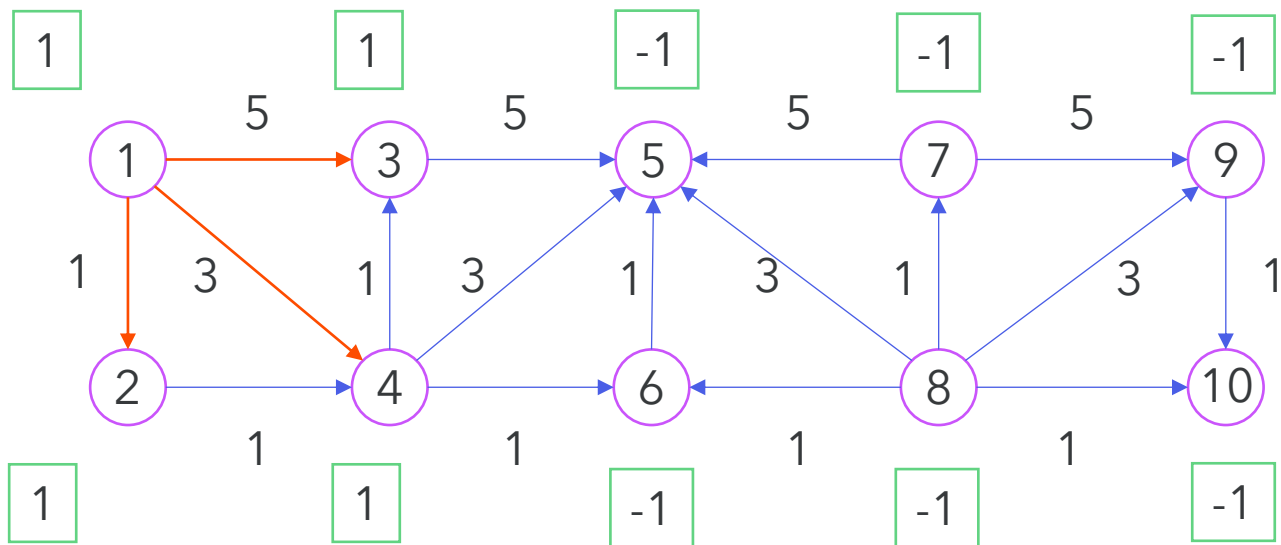
INESISTENZA: VISITA DI UN GRAFO



$Q = \{1\}$

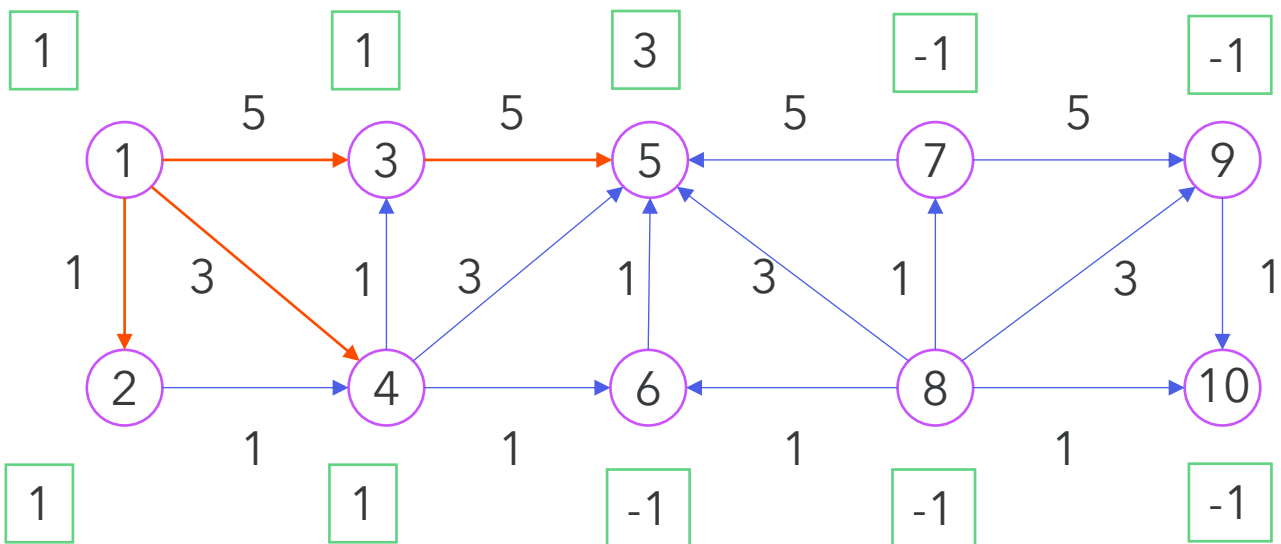
Creo un **vettore di etichette**:
mi dice, per ogni nodo, da
quale nodo ci sono arrivato.
Inizializzo tutto a -1.

Creo una lista di nodi visitati.
Inizializzo al primo nodo
 $Q = \{1\}$



$Q = \{2, 3, 4\}$

Tolgo 1 dalla lista e aggiorno il vettore di etichette e la lista stessa.

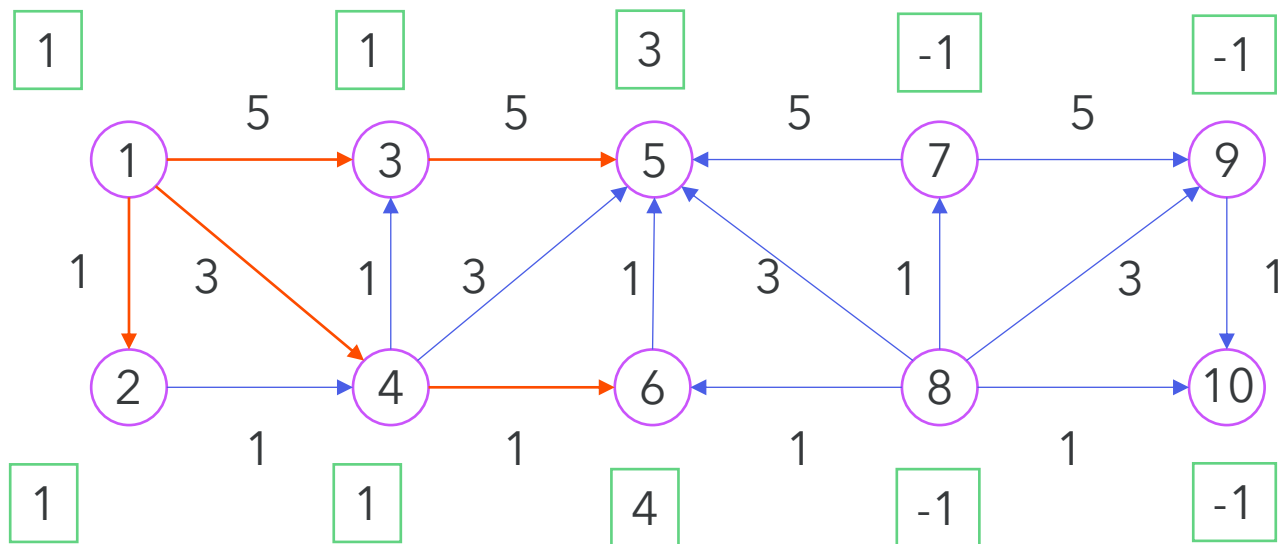


$Q = \{3, 4\}$

Tolgo 2 dalla lista. Q e le etichette non cambiano.

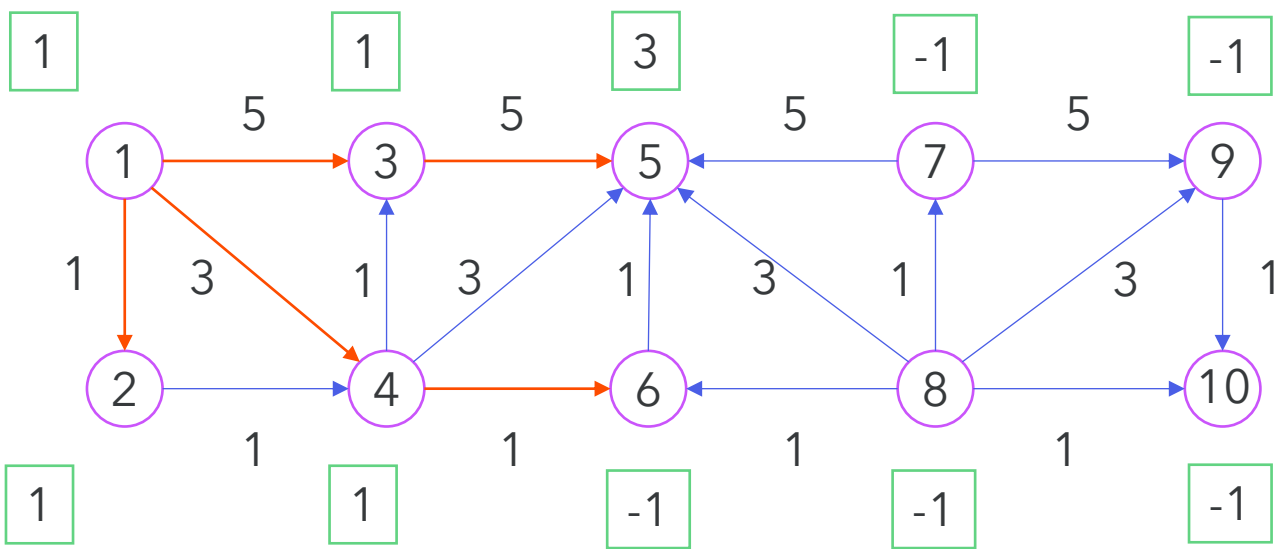
$Q = \{4, 5\}$

Tolgo 3 dalla lista e aggiorniamo il vettore di etichette e di Q .



$Q = \{5,6\}$

Tolgo 4 dalla lista e aggiorno il vettore di etichette e la lista stessa.



$Q = \{6\}$

Tolgo 5 dalla lista. Q e le etichette non cambiano.

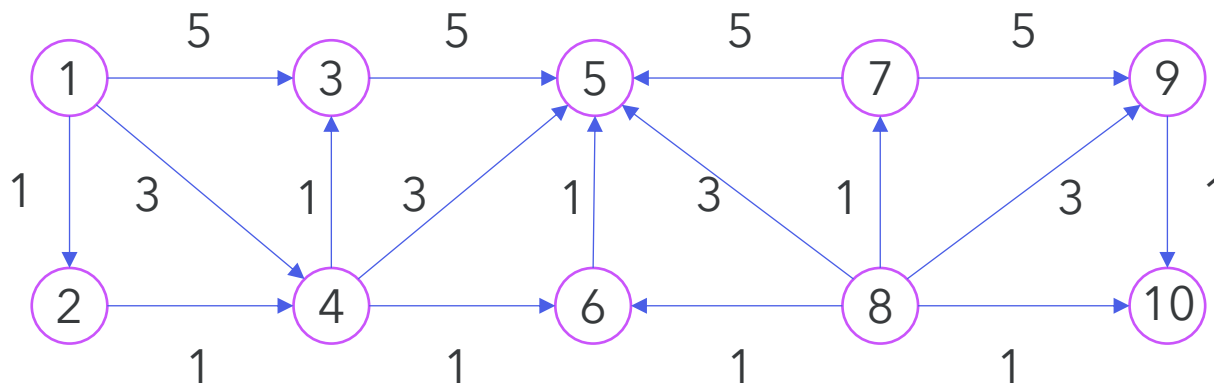
$Q = \emptyset$

Tolgo 6 dalla lista e non riesco a visitare altri nodi.

INESISTENZA: VISITA DI UN GRAFO

Una procedura di visita del grafo costa $O(m)$, con m numero di archi (se il grafo è descritto per stelle uscenti).

Deve visitare ogni arco esattamente una volta.

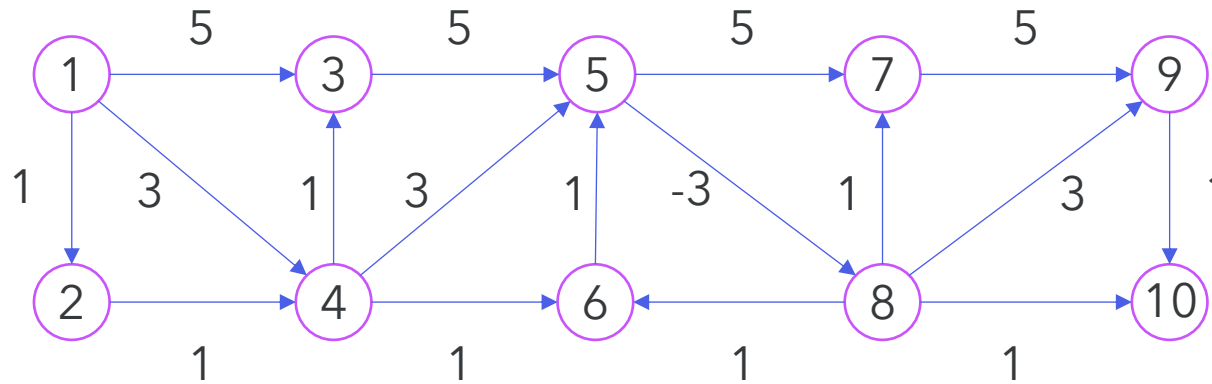


In questo caso il problema è vuoto. Potrebbe però esistere un cammino, ma non il cammino minimo.

ILLIMITATEZZA

Potrebbe esistere un cammino, ma non esistere il cammino minimo:

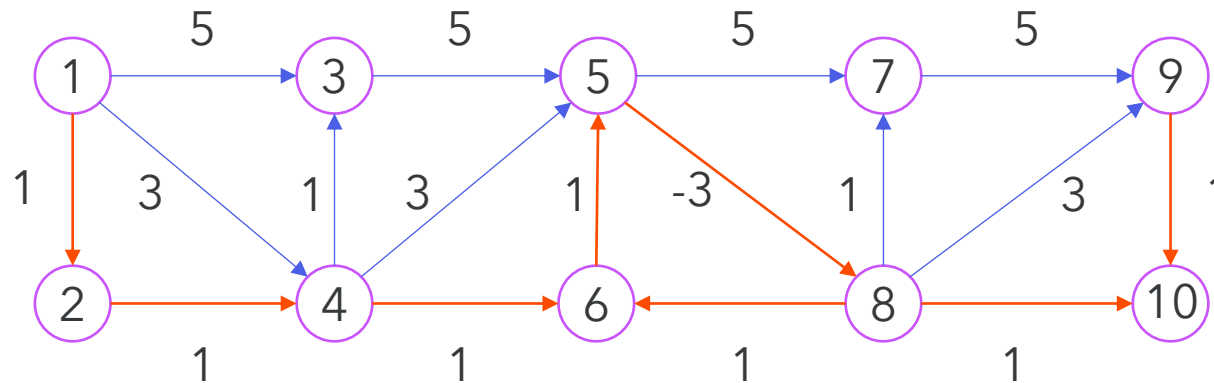
Potremmo avere dei costi negativi:



Com'è fatto il cammino da 1 a 9?

ILLIMITATEZZA

(SP) può essere **inferiormente illimitato**:



Se esiste un **ciclo orientato di costo negativo** raggiungibile da r e che raggiunge t

- Percorrendo il ciclo più volte, il costo totale diminuisce indefinitamente
- Il problema è *inferiormente illimitato*.

Occorre verificare assenza di cicli negativi:

è possibile verificare in tempo polinomiale se un grafo contiene oppure no un ciclo negativo.

CAMMINI SEMPLICI

Un cammino con un ciclo è un **cammino non semplice**.

Se il grafo **non contiene cicli negativi**:

- Una soluzione ottima del cammino di costo minimo è sempre un **cammino semplice**.
- Il **problema del cammino di costo minimo** coincide con il **problema del cammino semplice di costo minimo**.

Se il grafo contiene cicli negativi:

Problema del cammino semplice minimo diventa **NP-arduo**

Esempio: riduzione al problema del cammino Hamiltoniano minimo

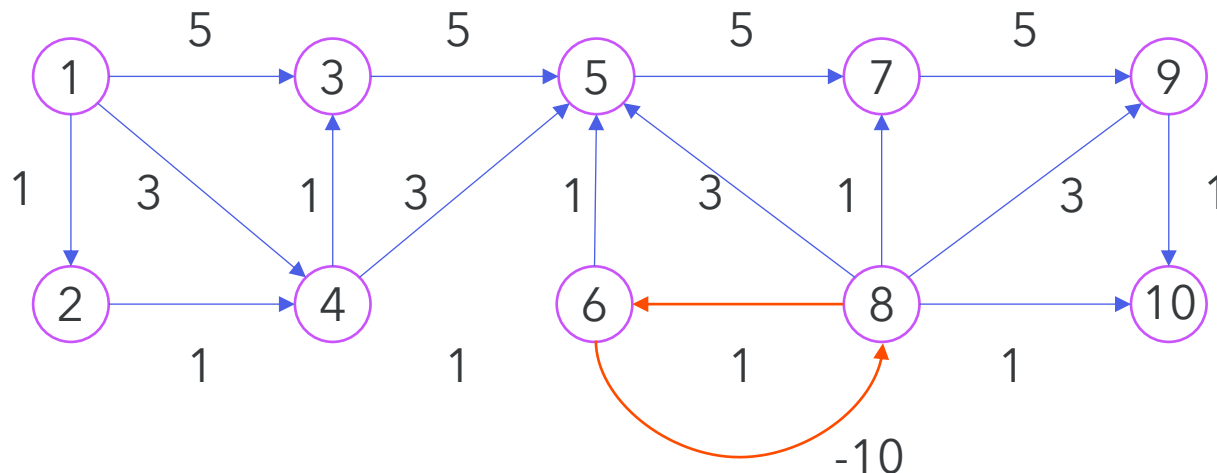
Problema **NP-arduo**:

- almeno difficile quanto ogni problema in NP;
- nessun algoritmo noto lo risolve in tempo polinomiale.

IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

Vogliamo risolvere il problema del cammino minimo non necessariamente semplice su un grafo, dato un nodo di partenza e uno di destinazione.

Potremmo avere una soluzione illimitata se ci fosse un ciclo negativo, ma potremmo non avere una soluzione illimitata anche se ci fosse un ciclo negativo sul grafo, basta che non lo incontri il cammino dalla partenza alla destinazione: *il ciclo potrebbe non essere raggiungibile*.

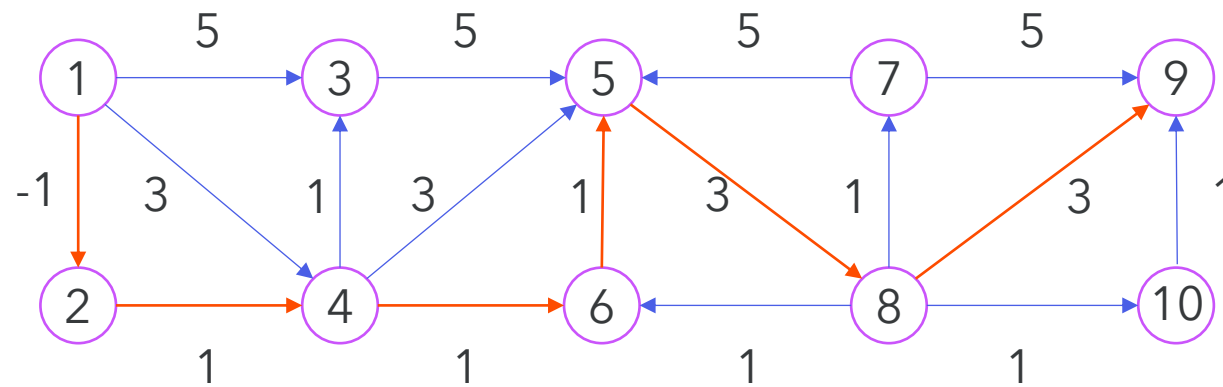


Un **problema di cammino di costo minimo** è **inferiormente illimitato**
 $\Leftrightarrow$
esiste un ciclo orientato di costo negativo raggiungibile dal nodo di partenza e dal quale si può raggiungere il nodo di arrivo.

IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

Una condizione sufficiente e non necessaria per non avere cicli orientati di costo negativo è che *tutti i costi siano non negativi*.

Potremmo anche avere costi negativi, ma non cicli di costo negativo:



IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

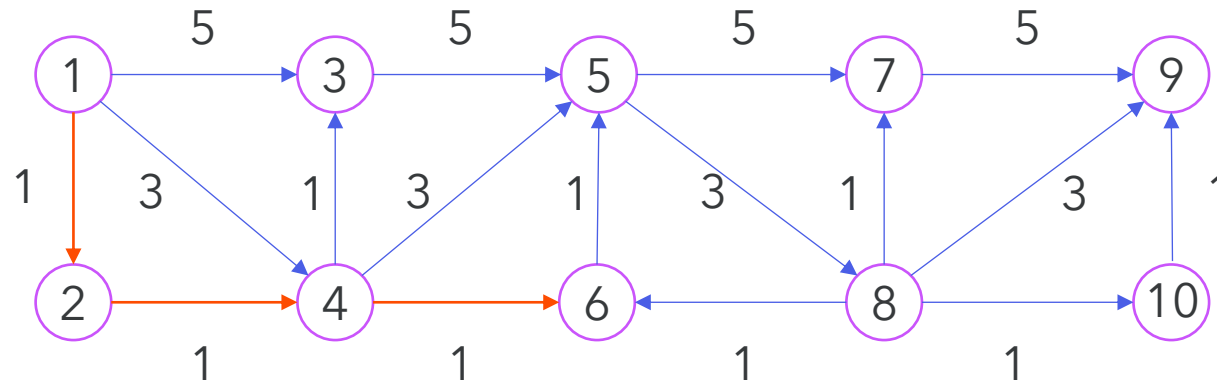
A queste condizioni possiamo trovare una soluzione senza un ciclo orientato di costo negativo.

Supponiamo di voler trovare un cammino minimo da 1 a 9 e da 1 a 6?

Non sappiamo ancora quale destinazione ci interessa.

Potrei calcolare lo SP da 1 a 6 e da 1 a 9 e valutare quale sia il migliore. Tuttavia questi problemi sono **correlati**.

Supponiamo che conosca il cammino da 1 a 6 e che sia 1, 2, 4, 6.



IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

Sottocammini di un cammino minimo

- Se conosco il cammino minimo da 1 a 6, allora conosco anche i cammini minimi da 1 a 4 e da 1 a 2.
- Tutti i **sottocammini** di un cammino minimo sono anch'essi **cammini minimi**.
- Infatti, se esistesse un cammino migliore da 1 a 4, allora il cammino da 1 a 6 **non sarebbe ottimo**.

Da un cammino a molti cammini

- Se il cammino minimo da 1 a 6 passa anche per 9, allora abbiamo trovato anche il cammino minimo da 1 a 9.
- In generale, calcolare un cammino minimo da un nodo significa **calcolare contemporaneamente** i cammini minimi verso tutti i nodi lungo il percorso.

IL PROBLEMA DEI CAMMINI DI COSTO MINIMO

Dal cammino singolo ai cammini multipli

- Allora considereremo il problema di trovare un cammino minimo **non** da un nodo all'altro, ma **tutti i cammini minimi dal nodo sorgente a tutti gli altri nodi**.
- Questo è il **problema dei cammini minimi** (da sorgente singola).

Come rappresentarli: l'albero dei cammini minimi

- Tutti i cammini minimi da una sorgente possono essere **organizzati in un albero orientato**.
- L'albero:
 - è **radicato nel nodo sorgente** r ,
 - è **orientato** verso gli altri nodi,
 - contiene **$n - 1$ archi**, uno per ogni nodo raggiungibile.
- In questo modo bastano **$n - 1$ informazioni** per descrivere tutti i cammini minimi.

ALBERO DEI CAMMINI MINIMI (SHORTEST PATH TREE, SPT)

Problema: Dato un nodo radice r , determinare il cammino di costo minimo da r a ogni altro nodo i .

Il problema può essere formulato come:

$$\min \left\{ \sum_{i \neq r} C(P_i) : P_i \in P_{ri}, i \neq r \right\}$$

Ogni P_i è un cammino minimo da r a i .

ALBERO DEI CAMMINI MINIMI (SHORTEST PATH TREE, SPT)

Formulazione come flusso di costo minimo

Possiamo formulare il problema come un **Minimum Cost Flow (MCF)**:

$$b_i = \begin{cases} -(n - 1), & \text{se } i = r \\ 1, & \text{altrimenti} \end{cases}$$

- La radice r è la **sorgente** di $n - 1$ unità di flusso
- Ciascun nodo $i \neq r$ è un **pozzo** con domanda 1
- Ogni arco (i, j) può essere usato più volte se compare in più cammini, x_{ij} rappresenta quante volte l'arco è usato.

ALBERO DEI CAMMINI MINIMI (SHORTEST PATH TREE, SPT)

Assenza di cicli di costo negativo e raggiungibilità

- Se il grafo **non ha cicli di costo negativo**, esiste una soluzione ottima finita.
- Se un nodo i **non è raggiungibile da** r , si può aggiungere un arco fittizio:

$$(r, i) \text{ con costo } M = (n - 1)c_{\max} + 1$$

- Il cammino minimo sarà costituito da questo arco **solo se non esistono cammini reali.**

ALBERO DEI CAMMINI MINIMI (SHORTEST PATH TREE, SPT)

Albero di cammini minimi (SPT)

L'unione di tutti i cammini minimi P_i può essere rappresentata come un **albero radicato in r** .

Proprietà:

Ogni arco è orientato **da** r verso le foglie.

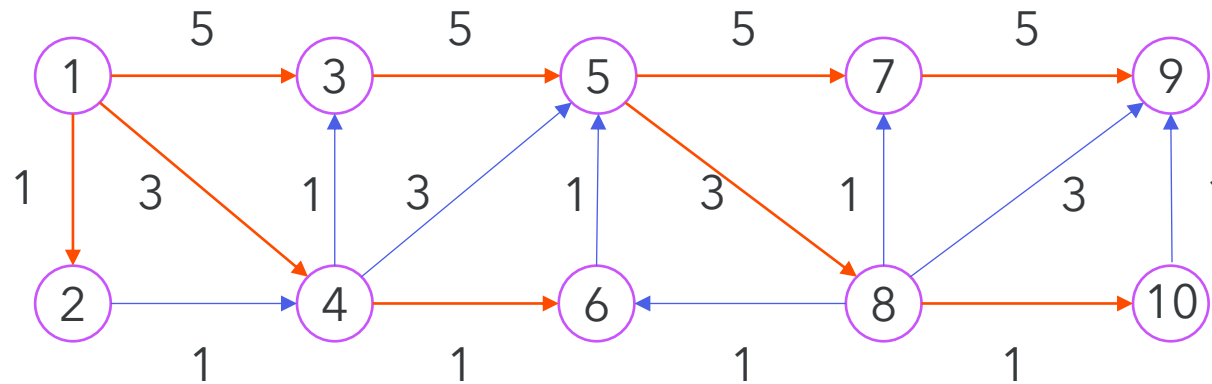
Il sottocammino da r a un nodo intermedio j è **a sua volta minimo**.

Questa struttura è detta **Shortest Path Tree (SPT) o Problema dei cammini minimi**.

ALBERO DEI CAMMINI MINIMI (SHORTEST PATH TREE, SPT)

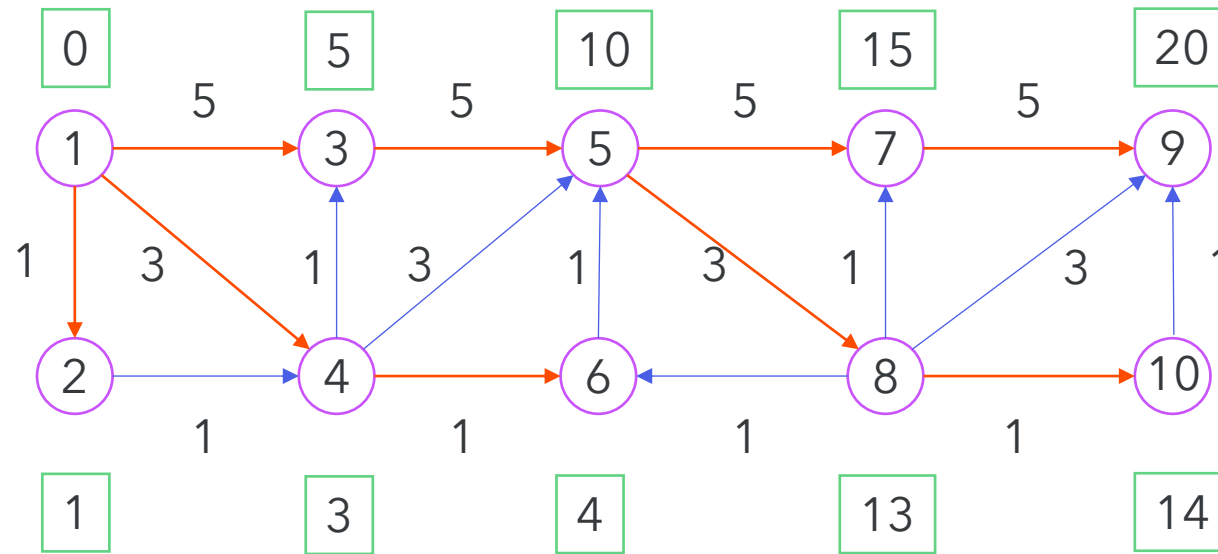
Dato un albero così fatto, posso determinare se è dei cammini minimi?

Per esempio, facciamo l'albero delle visite:



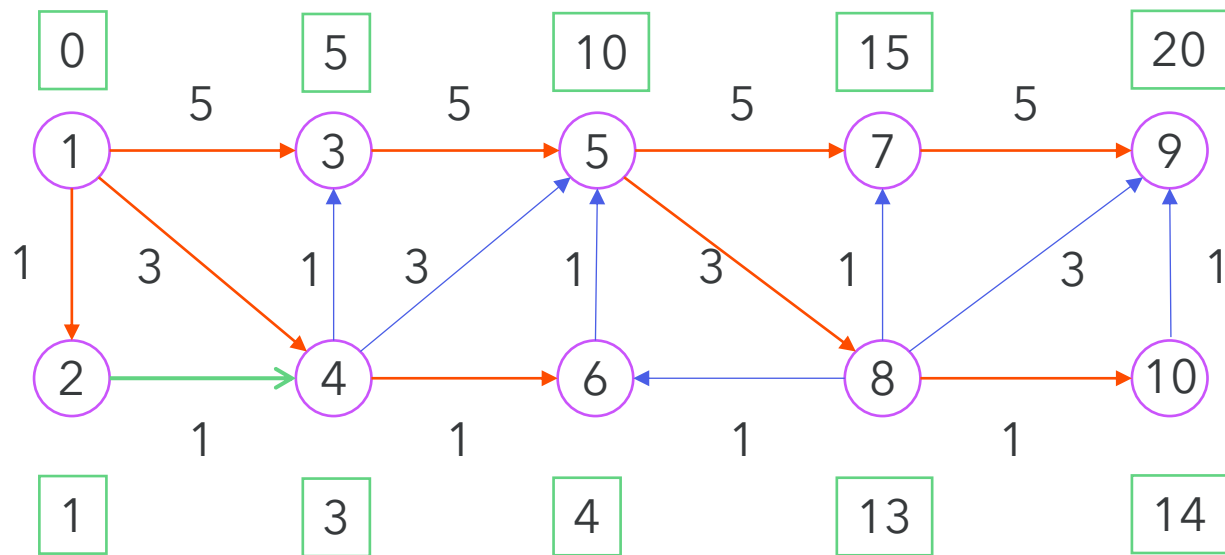
Esiste una
soluzione!

Già che c'ero potevo tenere conto delle lunghezze.



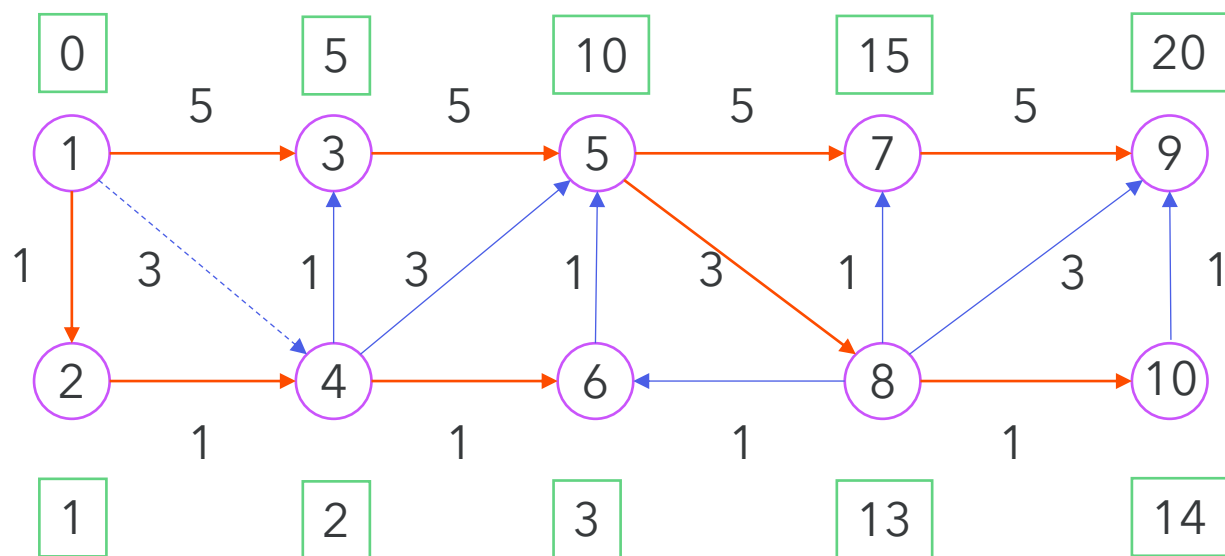
Quelle che ho usato sono le **etichette** dei nodi: un vettore di dimensione n che riporta il valore del cammino minimo sull'albero dalla radice r fino ad ogni nodo.

E' un albero con radice nella sorgente e che raggiunge tutti i nodi, però: **è dei cammini minimi?**

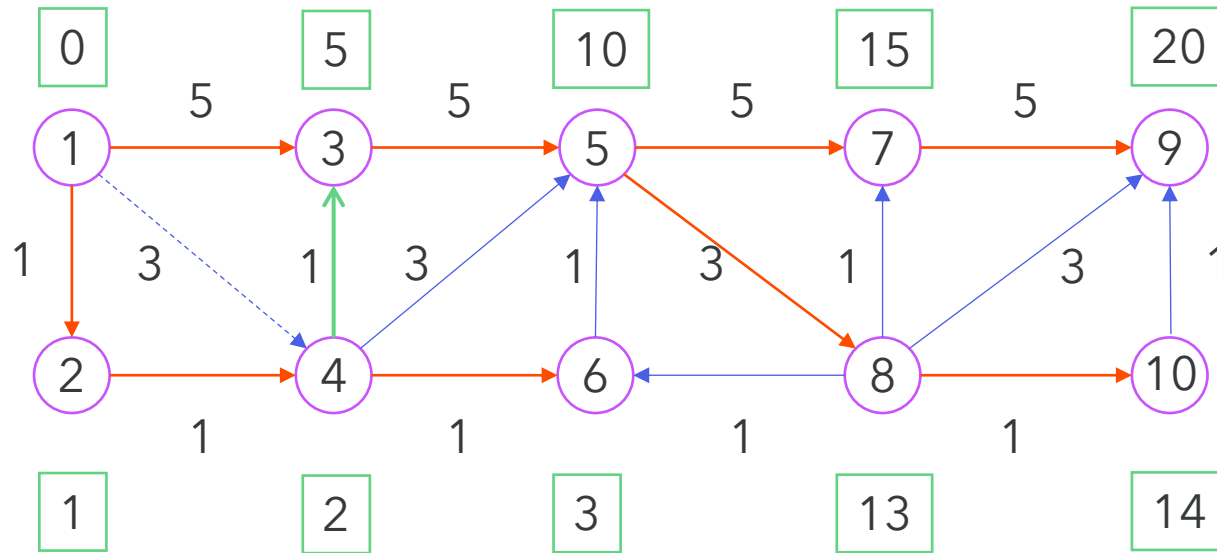


Valuto gli archi del grafo che non sono nell'albero. Migliorano la soluzione? Se sì cambio l'albero.

Valuto l'arco (2,4): migliora il cammino da r a 4. Aggiorno le etichette e l'albero.

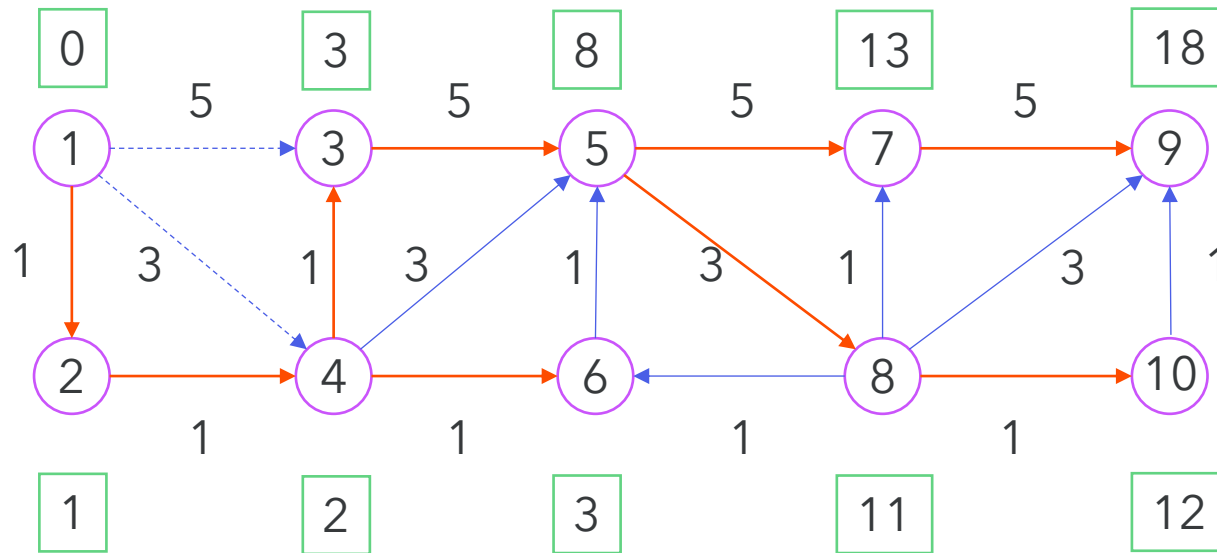


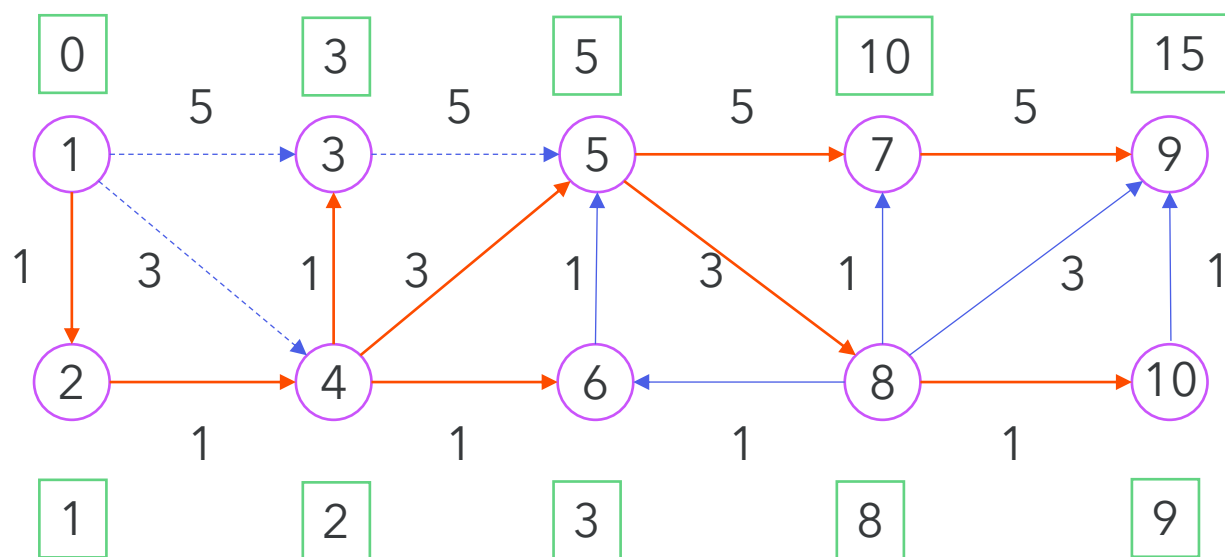
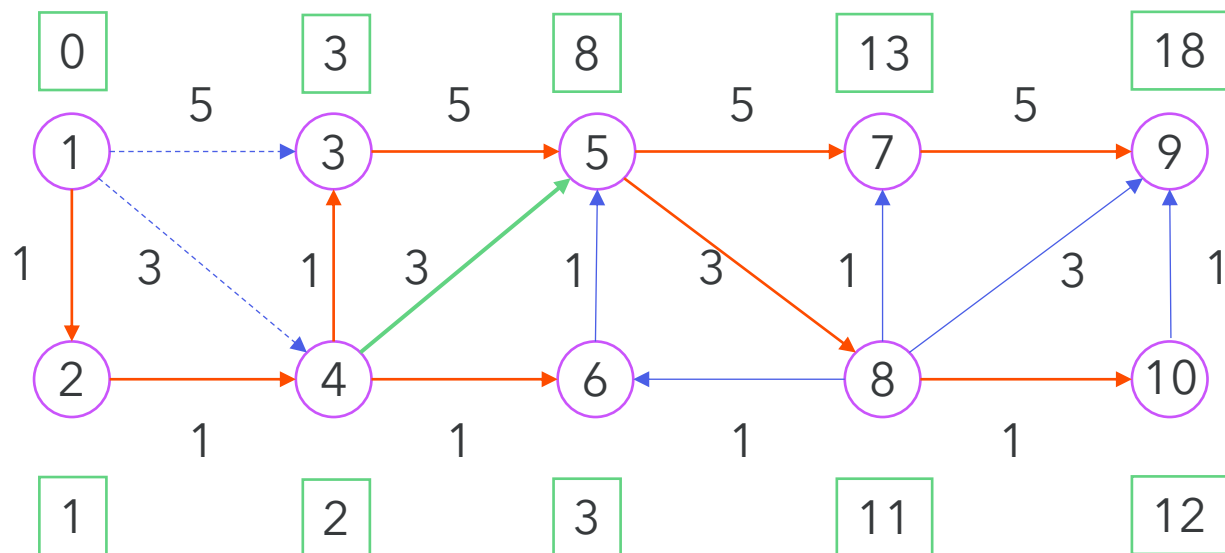
Devo cambiare anche le etichette a cascata da 4.



Valuto l'arco (4,3): migliora il cammino da r a 3. Aggiorno le etichette e l'albero.

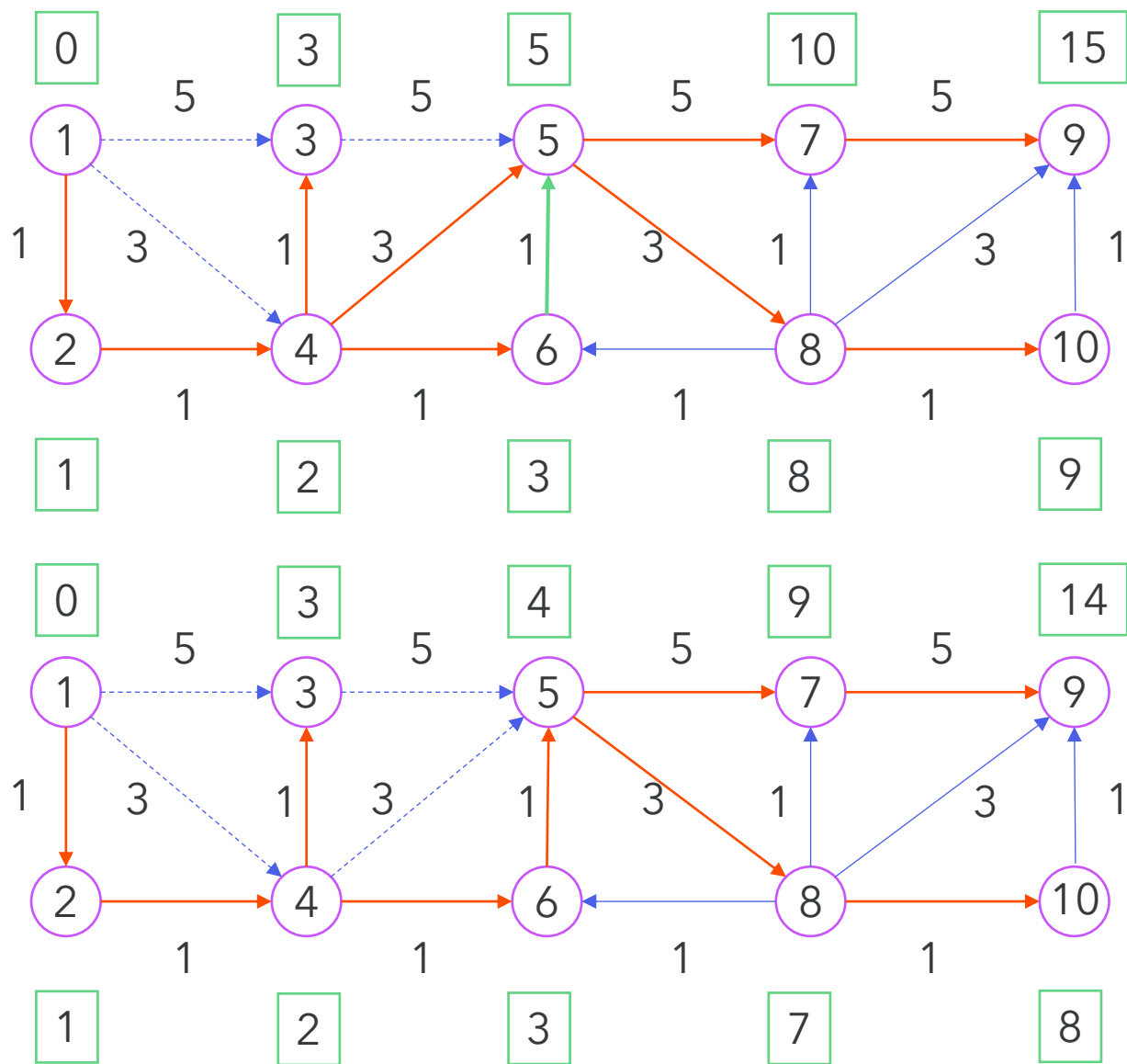
Devo cambiare anche le etichette a cascata da 3.





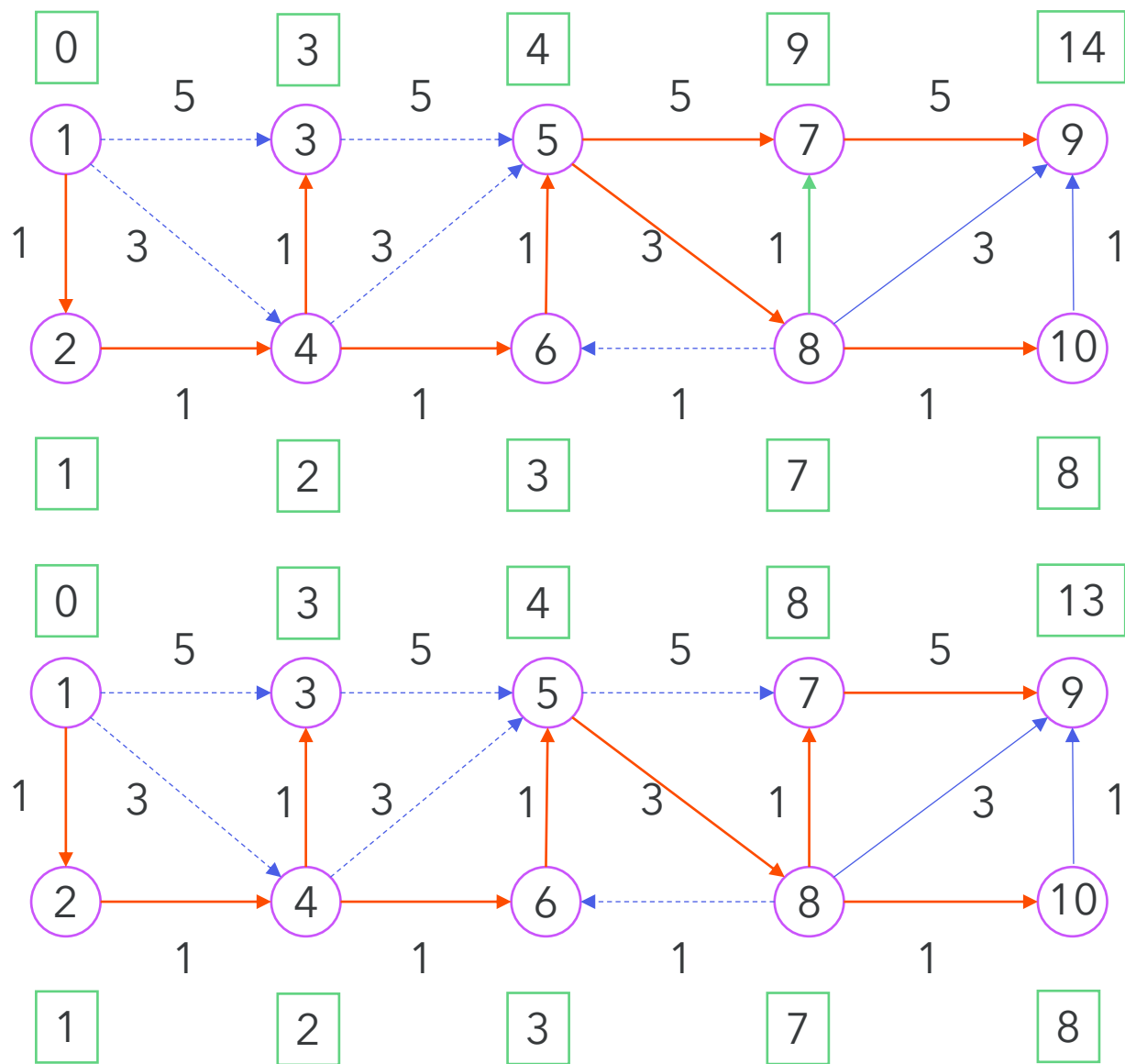
Valuto l'arco (4,5): migliora il cammino da r a 5. Aggiorno le etichette e l'albero.

Devo cambiare anche le etichette a cascata da 5.



Valuto l'arco (6,5): migliora il cammino da r a 5. Aggiorno le etichette e l'albero.

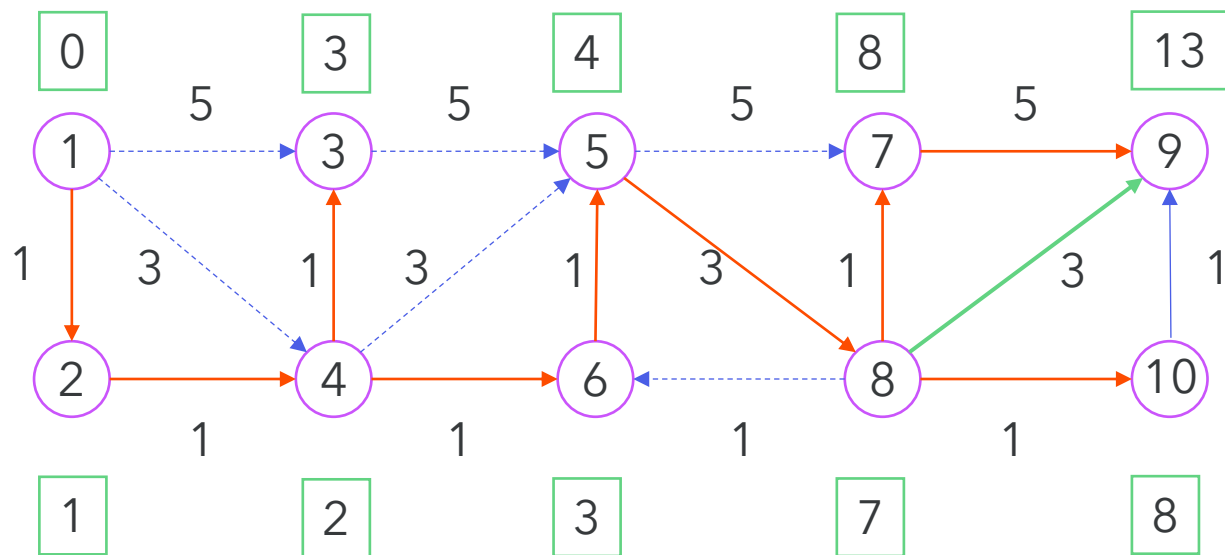
Devo cambiare anche le etichette a cascata da 5.



Valuto l'arco (8,6): non
migliora il cammino da r a 6.
Lo scarto.

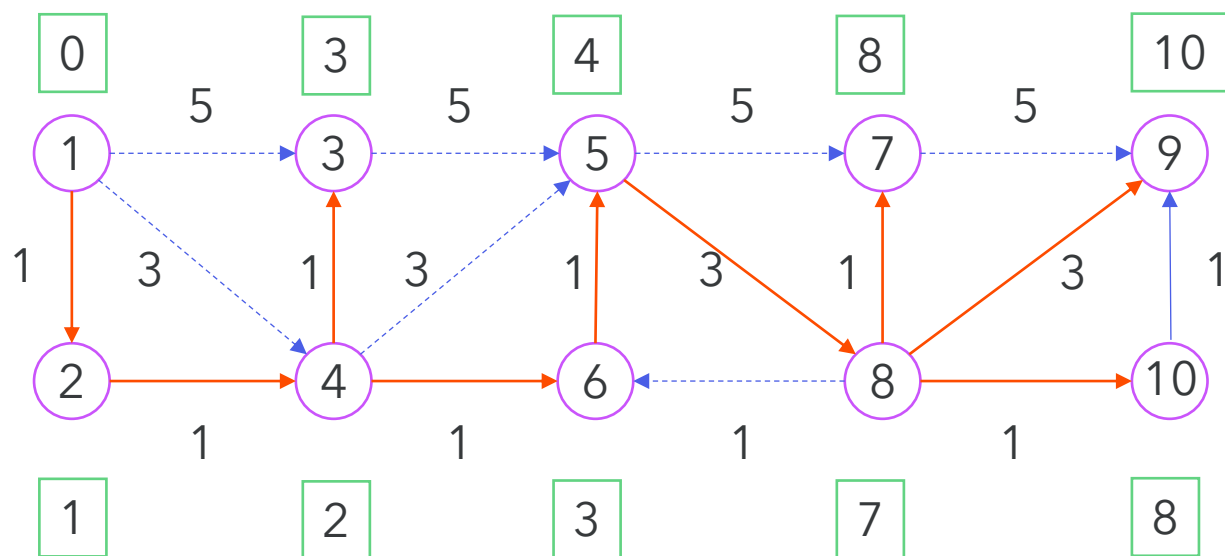
Valuto l'arco (8,7): migliora il
cammino da r a 7. Aggiorno
le etichette e l'albero.

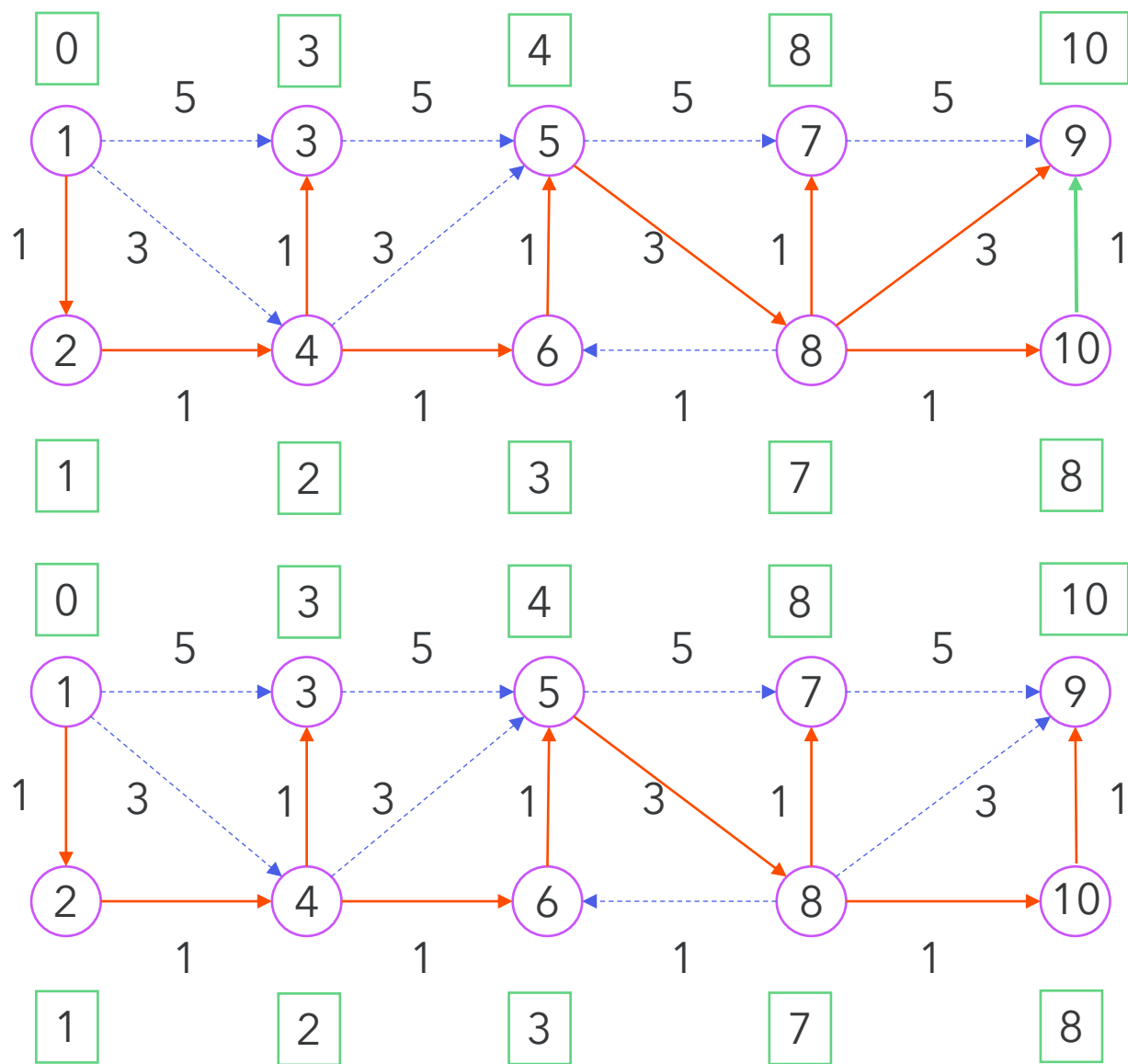
Devo cambiare anche le
etichette a cascata da 7.



Valuto l'arco (8,9): migliora il cammino da r a 9. Aggiorno le etichette e l'albero.

Cambio l'etichetta di 9.





Valuto l'arco (10,9): migliora il cammino da r a 9. Aggiorno le etichette e l'albero.

Cambio l'etichetta di 9.

ALBERI, ETICHETTE E CONDIZIONI DI OTTIMO

Obiettivo: verificare se un albero $T = (N, A_T)$ è **ottimo** per il problema (SPT).

- T è un **albero di copertura** radicato in r , con $|A_T| = n - 1$.
- Per ogni nodo $i \neq r$, esiste **un solo cammino** da r a i : P_i^T .
- T è **ottimo** se non esiste nessun cammino da r a i con costo inferiore a $C(P_i^T)$.

ALBERI, ETICHETTE E CONDIZIONI DI OTTIMO

Vettore delle etichette

Definiamo un vettore $d \in \mathbb{R}^n$ di etichette tale che:

$$\begin{aligned} d_i &= C(P_i^T), i \in N \\ d_r &= 0 \end{aligned}$$

d_i = costo del cammino da r a i nell'albero T .

Le etichette possono essere calcolate **percorrendo l'albero da r** .

Se l'arco $(i, j) \in A_T$, allora:

$$d_i + c_{ij} = d_j$$

ALBERI, ETICHETTE E CONDIZIONI DI OTTIMO

Verifica dell'ottimalità

Per ogni arco $(i, j) \notin A_T$, controlliamo se:

$$d_i + c_{ij} < d_j$$

- Se **sì**: esiste un cammino migliore di P_j^T e T **non è ottimo**.
- Se **no** per tutti gli archi: T **è ottimo**.

Sostituendo nell'albero l'arco (h, j) con (i, j) si ottiene un nuovo albero T' di costo inferiore.

CONDIZIONI DI BELLMAN

Le condizioni di **Bellman** per un vettore d sono:

$$\begin{aligned}d_i + c_{ij} &\geq d_j \quad \forall (i, j) \in A \\ d_r &= 0\end{aligned}$$

Se un vettore d soddisfa queste disuguaglianze:

Teorema di ottimalità (di Bellman)

Un albero $T = (N, A_T)$ è un **albero dei cammini minimi** di radice r

$\Leftrightarrow$

Il vettore di etichette d corrispondente **verifica le condizioni di Bellman**.

CONDIZIONI DI BELLMAN

Dimostrazione: SE

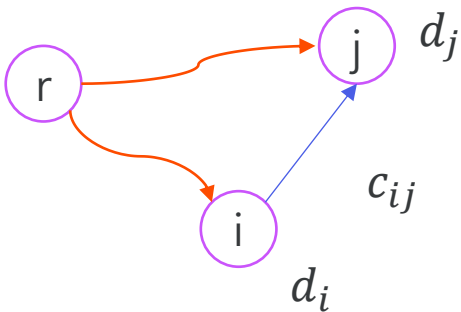
Un albero $T = (N, A_T)$ è un **albero dei cammini minimi** di radice r
 $\Rightarrow$
Il vettore di etichette d corrispondente **verifica le condizioni di Bellman**.

Dimostriamo per assurdo: non sono rispettate le condizioni di Bellman

Vuol dire che, dato un albero T con radice r , esistono un nodo i e un nodo j per cui:

$$d_i + c_{ij} < d_j$$

In T avrò un cammino da r a i e uno da r a j , ma se le condizioni di Bellman non sono rispettate, allora il valore d_j non è il valore del cammino minimo da r a j .



Allora T non è l'albero dei cammini ottimi

CONDIZIONI DI BELLMAN

Dimostrazione: SOLO SE

Un vettore di etichette d **verifica le condizioni di Bellman**

$\Rightarrow$

L'albero corrispondente $T = (N, A_T)$ di radice r è un **albero dei cammini minimi**.

Dimostrazione attraverso il Lemma:

Lemma di Bellman

Se un vettore d verifica le condizioni di Bellman, allora per ogni $i \neq r$:

$$d_i \leq C(P_i)$$

per ogni cammino P_i da r a i .

Ossia, se un vettore d rispetta le condizioni di Bellman (e $d_r = 0$), allora preso qualunque cammino (anche non semplice) da r a i su un albero dato, il costo di quel cammino non può superare il valore dell'etichetta d_i .

CONDIZIONI DI BELLMAN

Dimostrazione: SOLO SE

Un vettore di etichette d **verifica le condizioni di Bellman**

$\Rightarrow$

L'albero corrispondente $T = (N, A_T)$ di radice r è un **albero dei cammini minimi**.

Il cammino P_{ri} è formato da archi:

$$(i_1, i_2), (i_2, i_3), \dots, (i_{k-1}, i_k) \\ \text{Con } i_1 = r \text{ e } i_k = i.$$

Scriviamo le condizioni di Bellman per tutti gli archi:

$$\begin{aligned} d_{i_1} + c_{i_1 i_2} &\geq d_{i_2} \\ d_{i_2} + c_{i_2 i_3} &\geq d_{i_3} \\ &\dots \\ d_{i_{k-1}} + c_{i_{k-1} i_k} &\geq d_{i_k} \end{aligned}$$

Sommando le disequazioni (che si mantengono) e ricordando che $d_r = 0$ otteniamo:

$$d_{i_1} + c(P_{i_1 i_k}) \geq d_{i_k}$$



$$c(P_{ir}) \geq d_i$$

Lemma
dimostrato

CONDIZIONI DI BELLMAN

Dimostrazione: SOLO SE

Un vettore di etichette d **verifica le condizioni di Bellman**

$\Rightarrow$

L'albero corrispondente $T = (N, A_T)$ di radice r è un **albero dei cammini minimi**.

Dimostrando il Lemma, sappiamo che dato un vettore di etichette che rispetta le condizioni di Bellman, non possiamo trovare cammini di costo minore rispetto alle etichette, perciò l'albero corrispondente è quello dei cammini minimi.

Ora siamo pronti per scrivere un algoritmo!

VERIFICA D'OTTIMALITÀ DI UN ALBERO DEI CAMMINI MINIMI

L'albero dei cammini minimi deve rispettare le condizioni di Bellman.

Al momento abbiamo visto che possiamo:

1. Trovare un albero $T(N, A_T)$ radicato in r :
 - a. Facendo una visita del grafo A
 - b. E calcolando le etichette d nel frattempo.
2. Prendere gli archi del grafo una alla volta e verificare
 - a. Se trovo un arco che viola le condizioni di Bellman devo aggiornare l'albero e le etichette a cascata.
 - b. Se tutti gli archi verificano le condizioni di Bellman allora l'albero è ottimo.

In pratica devo fare molte visite del grafo e vorrei un algoritmo più efficiente

ALGORITMO SPT

```
procedure (p, d) = SPT(G, c, r) {  
  // Inizializzazione  
  foreach (i ∈ N) do { p[i] = r; d[i] = M; }  
  d[r] = 0;  
  Q = { r };  
  
  // Costruzione dell'albero dei cammini  
  minimi  
  do {  
    select i from Q;  
    Q = Q \ { i };  
  
    foreach ((i, j) ∈ FS(i)) do  
      if (d[i] + cij < d[j]) then {  
        d[j] = d[i] + cij;  
        p[j] = i;  
        Q = Q ∪ { j };  
      }  
  } while (Q ≠ ∅);  
}
```

Input: Grafo G , costi degli archi c e nodo radice r .

Output:

p : etichette dei predecessori

d : etichette della lunghezza dei cammini

Inizializzazione:

Inizializziamo tutte le etichette p al nodo radice r .
E tutte le etichette d ad un valore molto grande M .
Poi il costo per arrivare in r lo fissiamo a 0
E inizializziamo una lista Q a r .

Costruzione dell'albero:

Prendiamo un nodo i dalla lista Q e valutiamo gli archi uscenti (i, j) dal nodo i . Se c'è un arco (i, j) che non rispetta le condizioni di Bellman, allora aggiorniamo l'etichetta dei costi d_j e fissiamo il predecessore di j a i . Aggiungo il nodo j alla lista Q . Proseguo fino allo svuotamento della lista.

INIZIALIZZAZIONE

Ricordiamo che $r = 1$.

Inizializziamo mettendo le etichette d tutte ad un valore molto alto (per esempio $M = (n - 1)c_{\max} + 1$).

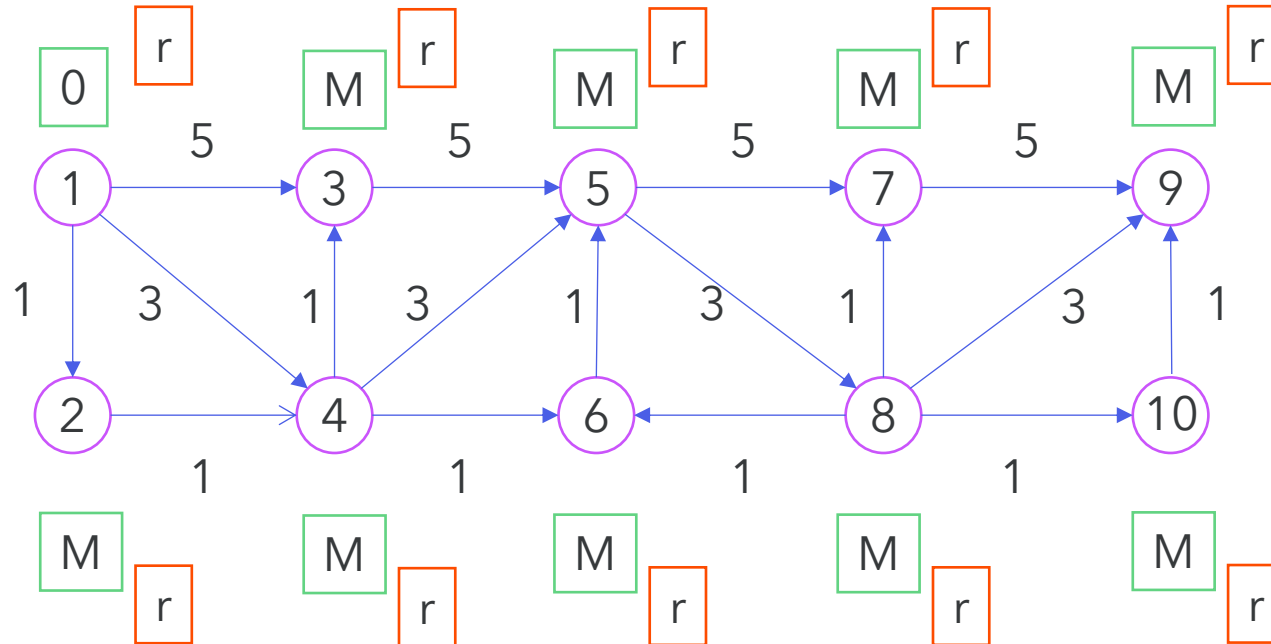
Inizializziamo mettendo tutte le etichette p a r .

E' come inserire un arco fittizio da r ad ogni i con costo M .
Se poi un nodo alla fine dell'algoritmo rimane con l'etichetta M allora quel nodo non era raggiungibile.

Poi il costo per arrivare in r lo fissiamo a 0.

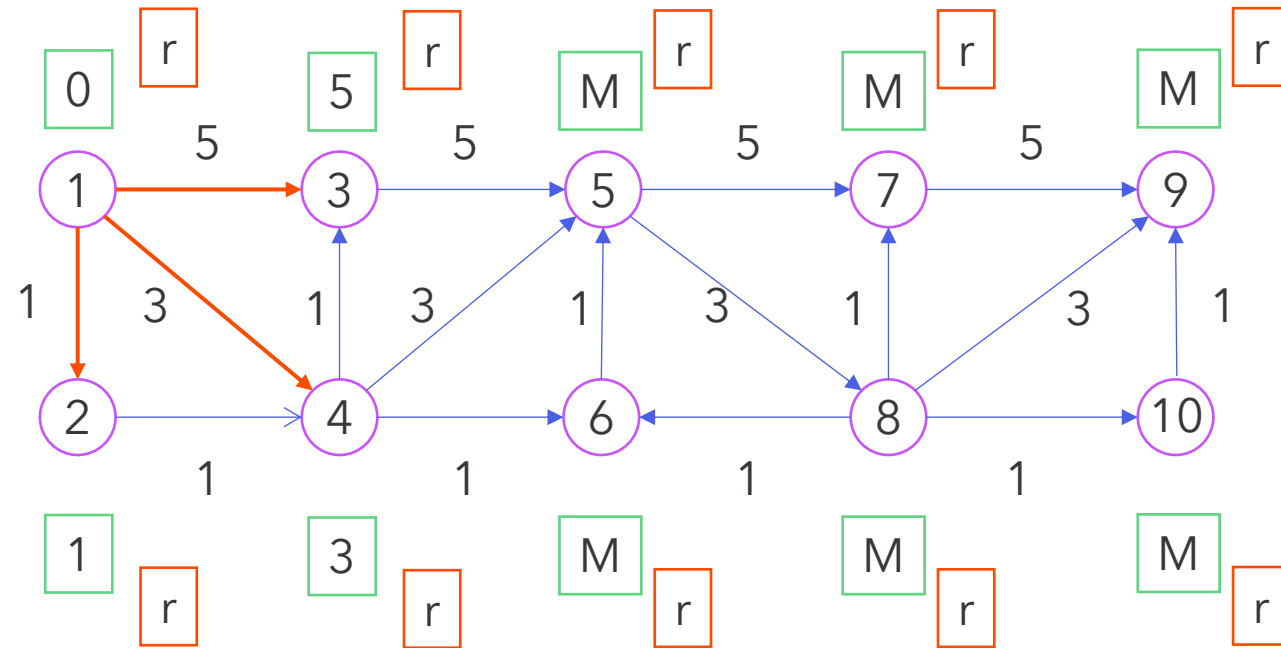
E' come dire che c'è un arco (r, r) di costo 0.

Nella lista Q c'è solo r .



$Q = \{1\}$

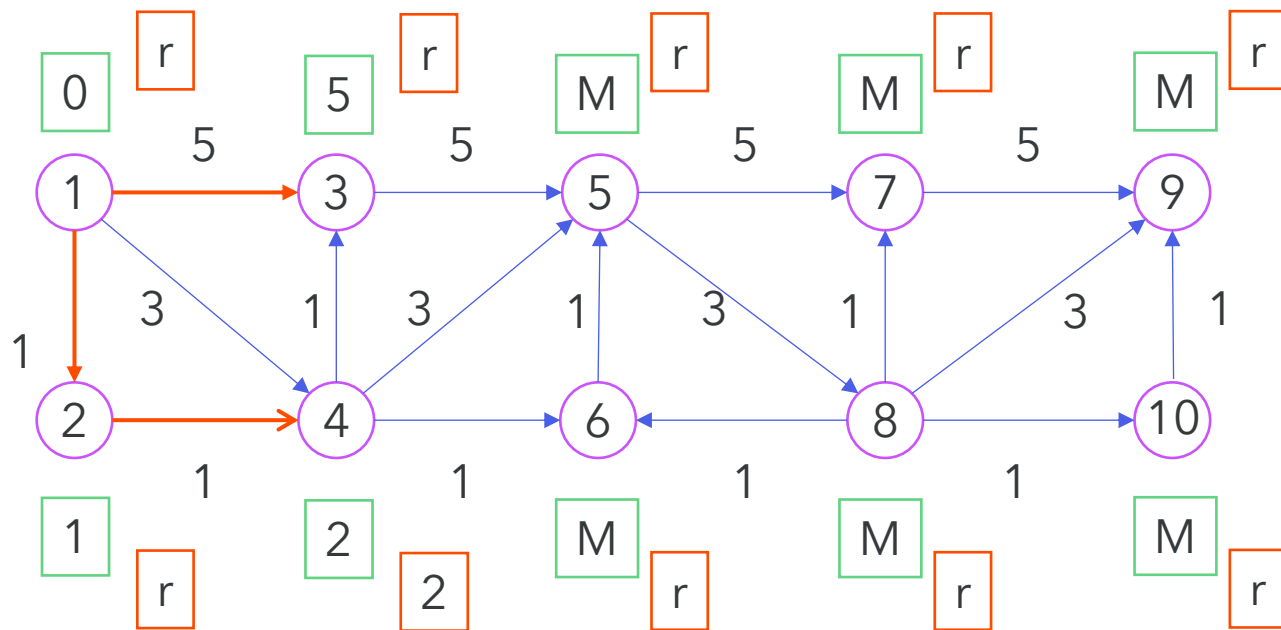
COSTRUZIONE DELL'ALBERO



$Q = \{1\}$ → $Q = \{2,3,4\}$

Nella lista Q c'è solo r .
Valuto la stella uscente di r .
Come lo faccio influirà sulla visita della coda Q . Noi fissiamo un ordinamento:
ordine lessicografico

Gli archi uscenti da 1 violano tutti le condizioni di Bellman.
Aggiorno tutte le etichette d , aggiornò i predecessori p e la lista Q .

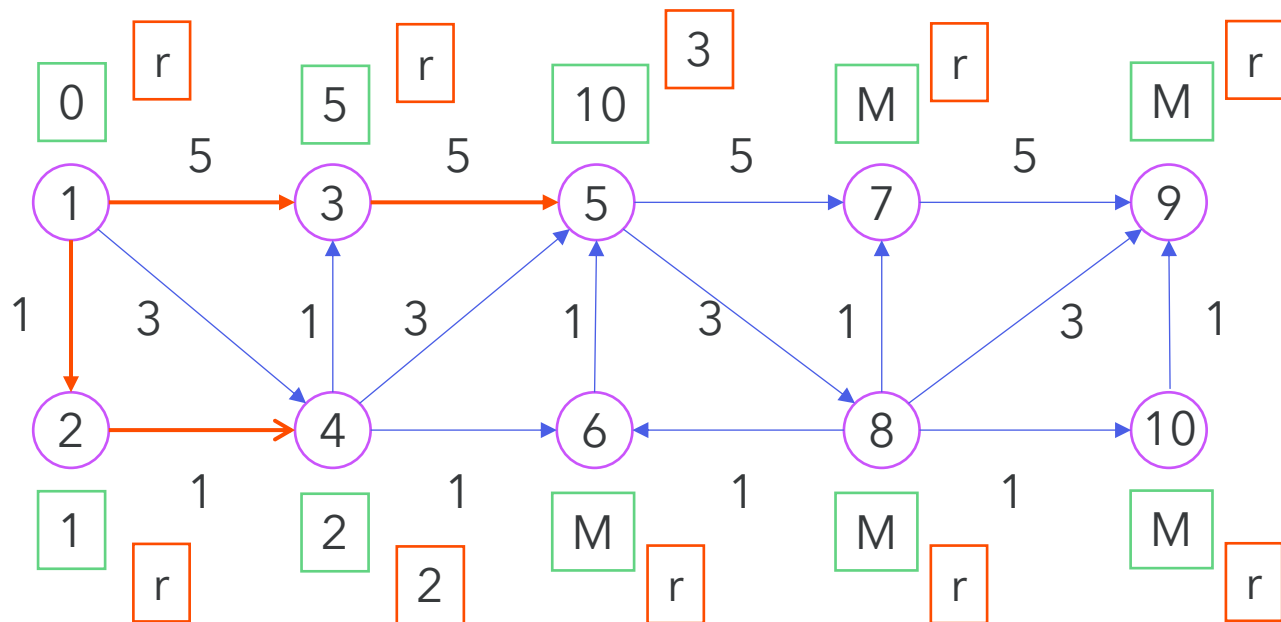


$Q = \{2,3,4\}$



$Q = \{3,4\}$

Passo al seguente nodo in lista: 2.
Non si rispettano le condizioni di Bellamn: aggiorno le etichette di 4 e la lista.

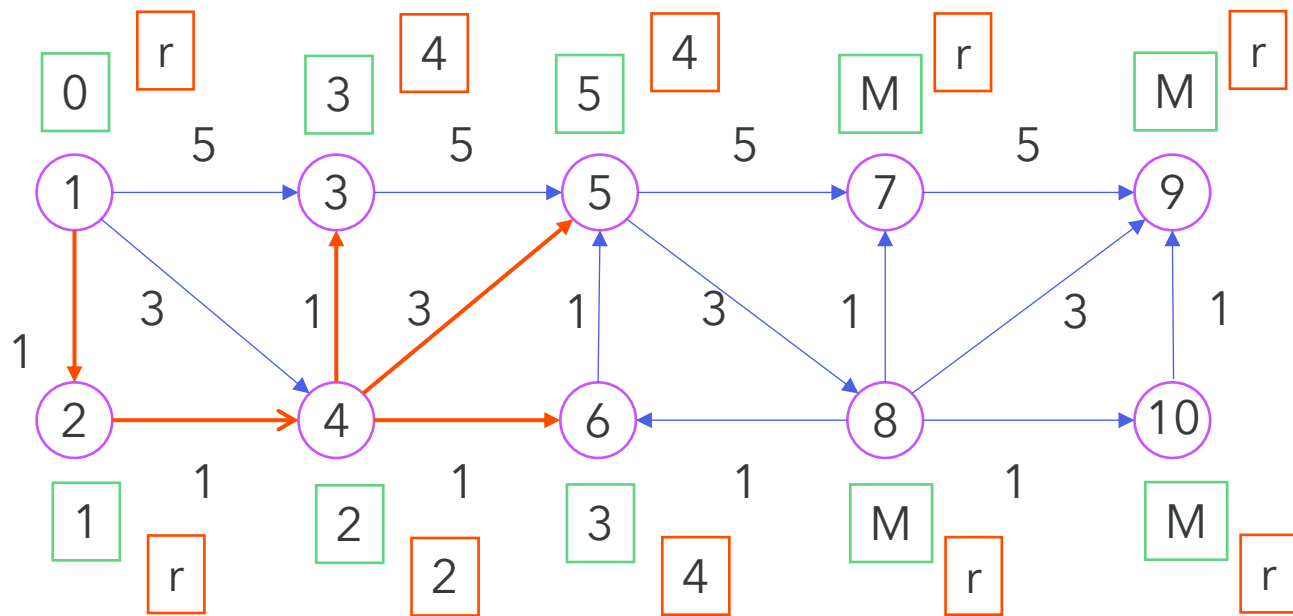


$Q = \{3,4\}$



$Q = \{4,5\}$

Passo al seguente nodo in lista: 3. Non si rispettano le condizioni di Bellman: aggiorno le etichette di 5 e la lista.



$Q = \{4,5\}$



$Q = \{3,5,6\}$

Passo al seguente nodo in lista: 2.
Non si rispettano le condizioni di Bellman: agguirno le etichette di 3, 5, e 6 e la lista.

$Q = \{3,5,6\}$



$Q = \{5,6\}$

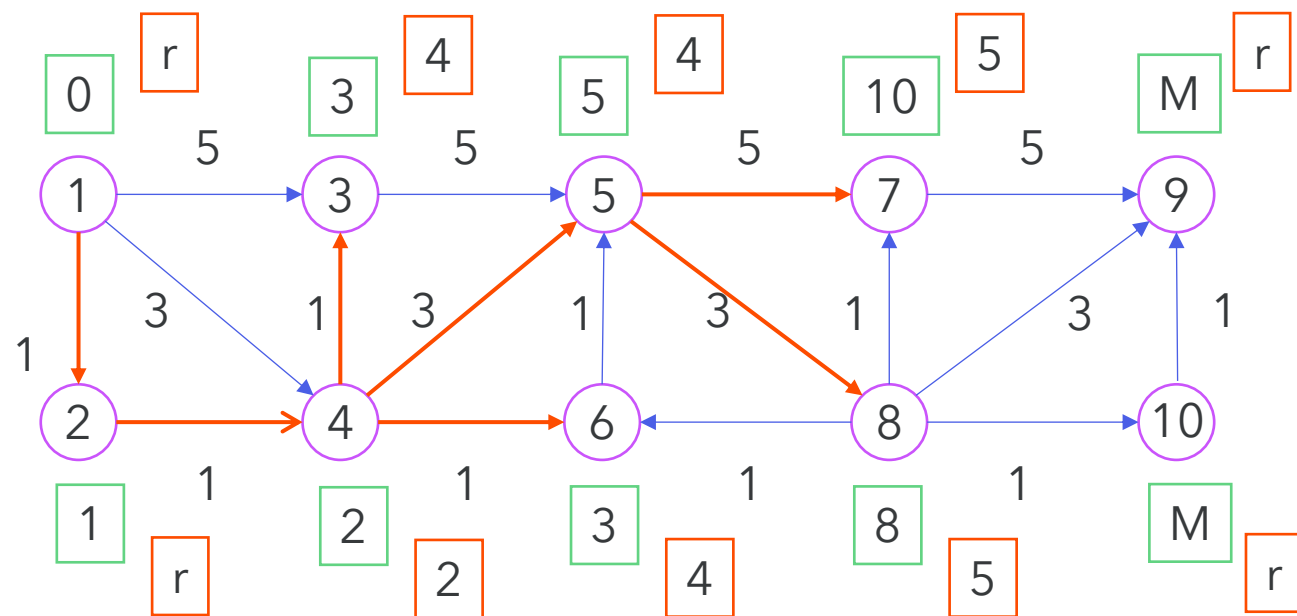
Da 3 non ci sono archi che violino le condizioni di Bellman

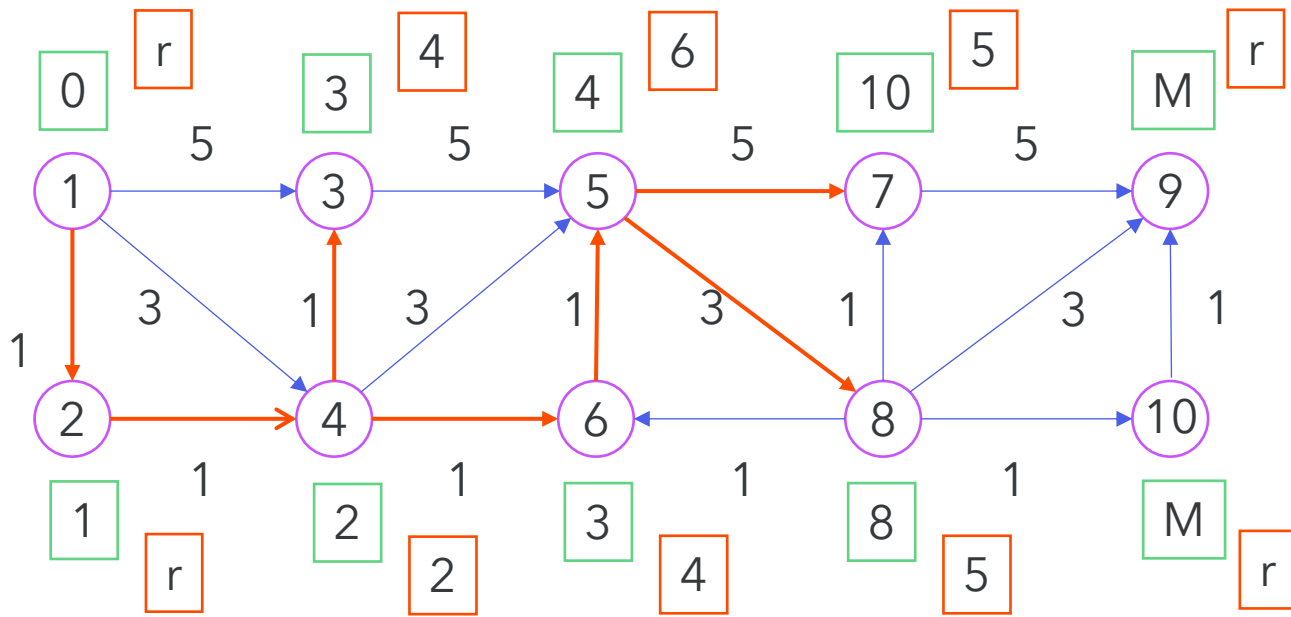
$Q = \{5,6\}$



$Q = \{6,7,8\}$

Passo al seguente nodo in lista: 5. Non si rispettano le condizioni di Bellman: agguirno le etichette di 7 e 8 e la lista.





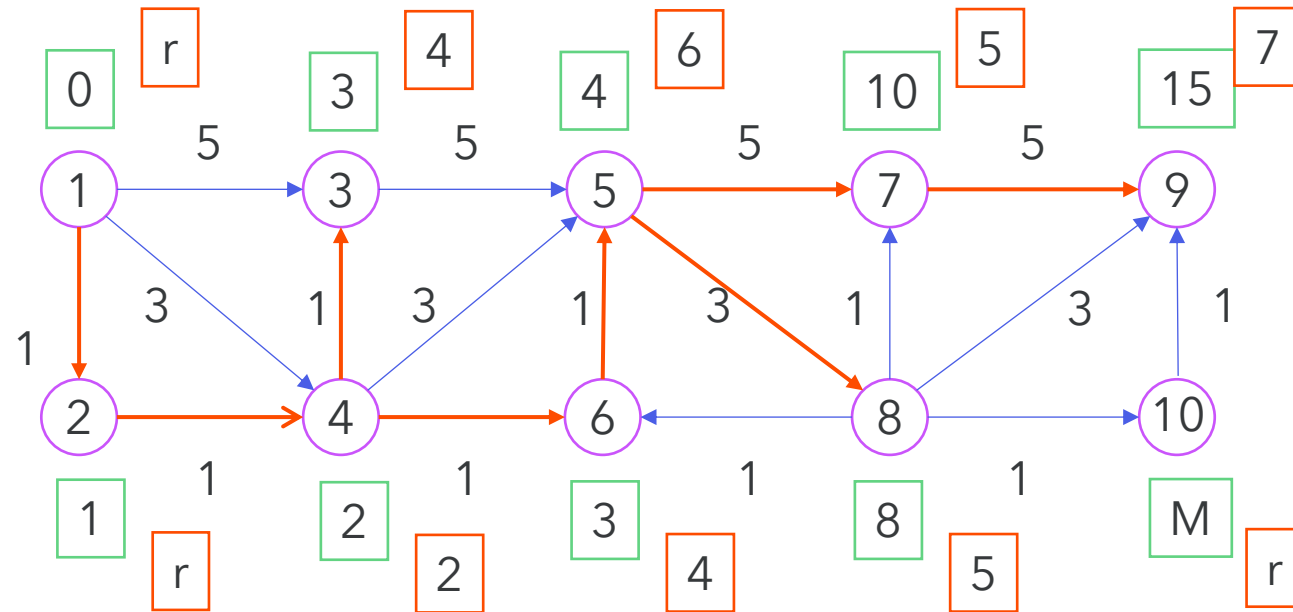
$Q = \{6,7,8\}$



$Q = \{7,8,5\}$

Passo al seguente nodo in lista: 6. Non si rispettano le condizioni di Bellman: aggiornò le etichette di 5, che torna in lista.

Non si aggiornano le etichette a cascata, perché le aggiornerò introducendo più volte i nodi in lista.

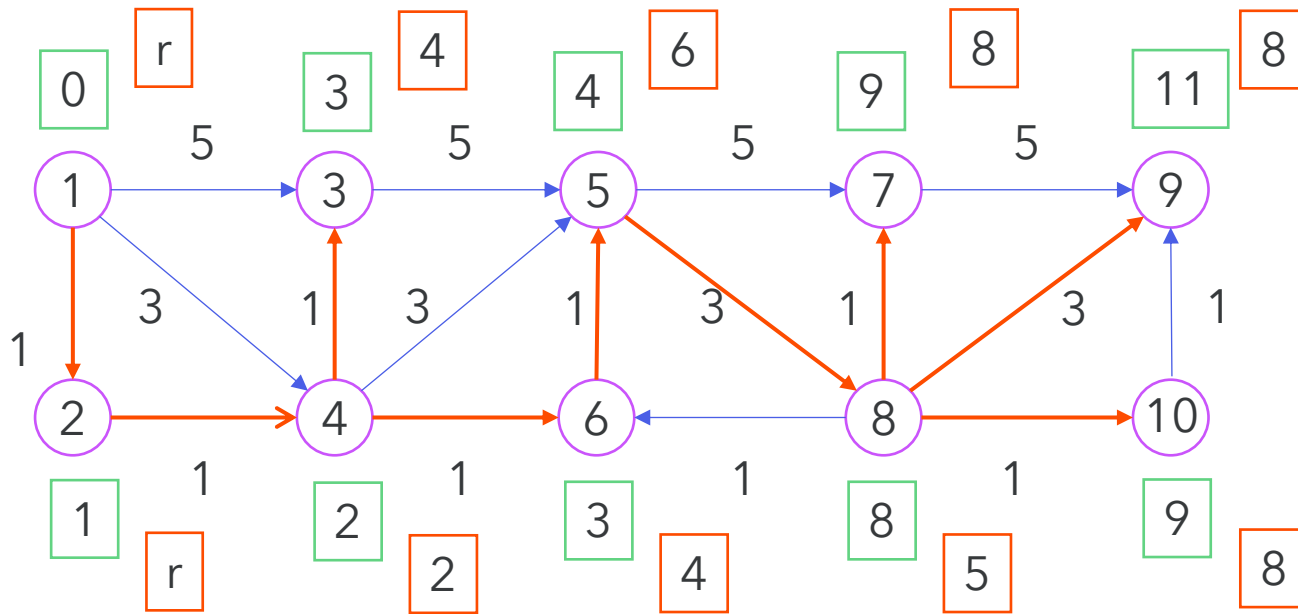


$Q = \{7,8,5\}$



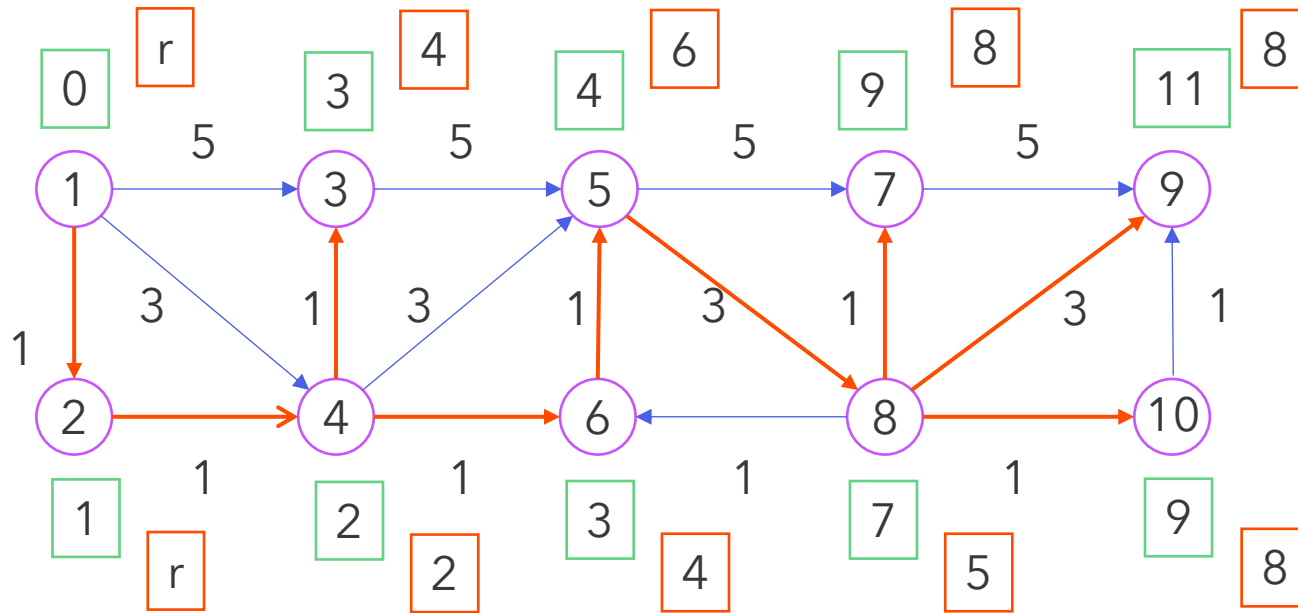
$Q = \{8,5,9\}$

Passo al seguente nodo in lista: 7. Le etichette non rispettano le condizioni di Bellman: aggiornò le etichette di 9 e la lista.



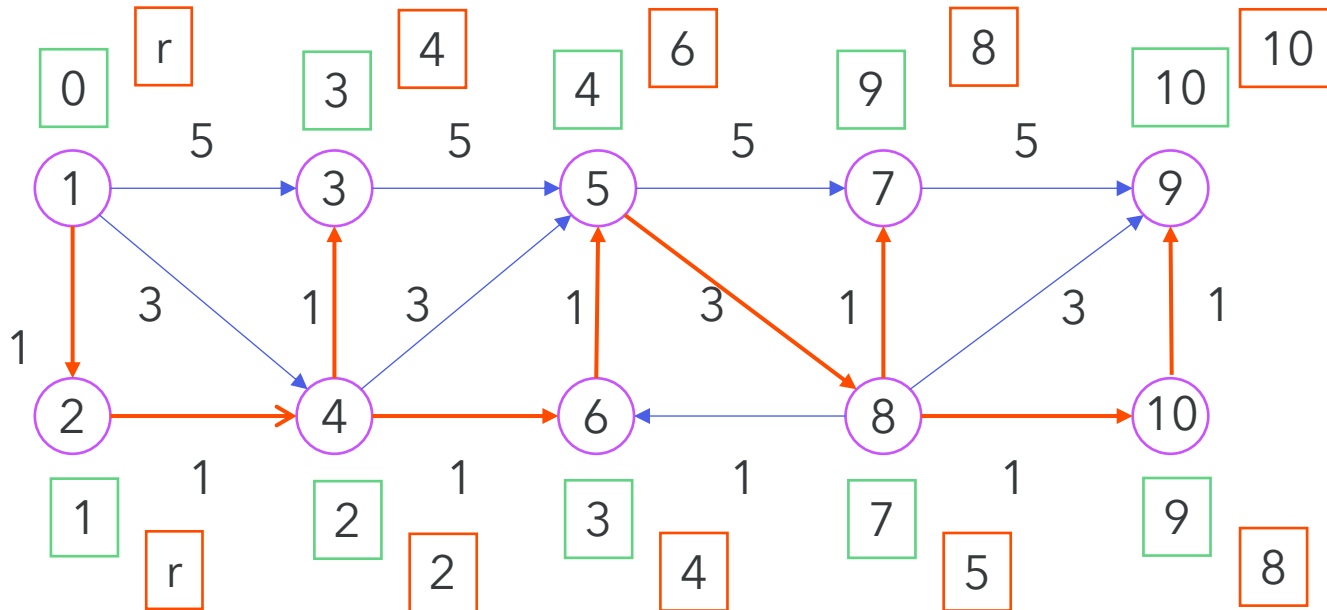
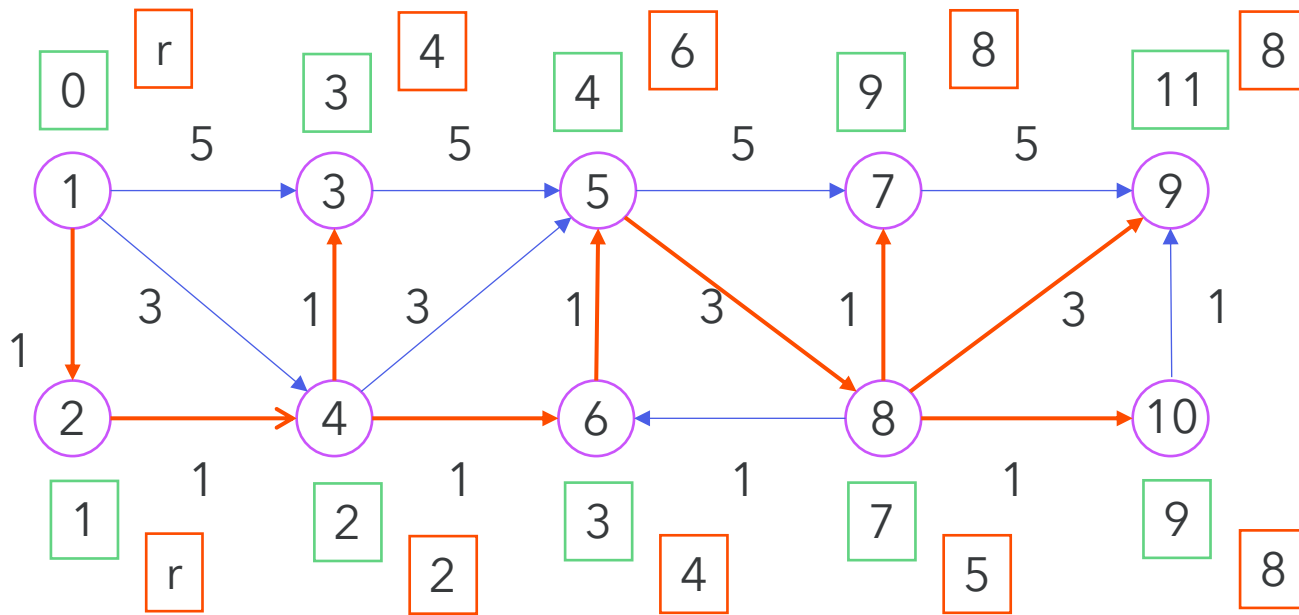
$Q = \{8,5,9\}$ $\rightarrow$ $Q = \{5,9,7,10\}$

Passo al seguente nodo in lista: 8. Non si rispettano le condizioni di Bellman: aggiorniamo le etichette di 7, 9 e 10, e la lista.



$Q = \{5,9,7,10\}$ $\rightarrow$ $Q = \{9,7,10,8\}$

Passo al seguente nodo in lista: 5. Le etichette non rispettano le condizioni di Bellman: aggiorniamo le etichette di 8 e la lista.



$Q = \{9, 7, 10, 8\}$ $\Rightarrow$ $Q = \{7, 10, 8\}$

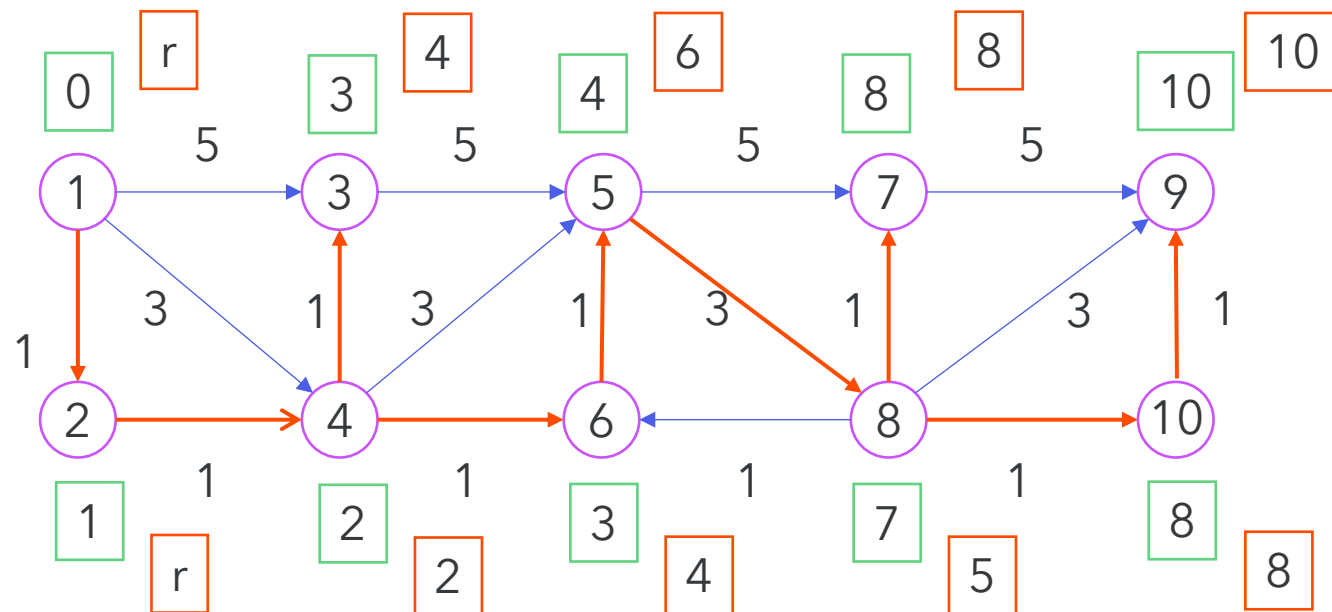
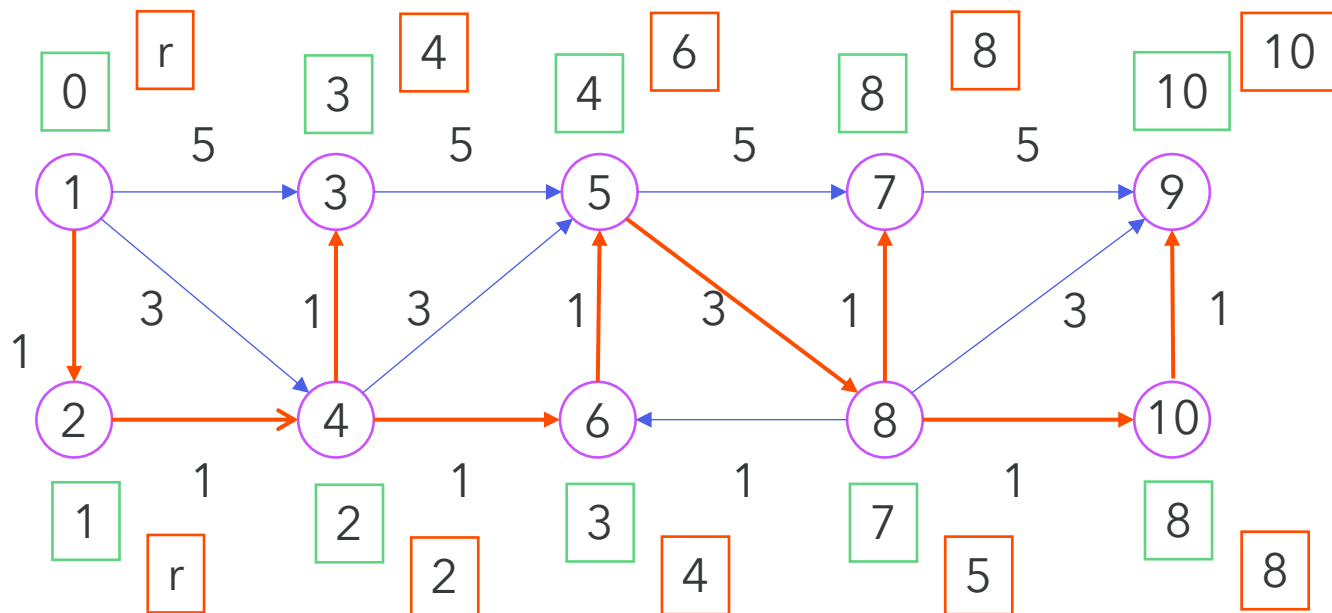
Passo al seguente nodo in lista: 9. Non ha nodi uscenti.

$Q = \{7, 10, 8\}$ $\Rightarrow$ $Q = \{10, 8\}$

Passo al seguente nodo in lista: 7. Le etichette rispettano le condizioni di Bellman.

$Q = \{10, 8\}$ $\Rightarrow$ $Q = \{8, 9\}$

Passo al seguente nodo in lista: 10. Le etichette non i rispettano le condizioni di Bellman: aggiorno le etichette di 9 e la lista.



$Q = \{8, 9\}$ $\Rightarrow$ $Q = \{9, 7, 10\}$

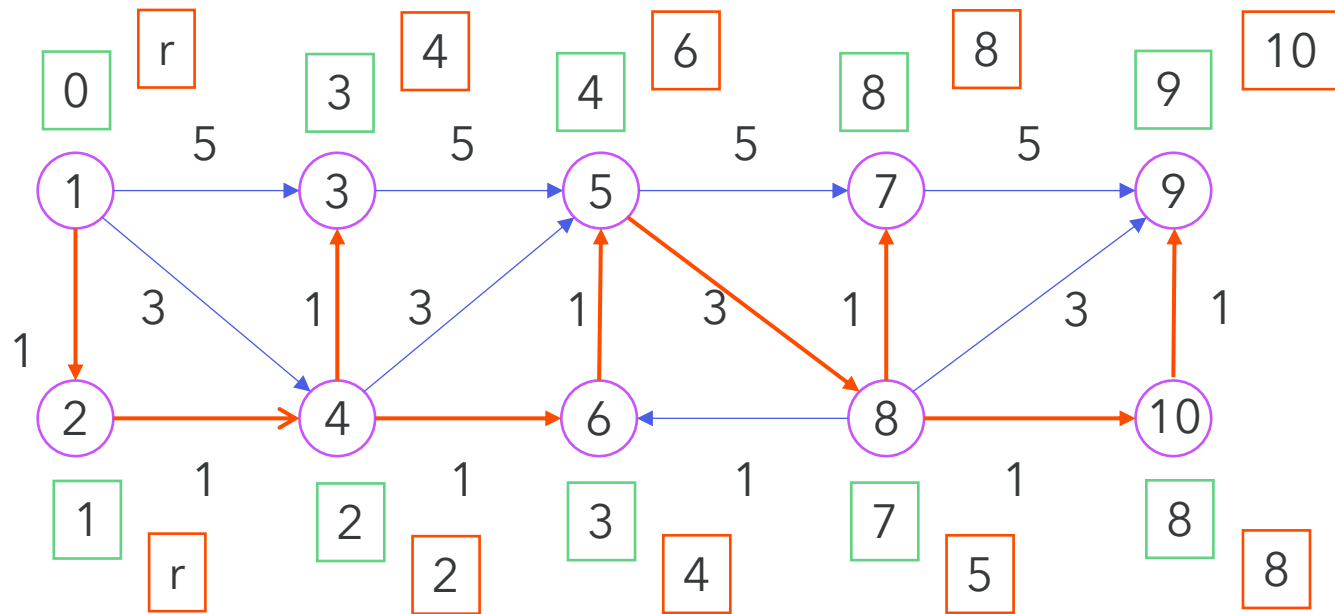
Non si rispettano le condizioni di Bellman: aggiornò le etichette di 7 e 10, e la lista.

$Q = \{9, 7, 10\}$ $\Rightarrow$ $Q = \{7, 10\}$

Passo al seguente nodo in lista: 9. Non ha archi uscenti.

$Q = \{7, 10\}$ $\Rightarrow$ $Q = \{10\}$

Passo al seguente nodo in lista: 7. Le etichette rispettano le condizioni di Bellman



$Q = \{10\}$ → $Q = \{9\}$

Passo al seguente nodo in lista: 10.
Non si rispettano le condizioni di Bellman: aggiorno le etichette di 9 e la lista.

$Q = \{9\}$ → $Q = \emptyset$

Passo al seguente nodo in lista: 9. Non ci sono archi uscenti. La lista è vuota, termina l'algoritmo.

Ho ottenuto **l'albero dei cammini minimi**

ALGORITMO SPT.L.QUEUE (BELLAMN)

Questo algoritmo si chiama **SPT.L.Queue** o Algoritmo di Bellamn-Ford.

Abbiamo notato che i nodi possono rientrare in coda.

L'algoritmo può terminare?

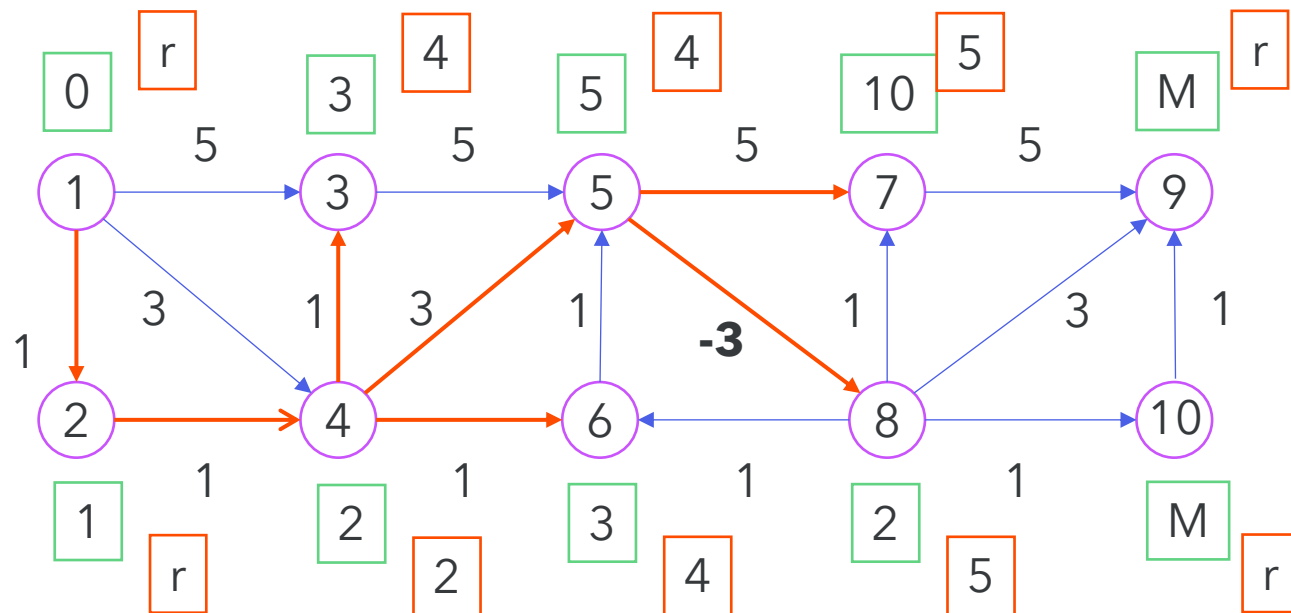
ALGORITMO SPT.L.QUEUE (BELLAMN)

Questo algoritmo si chiama **SPT.L.Queue** o Algoritmo di Bellamn-Ford.

Abbiamo notato che i nodi possono rientrare in coda.

L'algoritmo può terminare?

L'algoritmo non termina quando ci sono dei cicli orientati di costo negativo.



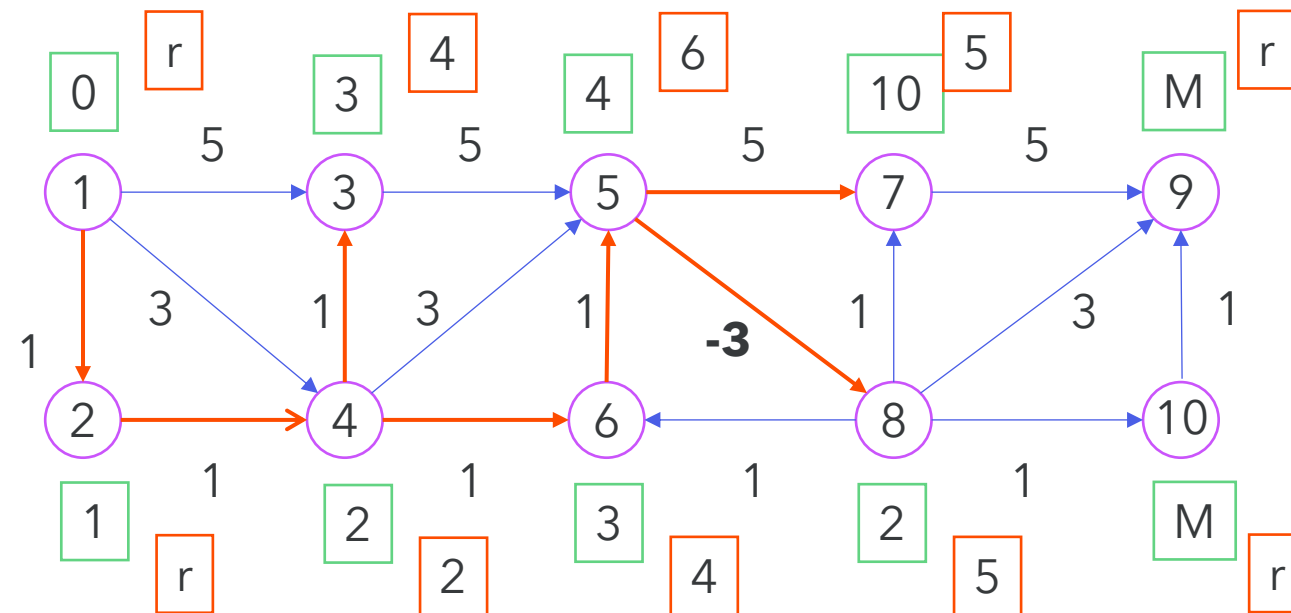
Supponiamo di essere arrivati a visitare il nodo 5 e la sua stella uscente.

$Q = \{5, 6\}$



$Q = \{6, 7, 8\}$

Passo al seguente nodo in lista: 5. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorniamo le etichette di 7 e 8 e la lista.

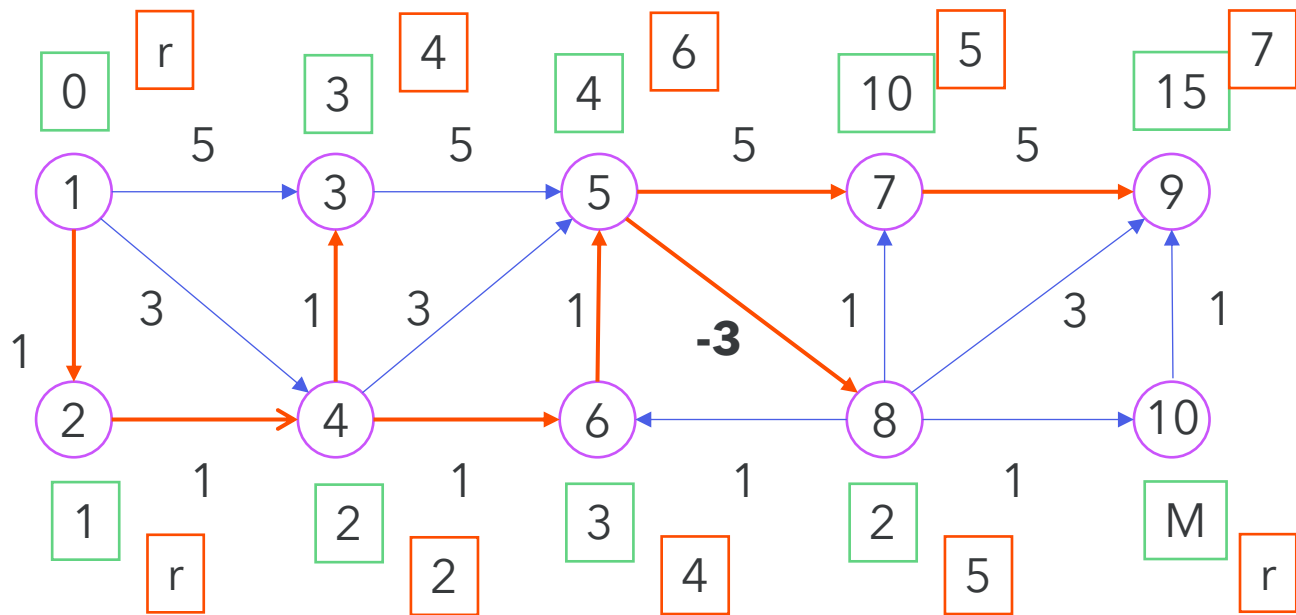


$Q = \{6, 7, 8\}$



$Q = \{7, 8, 5\}$

Passo al seguente nodo in lista: 6. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorniamo le etichette di 5 e la lista.

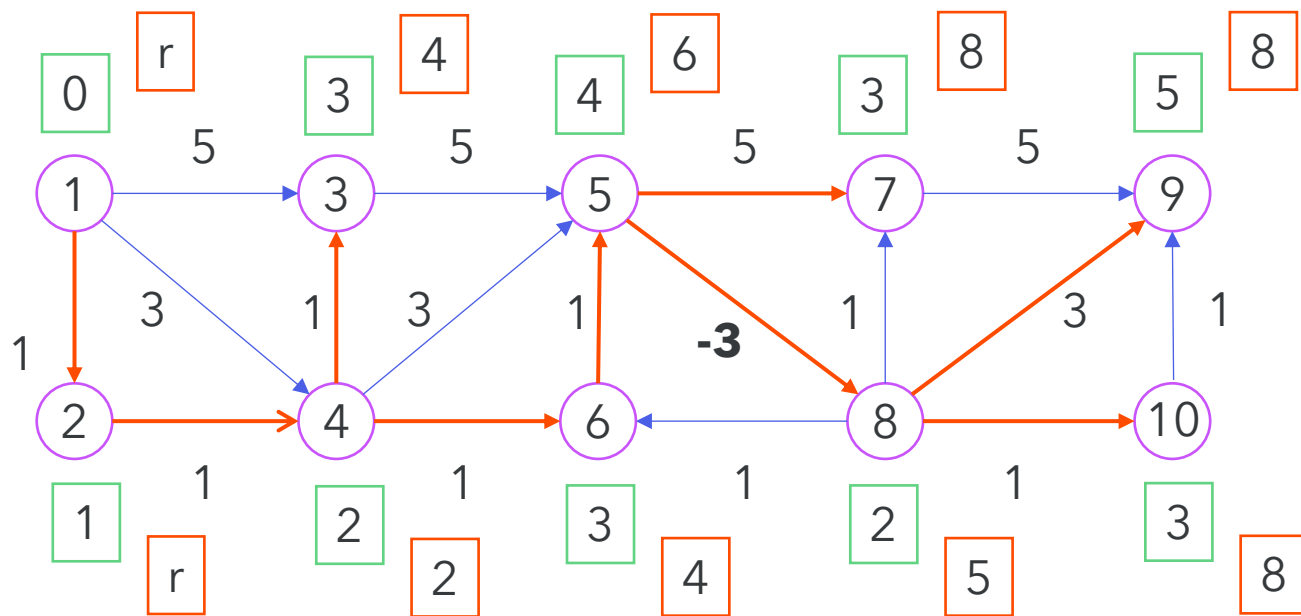


$Q = \{7, 8, 5\}$



$Q = \{8, 5, 9\}$

Passo al seguente nodo in lista: 7. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorniamo le etichette di 9 e la lista.

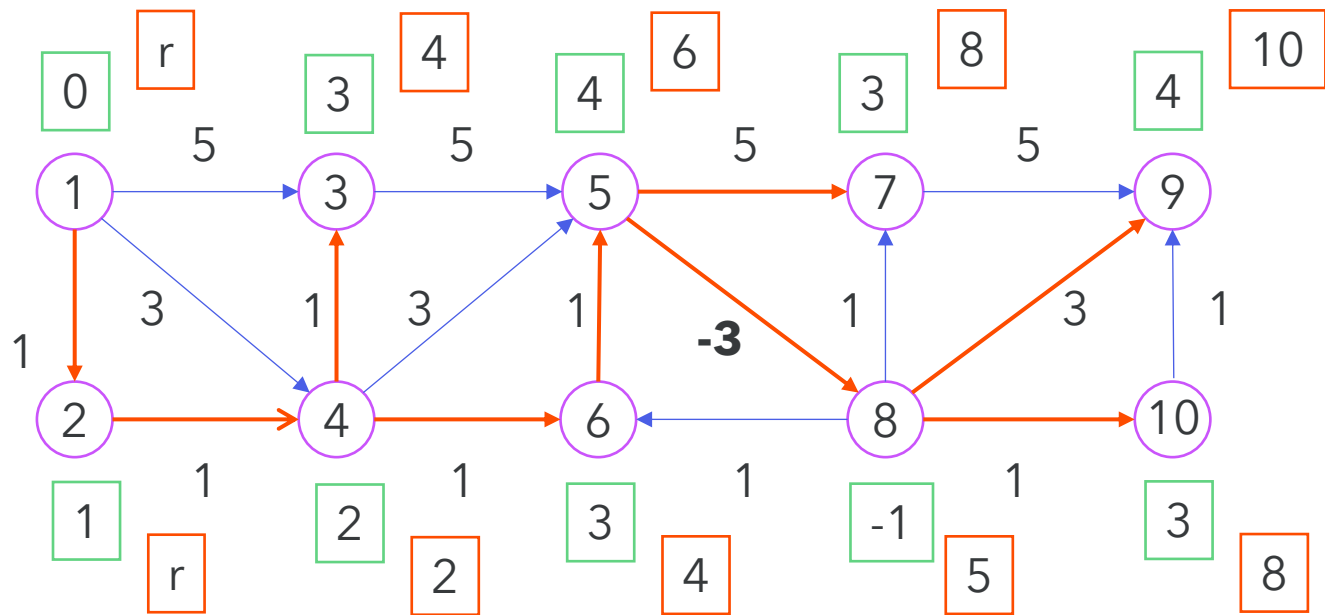
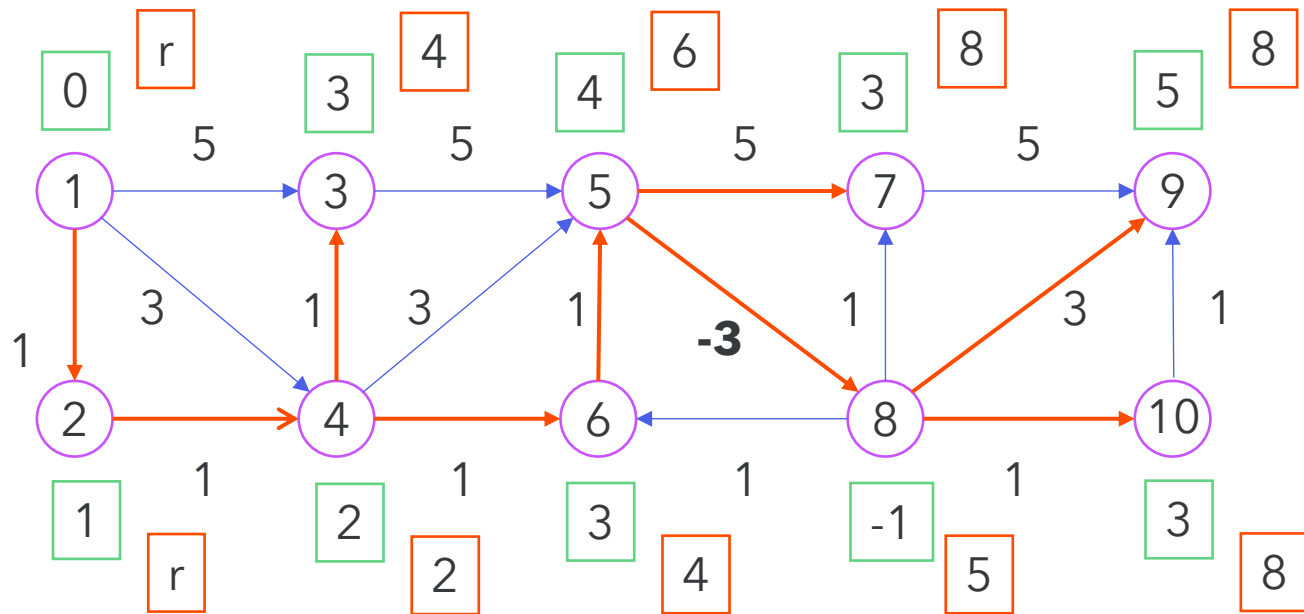


$Q = \{8, 5, 9\}$



$Q = \{5, 9, 7, 10\}$

Passo al seguente nodo in lista: 8. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorniamo le etichette di 7, 9 e 10 e la lista.



$Q = \{5, 9, 7, 10\}$



$Q = \{9, 7, 10, 8\}$

Passo al seguente nodo in lista: 5. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorno le etichette di 8 e la lista.

$Q = \{9, 7, 10, 8\}$



$Q = \{7, 10, 8\}$

Passo al seguente nodo in lista: 9. Non ha archi uscenti.

$Q = \{7, 10, 8\}$



$Q = \{10, 8\}$

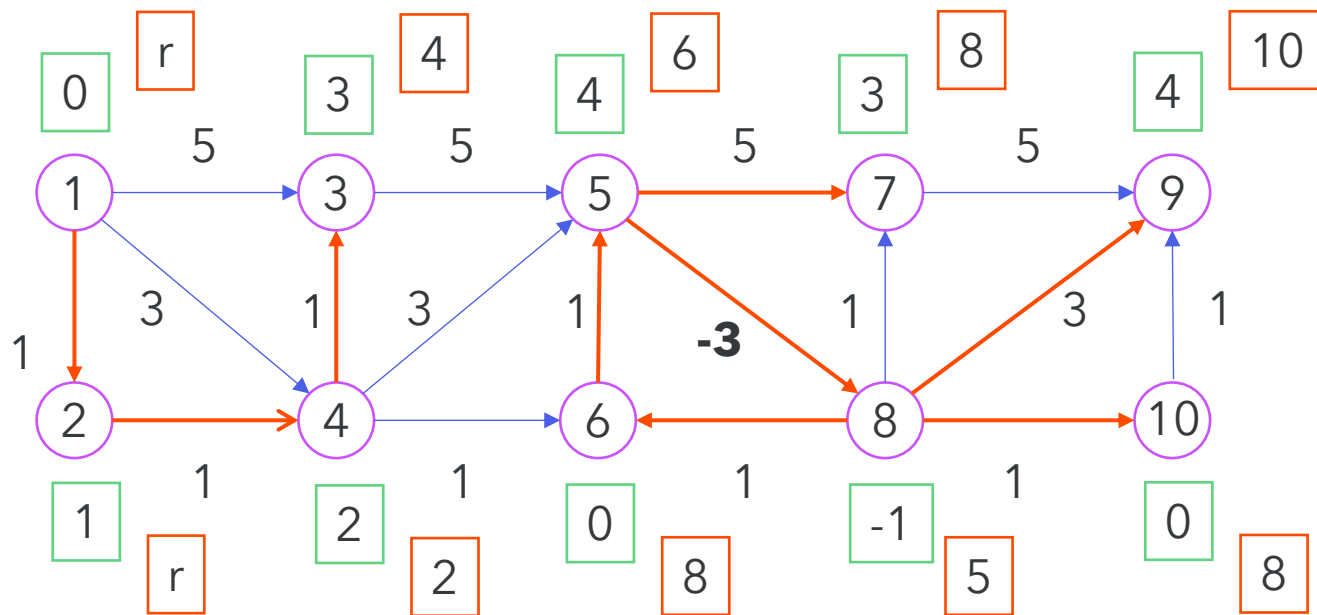
Passo al seguente nodo in lista: 7. Condizioni rispettate.

$Q = \{10, 8\}$



$Q = \{8, 9\}$

Passo al seguente nodo in lista: 10. Gli archi uscenti non rispettano le condizioni di Bellman: aggiorno le etichette di 9 e la lista.


 $Q = \{8, 9\}$

 $Q = \{9, 6, 10\}$

Passo al seguente nodo in lista: 8. Gli archi uscenti non rispettano le condizioni di Bellman: aggiornò le etichette di 6, 10 e la lista.

Il ciclo proseguirebbe all'infinito: non ho nemmeno più un albero!

CICLI NEGATIVI E CONDIZIONI DI BELLMAN

Se esiste un **ciclo orientato di costo negativo**, non può esistere alcun vettore d che rispetti le condizioni di Bellman.

Dimostrazione per assurdo:

Sommando le disuguaglianze lungo un ciclo $L = (i_1, i_2, \dots, i_k, i_1)$:

$$0 \leq C(L)$$

Ma per un ciclo negativo $C(L) < 0$, dunque abbiamo una **contraddizione**.

Quindi, in presenza di cicli negativi, il problema dei cammini minimi **non ha soluzione finita**.

ALGORITMO SPT.L.QUEUE: TERMINAZIONE

Se esiste un ciclo orientato di costo negativo raggiungibile da r , l'algoritmo **non termina**.

Se esiste un ciclo negativo ma non raggiungibile da r , l'algoritmo **termina**, restituendo l'albero dei cammini minime solo per i vertici raggiungibili da r .

Se non esistono cicli orientati di costo negativo: **l'algoritmo termina**.

Lemma:

- In ogni momento dell'algoritmo le etichette sono un costo di un cammino sul grafo, non necessariamente quello di costo minimo.
- Ad ogni iterazione le etichette non aumentano: diminuiscono o non cambiano.
- Se non esistono cicli orientati di costo negativo, le etichette non possono diminuire indefinitamente.
- Pertanto, dopo un numero finito di iterazioni, le etichette raggiungono il valore del cammino minimo e **l'algoritmo termina**.

ALGORITMO SPT.L.QUEUE

Visto come l'abbiamo implementato, l'algoritmo termina in un numero polinomiale di iterazioni:

Abbiamo implementato l'algoritmo con Q come fila (FIFO).

Questa scelta non è l'unica e nemmeno la migliore, però è ragionevole e ci permette di rendere l'algoritmo completo (trovare un metodo per tener conto dei cicli orientati di costo negativo) e di rendere l'algoritmo polinomiale.

Proprietà:

Se non esistono cicli orientati di costo negativo, allora nessun nodo può entrare in coda più di $n - 1$ volte, qualora Q sia una fila (FIFO).

ALGORITMO SPT.L.QUEUE

Proprietà:

Se non esistono cicli orientati di costo negativo, allora nessun nodo può entrare in coda più di $n - 1$ volte, qualora Q sia una fila (FIFO).

Le file (FIFO) fanno visite del grafo a ventaglio.

Quando un nodo i esce dalla coda e vi rientra è perché ha un costo minore, ma una lunghezza maggiore (in termini di archi nel cammino).

E i cammini semplici possono essere lunghi al massimo $n - 1$.

Quindi gli aggiornamenti possono essere fatti al massimo $n - 1$ volte per ogni nodo, qualora non ci siano cicli orientati di costo negativo.

ALGORITMO SPT.L.QUEUE: COMPLESSITÀ

Ogni nodo possiamo vederlo al massimo $n - 1$ volte, e ogni volta che vediamo un nodo visitiamo gli archi uscenti, che sono al più m :

Complessità Polinomiale: $O(nm)$

Per grafi particolarmente densi m è circa n^2 , perciò la complessità è al più di $O(n^3)$: non molto efficiente.

ALGORITMO SPT.L.QUEUE: CICLI NEGATIVI

Se non esistono cicli orientati di costo negativo, allora nessun nodo esce dalla coda più di $n - 1$ volte.

Allora ho un metodo per valutare se esistano cicli orientati di costo negativo:

- **Conto** quante volte un nodo riappare nella coda
- Se esce dalla coda **n volte: ho un ciclo negativo**
- **Termino** l'algoritmo con una soluzione **illimitata**.

ALGORITMO SPT: DEQUEUE

In pratica l'algoritmo visto finora non è molto efficiente, perché si fanno dei **treni di aggiornamento**. Allora si possono valutare altri metodi per considerare quale nodo visitare:

Deque: double ended queue

- Le estrazioni dei nodi avvengono dalla testa della coda
- Le inserzioni dei nodi possono avvenire dalla testa o dalla coda:
 - Se l'**etichetta** del nodo entrante è **infinita** (M) allora lo inserisco in **coda** (è la prima volta che lo visito).
 - Se l'**etichetta** del nodo entrante è **non infinita** ($< M$) allora lo inserisco in **testa** (potrebbe avere dei figli con etichetta da aggiornare, se lo mettessi in coda creerei un treno di aggiornamento).

L'algoritmo è più efficiente, però perdiamo la proprietà che ci permette di trovare facilmente i cicli orientati di costo negativo, perché dipendeva dalla fila FIFO.

Complessità teorica di $O(n2^n)$, però in pratica è più efficiente e si può usare in casi dove non ci sono cicli orientati di costo negativo.

ALGORITMO SPT: ALTRE CODE

LIFO

L'ultimo che entra in coda è il primo a uscire.
Complessità di $O(n2^n)$.

2-QUEUE

Usa due code:

- Coda prioritaria: nodi con etichette con valore minore di infinito ($<M$). Nodi già visitati in precedenza.
- Coda non prioritaria: nodi con etichette con valore pari a infinito (M), nodi visitati per la prima volta.

Prima si usa la coda prioritaria, quando finisce si usa la coda non prioritaria.

Complessità $O(nm^2)$.

Threshold

Si introduce un valore soglia T e si usano due code:

- Coda prioritaria: nodi con etichette $d_i \leq T$.
- Coda non prioritaria: nodi con etichette $d_i > T$.

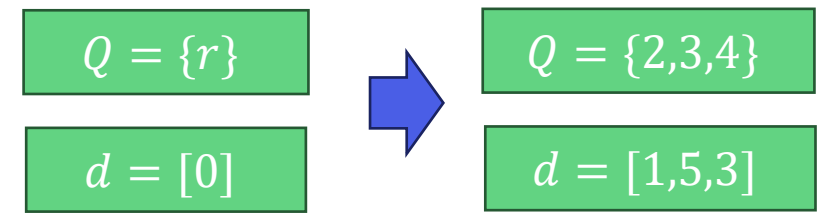
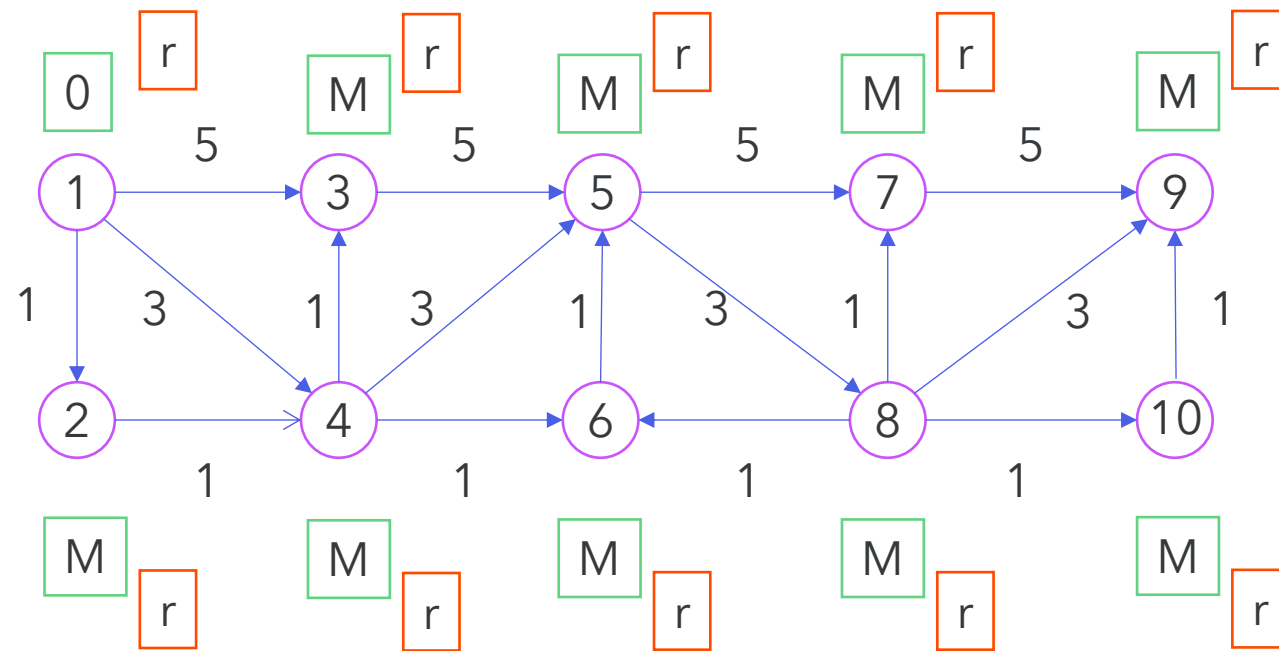
Con un T scelto adeguatamente l'algoritmo risulta molto efficiente.

ALGORITMO SPT.S

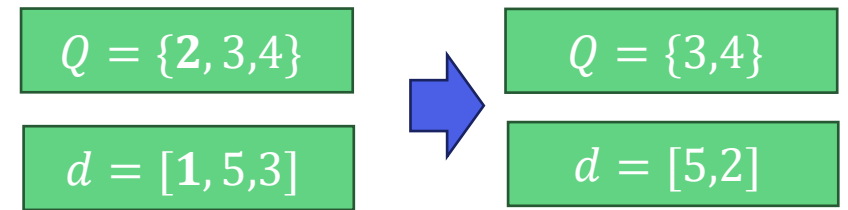
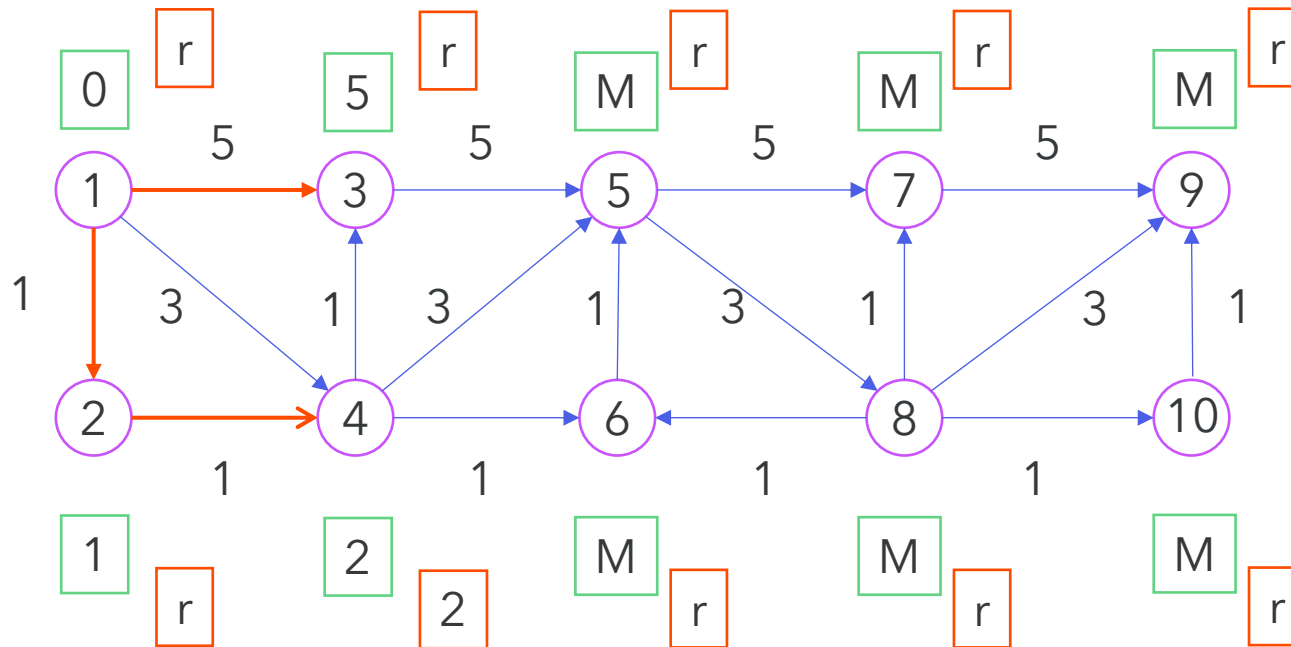
Perché non discriminare direttamente sul valore delle etichette?

Estraiamo prima il nodo con **l'etichetta con valore più basso**.

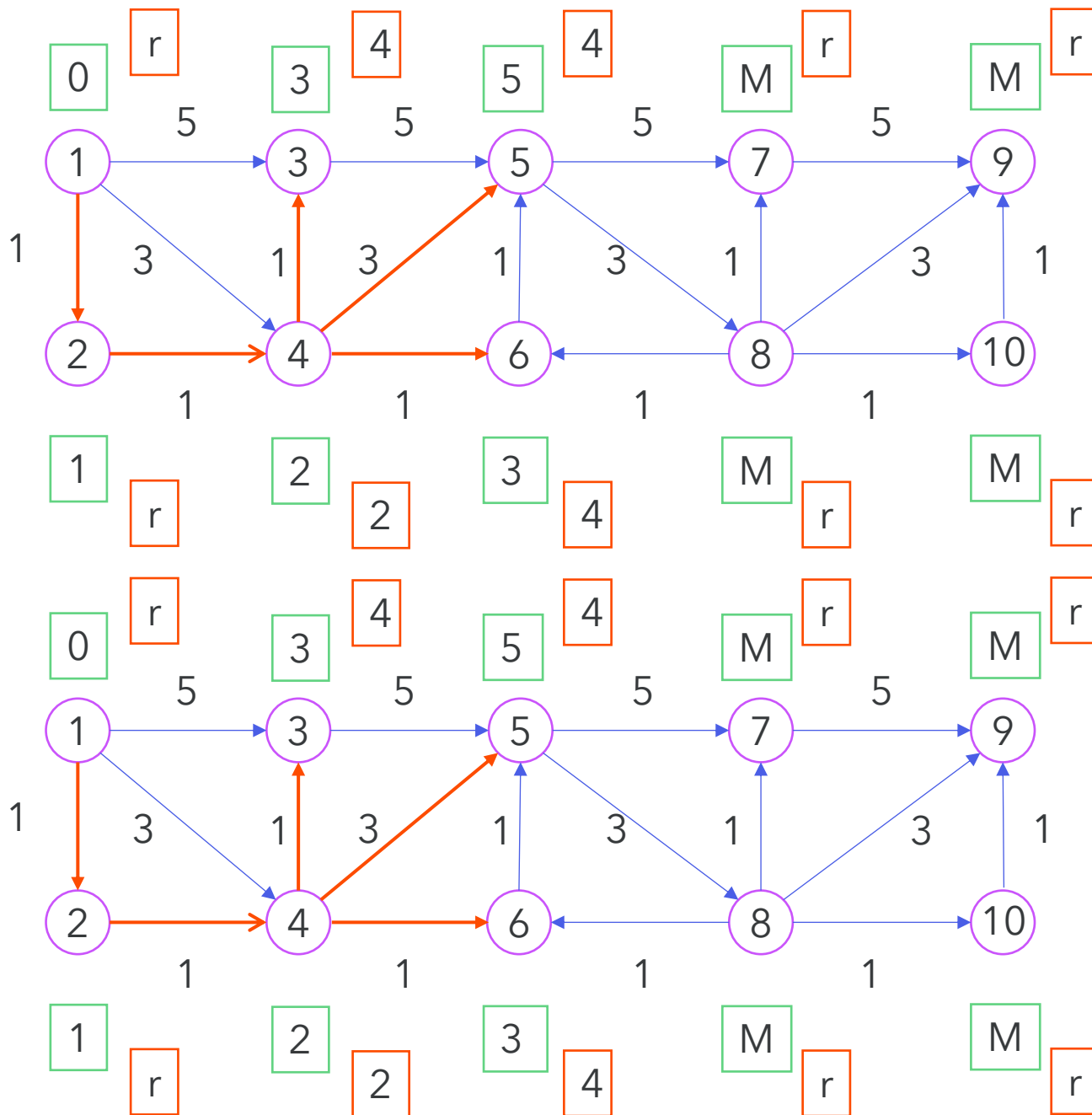
Otteniamo così: l'**Algoritmo di Dijkstra** (SPT.S (Shortest-first)).



Inizializziamo tutte le etichette d a M e dei predecessori a r .
 Selezioniamo il nodo 0 e valutiamo gli archi uscenti.



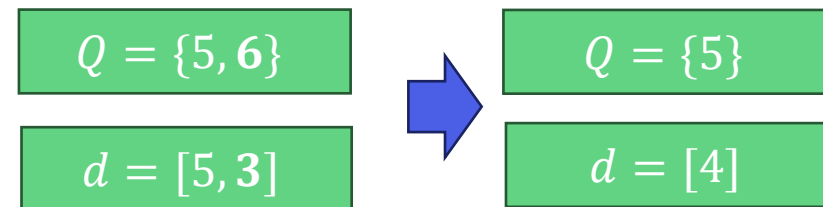
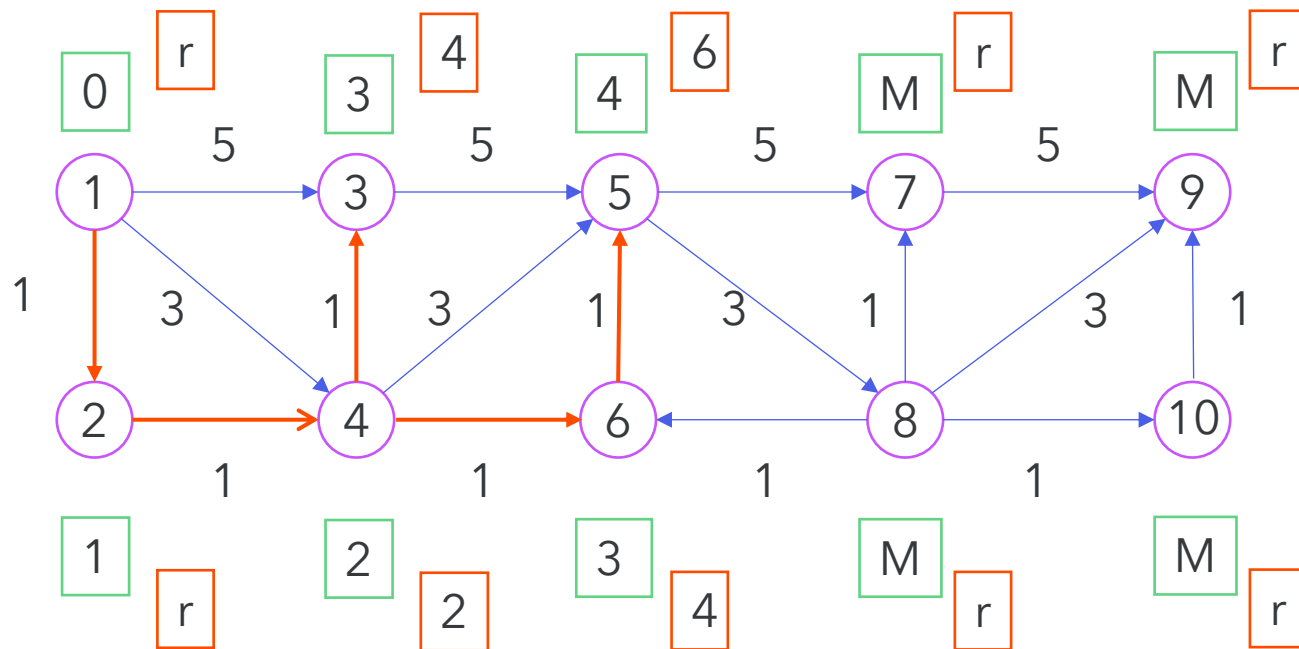
Passo al seguente nodo in lista con valore dell'etichetta più basso: 2.
 Esploro gli archi uscenti e aggiorno le etichette di 4 e la lista.


 $Q = \{3, 4\}$
 $d = [5, 2]$
 $Q = \{3, 5, 6\}$
 $d = [3, 5, 3]$

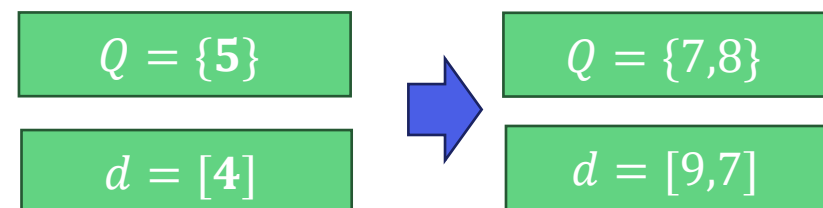
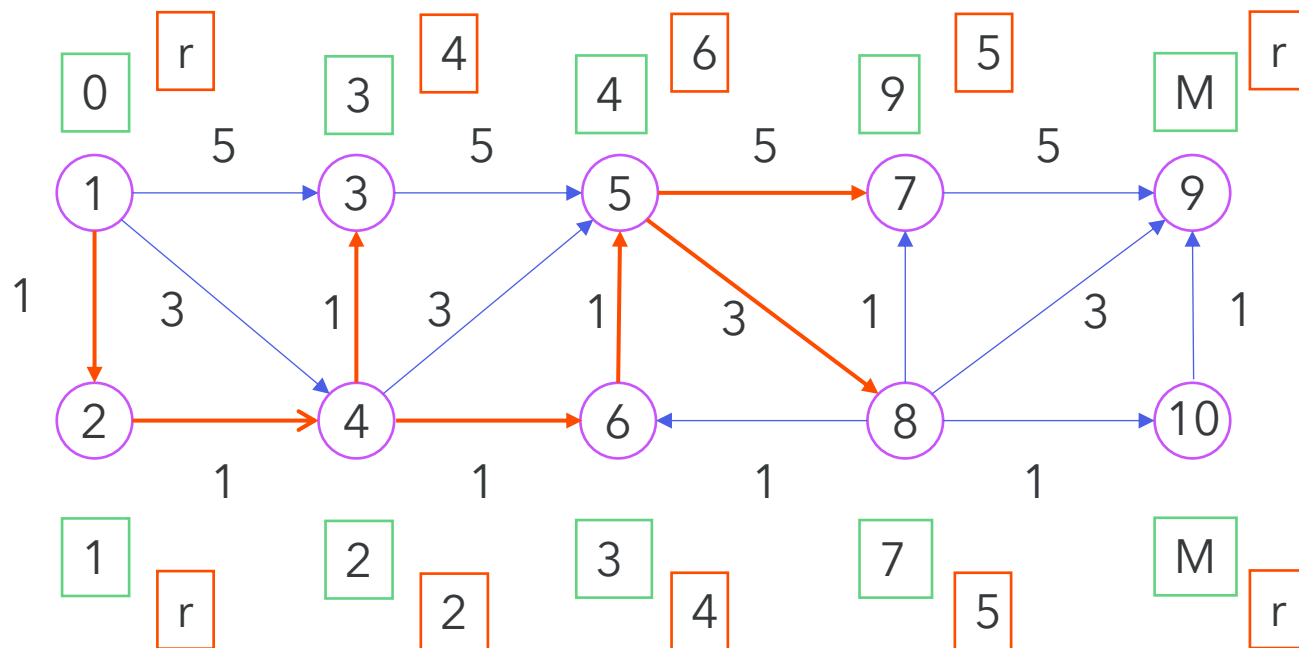
Passo al seguente nodo in lista con valore dell'etichetta più basso: 4. Esploro gli archi uscenti e aggiorno le etichette e la lista.

 $Q = \{3, 5, 6\}$
 $d = [3, 5, 3]$
 $Q = \{5, 6\}$
 $d = [5, 3]$

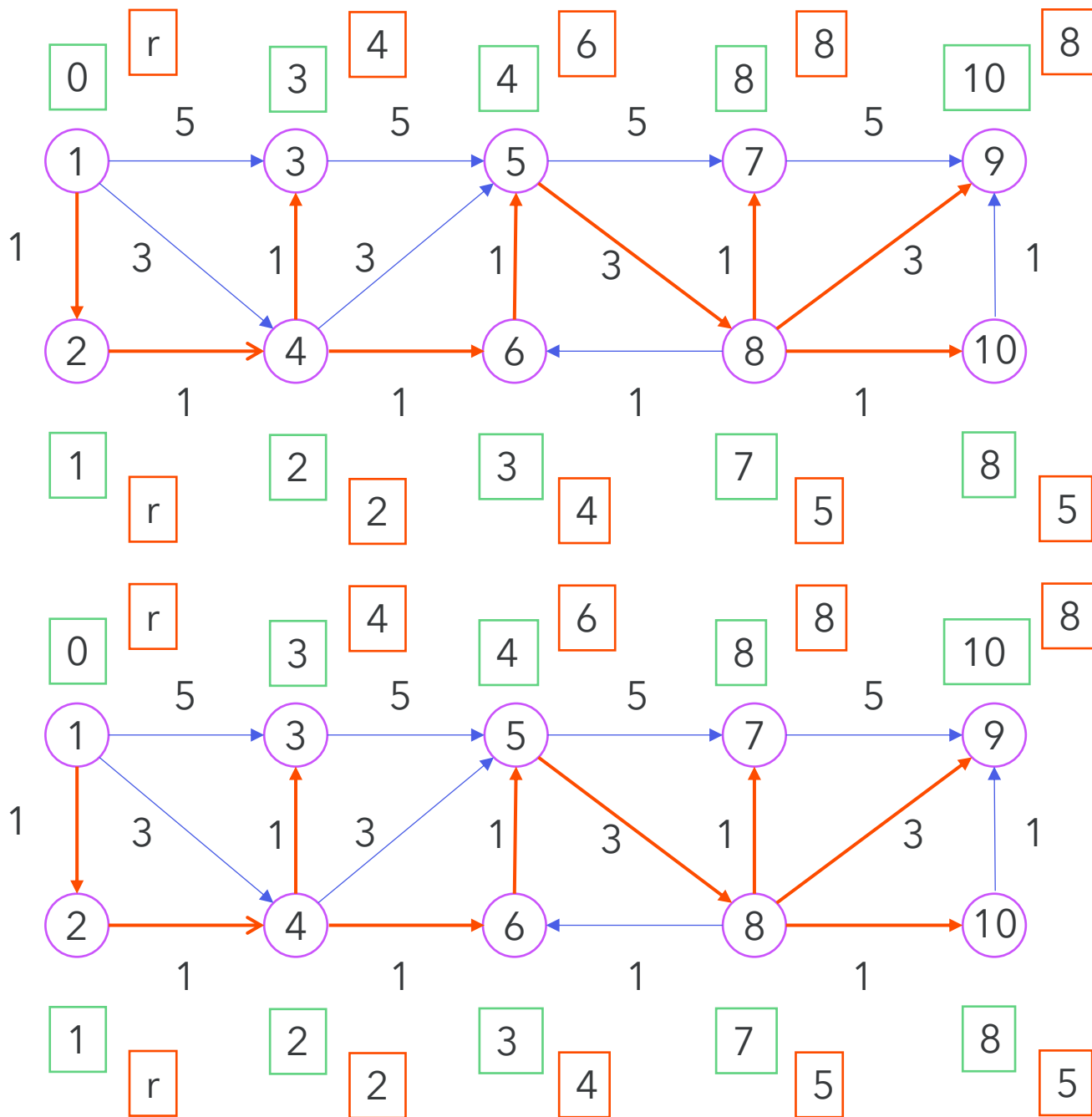
Passo al seguente nodo in lista con valore dell'etichetta più basso: 3. Esploro gli archi uscenti: non cambiano le etichette.



Passo al seguente nodo in lista con valore dell'etichetta più basso: 6. Esploro gli archi uscenti: aggiornò l'etichetta di 5 e la lista.



Passo all'unico nodo in lista: 5. Esploro gli archi uscenti: aggiornò l'etichetta di 5 e la lista.



$Q = \{7, 8\}$

$d = [9, 7]$



$Q = \{7, 9, 10\}$

$d = [8, 10, 8]$

Passo al seguente nodo in lista con valore dell'etichetta più basso: 8. Esploro gli archi uscenti: aggiornò l'etichetta di 7, 9 e 10 e la lista.

$Q = \{7, 9, 10\}$

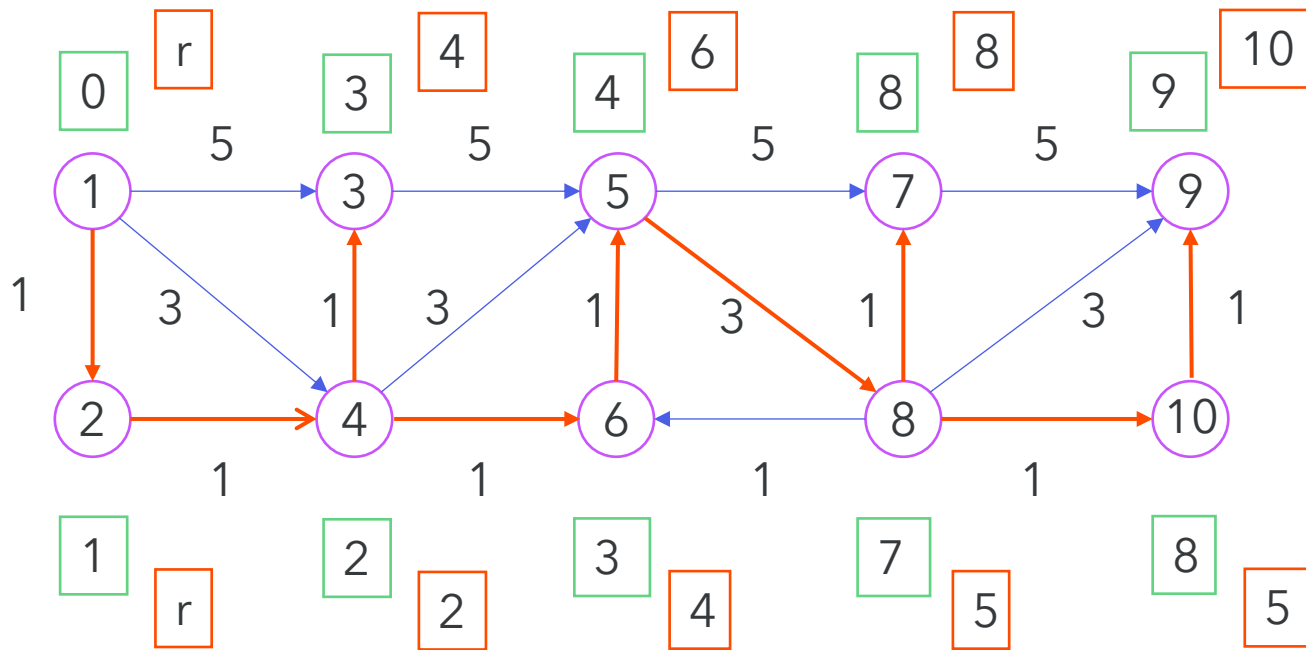
$d = [8, 10, 8]$



$Q = \{9, 10\}$

$d = [10, 8]$

Passo al primo nodo in lista con valore dell'etichetta più basso: 7. Esploro gli archi uscenti: non miglioro etichette.


 $Q = \{9, 10\}$
 $d = [10, 8]$
 $Q = \{9\}$
 $d = [9]$

Passo al primo nodo in lista con valore dell'etichetta più basso: 10.
Esploro gli archi uscenti: aggiornò l'etichetta di 9.

 $Q = \{9\}$
 $d = [9]$
 $Q = \emptyset$

9 non ha archi uscenti. Ho terminato l'algoritmo visitando ogni nodo una sola volta.

ALGORITMO SPT.S: TEOREMA DI DIJKSTRA

Teorema di Dijkstra

Nei grafi con costi **non negativi**, ogni nodo viene **inserito ed estratto dalla coda al più una volta**.

Al più una volta perché un nodo potrebbe non entrare mai nella coda, nel caso che non sia raggiungibile dalla radice.

Lemma del teorema di Dijkstra:

Se tutti i costi sono non negativi, allora l'ordine delle etichette una volta che abbiamo fissato il nodo sono non decrescenti. Ovvero, se indichiamo con d_i^k l'etichetta del nodo i_k fissato alla k -esima iterazione, allora:

$$d_{i_1}^1 \leq d_{i_2}^2 \leq \dots \leq d_{i_k}^k \leq \dots \leq d_{i_n}^n$$

ALGORITMO SPT.S: TEOREMA DI DIJKSTRA

Dimostrazione per assurdo

Supponiamo, per assurdo, che un nodo i venga estratto due volte dalla coda:

- la prima all'iterazione k con etichetta d_i^k
- la seconda all'iterazione $h > k$ con etichetta d_i^h

Poiché tutti i pesi sono non negativi, le **etichette non decrescono** dopo essere state fissate (lemma):

$$d_i^k \leq d_i^h$$

Tuttavia, $d_i^h > d_i^k$ con $h > k$ non può accadere: perché non effettueremmo l'algoritmo non effettuerebbe l'aggiornamento qualora facesse aumentare le etichette.

ALGORITMO SPT.S: COMPLESSITÀ

Se $c_{ij} \geq 0$ per ogni $(i, j) \in A$, l'algoritmo **SPT.S**

- visita ogni nodo **una sola volta** (n iterazioni)
- e considera **tutti gli archi uscenti** di ciascun nodo, quindi ogni arco viene visitato al massimo una volta (m archi totali).
- Ad ogni iterazione si sceglie il nodo con etichetta minima:

$$i = \operatorname{argmin}\{d_j : j \in Q\},$$

operazione di costo $O(n)$ (senza altro ordinamento)

Pertanto la complessità totale è

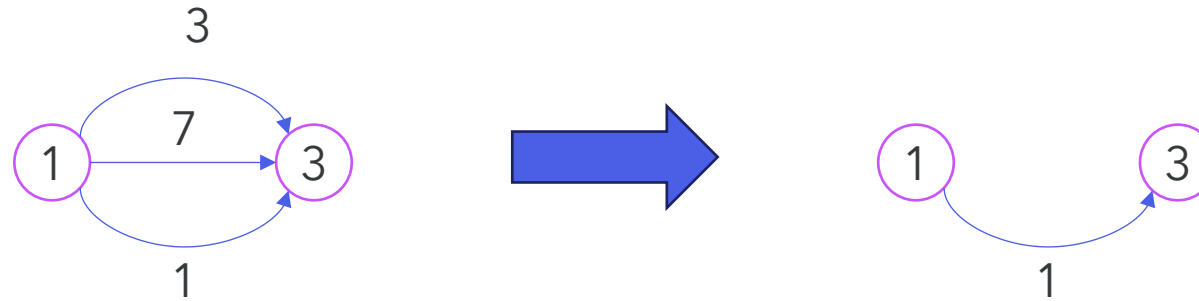
$$O(n^2 + m) = O(n^2)$$

poiché $n^2 \geq m$.

Iterazioni e ordinamento +
archi uscenti

ALGORITMO SPT.S: COMPLESSITÀ

Questa cosa è vera se non esistono gli **archi paralleli**, ma se esistono aggiorniamo il grafo includendo solo l'arco con costo minore.



ALGORITMI SPT.S E SPT.L: COMPLESSITÀ

L'algoritmo **SPT.S** (tipo Dijkstra) ha complessità $O(n^2)$, mentre **SPT.L** (tipo Bellman-Ford) ha complessità $O(nm)$.

Se il grafo è **denso** ($m \approx n^2$) allora

$$O(nm) = O(n^3)$$

e quindi *SPT.S* è più efficiente, poiché $O(n^2)$ è asintoticamente migliore.

Se il grafo è **sperso** ($m \approx n$) allora:

si può migliorare l'efficienza di *SPT.S* utilizzando una struttura dati più sofisticata per selezionare il nodo con etichetta minima.

In particolare, usando un **heap binario non bilanciato**, la scelta del nodo minimo costa $O(\log n)$. Poiché si effettuano $O(m)$ operazioni, la complessità complessiva diventa:

$$O(m \log n)$$

che risulta più efficiente di $O(n^2)$ nei grafi sparsi.

ALGORITMI SPT.S E SPT.L: COMPLESSITÀ

In sintesi:

Grafi densi → usare **SPT.S** estraendo il nodo con etichetta minore → $O(n^2)$

Grafi sparsi → usare **SPT.S** con heap → $O(m \log n)$

SPT.L è generalmente meno efficiente ($O(nm)$): è utile solo in presenza di **archi di costo negativo**.

ALGORITMO SPT.S: SOTTOINSIEMI DI DESTINAZIONI

L'algoritmo di **Dijkstra** è efficiente non solo per il calcolo dei cammini minimi da una sorgente r a **tutti** i nodi del grafo, ma anche quando si desidera conoscere il cammino minimo **solo verso un nodo specifico i** .

Infatti, se tutti i costi $c_{ij} \geq 0$, nel momento in cui il nodo i viene estratto dalla coda, la sua etichetta d_i è già **definitiva**: corrisponde al costo minimo del cammino da r a i .

A quel punto l'algoritmo può **terminare anticipatamente**, senza completare l'esplorazione del grafo.

Lo stesso principio vale se si vogliono calcolare i cammini minimi verso un **sottoinsieme di destinazioni** $S \subseteq N \setminus \{r\}$: l'algoritmo può arrestarsi non appena **tutti i nodi di S** siano stati estratti dalla coda.

CAMMINI MINIMI CON RADICI MULTIPLE

Se si vogliono calcolare i cammini minimi da un insieme di sorgenti

$$R = \{r_1, \dots, r_k\}$$

verso una destinazione t ,

si introduce un **nodo sorgente fittizio** r_0 , collegato a ciascuna sorgente $r_i \in R$ con un **arco di costo nullo**:

$$c_{r_0 r_i} = 0 \quad \forall r_i \in R.$$

Si ottiene così un grafo esteso su cui si calcolano i cammini minimi **a partire da** r_0 .

L'algoritmo può **terminare non appena** il nodo t viene estratto dalla coda.

Le distanze minime da r_0 corrispondono quindi ai **cammini minimi da una qualunque sorgente in R verso t** .

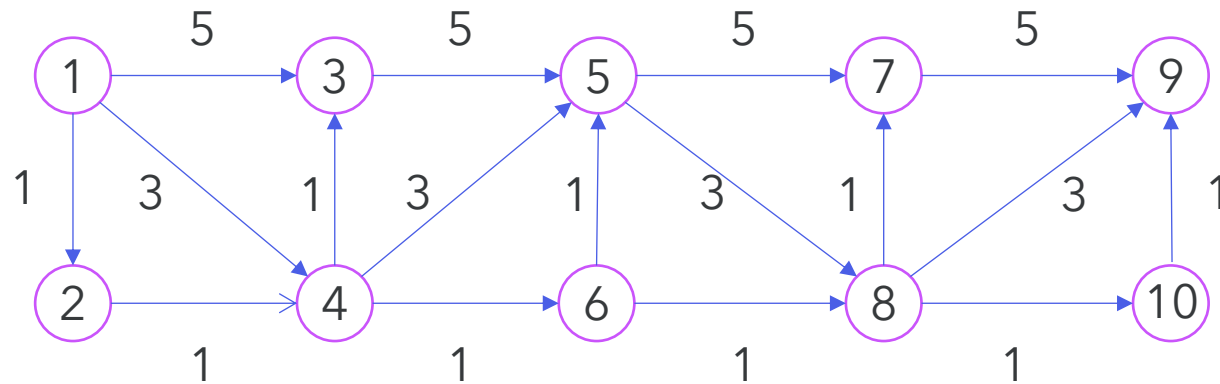


CAMMINI MINIMI SU GRAFI ACICLICI

DA PAG 64 DISPENSE

CAMMINI MINIMI SU GRAFI ACICLICI

Qualora avessimo dei grafi aciclici potremmo fare di meglio in termini di complessità.



Come faccio a capire se un grafo è aciclico?

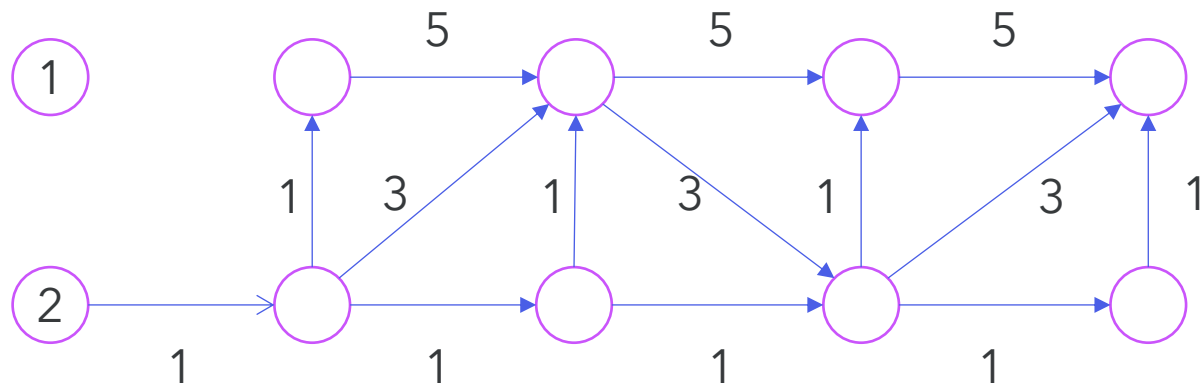
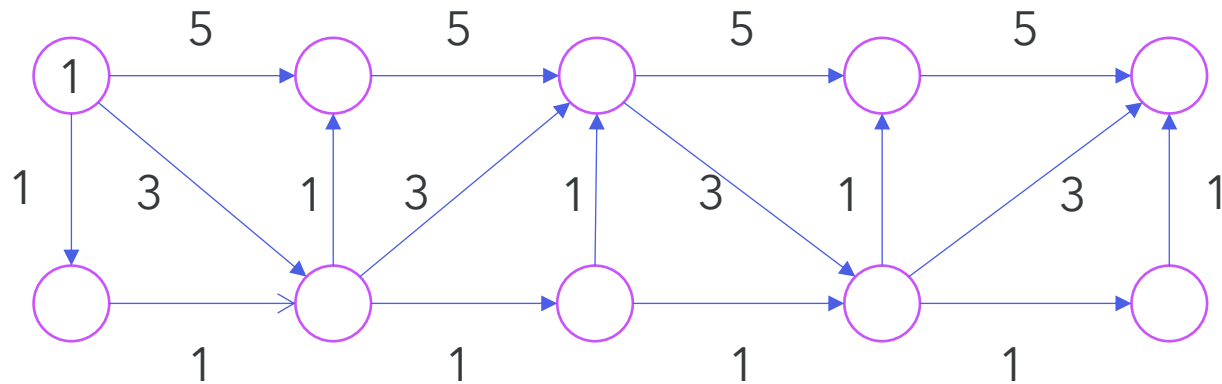
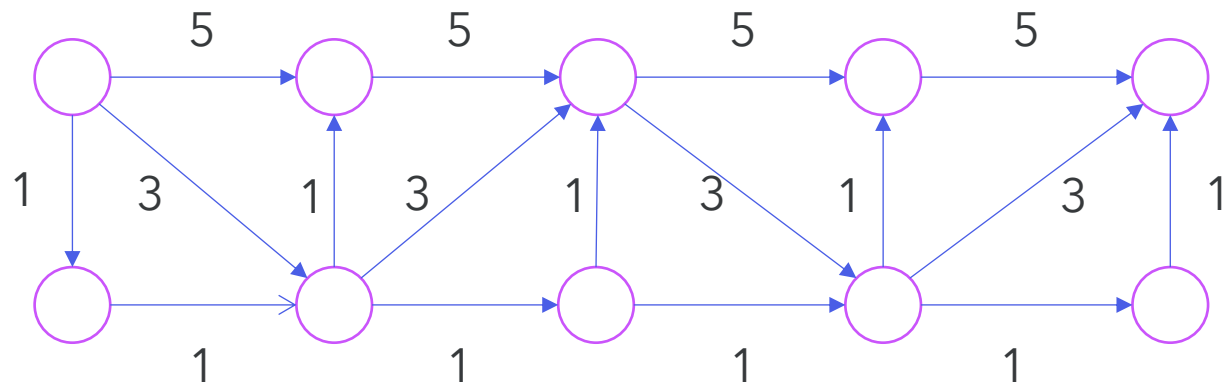
NUMERAZIONE GRAFI ACICLICI

Un grafo orientato è **aciclico** se e solo se esiste una **numerazione topologica** dei nodi.

Ossia, è possibile ordinare i nodi $1, 2, \dots, n$ in modo tale che:

per ogni arco $(i, j) \in A$ si abbia $i < j$

Se esiste un ciclo, nessuna numerazione può rispettare questa condizione.

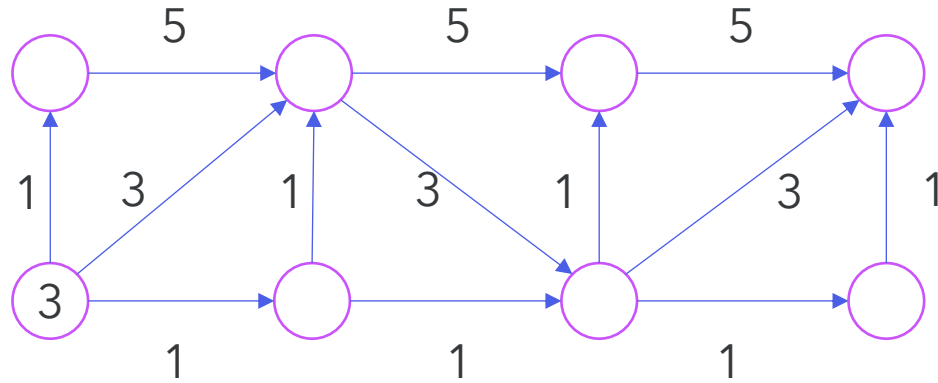


Passo 1: Se esiste almeno un nodo nel grafo che **non ha archi entranti** gli associo il numero non ancora usato in ordine crescente a partire da 1.

Passo 2: Una volta numerato un nodo, cancello tutti gli archi uscenti e valuto il grafo ottenuto. Poi torno al Passo 1.

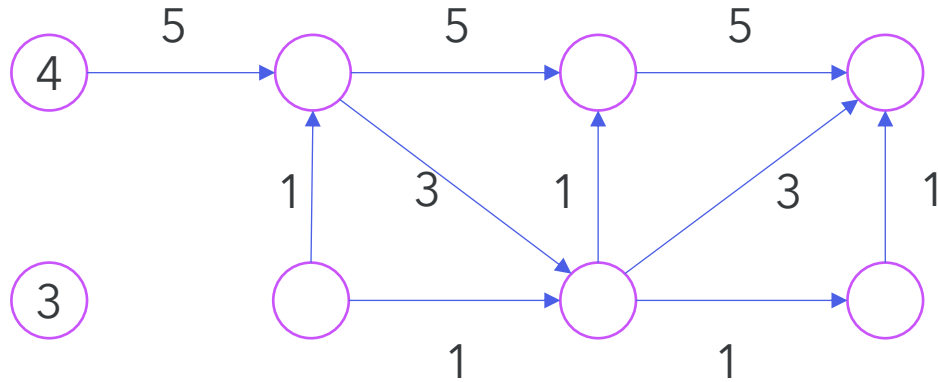
Qualora non trovassi nessun nodo privo di archi entranti: **il grafo non è aciclico.**

1



2

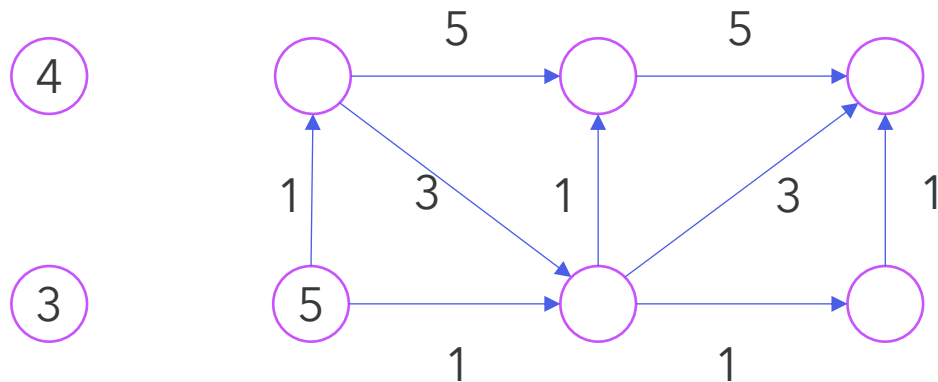
1



2

3

1



2

3

Passo 1: Se esiste almeno un nodo nel grafo che non ha archi entranti gli associo il numero non ancora usato in ordine crescente a partire da 1.

Passo 2: Una volta numerato un nodo, cancello tutti gli archi uscenti e valuto il grafo ottenuto. Poi torno al Passo 2.

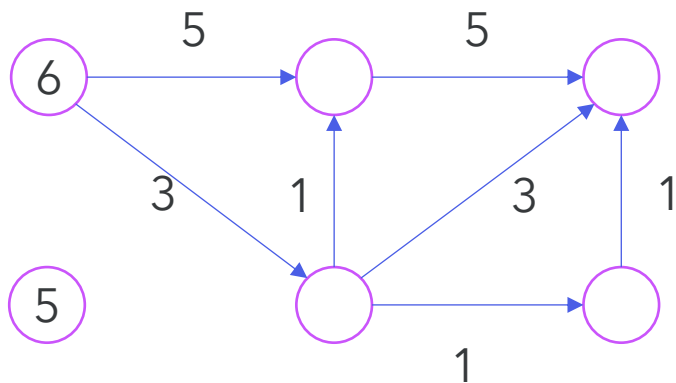
Qualora non trovassi nessun nodo privo di archi entranti: **il grafo non è aciclico.**

1

4

2

3



1

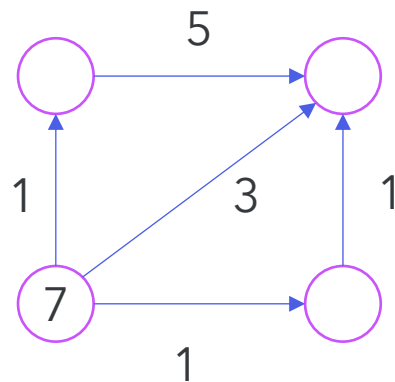
4

2

3

6

5



1

4

6

8

5

1

2

3

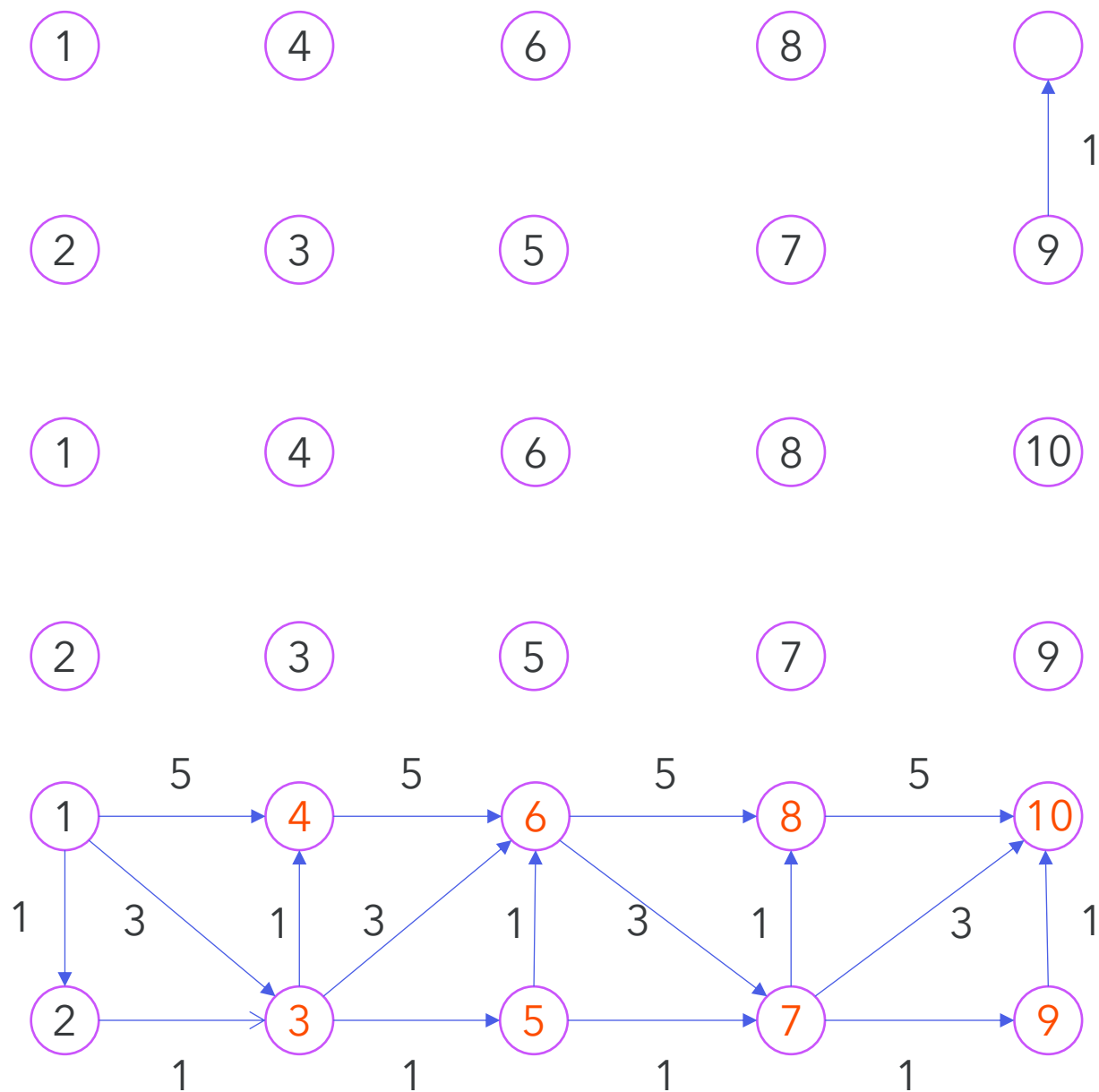
5

7

Passo 1: Se esiste almeno un nodo nel grafo che non ha archi entranti gli associo il numero non ancora usato in ordine crescente a partire da 1.

Passo 2: Una volta numerato un nodo, cancello tutti gli archi uscenti e valuto il grafo ottenuto. Poi torno al Passo 2.

Qualora non trovassi nessun nodo privo di archi entranti: **il grafo non è aciclico.**



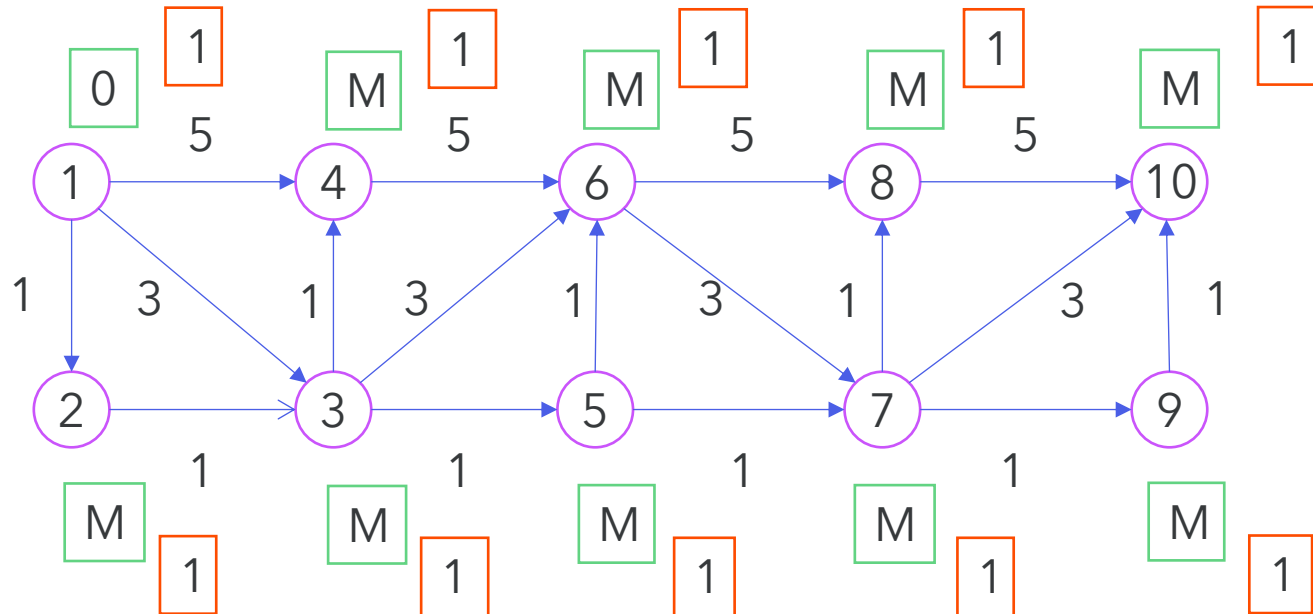
Passo 1: Se esiste almeno un nodo nel grafo che non ha archi entranti gli associo il numero non ancora usato in ordine crescente a partire da 1.

Passo 2: Una volta numerato un nodo, cancello tutti gli archi uscenti e valuto il grafo ottenuto. Poi torno al Passo 2.

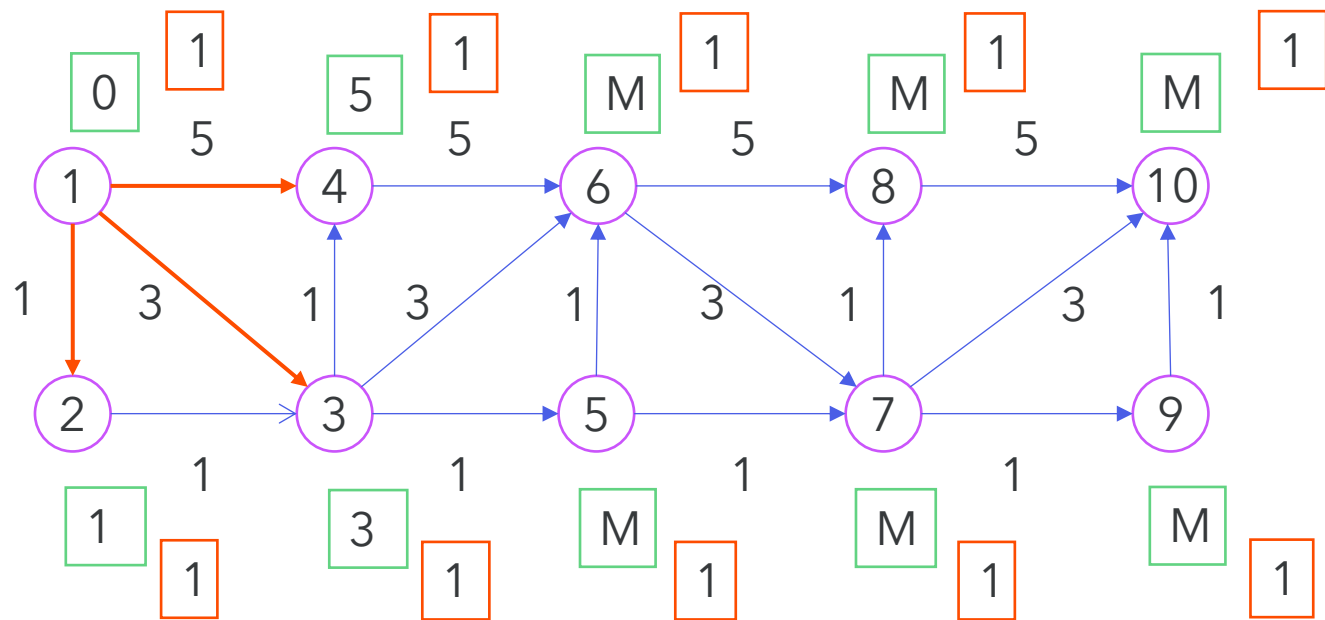
Qualora non trovassi nessun nodo privo di archi entranti: **il grafo non è aciclico.**

CAMMINI MINIMI SU GRAFI ACICLICI

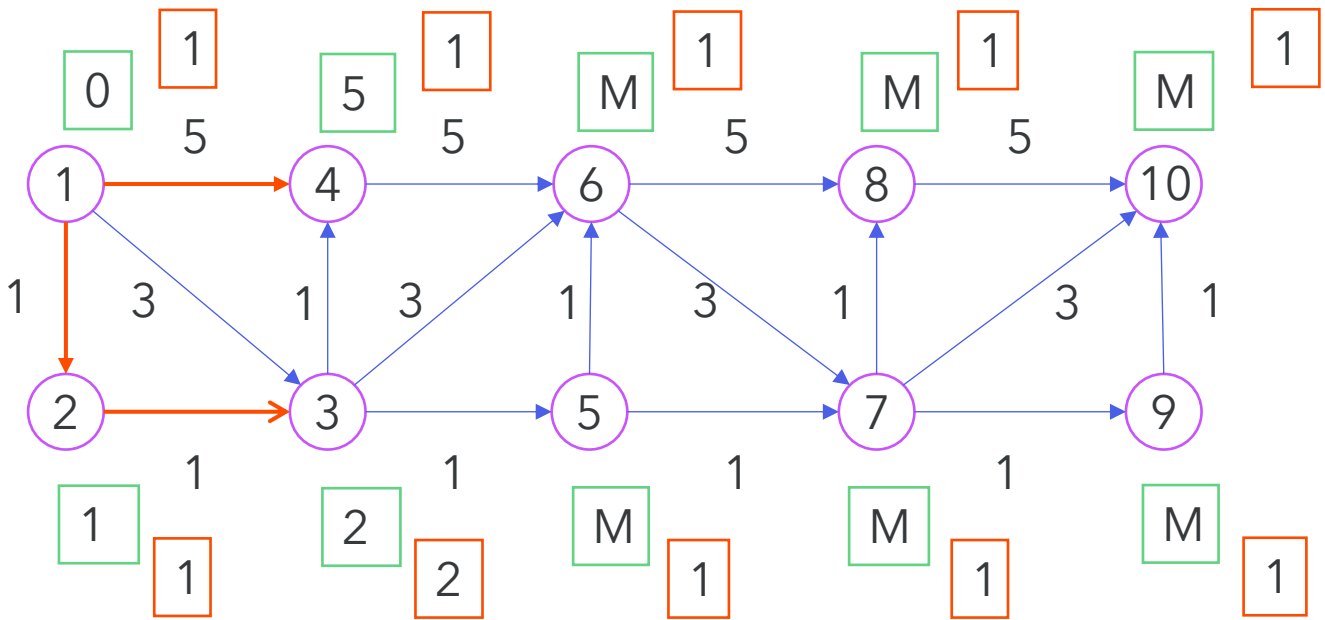
Una volta che ho ordinato topologicamente il grafo posso risolvere il problema dei cammini minimi su un grafo aciclico senza usare le etichette per fare l'ordinamento, ma **usando la numerazione: STP.Acyclic**



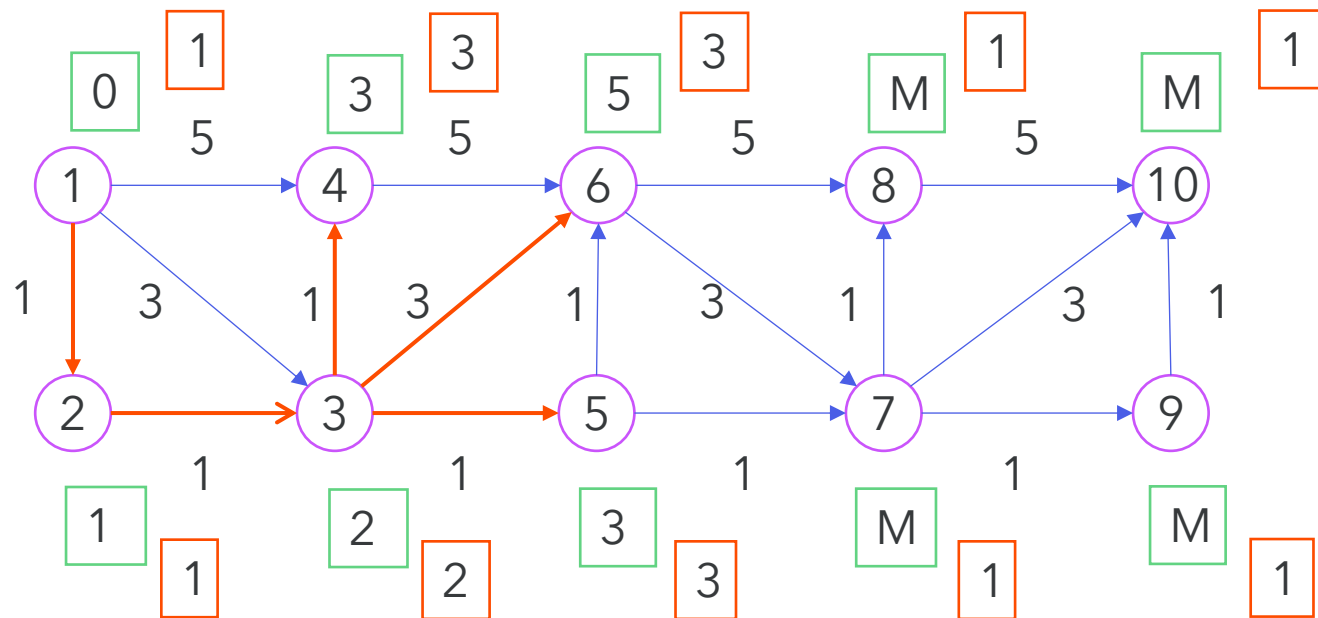
Inizializziamo tutte le etichetta d a M (0 alla radice) e tutte i predecessori a 1.



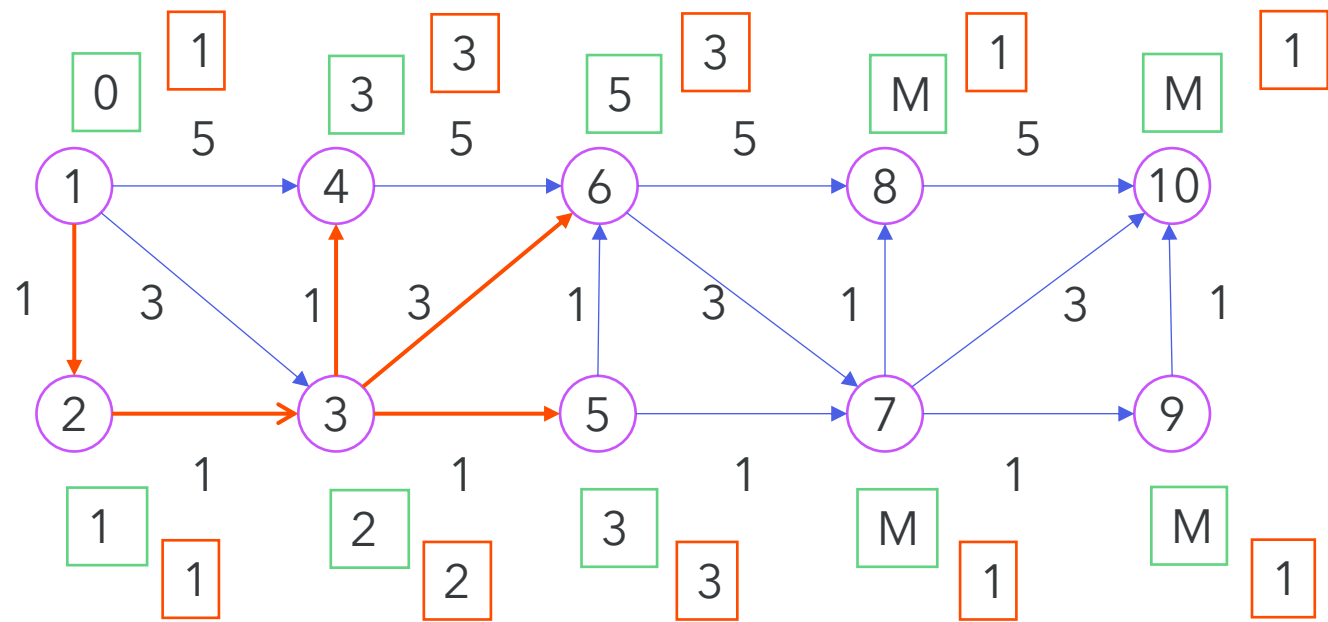
Valutiamo gli archi uscenti da 1 e aggiorniamo le etichette.



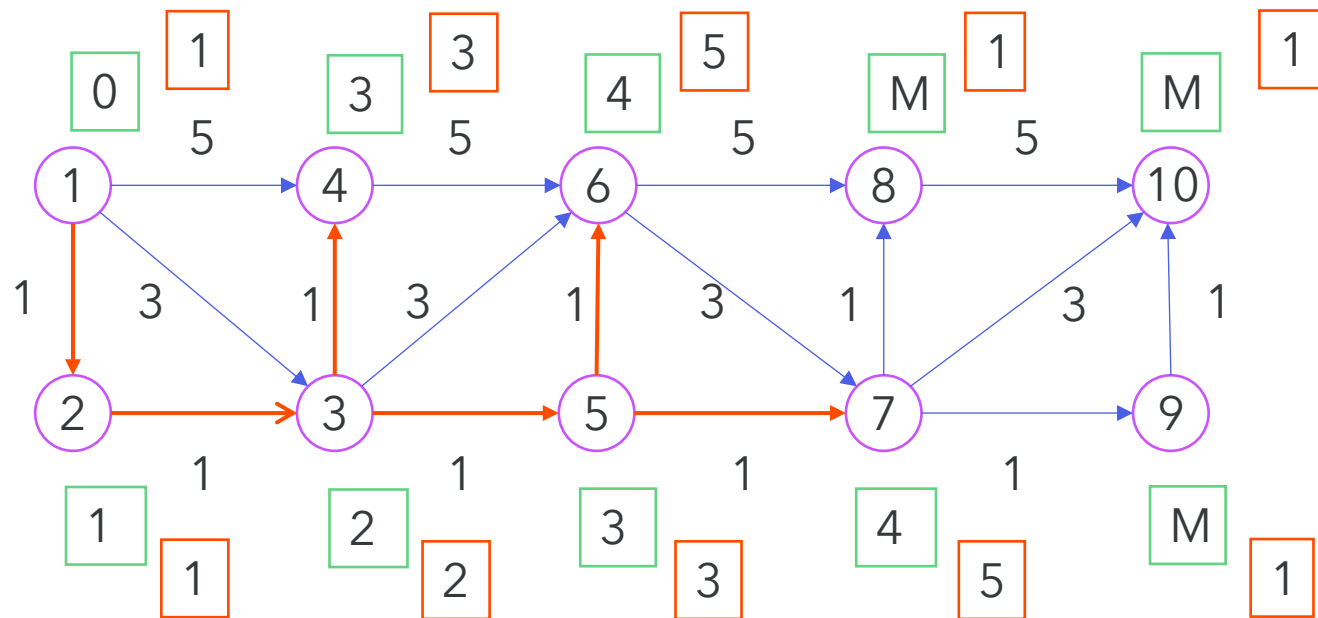
Valutiamo gli archi uscenti da 2 e aggiorniamo le etichette.



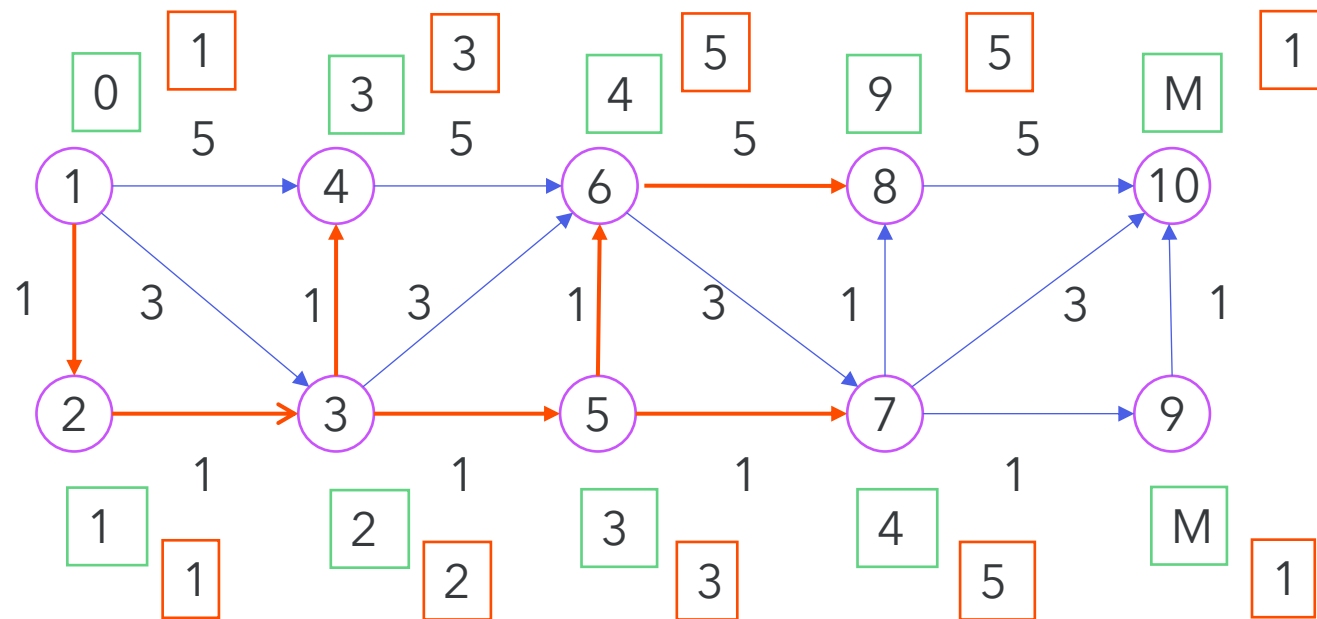
Valutiamo gli archi uscenti da 3 e aggiorniamo le etichette.



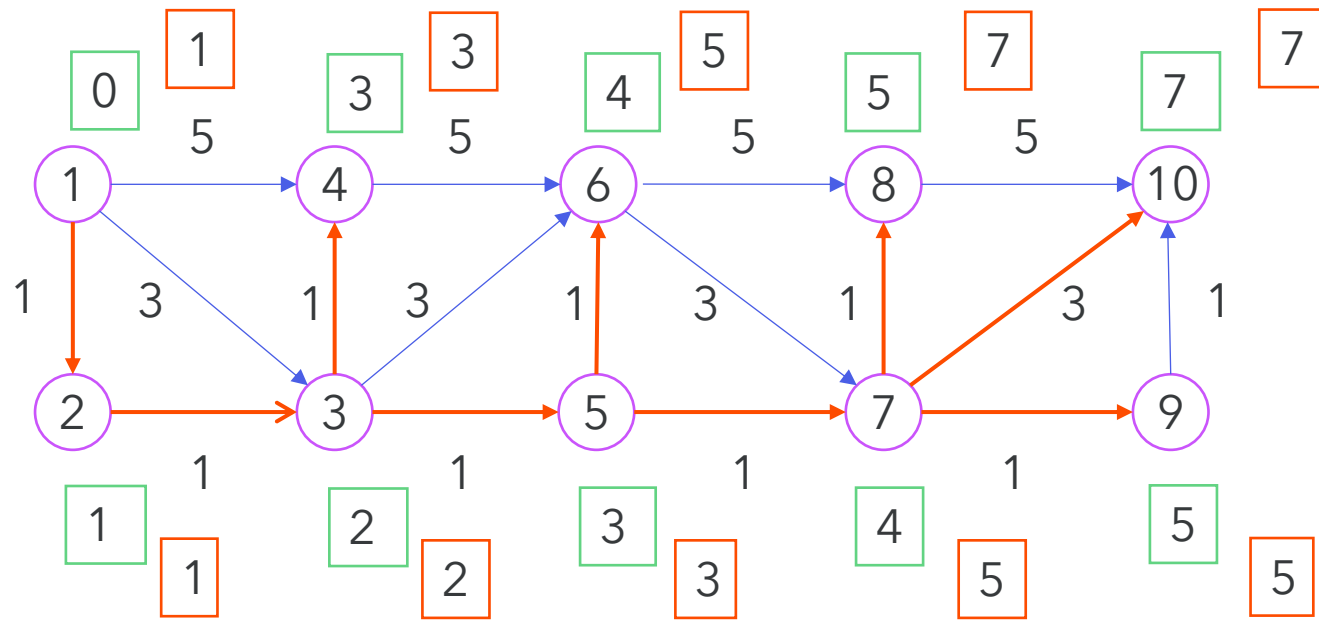
Valutiamo gli archi uscenti da 4. Non si aggiornano le etichette.



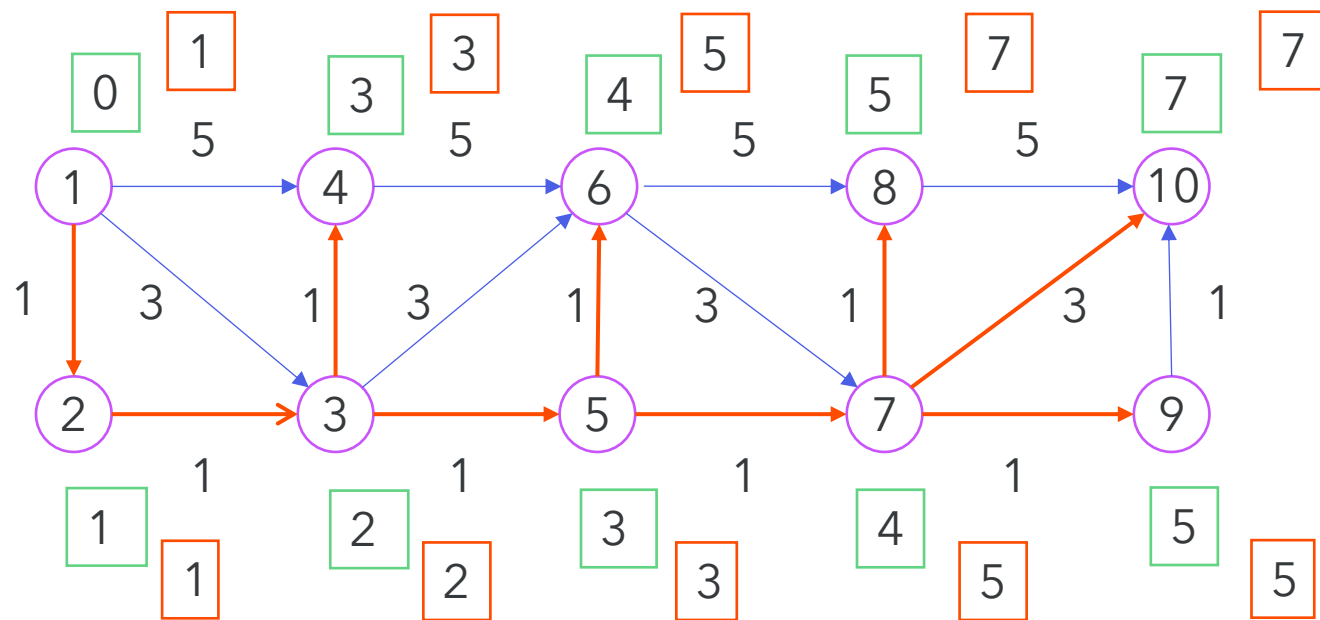
Valutiamo gli archi uscenti da 5 e aggiorniamo le etichette.



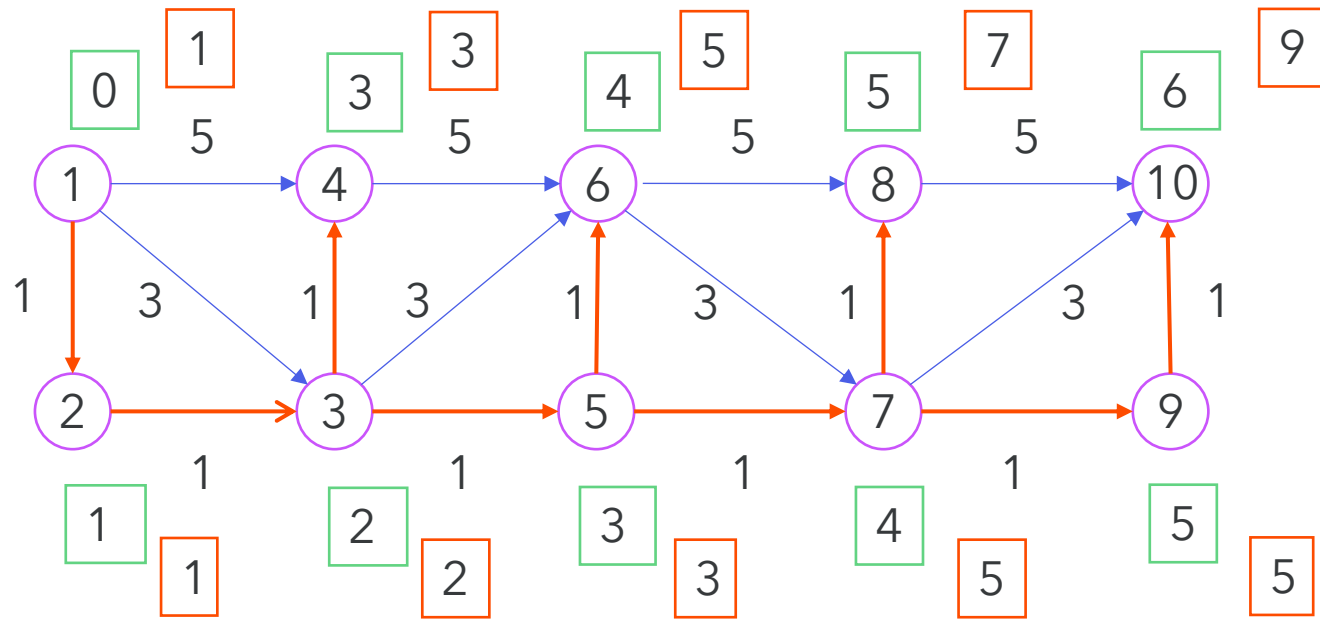
Valutiamo gli archi uscenti da 6 e aggiorniamo le etichette.



Valutiamo gli archi uscenti da 7 e aggiorniamo le etichette.



Valutiamo gli archi uscenti da 8. Non ci sono etichette da aggiornare.



Valutiamo gli archi uscenti da 9 e aggiorniamo le etichette.

Il nodo 10 non ha archi uscenti

Valutiamo gli archi uscenti da 8. Non ci sono etichette da aggiornare.

Questo algoritmo è di ordine $O(m)$.

L'ordinamento costa anch'esso $O(m)$.

ALTRI CASI SU GRAFI ACICLICI

Costi Negativi

Su un **grafo aciclico**, possiamo risolvere il problema dei cammini minimi anche con archi con **costi negativi** (non ci sono cicli e quindi non ci sono nemmeno cicli di costo negativo).
Rimane la stessa complessità lineare.

Cammini di costo massimo

Allora possiamo anche risolvere il problema dei **cammini costo massimo**: moltiplicando i costi di tutti gli archi per -1 e applicando l'algoritmo **STP.Acyclic** sul grafo modificato.

Altre radici

Se volessimo l'albero dei cammini minimi con altra radice: possiamo considerare solo i nodi dalla numerazione della radice in avanti.



ALBERO DI COPERTURA DI COSTO MINIMO

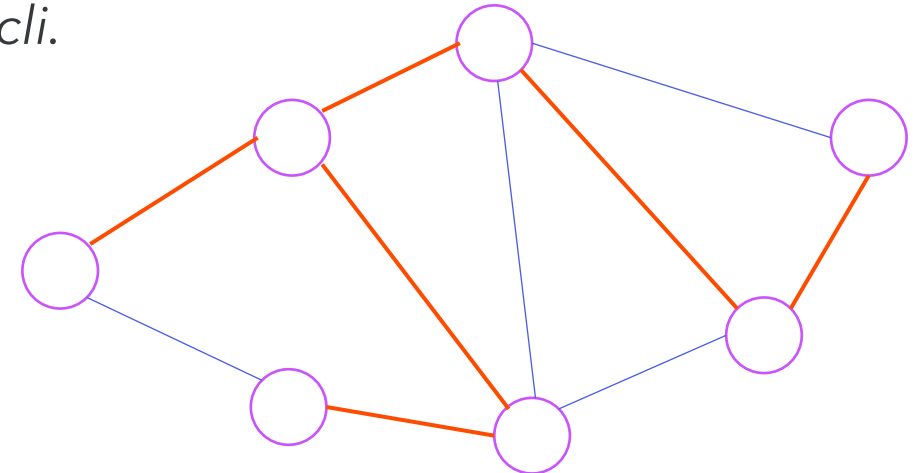
DA PAG 67 DISPENSE

ALBERO DI COPERTURA DI COSTO MINIMO (MST)

Problema: dato un grafo non orientato $G = (V, E)$ con costi c_{ij} sui lati, trovare un albero di copertura $T = (V, E_T)$ che minimizzi il costo totale:

$$c(T) = \sum_{\{i,j\} \in E_T} c_{ij}$$

Un albero di copertura collega tutti i nodi senza creare cicli.



MOTIVAZIONE

Algoritmo greedy ottimale:

È uno dei pochi problemi di ottimizzazione risolvibili al **100% ottimamente** con un approccio *greedy*.

Somiglianze con il problema del percorso minimo (SPT):

Anche se MST è definito su grafi **non orientati**, alcune strategie sono simili agli algoritmi per SPT.

Applicazioni pratiche:

Reti elettriche, telecomunicazioni, progettazione di infrastrutture minimizzando costi di collegamento.

PROPRIETÀ DI UN ALBERO DI COPERTURA

Per semplificare la trattazione, assumiamo:

- G è **connesso**.
- Tutti i costi $c_{ij} > 0$.

$$E_T \subset E$$
$$n = |V|$$

Un **albero di copertura** T è un sottografo $T = (V, E_T)$ di G che:

- Contiene **tutti i nodi** del grafo (è *di copertura*).
- È **connesso** (esiste un cammino tra ogni coppia di nodi).
- **Non contiene cicli**.

Proprietà: Ogni albero di copertura su n nodi ha esattamente $n - 1$ lati

GRAFO NON CONNESSO

Se G non è connesso l'albero di copertura di costo minimo non esiste.

Si parla di **foresta di copertura minima**.

Si aggiungono **lati fittizi**:

- Tra un nodo e un altro nodo non raggiungibile
- Con costo molto grande, per esempio:

$$M = c_{\max} + 1$$

dove $c_{\max}$ è il costo più alto tra i lati reali.

- L'algoritmo userà i lati fittizi solo se necessario.

COSTI NEGATIVI

Proprietà: la soluzione ottima non cambia se si aumentano tutti i costi della stessa quantità.

In caso di costi negativi:

- aggiungere una costante $\delta = -c_{\min} + 1$ a tutti i lati del grafo per renderli positivi senza cambiare la soluzione ottima.
- $c_{\min}$ è il costo minimo tra tutti i lati.

ALGORITMO GREEDY-MST (SCHEMA GENERALE)

Idea principale: costruire l'albero passo passo, decidendo quali lati includere o escludere.

Mantiene due insiemi:

S : lati **già inclusi** nell'albero.

R : lati **scartati**, che **non entreranno mai** nell'albero.

**Durante tutte le
iterazioni si deve
mantenere:**

$$S \subset E, R \subset E, S \cap R = \emptyset$$

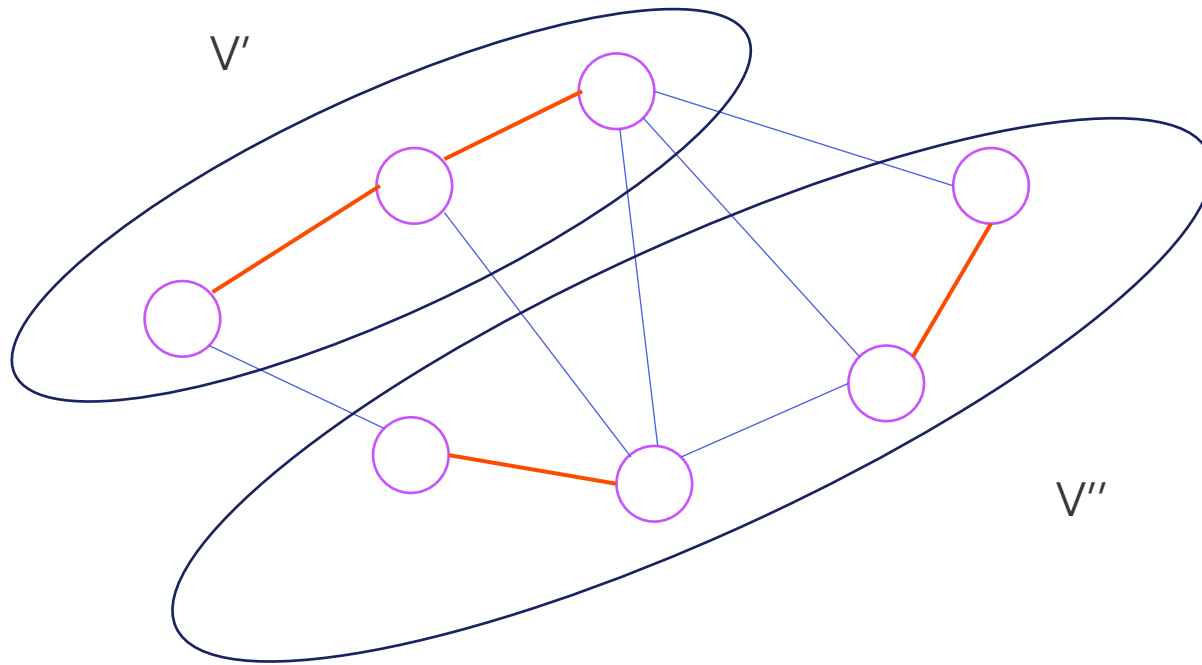
Operazioni disponibili:

Inserzione: scegliere un lato minimo che collega due componenti ancora separate.

Cancellazione: rimuovere il lato massimo da un ciclo per evitare cicli nell'albero.

ALGORITMO GREEDY-MST (SCHEMA GENERALE)

Inserzione: scegliere un lato minimo che collega due componenti ancora separate.



- Scegliere il lato $\{i, j\} = \operatorname{argmin}\{c_{ij} : \{i, j\} \in E(V', V'') \setminus R\}$ nel taglio definito da $E(V', V'') \cap S = \emptyset$
- Aggiungerlo ad S , $S = S \cup \{i, j\}$

Taglio di un grafo:

Presi due sottoinsiemi **non vuoti e disgiunti** V' e V'' tali che:

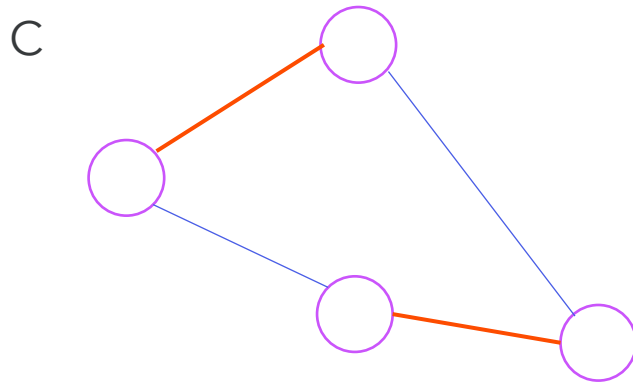
$$V' \cup V'' = V, V' \cap V'' = \emptyset$$

Il **taglio** è l'insieme di **lati che collegano i nodi appartenenti i due insiemi**:

$$E(V', V'') = \{\{i, j\} \in E : i \in V', j \in V''\}$$

ALGORITMO GREEDY-MST (SCHEMA GENERALE)

Cancellazione: rimuovere il lato massimo da un ciclo per evitare cicli nell'albero.



Scegliere un ciclo $C \subseteq G$ tale che $C \cap R = \emptyset$,
e scegliere il lato $\{i, j\} = \operatorname{argmax}\{c_{ij} : \{i, j\} \in C \setminus S\}$
cioè il lato **di costo massimo** tra quelli del ciclo non
ancora presenti in S .

Aggiungerlo all'insieme dei lati scartati:

$$R = R \cup \{i, j\}$$

ALGORITMO GREEDY-MST (SCHEMA GENERALE)

Lemma di correttezza:

Dati due insiemi di lati:

- S : i lati **già scelti** (che fanno sicuramente parte di almeno un albero di copertura ottimo),
- R : i lati **scartati** (che non possono appartenere ad alcun albero di copertura ottimo),

e se **esiste almeno un albero di copertura minimo** T^* tale che:

$$S \subseteq E_{T^*} \text{ e } R \cap E_{T^*} = \emptyset$$

cioè T^* include tutti i lati di S e nessuno di quelli di R , allora, **dopo un'operazione di inserzione o di cancellazione**, i nuovi insiemi S' e R' ottenuti **mantengono la stessa proprietà**.

Se partiamo da un insieme vuoto e facciamo queste operazioni in qualsiasi ordine, quando troviamo un albero di copertura è quello di costo minimo, se esiste.

Intuizione per un algoritmo

PROCEDURA GREEDY-MST

```
procedure S = Greedy-MST(G, c){  
  S =  $\emptyset$ ; R =  $\emptyset$ ;  
  do applica Inserzione o Cancellazione  
  while(S  $\cup$  R  $\subsetneq$  E and  $|S| < n - 1$ )  
  end
```

Inizializzo i due insiemi all'insieme vuoto.

Applica iterativamente una delle due operazioni fino a che i lati inclusi nell'albero non sono tanti quanti ne voglio.

Condizioni di terminazione:

1. $S \cup R = E \rightarrow$ tutti i lati sono stati classificati.
2. $|S| = n - 1 \rightarrow$ l'albero è completo.

Alla fine, S rappresenta un **albero di copertura di costo minimo**.

PROCEDURA GREEDY-MST

Conseguenza del lemma di correttezza:

- Per induzione, Greedy-MST termina sempre con un MST ottimale (se esiste).
- L'ordine delle operazioni **non influisce** sul risultato finale.

Complessità: $O(m)$

IMPLEMENTAZIONI PRATICHE

Esistono diverse varianti di Greedy-MST a seconda dell'ordine di inserzione e cancellazione:

Kruskal: seleziona sempre il lato minimo disponibile che non forma cicli.

Prim: costruisce l'albero aggiungendo nodi vicini al sottografo già costruito.

Nota: entrambe rispettano la logica del lemma, quindi garantiscono ottimalità.

ALGORITMO DI KRUSKAL

L'**algoritmo di Kruskal** è un caso particolare di Greedy-MST in cui:

Tutti i lati del grafo vengono **ordinati in ordine non decrescente di costo**:

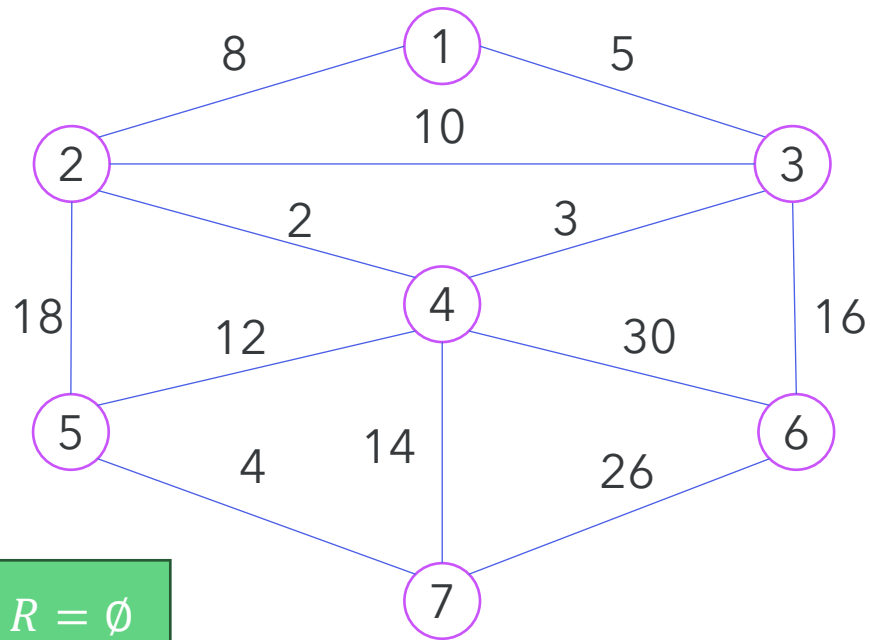
$$c_1 \leq c_2 \leq \dots \leq c_m$$

Si esaminano i lati **uno per volta** in questo ordine:

Se il lato **non forma un ciclo** con i lati già in S , lo **si inserisce** in S (operazione di *inserzione*).

Se invece **forma un ciclo**, lo **si scarta** mettendolo in R (operazione di *cancellazione*)

Alla fine, quando $|S| = n - 1$, S contiene i lati dell'**albero di copertura minimo**.



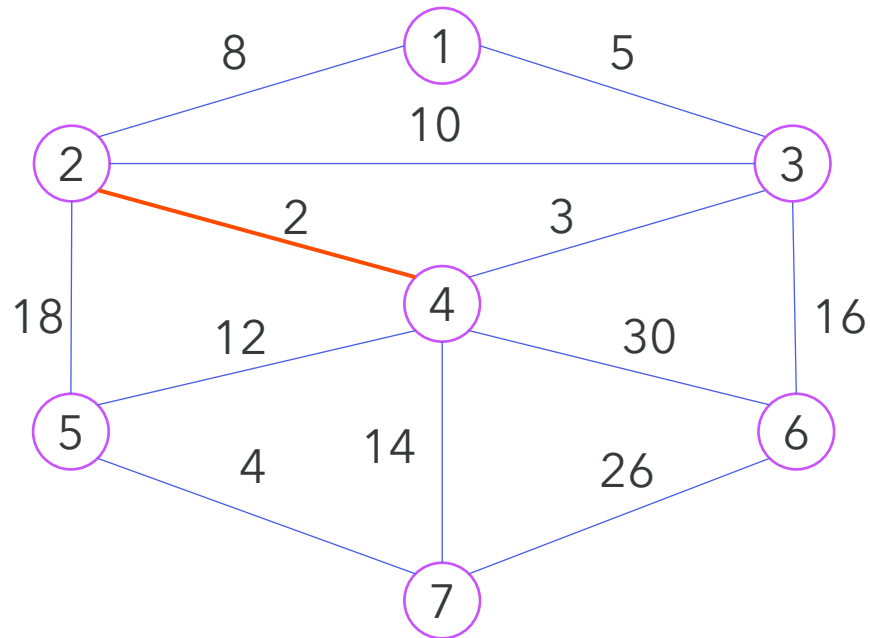
$S = \emptyset, R = \emptyset$

Lato	Costo
{2,4}	2
{3,4}	3
{5,7}	4
{1,3}	5
{1,2}	8
{2,3}	10
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

Effettuo **l'ordinamento** degli archi per costo non decrescente.

Prendo il primo lato dalla lista: {2,4}
Non crea un ciclo, perciò lo inserisco in S .
Questa è un'operazione di **inserzione**:
esiste un taglio che separa 2 da 4 in cui non ci sono lati di S .

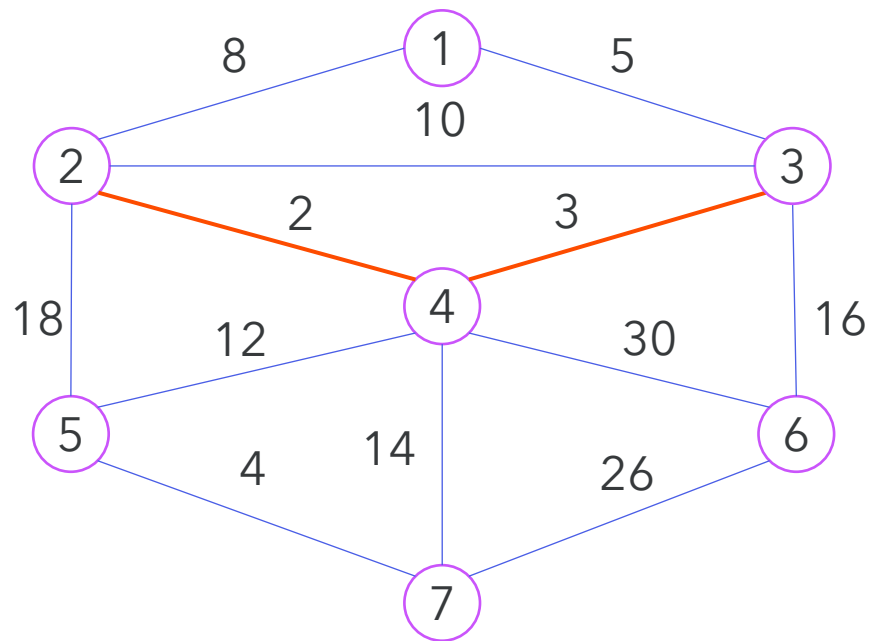
Cancello il lato {2,4} dalla lista.



$S = \{\{2,4\}\}, R = \emptyset$

Lato	Costo
{3,4}	3
{5,7}	4
{1,3}	5
{1,2}	8
{2,3}	10
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

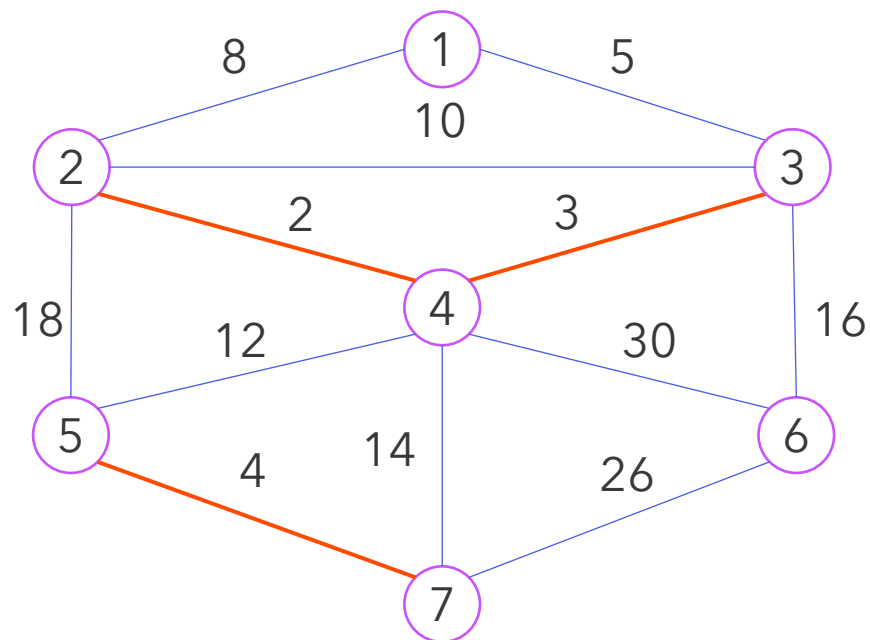
Prendo il primo nodo dalla lista: {3,4}
Non crea un ciclo, perciò lo inserisco in S .
Esiste un taglio che separa 2 da 4 in cui non ci sono lati di S : inserzione.



Lato	Costo
{5,7}	4
{1,3}	5
{1,2}	8
{2,3}	10
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}\}, R = \emptyset$$

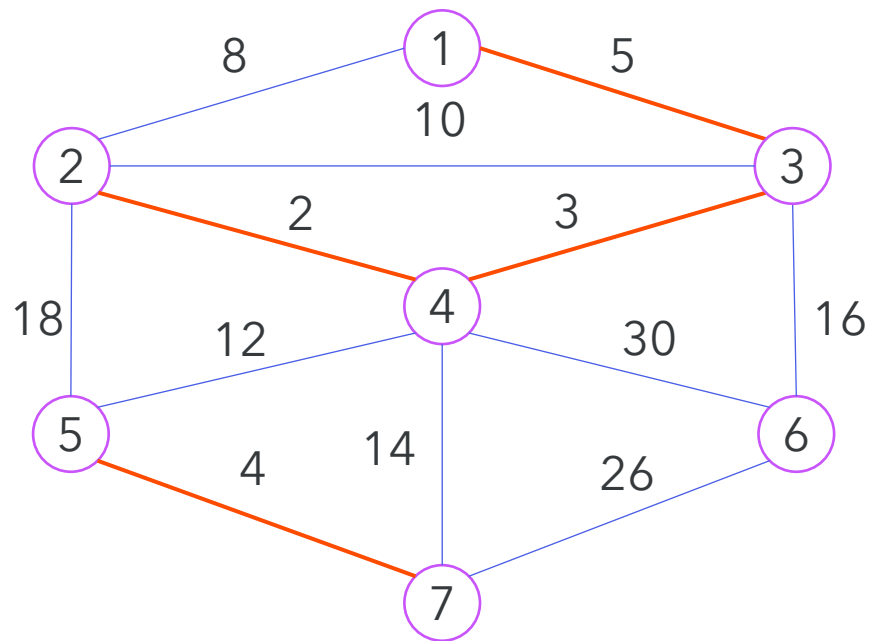
Prendo il primo nodo dalla lista: {5,7}
Non crea un ciclo, perciò lo inserisco in S .
Esiste un taglio che separa 5 da 7 in cui non ci sono lati di S : inserzione.



Lato	Costo
{1,3}	5
{1,2}	8
{2,3}	10
{4,5}	12
{4,7}	14
{5,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}\}, R = \emptyset$$

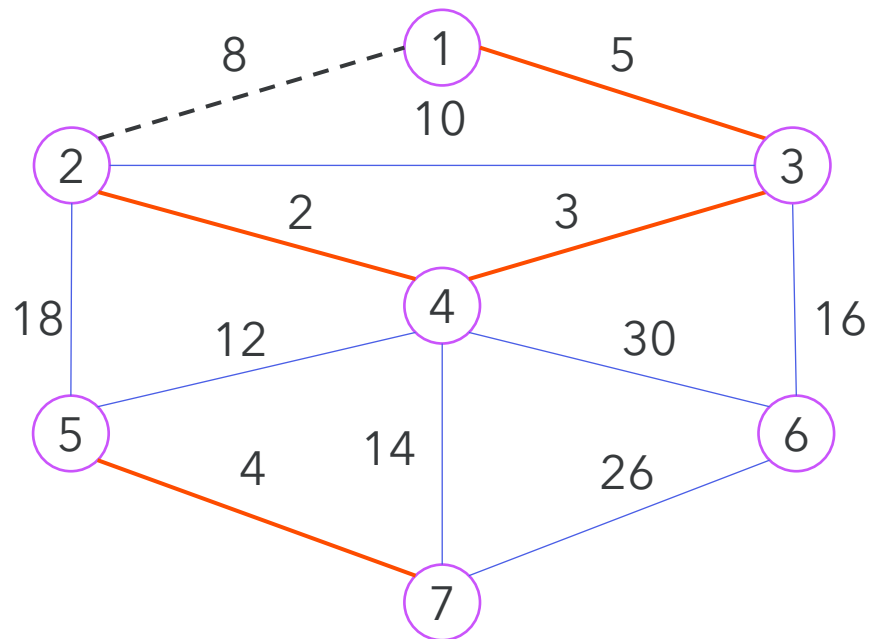
Prendo il primo nodo dalla lista: {1,3}
Non crea un ciclo, perciò lo inserisco in S .
Esiste un taglio che separa 1 da 3 in cui non ci sono lati di S : inserzione.



Lato	Costo
{1,2}	8
{2,3}	10
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}\}, R = \emptyset$$

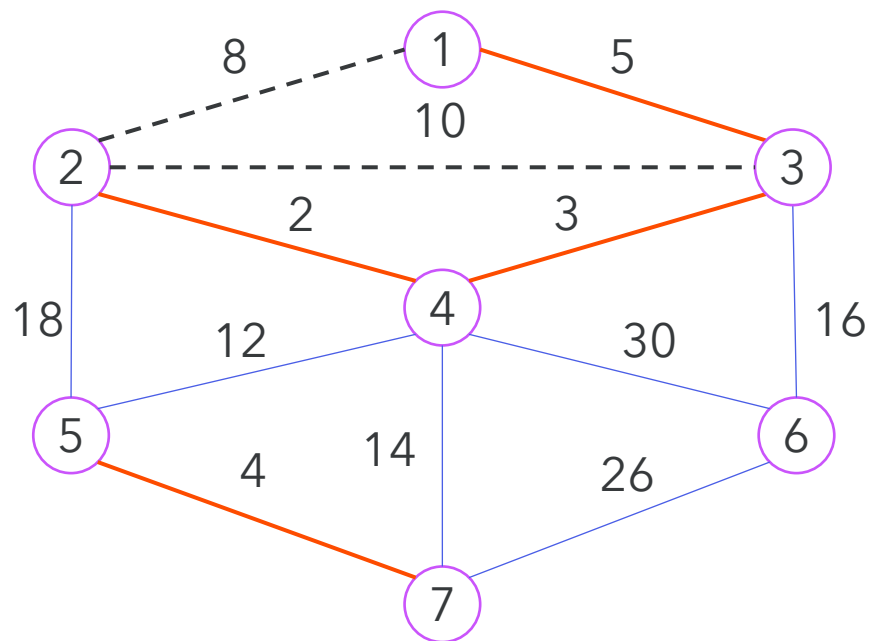
Prendo il primo nodo dalla lista: {1,2}
 Crea un **ciclo**, non può essere inserito in S .
 Prendo il ciclo 1,3,4,2: il lato con costo maggiore è {1,2}, lo inserisco in R .
 Operazione di **cancellazione**.



Lato	Costo
{2,3}	10
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}\}, \\ R = \{\{1,2\}\}$$

Prendo il primo nodo dalla lista: {2,3}
 Crea un **ciclo**, non può essere inserito in S .
 Prendo il ciclo 3,4,2: il lato con costo maggiore è {2,3}, lo inserisco in R .



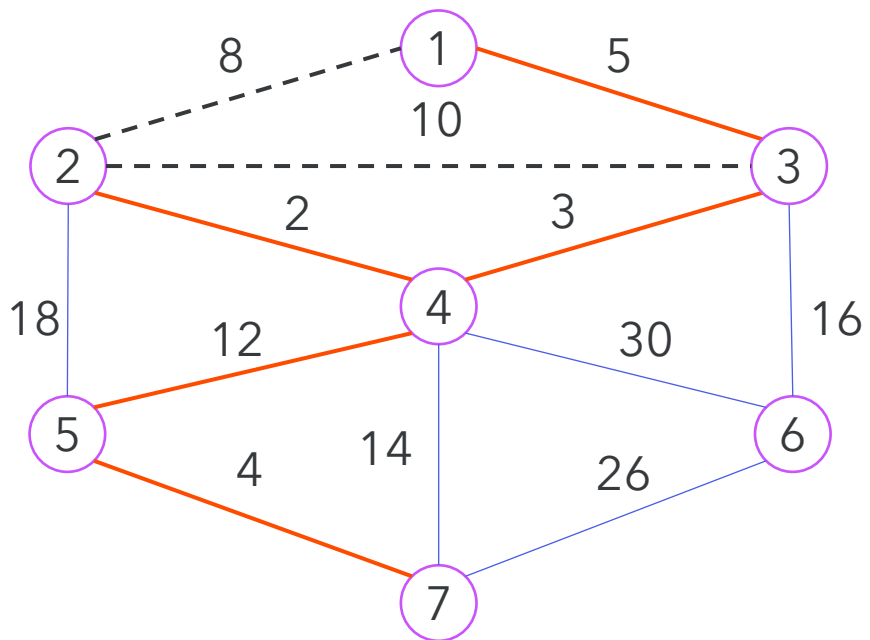
Lato	Costo
{4,5}	12
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}\},$$

$$R = \{\{1,2\}, \{2,3\}\}$$

Prendo il primo nodo dalla lista: {4,5}
Non crea un ciclo, perciò lo inserisco in S .

Esiste un taglio che separa 4 da 5 in cui non ci sono lati di S : inserzione.



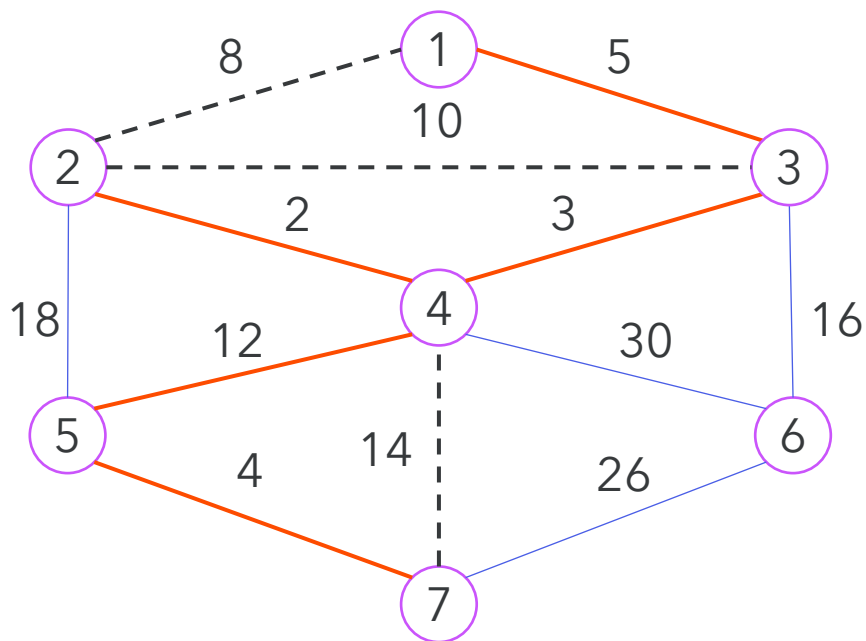
Lato	Costo
{4,7}	14
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}, \{4,5\}\},$$

$$R = \{\{1,2\}, \{2,3\}\}$$

Prendo il primo nodo dalla lista: {4,7}
Crea un **ciclo**, non può essere inserito in S .

Prendo il ciclo 4,5,7: il lato con costo maggiore è {4,7}, lo inserisco in R .



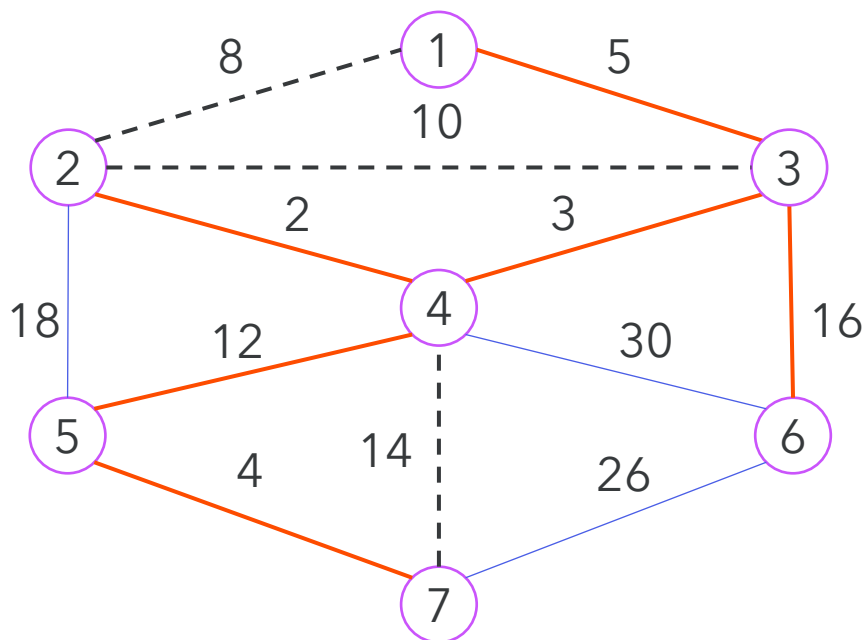
Lato	Costo
{3,6}	16
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}, \{4,5\}\},$$

$$R = \{\{1,2\}, \{2,3\}, \{4,7\}\}$$

Prendo il primo nodo dalla lista: {3,6}
Non crea un ciclo, perciò lo inserisco in S .

Esiste un taglio che separa 3 da 6 in cui non ci sono lati di S : inserzione.



Lato	Costo
{2,5}	18
{6,7}	26
{4,6}	30

$$S = \{\{2,4\}, \{3,4\}, \{5,7\}, \{1,3\}, \{4,5\}, \{3,6\}\},$$

$$R = \{\{1,2\}, \{2,3\}, \{4,7\}, \{2,5\}\}$$

Raggiungo la condizione di terminazione: $|S| = n - 1$

$T^* = S$ albero di copertura di costo minimo del grafo

Con costo:
 $C(T^*) = 2 + 3 + 4 + 5 + 12 + 16 = 42$

ALGORITMO DI KRUSKAL: COMPLESSITÀ

L'algoritmo di Kruskal può essere visto come:

- **Una fase di ordinamento** di tutti i lati per costo crescente.
- **Una fase di selezione** in cui si scansionano i lati.

L'algoritmo fa al massimo $O(m)$ iterazioni perché scansiona al massimo tutti i lati, però l'ordinamento costa $O(m \log(m))$ che domina la complessità della scansione.

Per evitare cicli, si utilizza una **struttura dati Union-Find** che permette di farlo a costo quasi costante (inversa della funzione di Ackermann).

ALGORITMO DI KRUSKAL: COMPLESSITÀ

In un **grafo connesso**: $m \geq n - 1$

Inoltre, in un grafo **completo**, il numero massimo di lati è: $\frac{n(n-1)}{2} \approx O(n^2)$

Quindi, in generale:

$$n - 1 \leq m \leq O(n^2)$$

Dal fatto che $m \leq O(n^2)$, otteniamo: $\log m \leq \log(O(n^2)) = O(\log n^2) = O(2 \log n) = O(\log n)$

E quindi posso anche scrivere:

$$O(m \log m) = O(m \log n)$$

ALGORITMO DI PRIM

```
procedure p = Prim( $G, c, r$ ) {  
  foreach( $i \in V$ ) do {  $p[i] = r$ ;  $d[i] = M$ ; }  
   $d[r] = -M$ ;  $Q = \{ r \}$ ;  
  do {  
    // Seleziona nodo  $u$  con distanza minima  
    seleziona  $u \in Q$  tale che  $d[u] = \min\{d[j] : j \in Q\}$ ;  
     $d[u] = -M$ ;  
     $Q = Q \setminus \{ u \}$ ;  
    // Aggiorna i vicini di  $u$   
    foreach( $\{u, v\} \in S(u)$ ) do  
      if( $c[u, v] < d[v]$ ) then {  
         $d[v] = c[u, v]$ ;  
         $p[v] = u$ ;  
        if( $v \notin Q$ ) then {  $Q = Q \cup \{ v \}$ ; }  
      }  
  } while( $Q \neq \emptyset$ );  
}
```

Scelto un nodo qualsiasi come radice r .

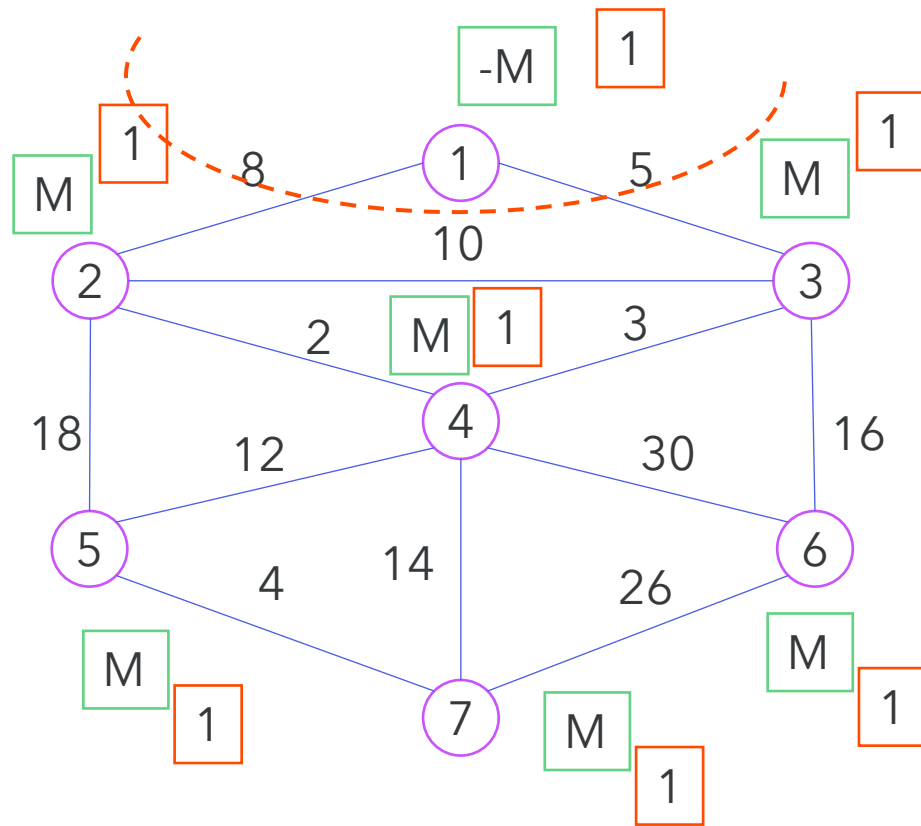
Inizializzazione:

- per tutti i nodi, porre l'etichetta predecessore uguale alla radice e l'etichetta d uguale a $+\infty$ (M).
- per il nodo radice porre l'etichetta d a $-\infty$ ($-M$).
- Inserire r nella coda di priorità Q .

Continuiamo il **ciclo** fino a che la coda non è vuota:

- Prendiamo il vertice u con etichetta minima dalla coda di priorità Q e poniamo la sua etichetta d a $-M$.
- Tra tutti i lati $\{u, v\}$ incidenti u , se il costo c_{uv} è minore dell'etichetta di v :
 - Poniamo l'etichetta di v al costo del lato
 - Fissiamo il predecessore di v a u .
 - Se v non è in coda lo inseriamo.

I predecessori permetteranno di costruire l'albero. La somma dei costi dei lati nell'albero sarà il costo della soluzione ottima.



Scegliamo una radice, per esempio 1.

Mettiamo tutte le etichette d dei nodi a M e tutti i predecessori a $r = 1$.

Mettiamo l'etichetta di 1 a $-M$ e aggiorniamo lo coda $Q = \{1\}$.

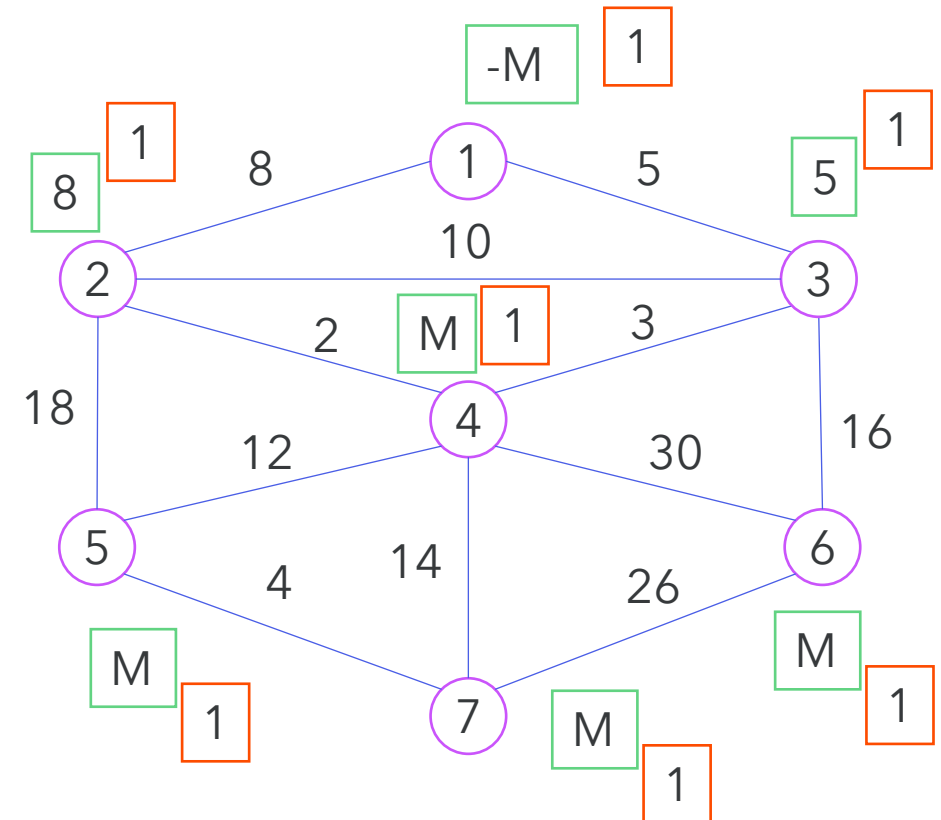
$Q = \{1\}$

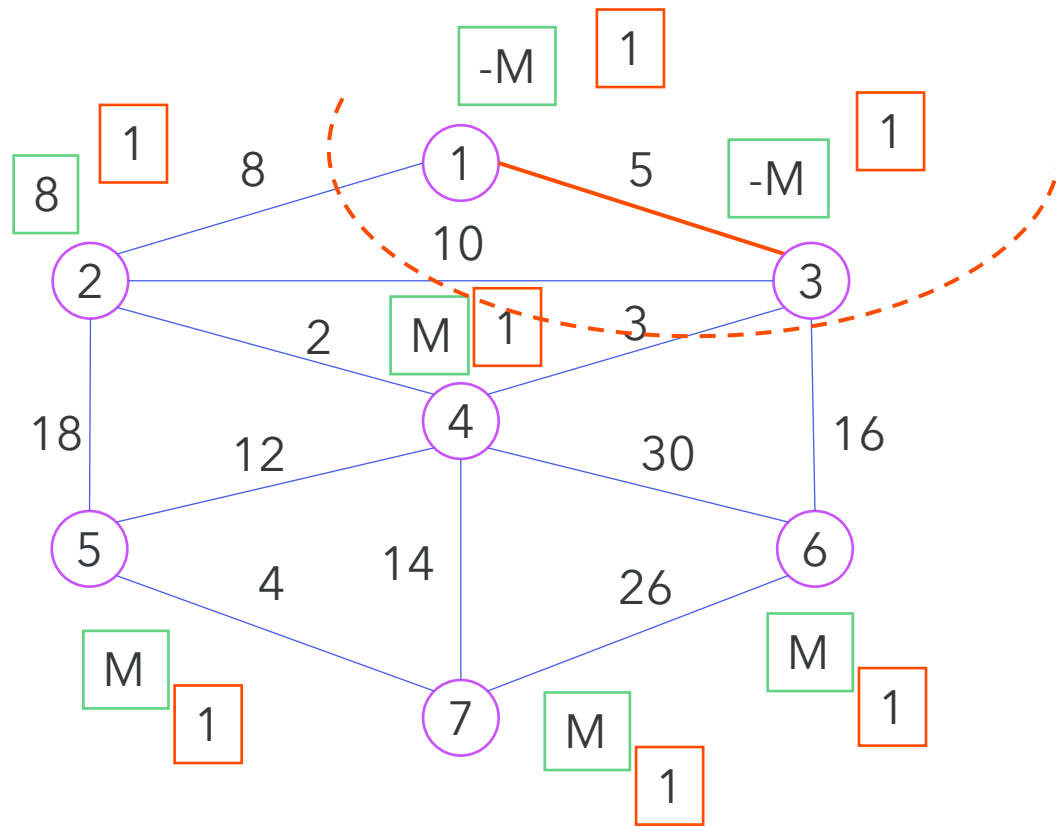
Selezioniamo il nodo in coda con etichetta minore: 1.

Valutiamo i lati incidenti a 1:

- $\{1,3\}$: l'etichetta di 3 è maggiore del costo c_{13} , perciò aggiorniamo l'etichetta $d_3 = 5$ e il suo predecessore a $p_3 = 1$. Inoltre metto 3 in coda: $Q = \{3\}$.
- $\{1,2\}$: $d_2 > c_{12}$, perciò $d_2 = 8$, $p_2 = 1$, $Q = \{3,2\}$.

Nella prima iterazione non si fanno né cancellazioni né inserzioni.





$Q = \{3,2\}$

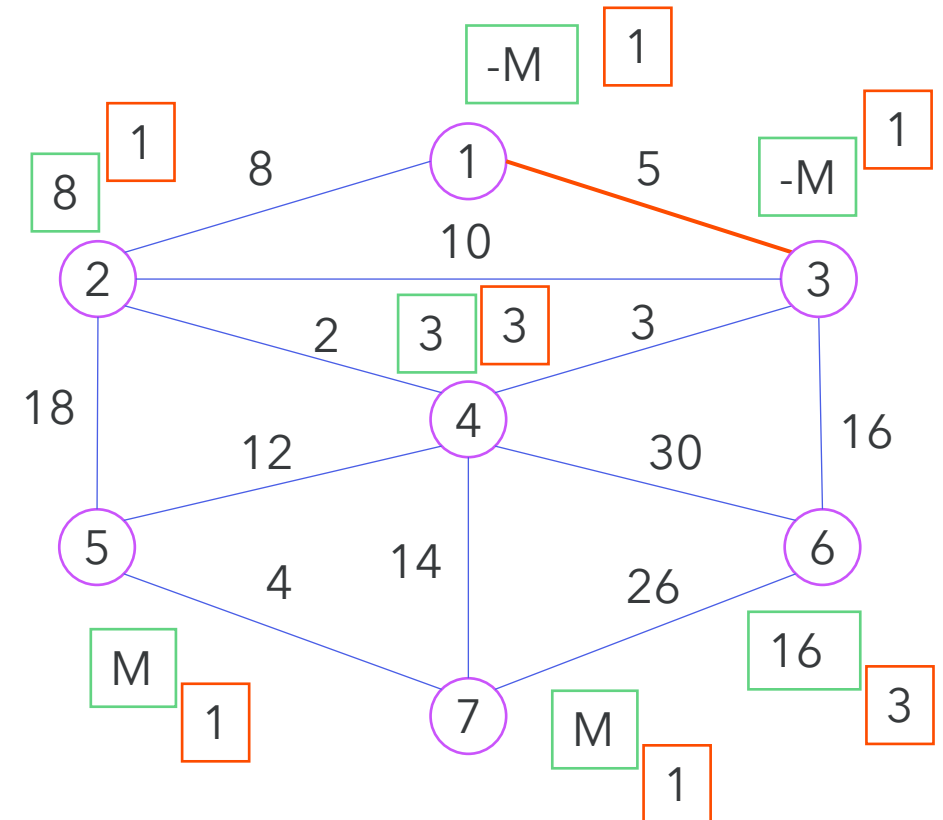
$c = 5$

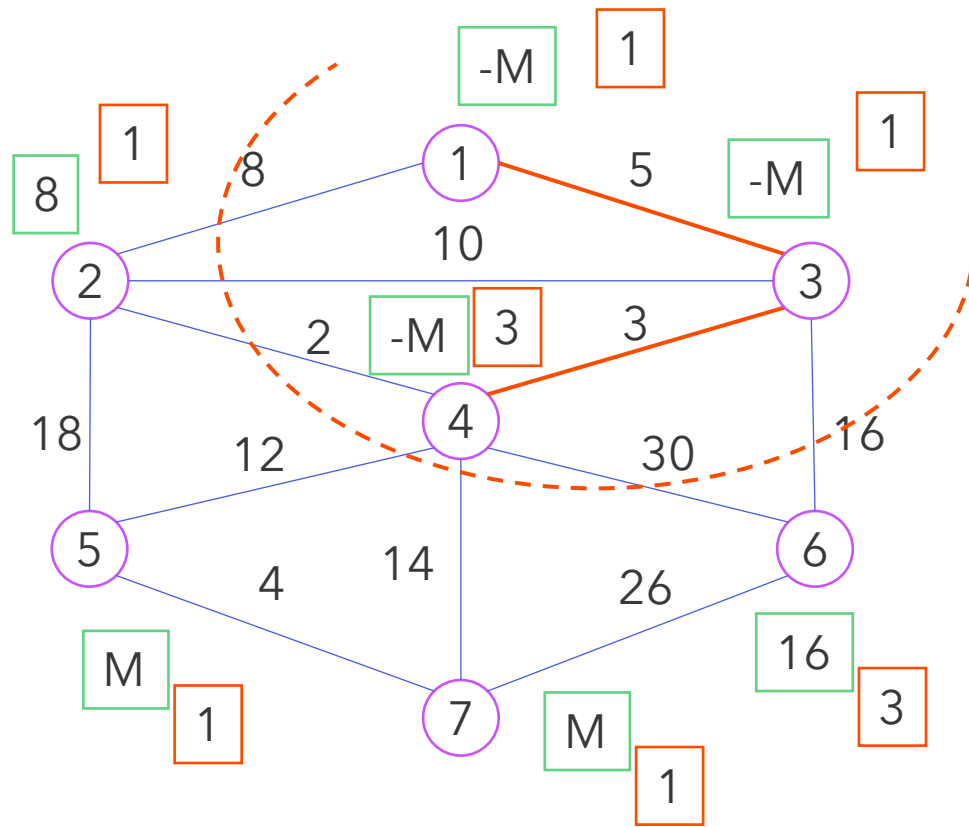
Il prossimo nodo che esce dalla coda (ha etichetta minore) è 3.
Pongo l'etichetta di 3 a $-M$.

Si effettua un'**inserzione**: il taglio è tra i nodi con etichetta $-M$ da un lato e tutti gli altri nodi dall'altro.

Valutiamo i lati del taglio incidenti a 3:

- $\{2,3\}$: l'etichetta di 2 è minore del costo c_{23} , perciò non aggiorno l'etichetta d_2 .
- $\{3,4\}$: $d_4 > c_{32}$, perciò $d_4 = 3$, $p_4 = 3$, $Q = \{4,2\}$.
- $\{3,6\}$: $d_6 > c_{36}$, perciò $d_6 = 16$, $p_6 = 3$, $Q = \{4,2,6\}$.
- $\{1,3\}$: $d_1 = -M < c_{13}$, non lo considero più.





$Q = \{2,5,7,6\}$

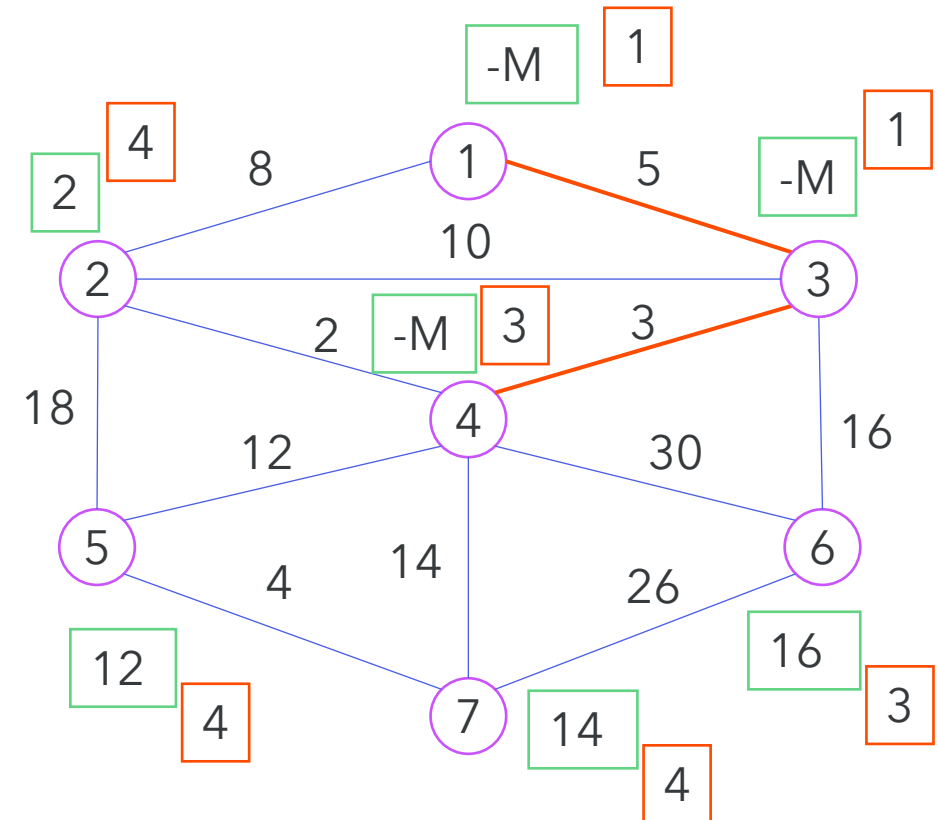
$c = 8$

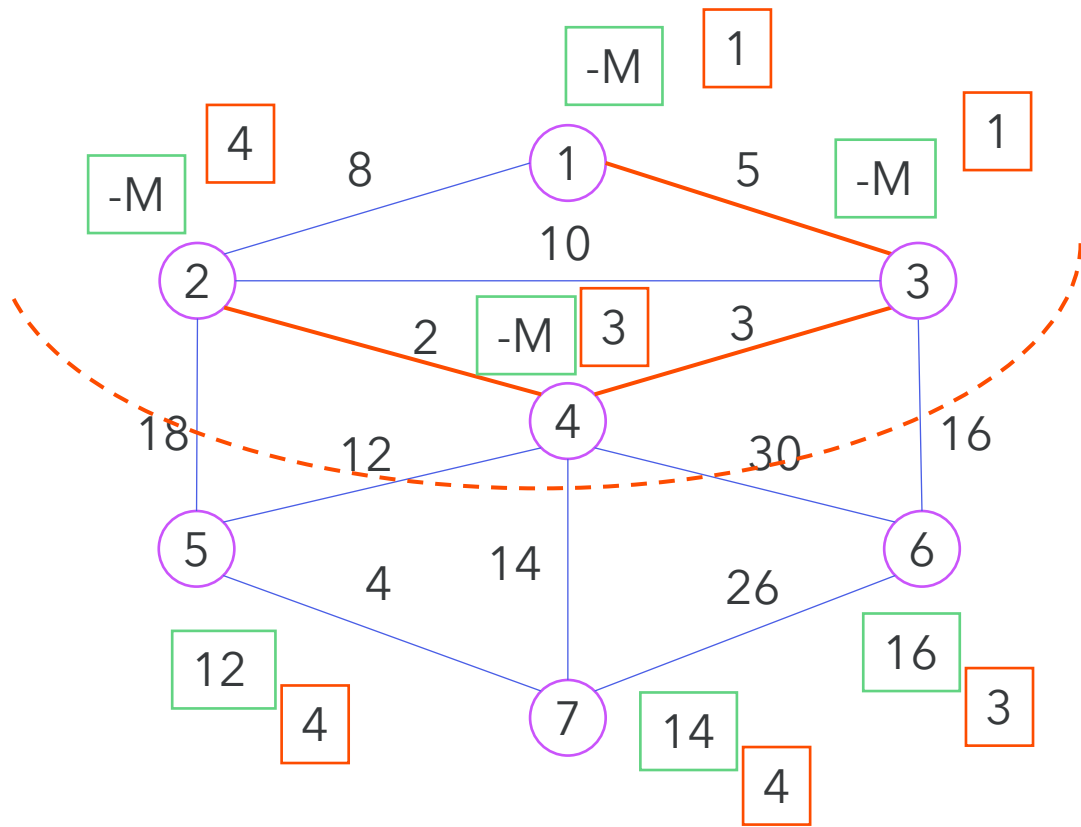
Il prossimo nodo che esce dalla coda (ha etichetta minore) è 4. Pongo l'etichetta di 4 a $-M$.

Si effettua un'**inserzione**: il taglio è tra i nodi con etichetta $-M$ da un lato e tutti gli altri nodi dall'altro.

Valutiamo i lati del taglio incidenti a 4:

- $\{2,4\}$: $d_2 > c_{24}$, perciò $d_2 = 2$, $p_2 = 4$, $Q = \{2,6\}$.
- $\{4,5\}$: $d_5 > c_{45}$, perciò $d_5 = 12$, $p_5 = 4$, $Q = \{2,5,6\}$.
- $\{4,6\}$: $d_6 < c_{36}$, non aggiorno.
- $\{4,7\}$: $d_7 > c_{47}$, perciò $d_7 = 14$, $p_7 = 4$, $Q = \{2,5,7,6\}$.
- $\{4,3\}$: $d_3 = -M < c_{43}$, non lo considero più.





Valutiamo i lati del taglio incidenti a 2:

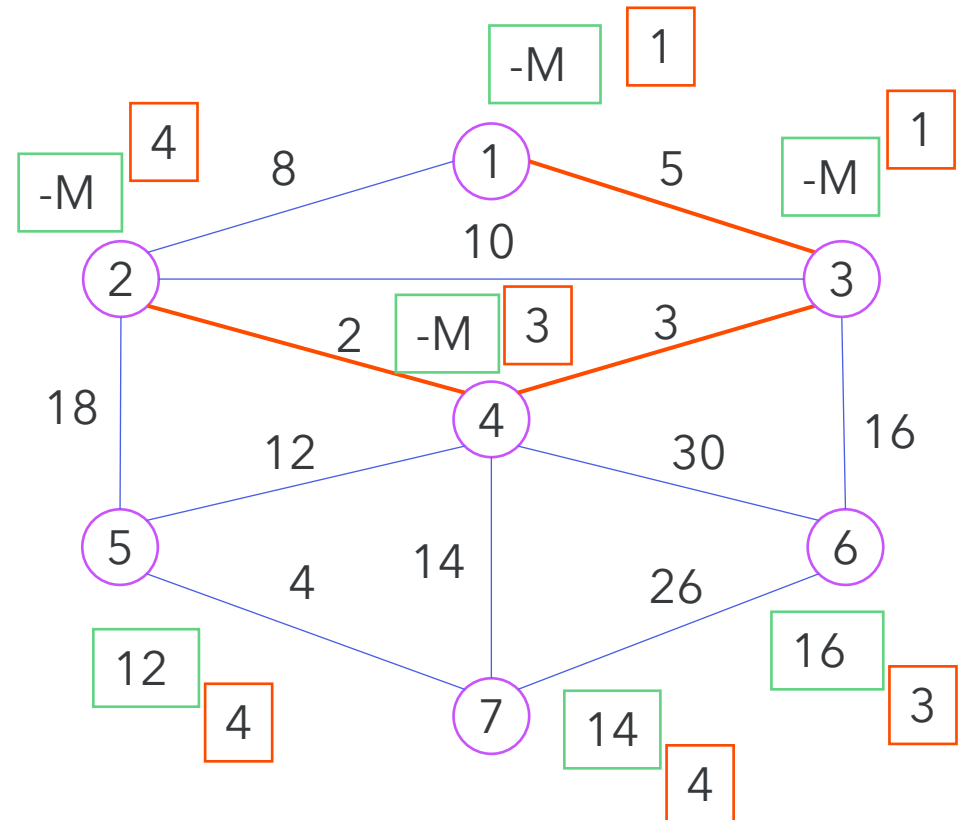
- $\{2,5\}$: $d_5 < c_{25}$, non aggiorno.
- $\{2,4\}$: $d_4 = -M < c_{24}$, non lo considero più.
- $\{2,3\}$: $d_3 = -M < c_{23}$, non lo considero più.
- $\{2,8\}$: $d_8 = -M < c_{28}$, non lo considero (cancellazione).

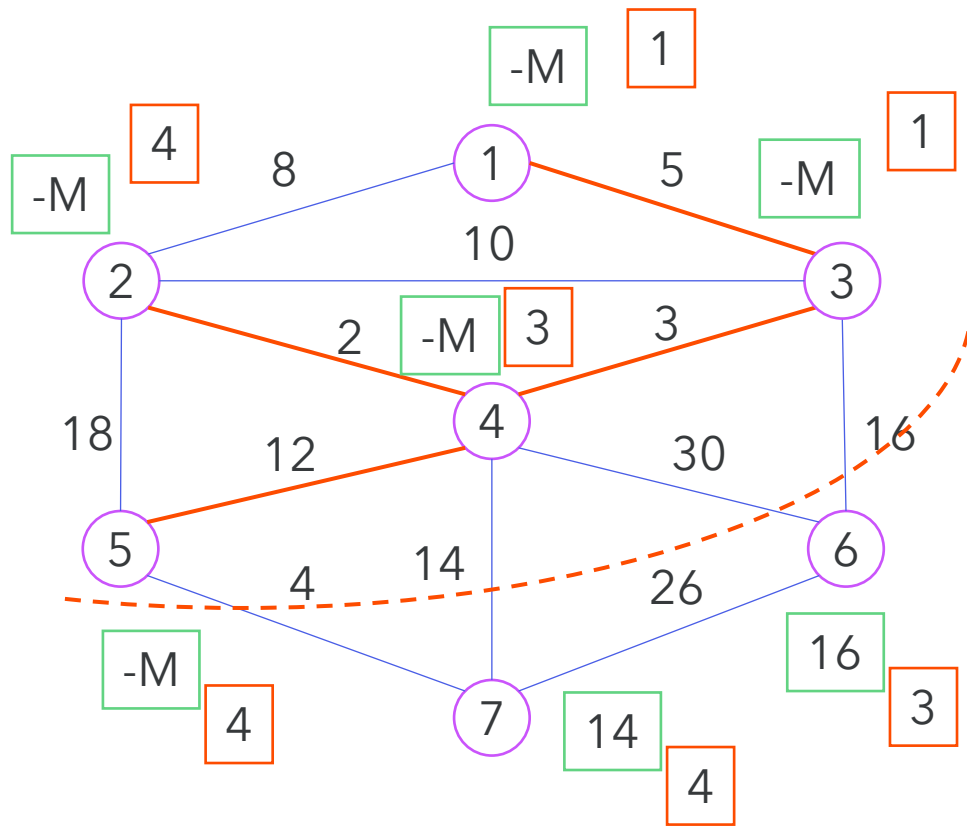
$Q = \{2,5,7,6\}$

$c = 10$

Il prossimo nodo che esce dalla coda (ha etichetta minore) è 2.
Pongo l'etichetta di 2 a $-M$.

Si effettua un'**inserzione**: il taglio è tra i nodi con etichetta $-M$ da un lato e tutti gli altri nodi dall'altro.





$Q = \{5, 7, 6\}$

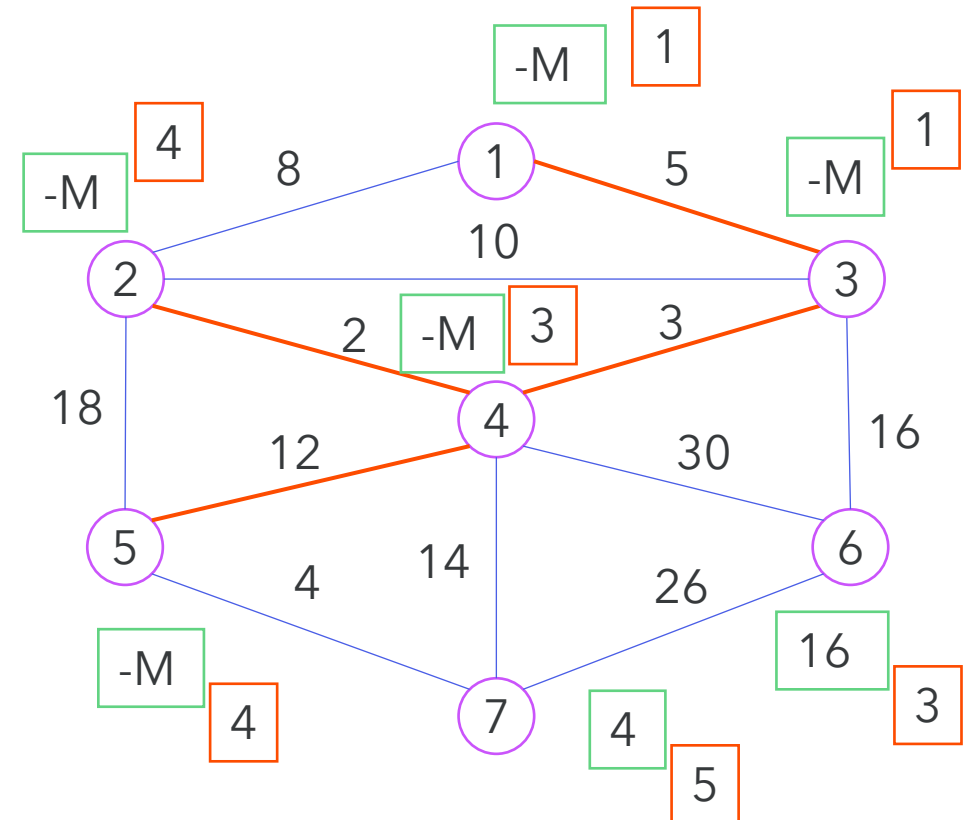
$c = 22$

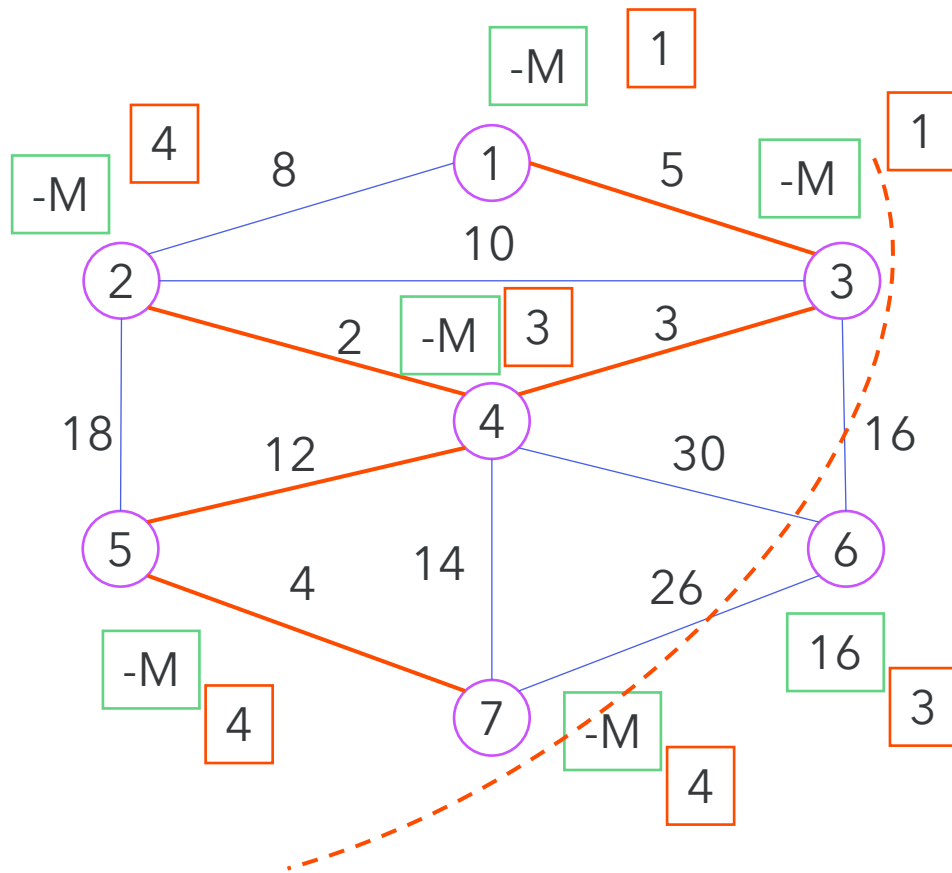
Il prossimo nodo che esce dalla coda (ha etichetta minore) è 5.
Pongo l'etichetta di 5 a $-M$.

Si effettua un'**inserzione**: il taglio è tra i nodi con etichetta $-M$ da un lato e tutti gli altri nodi dall'altro.

Valutiamo i lati del taglio incidenti a 5:

- $\{5, 7\}$: $d_7 > c_{57}$, perciò $d_7 = 4$, $p_7 = 5$, $Q = \{7, 6\}$.
- $\{4, 5\}$: $d_5 = -M < c_{45}$, non lo considero più.
- $\{2, 5\}$: $d_2 = -M < c_{25}$, non lo considero (cancellazione).





$Q = \{7,6\}$

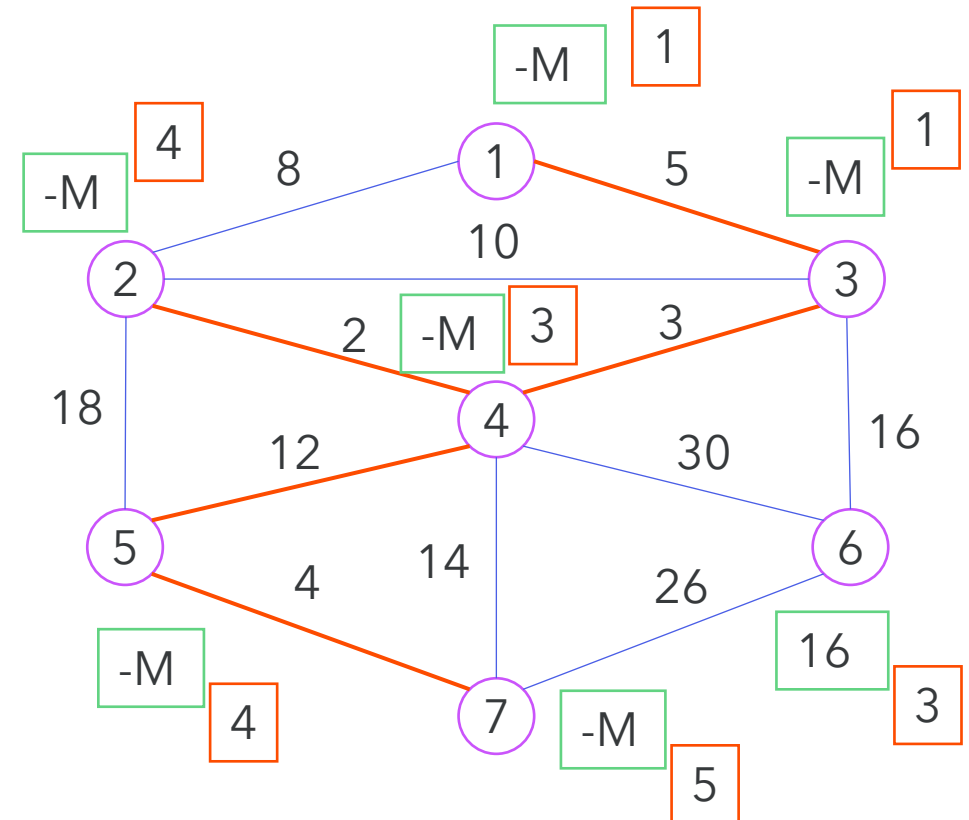
$c = 26$

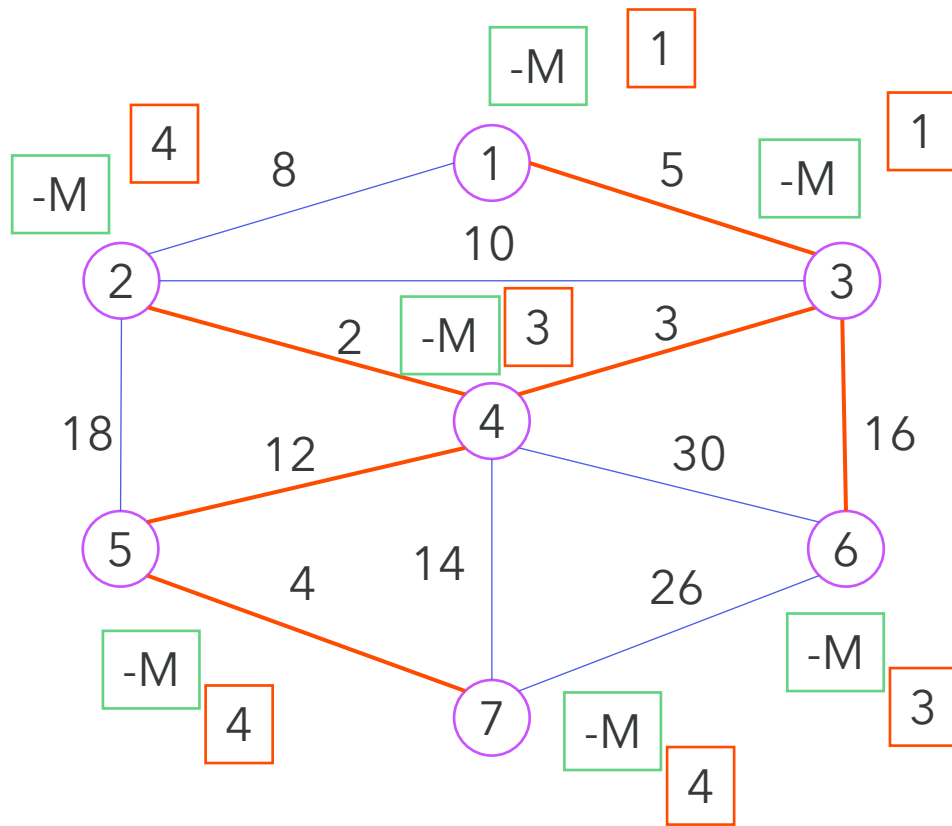
Il prossimo nodo che esce dalla coda (ha etichetta minore) è 7.
Pongo l'etichetta di 7 a $-M$.

Si effettua un'**inserzione**: il taglio è tra i nodi con etichetta $-M$ da un lato e tutti gli altri nodi dall'altro.

Valutiamo i lati del taglio incidenti a 7:

- $\{6,7\}$: $d_6 < c_{67}$, non aggiorno.
- $\{4,7\}$ e $\{5,7\}$ non hanno conviene l'aggiornamento.





$$Q = \{6\}$$

$$c = 42$$

Il prossimo nodo che esce dalla coda (ha etichetta minore) è 6.
Pongo l'etichetta di 6 a $-M$.

Valutiamo i lati del taglio incidenti a 6:

- Non ci sono lati che portano a nodi etichette promettenti.

$$Q = \emptyset$$

$$c = 42$$

Ho ottenuto l'albero di costo minimo

$E_T = \{\{1,3\}, \{2,4\}, \{3,4\}, \{3,6\}, \{4,5\}, \{5,7\}\}$
con costo 45.

Sono sicuro che quello trovato è l'albero ricoprente di costo minimo perché ho effettuato un'operazione di inserzione corretta ad ogni iterazione.

La correttezza deriva dal fatto che ad ogni operazione ho preso il taglio tra i nodi con etichetta $-M$ e gli altri e tra i lati del taglio ho scelto quello di costo minimo.

ALGORITMO DI PRIM: COMPLESSITÀ

La complessità dell'algoritmo dipende dall'ordinamento della coda Q :

- Se l'ordinamento è fatto con un **heap binario**: $O(m \log n)$
 - che è la stessa complessità dell'algoritmo di Kruskal.
- Con una coda non ordinata: $O(n^2)$
 - che in un grafo denso è meglio di $O(n^2 \log n)$ di Kruskal.



PROBLEMA DI FLUSSO MASSIMO

DA PAG 72 DISPENSE

PROBLEMA DI FLUSSO MASSIMO

Definizione generale

Dato un grafo orientato $G = (N, A)$, con:

N : insieme dei nodi

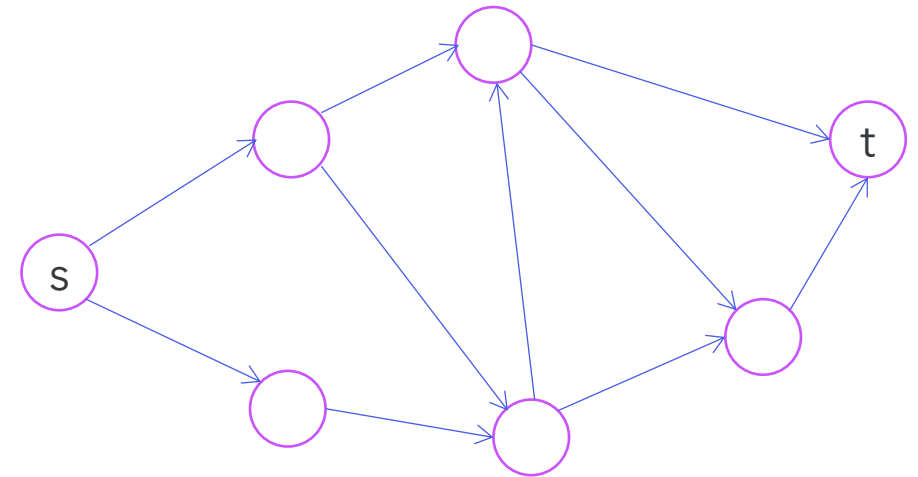
A : insieme degli archi orientati

$\mu = [\mu_{ij}]_{(i,j) \in A}$: capacità massima di ciascun arco

Si individuano due nodi speciali:

s: sorgente (origine del flusso)

t: pozzo (destinazione del flusso)



Obiettivo: Determinare la **massima quantità di flusso** v inviabile da **s** a **t**, rispettando i vincoli di capacità e conservazione del flusso.

FORMULAZIONE MATEMATICA

Si vuole determinare un **flusso ammissibile** $x = [x_{ij}]_{(i,j) \in A}$ tale che si rispettino:

- i vincoli *bilanciamento del flusso*:

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = \begin{cases} -v & i = s \\ v & i = t \\ 0 & \text{altrimenti} \end{cases}$$

- i vincoli di *capacità*:

$$0 \leq x_{ij} \leq \mu_{ij} \quad (i,j) \in A$$

Massimizzando v .

FLUSSO MASSIMO COME FLUSSO DI COSTO MINIMO

Il **problema del flusso massimo (MF)** è un caso particolare del **problema di flusso di costo minimo (MCF)**.

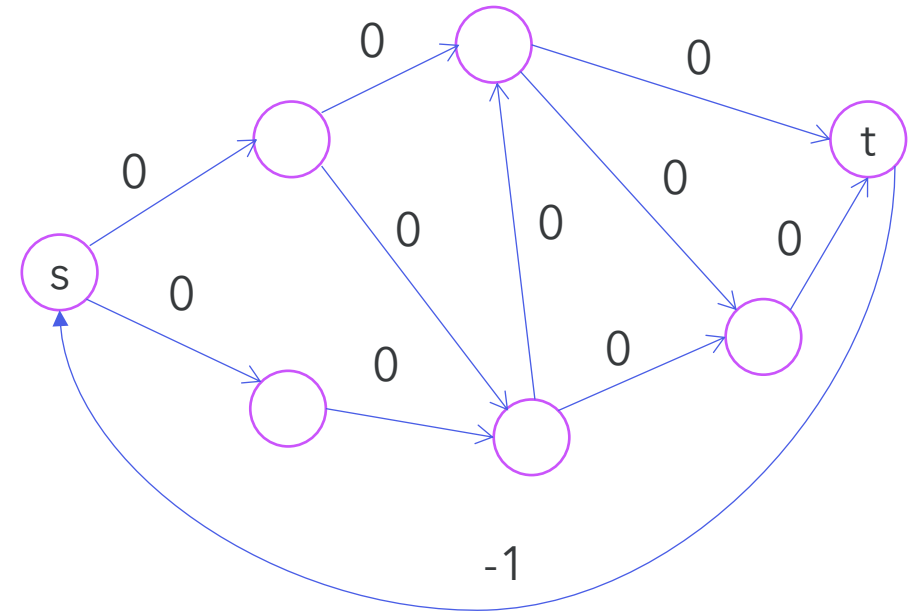
Costruzione di un grafo esteso G'

- Si aggiunge un **arco fittizio** (t, s) detto **arco di ritorno**
- Il flusso su questo arco è $x_{ts} = v$, ossia il valore del flusso massimo
- La capacità su (t, s) è infinita, mentre le altre capacità rimangono le stesse.

Costi

- Tutti gli archi reali hanno **costo 0**
- L'arco fittizio (t, s) ha **costo -1**

Minimizzare $-v \Leftrightarrow$ massimizzare v



In G' tutti i nodi (compresi s e t) hanno bilancio nullo $\rightarrow$ **problema di circolazione**

FLUSSO MASSIMO COME FLUSSO DI COSTO MINIMO

Nel grafo G' :

- Il flusso circola chiuso, ossia non vi sono sorgenti o pozzi reali
- Il valore del flusso massimo in G corrisponde al **flusso sull'arco di ritorno** (t, s)
- Minimizzando il costo totale in G' si ottiene implicitamente la massimizzazione del flusso in G

Vantaggio:

Questa formulazione permette di usare tecniche del flusso di costo minimo anche per calcolare il flusso massimo.

FLUSSI AMMISSIBILI E CONDIZIONE DI OTTIMALITÀ

Contesto:

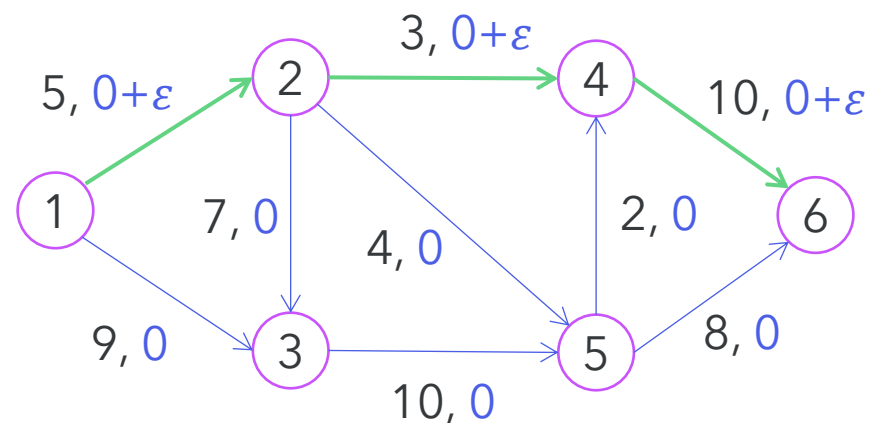
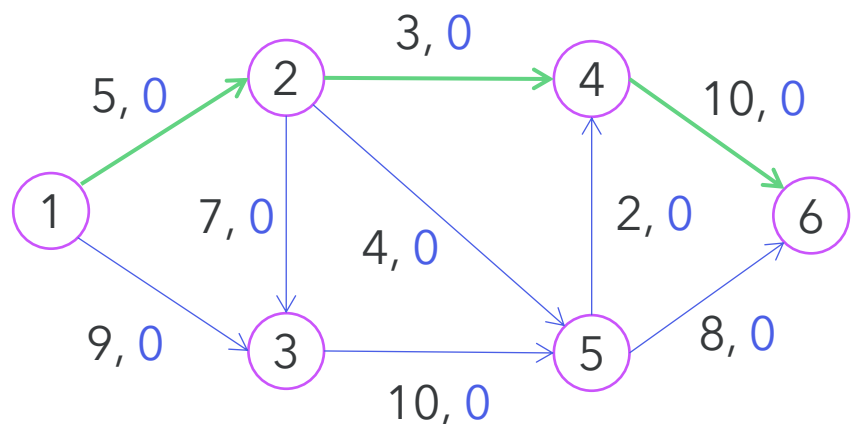
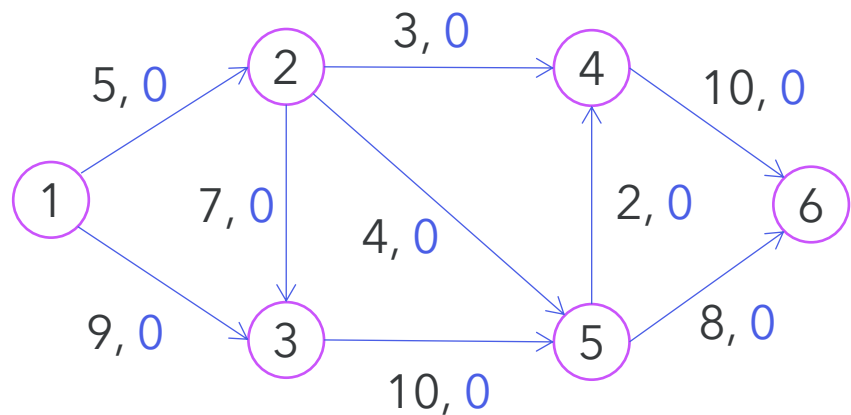
Abbiamo un flusso ammissibile x di valore v su un grafo orientato $G = (N, A)$.

Vogliamo capire se x è **ottimo**, cioè se non è possibile aumentare ulteriormente il flusso da s a t .

Idea intuitiva:

Se x **non è ottimo**, deve esistere un modo per inviare “altro flusso” da s a t .

Tale flusso addizionale può essere instradato lungo un **cammino** che collega s a t .



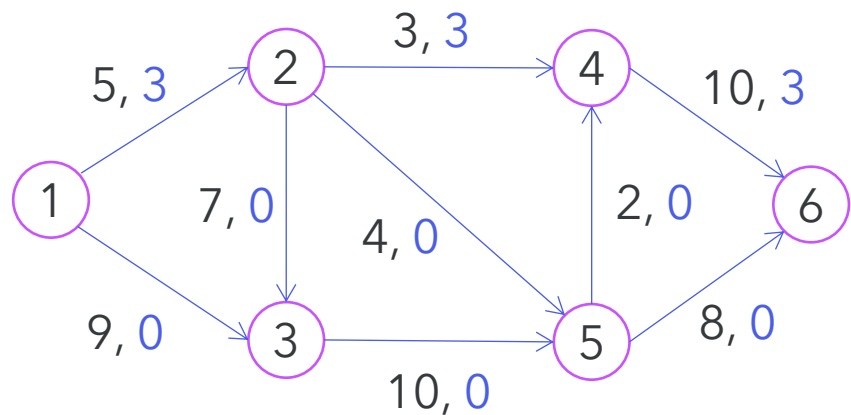
Dato un grafo $G = (N, A)$, $N = \{1, 2, 3, 4, 5, 6\}$, $A = \{(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 6), (5, 4), (5, 6)\}$, con capacità $\mu = (5, 9, 7, 5, 4, 10, 10, 2, 8)$, cerchiamo il flusso massimo tra la sorgente $s = 1$ e la destinazione $t = 6$.

Partiamo da un **flusso ammissibile**: tutto il flusso x è posto a 0. E' ammissibile perché rispetta le capacità e i vincoli di bilanciamento del flusso. Il flusso ha valore $v = 0$.

Non è il flusso ottimo: possiamo fare di meglio? Partiamo da s e vediamo se riusciamo a «portare» del flusso fino a t . Per esempio potremmo portare del flusso seguendo il **cammino** $(s, 2), (2, 4), (4, t)$.

Quanto possiamo portare? La capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(2, 4)$ con capacità 3. Allora aggiungo un flusso $\varepsilon = 3$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 3$.

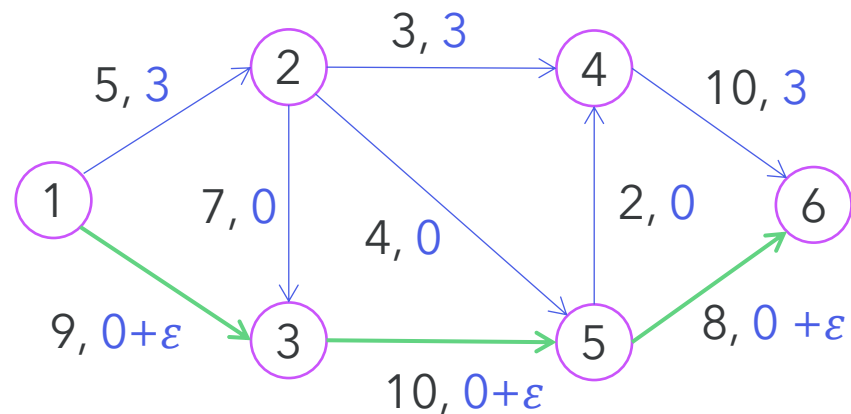
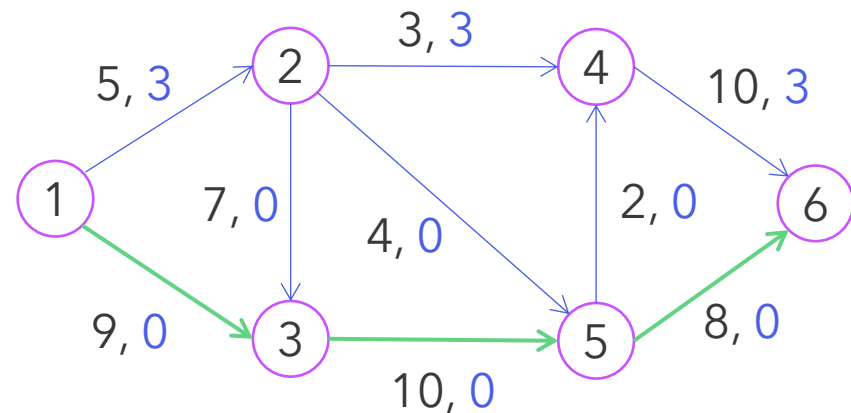


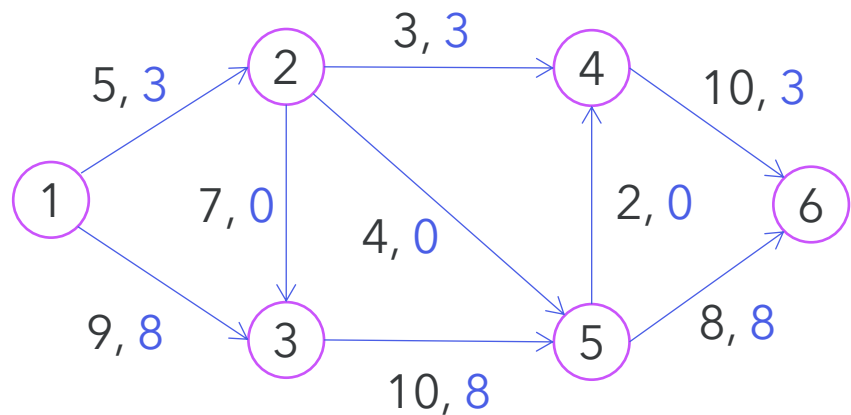
Abbiamo un flusso con valore $v = 3$. Possiamo migliorare?

Partiamo da s e vediamo se riusciamo a «portare» del flusso fino a t . Per esempio potremmo portare del flusso seguendo il **cammino** $(s,3), (3,5), (5,t)$.

Al massimo possiamo portare la capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(5,6)$ con capacità 8. Allora aggiungo un flusso $\varepsilon = 8$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 3 + 8 = 11$.



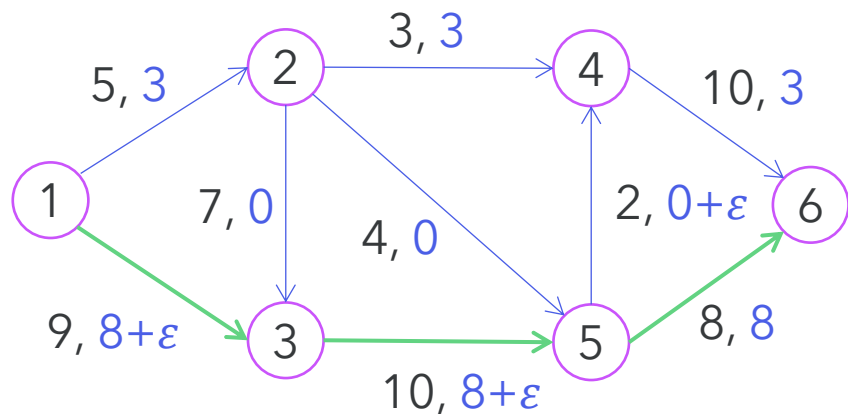
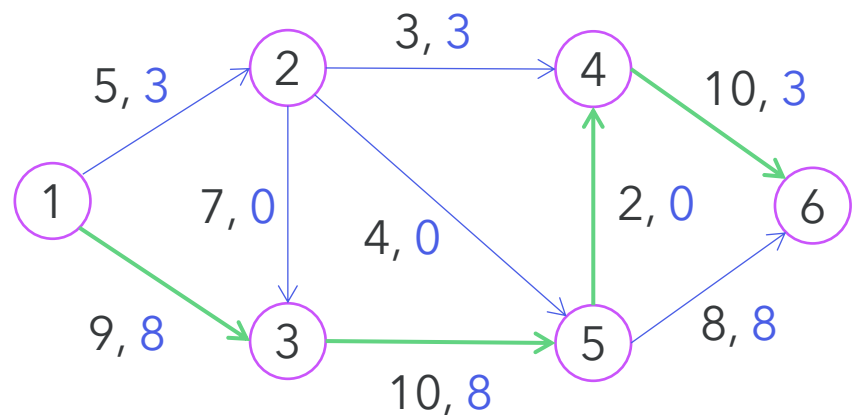


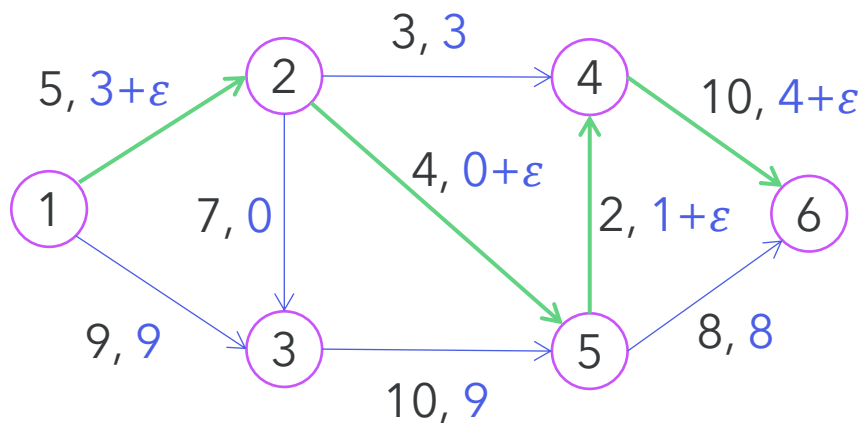
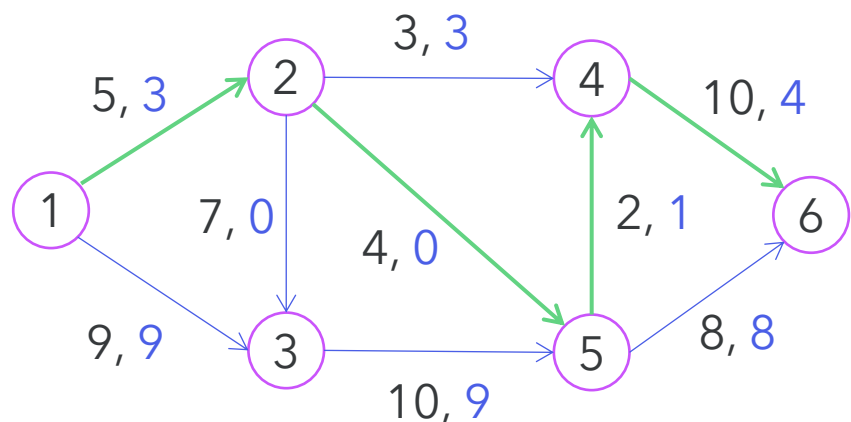
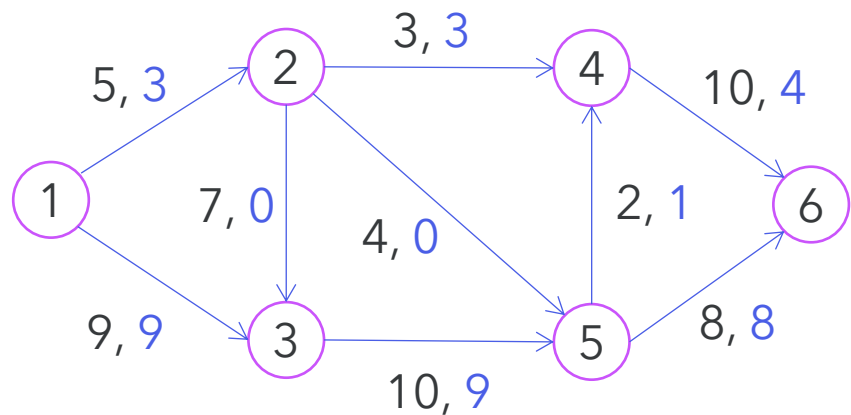
Abbiamo un flusso con valore $v = 11$. Possiamo migliorare?

Partiamo da s e vediamo se riusciamo a «portare» del flusso fino a t . Per esempio potremmo portare del flusso seguendo il **cammino** $(s,3), (3,5), (5,4), (4,t)$.

Al massimo possiamo portare la capacità residua minore tra gli archi del cammino. Il **collo di bottiglia** è $(1,3)$ con capacità residua 1. Allora aggiungo un flusso $\varepsilon = 1$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 11 + 1 = 12$.



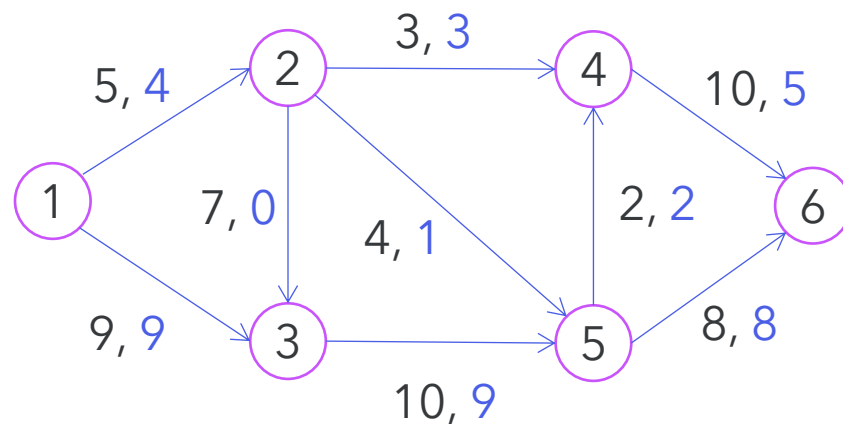


Abbiamo un flusso con valore $v = 12$. Possiamo migliorare?

Partiamo da s e vediamo se riusciamo a «portare» del flusso fino a t . Per esempio potremmo portare del flusso seguendo il **cammino** $(s,2), (2,5), (5,4), (5,t)$.

Al massimo possiamo portare la capacità residua minore tra gli archi del cammino. Il **collo di bottiglia** è $(5,4)$ con capacità residua 1. Allora aggiungo un flusso $\varepsilon = 1$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 12 + 1 = 13$.



È il flusso massimo?

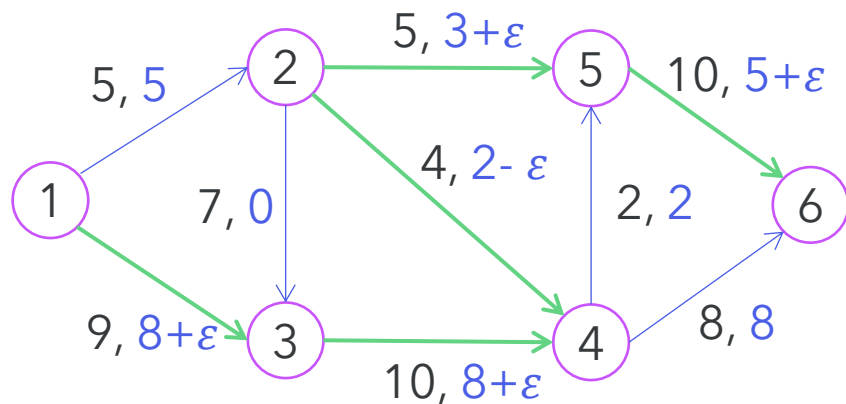
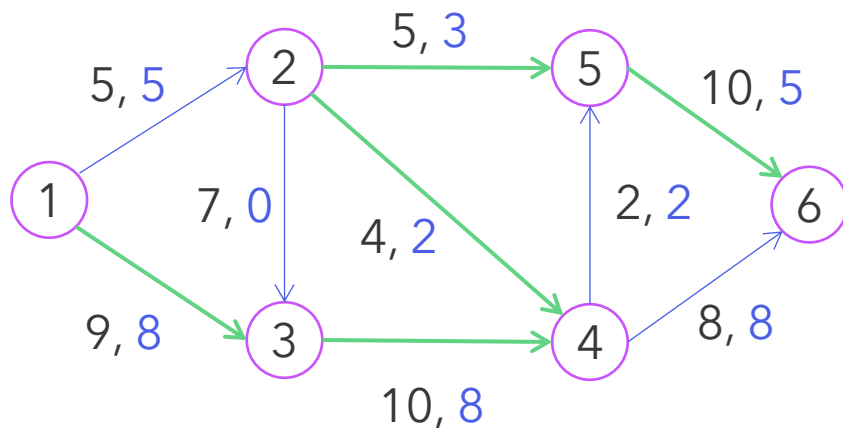
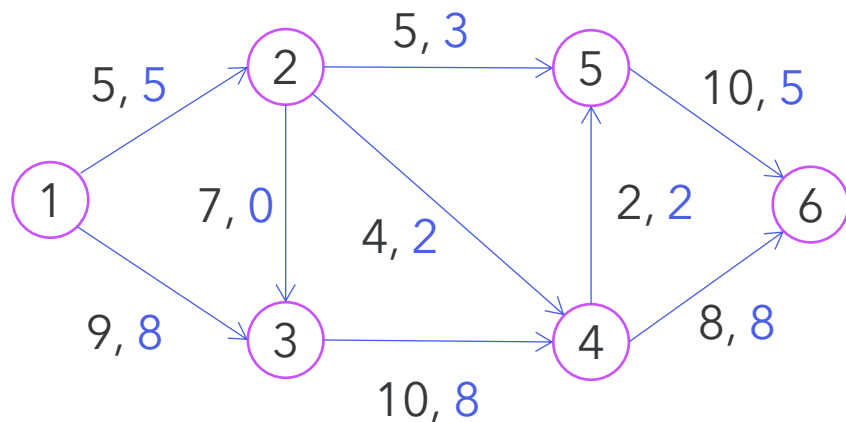
CAMMINI AUMENTANTI

Quello che abbiamo fatto fino ad ora era:

- Avere un flusso ammissibile ad ogni iterazione
- Cercare un cammino che aumenti il flusso da s a t

Il **cammino aumentante** non è orientato, anzi,

- può percorrere gli archi nel loro verso (*se non sono saturi*),
- ma anche nel verso contrario (*se sono percorsi da flusso*).

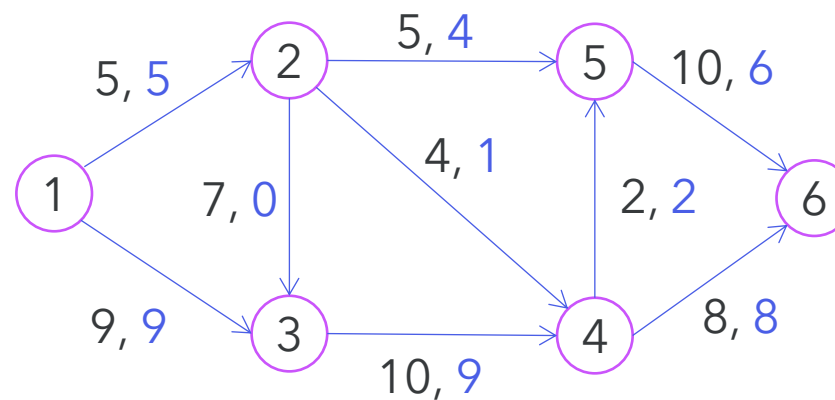


Dato il seguente grafo con un flusso x con valore $v = 13$.

Partiamo da s e vediamo se riusciamo a «portare» flusso fino a t . Per esempio potremmo seguire il **cammino** $(s,3), (3,4), (4,2), (2,5), (5,t)$.

Percorrere l'arco $(2,4)$ al contrario significa spostare il flusso dell'arco sul nodo 2, dal quale deve poi prendere un'altra strada. In pratica cambiamo la direzione di una quantità di flusso per permettere maggiore flusso da s a t .

Il **collo di bottiglia** in questo caso è definito dalla minima capacità residua sugli archi percorsi nel loro verso, o dalla minima quantità di flusso che posso spostare tra gli archi percorsi al contrario. In particolare, il minimo tra questi due valori. Nell'esempio è l'arco $(s,3)$, perciò avrò un valore $\epsilon = 1$. In particolare dovrò aggiungere questa quantità sugli archi percorsi nel loro verso e toglierla negli archi percorsi al contrario.



Abbiamo un flusso con valore $v = 13 + 1 = 14$.

CAMMINI AUMENTANTI

Un **cammino aumentante** è un

- cammino (non necessariamente orientato) P
- che porta flusso da s a t .

Gli archi di P si dividono in due insiemi:

- P^+ : archi **concordi**, percorsi nel **verso del flusso**
- P^- : archi **discordi**, percorsi nel **verso opposto al flusso**

$$P = P^+ \cup P^-, \\ P^+ \cap P^- = \emptyset$$

CAPACITÀ DI UN CAMMINO

Si definisce quindi la **capacità del cammino** P rispetto al flusso x :

$$\theta(P, x) = \min\{\min\{\mu_{ij} - x_{ij} : (i, j) \in P^+\} , \min\{x_{ij} : (i, j) \in P^-\}\} \geq 0$$

Interpretazione:

- Se $\theta(P, x) > 0$ il cammino è **aumentante**
- Se $\theta(P, x) = 0$ almeno un arco concorde è **saturo** o un arco discorde è **vuoto**: non riesco a portare flusso.

MODIFICA DEL FLUSSO LUNGO UN CAMMINO

Vogliamo aumentare o diminuire il flusso su P senza violare i vincoli di capacità.

Si definisce una **composizione** tra un flusso x e un cammino P :

$$x(\theta) = x \oplus \theta P$$

dove

$$x_{ij}(\theta) = \begin{cases} x_{ij} + \theta & \text{se } (i, j) \in P^+ \\ x_{ij} - \theta & \text{se } (i, j) \in P^- \\ x_{ij} & \text{altrimenti} \end{cases}$$

Aggiungiamo θ al flusso degli archi concordi e togliamo θ al flusso degli archi discordi.

Il nuovo flusso $x(\theta)$ **rispetta la conservazione del flusso** nei nodi intermedi.

Il **valore** del flusso diventa $v + \theta$.

MODIFICA DEL FLUSSO LUNGO UN CAMMINO

Teorema: dato un flusso ammissibile x di valore v , un cammino P di capacità $\theta(P, x)$ e uno scalare θ tale che $0 \leq \theta \leq \theta(P, x)$, allora $x(\theta) = x \oplus \theta P$ è un flusso ammissibile di valore $v + \theta$.

Il nuovo flusso è ammissibile per costruzione:

- Dove abbiamo aumentato il flusso non aumenta più della capacità residua, dove lo diminuiamo non può assumere valori negativi.
- Inoltre si mantiene il bilanciamento.

Come creiamo un cammino aumentante?

IL GRAFO RESIDUO $G_x = (N, A_x)$

Per individuare cammini aumentanti in modo sistematico, si costruisce il **grafo residuo**:

Si mantengono i nodi N .

Si cambiano gli archi:

$$A_x = A_x^+ \cup A_x^- = \{(i, j) \in A: x_{ij} < \mu_{ij}\} \cup \{(i, j): (j, i) \in A, x_{ji} > 0\}$$

Se un arco non è saturo $\rightarrow$ può accettare altro flusso $\rightarrow$ inserisco un **arco diretto (concorde)**

Se un arco non è vuoto $\rightarrow$ è possibile "restituire" flusso $\rightarrow$ inserisco un **arco inverso (discorde)**

Quindi, nel grafo residuo:

- Ogni arco può avere **due rappresentazioni**, una diretta e una inversa
- I cammini aumentanti corrispondono a **cammini orientati** in G_x

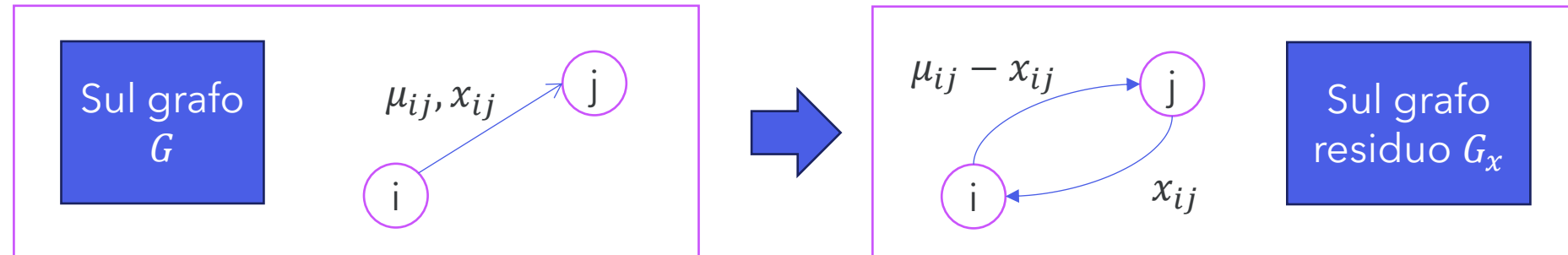
RICERCA DI CAMMINI AUMENTANTI

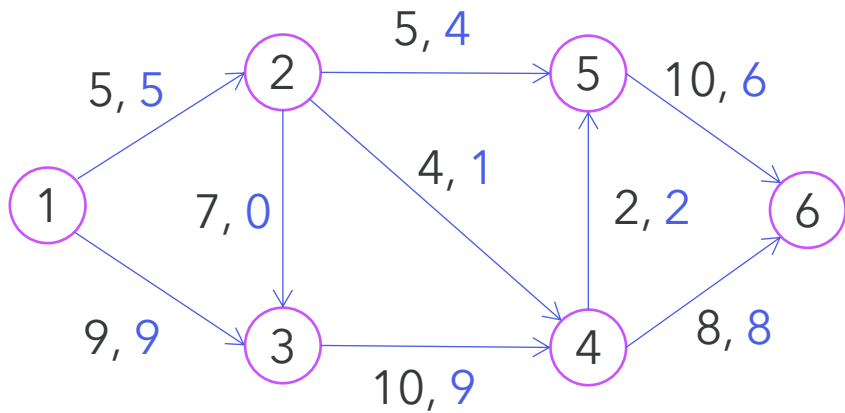
Lemma:

Per ogni cammino aumentante da s a t in G , esiste **uno e un solo cammino orientato** da s a t in G_x (grafo residuo).

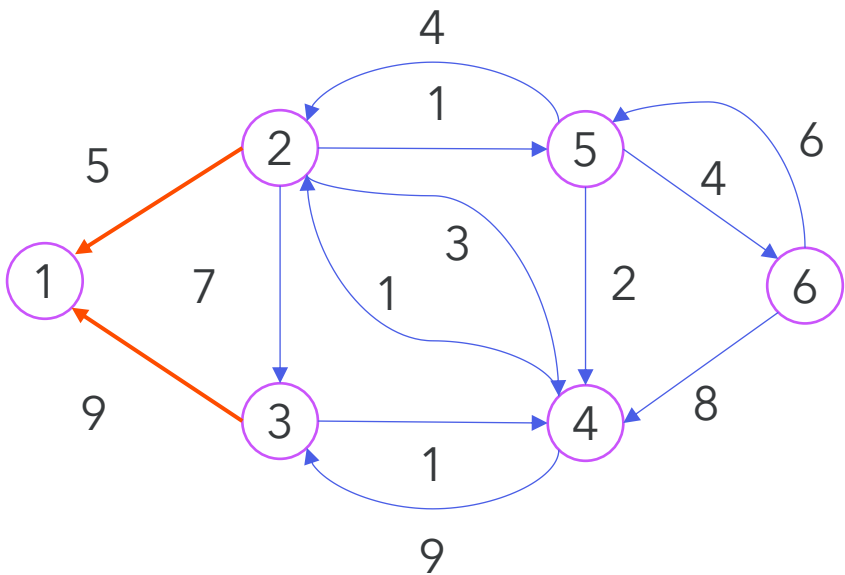
Conseguenza pratica:

- Un cammino aumentante può essere trovato tramite una **visita di G_x**
- Se la visita raggiunge t , esiste cammino aumentante e il flusso **non è ottimo**
- Se la visita **non raggiunge t , non esistono** cammini aumentanti e il flusso è **massimo**





Dato un grafo, costruiamo il grafo residuo corrispondente:



Il grafo residuo ci permette:

- di trovare facilmente cammini aumentanti e ci indica come aumentare il flusso
- di dire se abbiamo trovato la soluzione ottima, in caso non troviamo cammini aumentanti.

Allora possiamo immaginarci un **algoritmo**:

- che parta con una **soluzione ammissibile** (tutti i flussi a zero)
- e poi iterativamente cerchi un **cammino aumentante (sul grafo residuo)**.

Se lo trova si aumenta il flusso, se no trova la: **soluzione ottima**.

Nell'esempio ho ottenuto la soluzione ottima perché non posso più fare cammini aumentanti perché non posso più portare flusso. Gli archi (1,2) e (1,3) sono saturi. Questo si chiama **taglio**.

TAGLI NEL GRAFO

Un **taglio** (N_s, N_t) di G è una partizione dei nodi tale che:

$$\begin{aligned} s &\in N_s \\ t &\in N_t = N \setminus N_s \end{aligned}$$

Si definiscono:

Archi diretti del taglio:

$$A^+(N_s, N_t) = \{(i, j) \in A : i \in N_s, j \in N_t\}$$

Archi inversi del taglio:

$$A^-(N_s, N_t) = \{(i, j) \in A : i \in N_t, j \in N_s\}$$

FLUSSO E CAPACITÀ DI UN TAGLIO

Per un flusso x , il **flusso che attraversa il taglio**:

$$x(N_s, N_t) = \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}$$

Flusso che
attraversa il
taglio

Flusso che
torna indietro

Capacità massima del taglio:

$$\mu(N_s, N_t) = \sum_{(i,j) \in A^+(N_s, N_t)} \mu_{ij}$$

Massimo flusso che
può andare da una
parte all'altra del taglio

TEOREMA FONDAMENTALE

Teorema:

Per ogni flusso ammissibile x di valore v e ogni taglio (N_s, N_t) :

$$v = x(N_s, N_t) \leq \mu(N_s, N_t)$$

Il flusso non può mai superare la capacità di un taglio che separa s da t .

Il valore di qualsiasi flusso ammissibile è limitato dalla capacità di tutti i tagli del grafo.

Perciò il taglio con capacità minima limita il flusso di valore massimo.
Per l'esattezza hanno lo stesso valore.

TEOREMA FONDAMENTALE

Dimostrazione:

Il teorema dice: 1) la capacità di un taglio è minore o uguale al flusso di un taglio

$$x(N_s, N_t) \leq \mu(N_s, N_t)$$

Ma un taglio vale:

$$x(N_s, N_t) = \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} - \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij} \leq \mu(N_s, N_t)$$

$$\text{Ma } \sum_{(i,j) \in A^+(N_s, N_t)} x_{ij} \leq \sum_{(i,j) \in A^+(N_s, N_t)} \mu_{ij} = \mu(N_s, N_t),$$

$$\text{Inoltre tolgo una quantità } \sum_{(i,j) \in A^-(N_s, N_t)} x_{ij}$$

Vero

TEOREMA FONDAMENTALE

Dimostrazione:

Il teorema dice anche: 2) $v = x(N_s, N_t)$

Per i vincoli di conservazione del flusso

$$\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} = \begin{cases} -v & i = s \\ v & i = t \\ 0 & \text{altrimenti} \end{cases}$$

Per un flusso ammissibile sono tutte rispettate: adesso prendo solo quelle dei nodi di N_t e le sommo membro a membro:

$$\sum_{i \in N_t} \left(\sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} \right) = \sum_{i \in N_t} \begin{pmatrix} 0 & i \neq t \\ v & i = t \end{pmatrix} = v$$

TEOREMA FONDAMENTALE

Dimostrazione:

Siccome facciamo la somma su tutti gli N_t , gli archi che tra nodi di N_t una volta si sommano e una si sottraggono, perciò nella sommatoria rimangono solo gli archi del taglio:

$$x(N_s, N_t) = v$$

TEOREMA (FLUSSO MASSIMO – TAGLIO MINIMO) E OTTIMALITÀ

Se, nel grafo residuo G_x , t **non è raggiungibile** da s :

- I nodi visitati formano N_s
- Tutti gli archi $(i, j) \in A^+(N_s, N_t)$ sono **saturo**
- Tutti gli archi $(i, j) \in A^-(N_s, N_t)$ sono **vuoti**

Allora:

$$v = x(N_s, N_t) = \mu(N_s, N_t)$$

Il flusso v è **massimo** e

il taglio (N_s, N_t) è **di capacità minima**

Teorema (Flusso massimo - Taglio minimo):

Il valore massimo dei flussi ammissibili in un grafo è uguale alla minima capacità dei tagli che separano s da t .

TEOREMA (FLUSSO MASSIMO – TAGLIO MINIMO) E OTTIMALITÀ

Quando ottengo un flusso ottimo $v = x(N_s, N_t)$, non riesco a costruire un cammino aumentante sul grafo residuo G_x e definisco naturalmente un **taglio di capacità minima**:

- N_s sono i nodi che riesco a raggiungere da s .
- N_t sono i nodi che non riesco a raggiungere da s .

Gli archi che attraversano il taglio:

- Gli archi concordi A^+ sono saturi.
- Gli archi discordi A^- sono vuoti.

Problema di interdizione ottima:

Dato un grafo, tra tutti i tagli che separano s da t , voglio trovare quello di capacità minima.

$$\min\{\mu(N_s, N_t) : N_s \cup N_t = N, N_s \cap N_t = \emptyset, s \in N_s, t \in N_t\}$$

Risolvendo il problema di **flusso massimo** risolviamo anche il problema del **taglio di capacità minima**.

Sono problemi duali!

PROCEDURA GENERALE

Algoritmo Cammini-Aumentanti:

procedure *Cammini-Aumentanti*(G, s, t):

$x = 0$

while True:

$(p, \theta) = \text{Trova-Cammino}(G, s, t, x)$

if $p[t] == 0$:

break

$x = \text{Aumenta-Flusso}(x, p, s, t, \theta)$

Trova-Cammino: cerca un cammino aumentante e la sua capacità θ .

Aumenta-Flusso: invia θ lungo il cammino

Dato un grafo orientato e con capacità $G = (N, A)$, una sorgente s e un pozzo t , cerco il flusso massimo da s a t che rispetti le capacità

Parto con un flusso nullo su tutti gli archi: $x = 0$.

Finché possibile, cerco un cammino aumentante da s a t sul grafo residuo $G_x = (N, A_x)$.

Trova-cammino:

- costruisce G_x a partire dal grafo G e dal flusso x .
- Dà in output la capacità del cammino θ e un vettore dei predecessori p utile a ricostruire il cammino.

Se il predecessore del pozzo t rimane al valore di inizializzazione:

- non ho raggiunto t con un cammino aumentante,
- ho trovato il flusso ottimo x e termino l'algoritmo.

Altrimenti aggiorno il flusso x con il cammino aumentante trovato (*Aumenta-Flusso*).

PROCEDURA GENERALE

Trova-Cammino

Funziona sul **grafo residuo** G_x .

Memorizza per ogni nodo j :

Predecessore $p[j]$

Capacità residua $d(j)$ del cammino da s a j

Aggiornamento di $d(j)$:

Se si usa arco (i,j) : $d(j) = \min\{d(i), \mu(i,j) - x(i,j)\}$

Se si usa arco (j,i) : $d(j) = \min\{d(i), x(j,i)\}$

Inizializzazione: $d(s) = +\infty$

Aumenta-Flusso

Si percorre il cammino **inverso** da t a s tramite $p[\cdot]$.

Si incrementa il flusso x di θ lungo gli archi del cammino.

Operazione di composizione: $x = x \oplus \theta P$.

TEOREMA DI TERMINAZIONE

Teorema

Se tutte le capacità sono **interi**, l'algoritmo termina in **numero finito di iterazioni** e produce un **flusso massimo intero**.

Motivazione:

- I flussi iniziali e incrementi sono interi: ad ogni passo *aumenta il flusso di almeno 1 unità*.
- Il valore massimo del flusso è finito, *al massimo* si fanno $U = \max\{\mu_{ij} : (i, j) \in A\}$ iterazioni: perciò la terminazione è garantita.

Se le capacità **non sono intere**:

- L'algoritmo **potrebbe non terminare**
- Il flusso calcolato può **convergere a un valore inferiore al massimo**

In pratica, computer digitali usano numeri **interi o razionali**, quindi il problema è raro.

COMPLESSITÀ

Allora il massimo numero di iterazioni è

= numero di nodi

U = capacità massima sugli archi

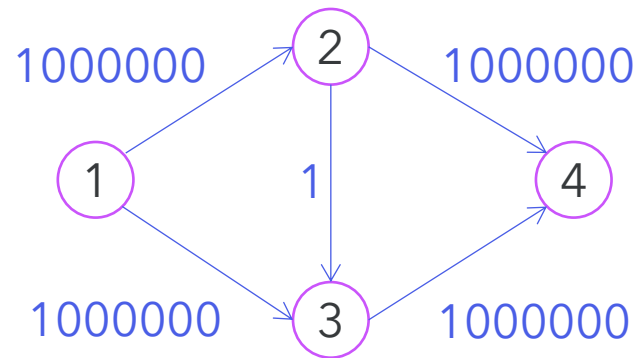
Ogni iterazione costa una visita del grafo: $O(m)$

Complessità: $O(mnU)$

Tale complessità è **pseudopolinomiale**, perché dipende dai valori numerici delle capacità (U).

COMPLESSITÀ

Esempio:



Potrei fare il cammino aumentante $(1,2), (2,3), (3,4)$ e poi fare il cammino aumentante $(1,3), (3,2), (2,4)$ e rifarli per 2 milioni di volte

Avendo una complessità pseudopolinomiale, questo algoritmo è veloce se le capacità degli archi sono piccole, ma può essere lento se sono grandi.

Il caso peggiore succede se scelgo male i cammini: questo dipende da come faccio le **visite del grafo**.

Possiamo fare le **visite a ventaglio** (*breadth first*) **usando una fila**, così possiamo dare una valutazione superiore polinomiale al numero di volte che si fa una visita.

ALGORITMO DI EDMONDS-KARP

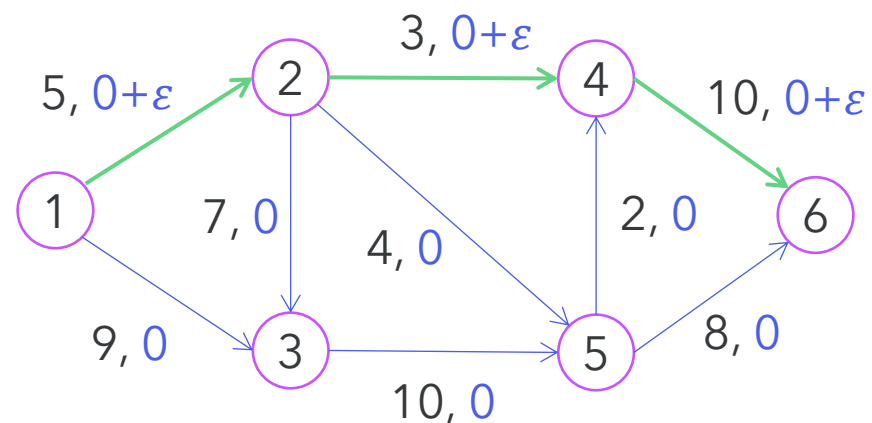
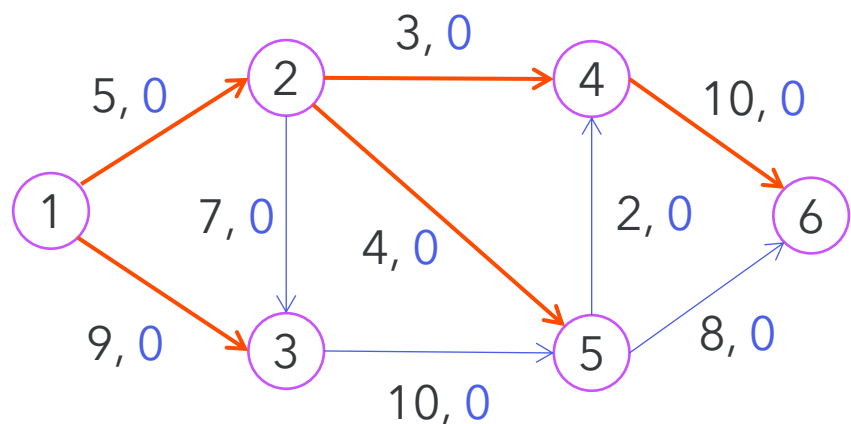
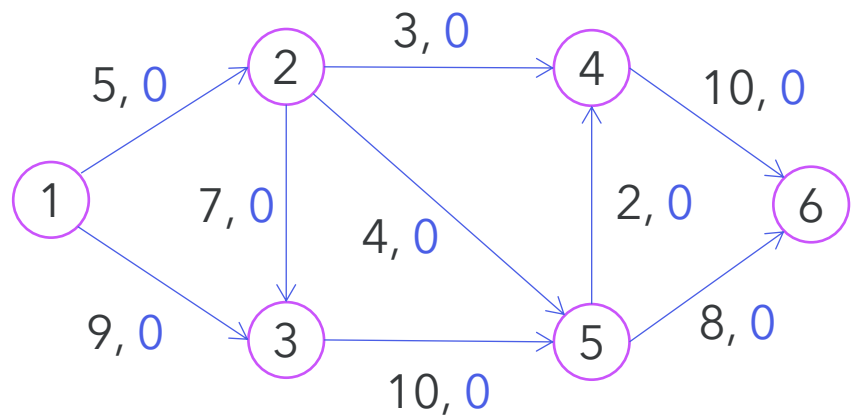
Variante dell'algoritmo dei cammini aumentanti.

Sceglie sempre il **cammino aumentante più corto** (minimo numero di archi): *attraverso delle visite a ventaglio e usando una fila.*

Effetto:

- Iterazioni iniziali usano cammini brevi

- Iterazioni successive cammini più lunghi



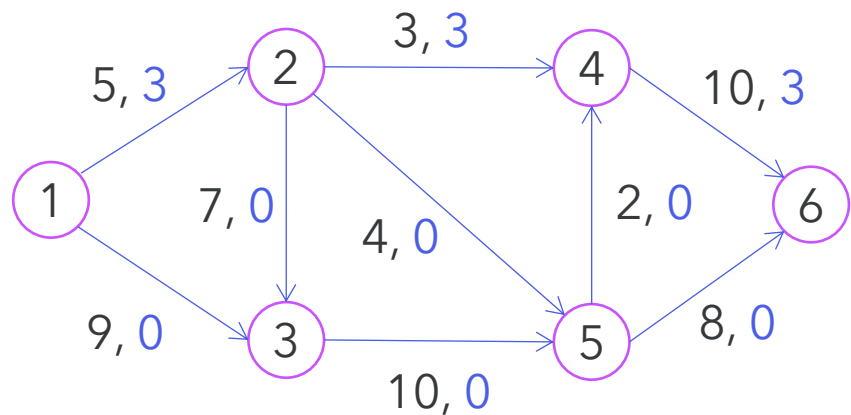
Dato un grafo $G = (N, A)$, $N = \{1, 2, 3, 4, 5, 6\}$, $A = \{(1, 2), (1, 3), (2, 3), (2, 4), (2, 5), (3, 5), (4, 6), (5, 4), (5, 6)\}$, con capacità $\mu = (5, 9, 7, 5, 4, 10, 10, 2, 8)$, cerchiamo il flusso massimo tra la sorgente $s = 1$ e la destinazione $t = 6$.

Partiamo da un **flusso ammissibile**: tutto il flusso x è a 0. E' ammissibile perché rispetta le capacità e i vincoli di bilanciamento del flusso. Il flusso ha valore $v = 0$.

Non è il flusso ottimo: scriviamo il **grafo residuo** G_x , che è identico al grafo G se il flusso è nullo, e esploriamolo *breadth first* a partire da s . Troviamo il **cammino aumentante** $(s, 2), (2, 4), (4, t)$.

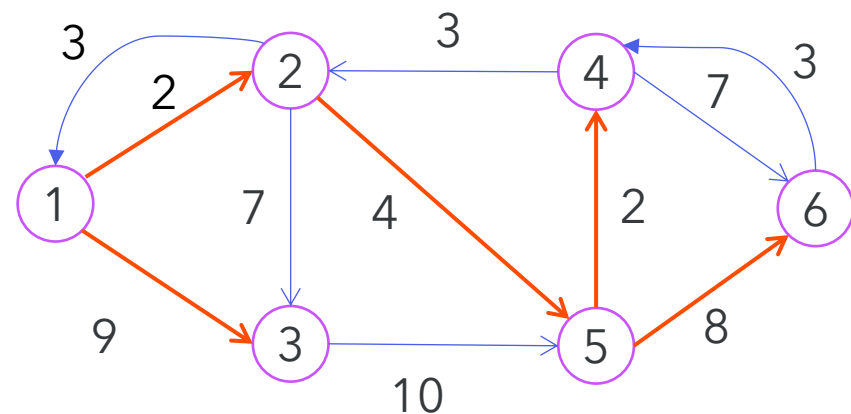
Quanto possiamo portare? La capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(2, 4)$ con capacità 3. Allora aggiungo un flusso $\varepsilon = 3$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 3$.



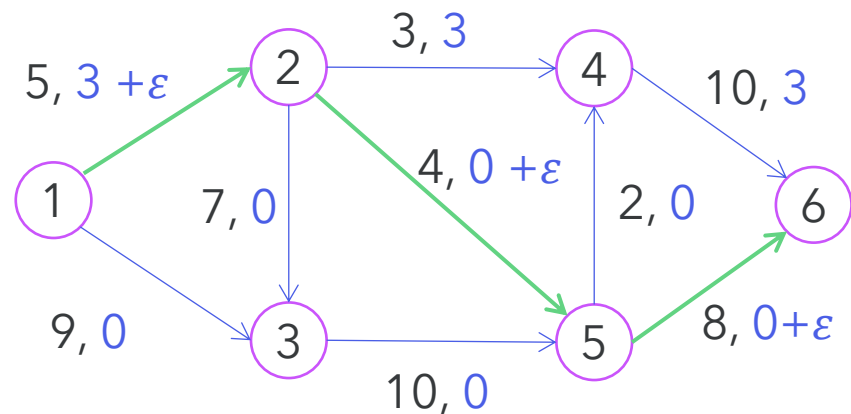
Abbiamo un flusso con valore $v = 3$. Possiamo migliorare?

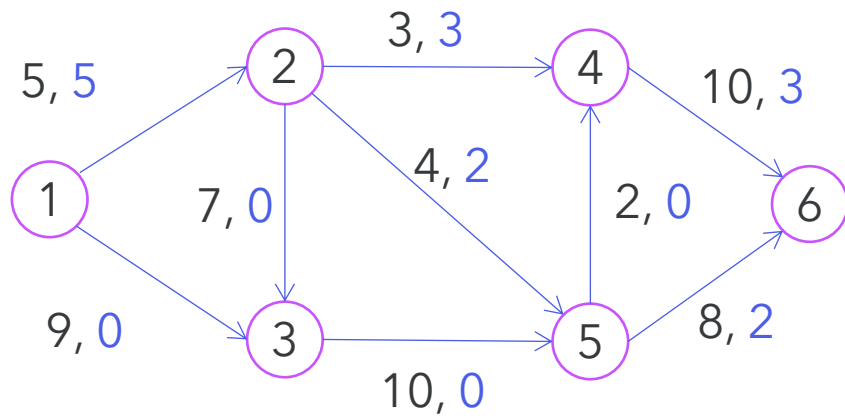
Scriviamo il **grafo residuo** e lo esploriamo *breadth first*. Troviamo il **cammino aumentante** $(s,2), (2,5), (5,t)$.



Al massimo possiamo portare la capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(1,2)$ con capacità 8. Allora aggiungo un flusso $\varepsilon = 2$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 3 + 2 = 5$.



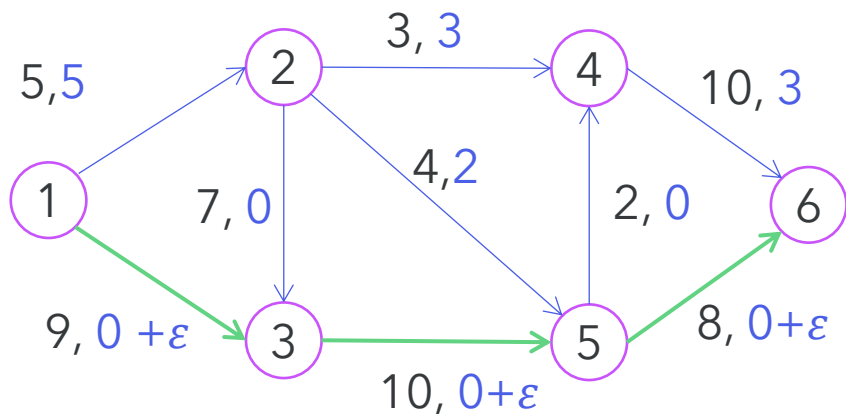
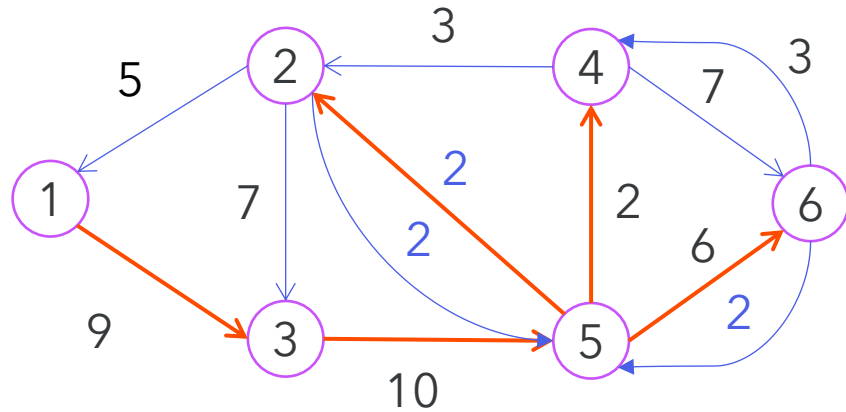


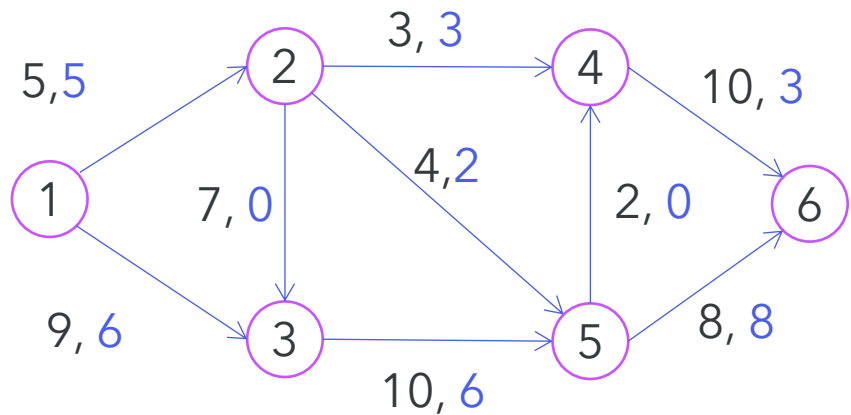
Abbiamo un flusso con valore $v = 5$. Possiamo migliorare?

Scriviamo il **grafo residuo** e lo esploriamo *breadth first*. Troviamo il **cammino aumentante** $(s,3), (3,5), (5,t)$.

Al massimo possiamo portare la capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(5,t)$ con capacità 6. Allora aggiungo un flusso $\varepsilon = 6$ a quello esistente su tutti gli archi del cammino.

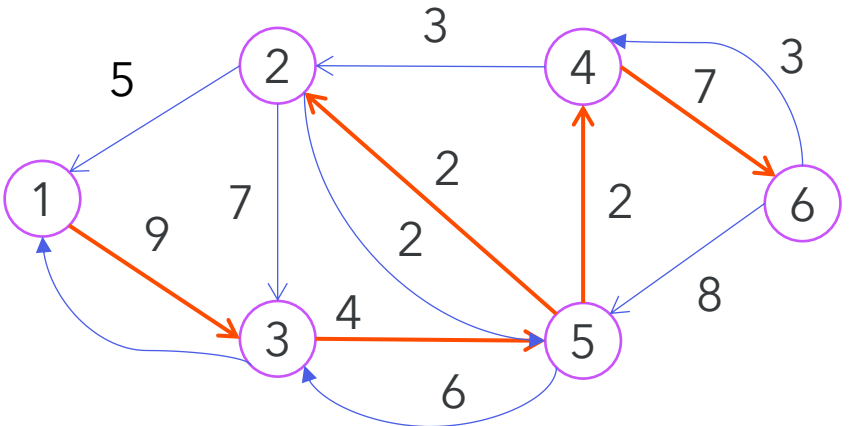
Abbiamo un flusso con valore $v = 5 + 6 = 11$.





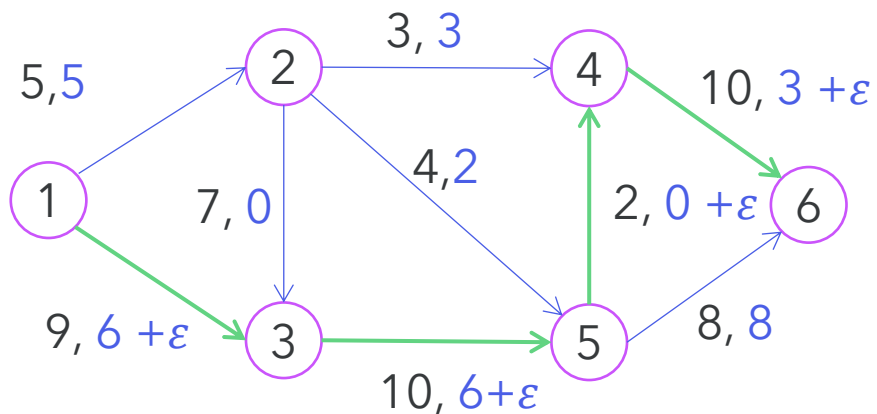
Abbiamo un flusso con valore $v = 11$. Possiamo migliorare?

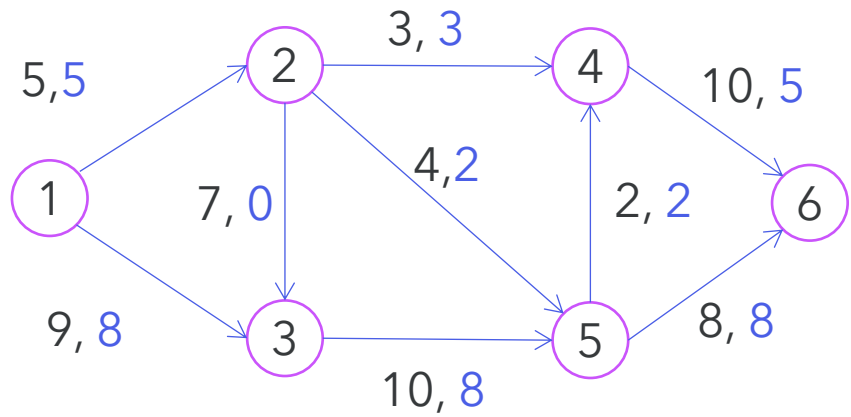
Scriviamo il **grafo residuo** e lo esploriamo *breadth first*. Troviamo il **cammino aumentante** $(s,3), (3,5), (5,4), (4,t)$.



Al massimo possiamo portare la capacità minore tra gli archi del cammino. Il **collo di bottiglia** è $(5,4)$ con capacità 2. Allora aggiungo un flusso $\varepsilon = 2$ a quello esistente su tutti gli archi del cammino.

Abbiamo un flusso con valore $v = 11 + 2 = 13$.



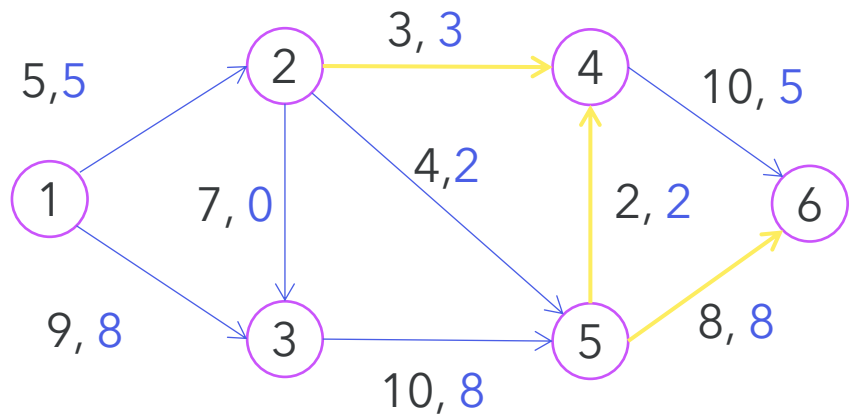
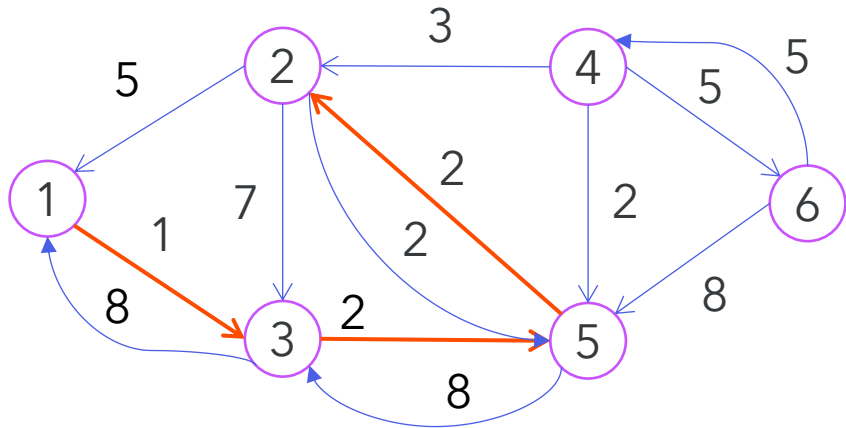


Abbiamo un flusso con valore $v = 11$. Possiamo migliorare?

Scriviamo il **grafo residuo** e lo esploriamo *breadth first*. Non esiste cammino aumentante da s a t .

Il flusso ottimo x vale $v = 13$.

Otteniamo anche il taglio di capacità minima che è $\{(2,4), (5,4), (5,6)\}$.



EDMONDS & KARP: COMPLESSITÀ

L'algoritmo di Edmonds & Karp permette di trovare **cammini aumentanti di lunghezza crescente** durante le iterazioni.

- Ogni arco m può cambiare direzione al più n volte.
- Le visite costano $O(m)$.



Edmonds & Karp ha complessità di **$O(m^2 n)$**

Vantaggi:

- Valido anche se le **capacità non sono intere**
- Garantisce terminazione **polinomiale**

Se un arco diventa saturo, poi gli possiamo cambiare direzione, ma con un cammino di due archi più lungo. Il numero massimo di archi in un cammino semplice è $n-1$: **$O(nm)$** .

Però su grafi densi: $O(n^5)$. Ci sono algoritmi che ottengono $O(n^3)$.

FLUSSO MASSIMO CON PIÙ SORGENTI / POZZI

Generalizzazione del problema del flusso massimo

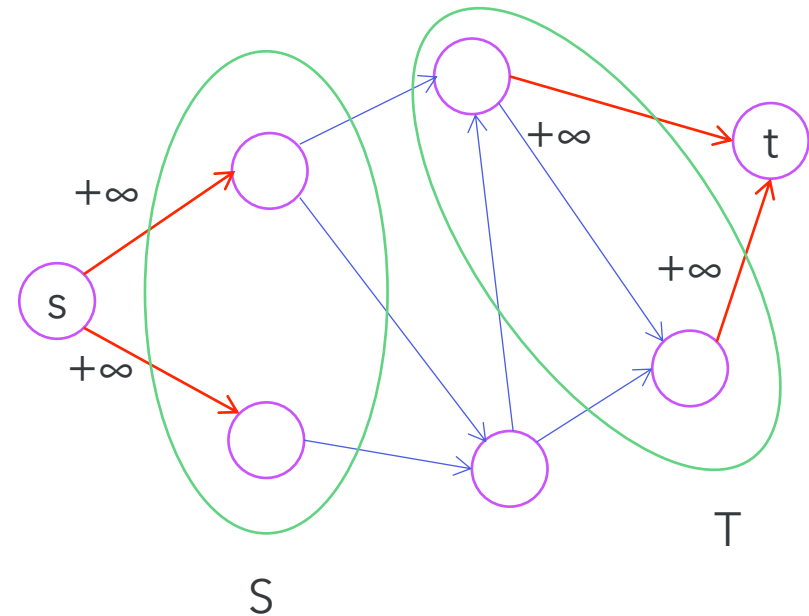
S = nodi sorgente

T = nodi pozzo

Obiettivo: massimizzare il flusso da **tutte le sorgenti a tutti i pozzi**.

Metodo classico: costruire un **grafo ampliato G'** con:

- *Super-sorgente s* collegata a tutte le sorgenti (archi a capacità $+\infty$ o $\max\{\mu_{ij}: (i, j) \in A\}$)
- *Super-pozzo t* collegato a tutti i pozzi (archi a capacità $+\infty$ o $\max\{\mu_{ij}: (i, j) \in A\}$)



CAMMINI AUMENTANTI CON SORGENTI MULTIPLE

È possibile **evitare di costruire G' esplicitamente.**

Modifica dell'algoritmo Cammini-Aumentanti:

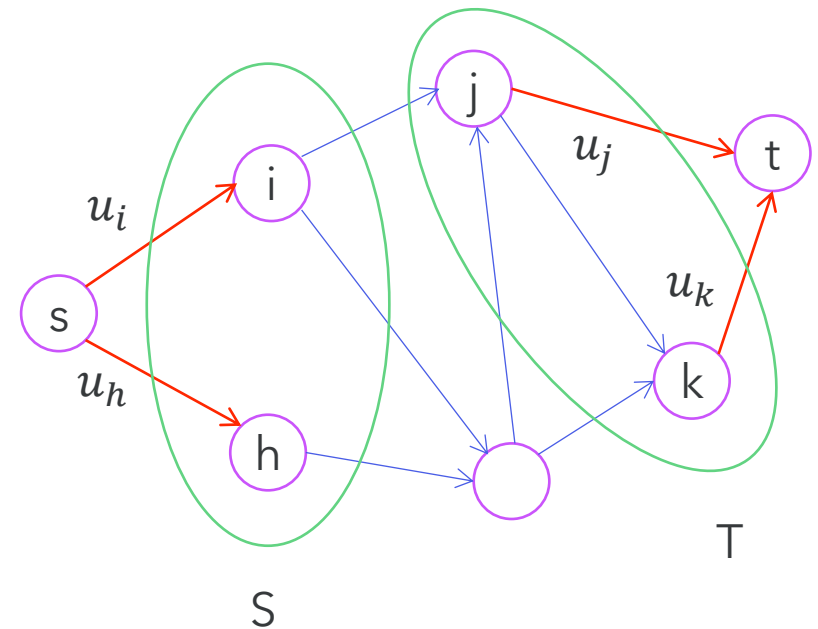
- Inizializziamo $Q = S$ (insieme di tutte le sorgenti)
Cioè possiamo mettere in coda tutti gli s .
- Appena si raggiunge un **qualsiasi nodo in T : cammino**
- Quando Q è vuoto: ho trovato un taglio saturato tra S e T

CAPACITÀ FINITE SULLE SORGENTI / POZZI

Sorgenti $i \in S$ e pozzi $j \in T$ possono avere **capacità finite** u_i

Implementazione:

- *Super-sorgente* s collegata a tutte le sorgenti
- *Super-pozzo* t collegato a tutti i pozzi
- Archi (s, i) con capacità u_i
- Archi (j, t) con capacità u_j





PROBLEMA DI FLUSSO DI COSTO MINIMO

DA PAG 79 DISPENSE

PROBLEMA DI FLUSSO DI COSTO MINIMO (MCF)

Sia dato un **grafo orientato** $G = (N, A)$ dove:

N è l'insieme dei nodi, $n = |N|$,

A è l'insieme degli archi, $m = |A|$.

Ogni **arco** $(i, j) \in A$ è caratterizzato da:

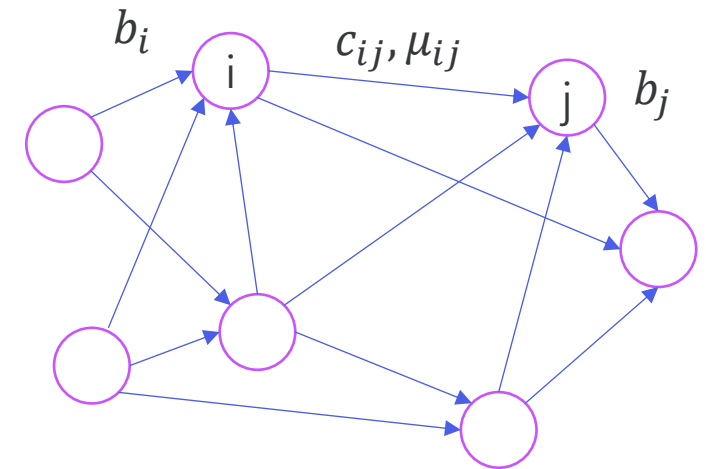
Costo per unità di flusso c_{ij} ,

Capacità massima μ_{ij} (quantità massima di flusso che può attraversarlo).

Ogni **nodo** $i \in N$ ha associata una **deficit** b_i :

$b_i < 0$: il nodo immette flusso nel grafo (sorgente o partenza),

$b_i > 0$: il nodo assorbe flusso dal grafo (pozzo o destinazione).



PROBLEMA DI FLUSSO DI COSTO MINIMO (MCF)

L'obiettivo è trovare un **flusso** x_{ij} **sugli archi** che:

Rispetti i **vincoli di conservazione del flusso** sui nodi:

$$\sum_{j:(j,i) \in A} x_{ji} - \sum_{j:(i,j) \in A} x_{ij} = b_i \quad i \in N$$

← Flusso

Rispetti i **vincoli di capacità**:

$$0 \leq x_{ij} \leq \mu_{ij} \quad (i,j) \in A$$

← Ammissibile

Minimizza il costo totale del flusso:

$$\min \sum_{(i,j) \in A} c_{ij} x_{ij}$$

← Di costo minimo

PROBLEMA DI FLUSSO DI COSTO MINIMO

Il problema del **flusso di costo minimo** è più *generale* di:

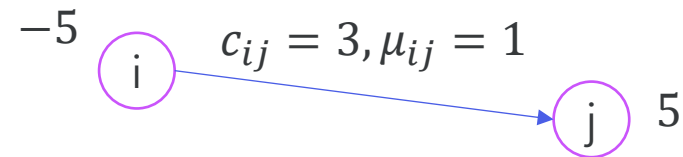
- **Albero dei cammini minimi** → costi ma non capacità
- **Flusso massimo** → capacità ma non costi

(MCF) combina entrambi gli aspetti: costi + capacità

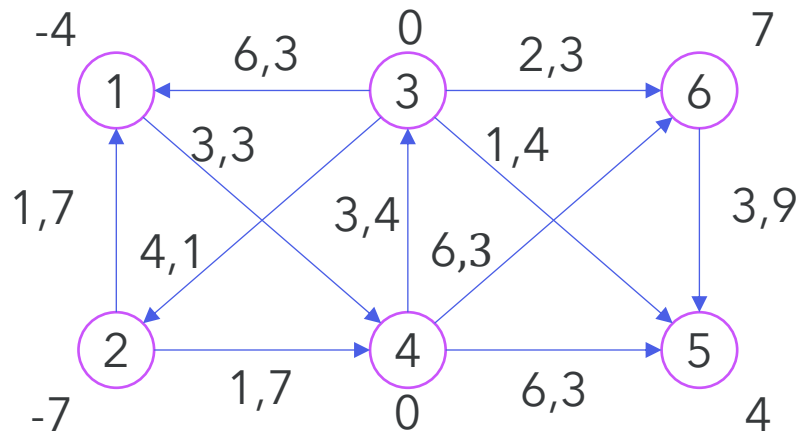
Gli algoritmi per MCF sfruttano metodi di SPT e MF per gestire gli aspetti di ottimalità e ammissibilità.

AMMISSIBILITÀ

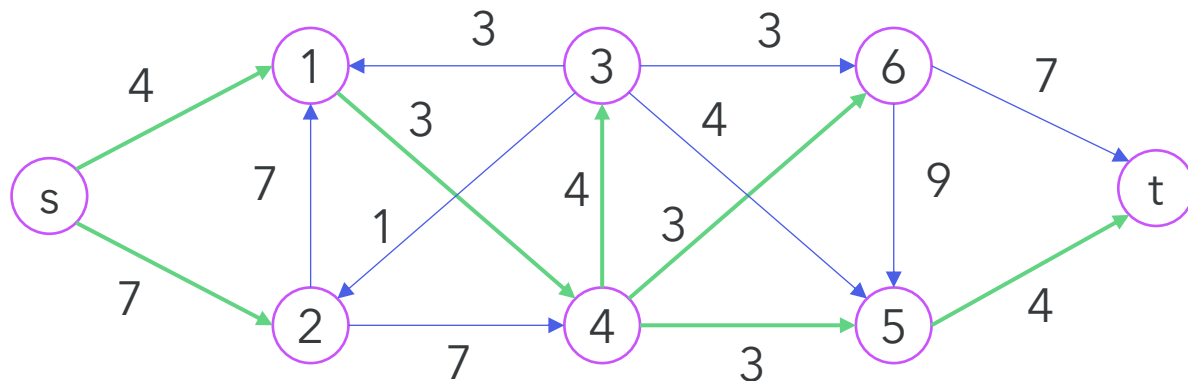
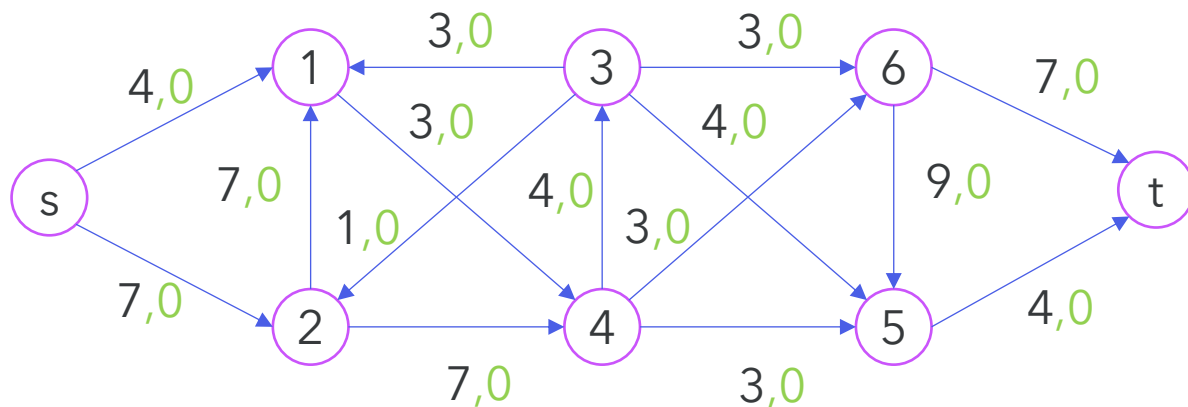
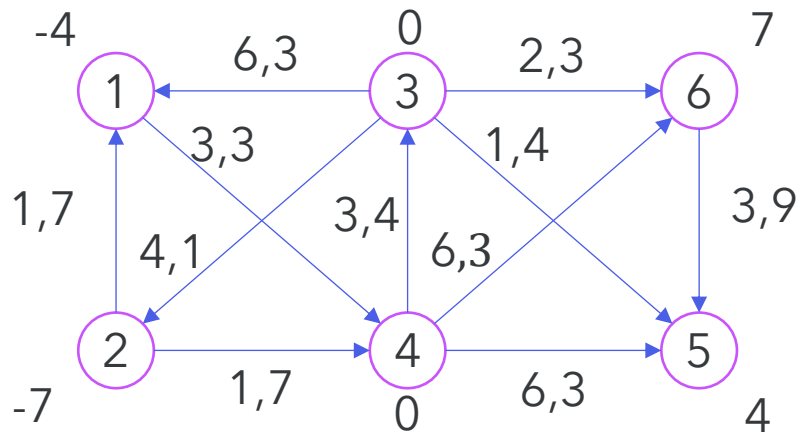
I flussi di costo minimo potrebbero non avere soluzione ammissibile:



Non è ovvio dire se esiste una soluzione ammissibile.



Come possiamo capire se esiste un flusso ammissibile?

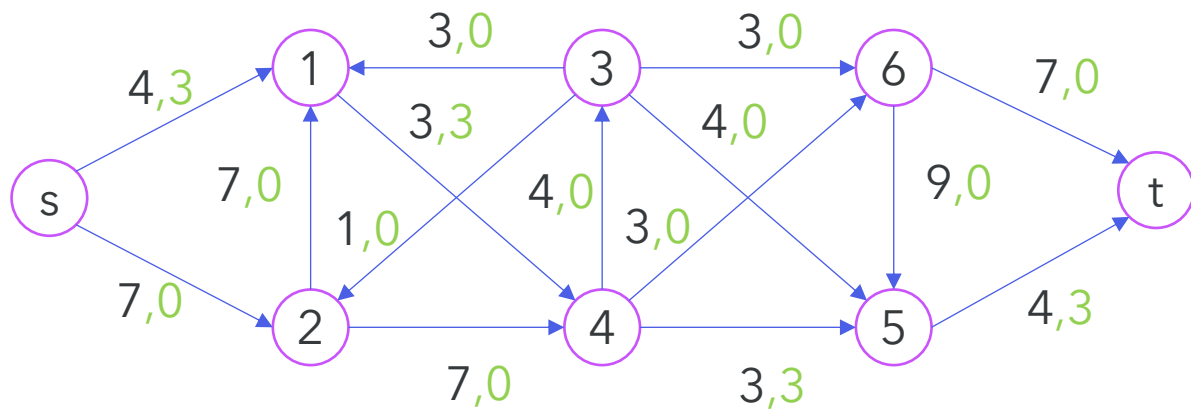


Risolviamo un problema di **flusso massimo su un grafo modificato**: se esiste un flusso allora esiste una soluzione ammissibile.

Modifichiamo il grafo:

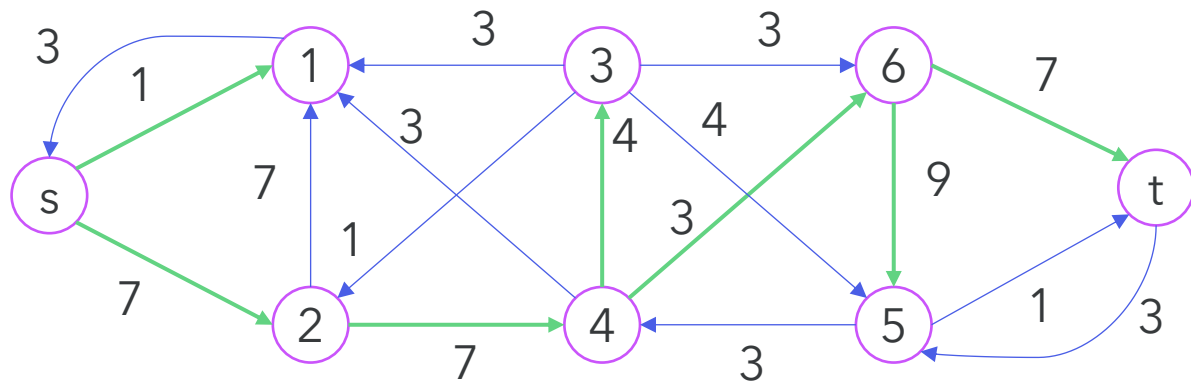
- Creando un nodo **supersorgente** e un nodo **superpozzo**.
- Colleghiamo la supersorgente alle sorgenti con *archi di capacità pari al deficit di ogni sorgente* (moltiplicato per -1).
- Colleghiamo i pozzi al superpozzo con *archi di capacità pari al deficit di ogni pozzo*.
- Non consideriamo i deficit dei nodi di transizione.
- Non consideriamo i costi, ma solo le **capacità**.

Dato flusso nullo, cerchiamo un cammino aumentante sul grafo, ovvero cerchiamo un cammino orientato sul grafo residuo, che nella prima iterazione coincidono. Troviamo $(s,1)$, $(1,4)$, $(4,3)$ $(3,t)$ con capacità: 3.



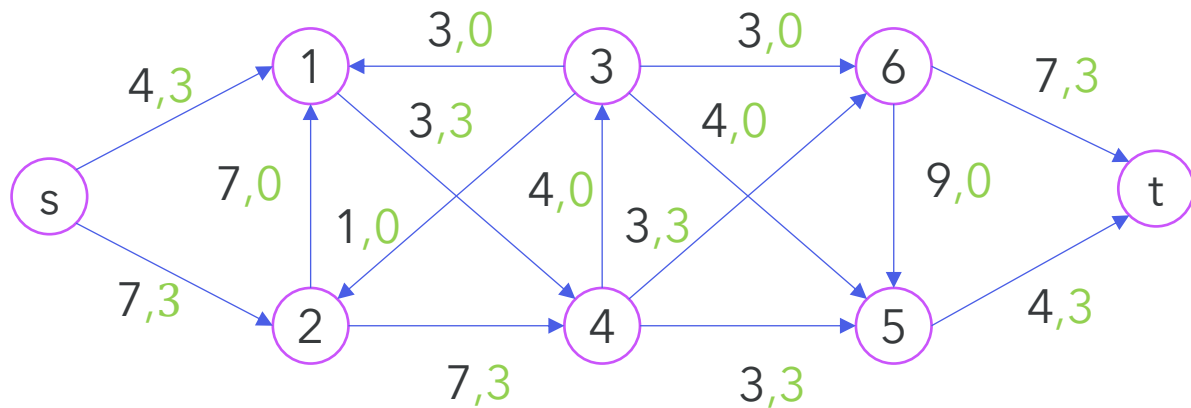
Aggiorniamo il grafo con il nuovo cammino aumentante: otteniamo un flusso $v = 3$.

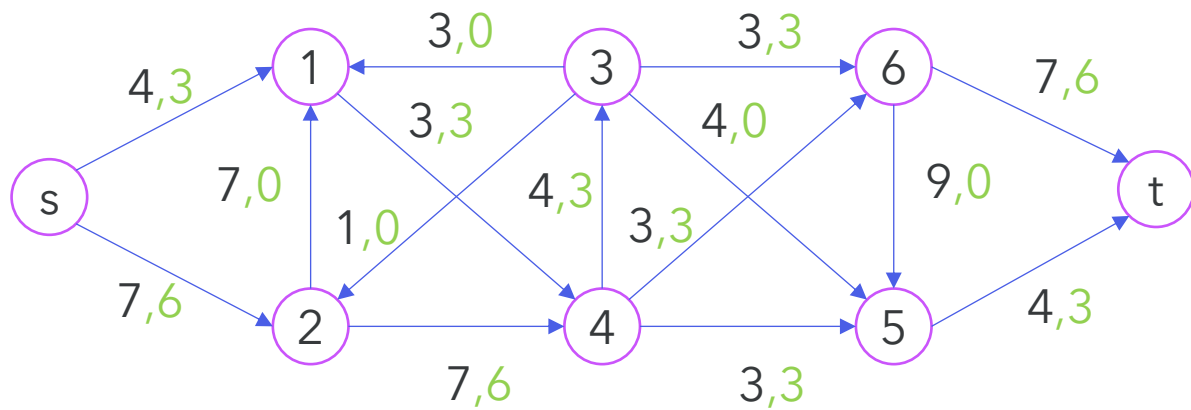
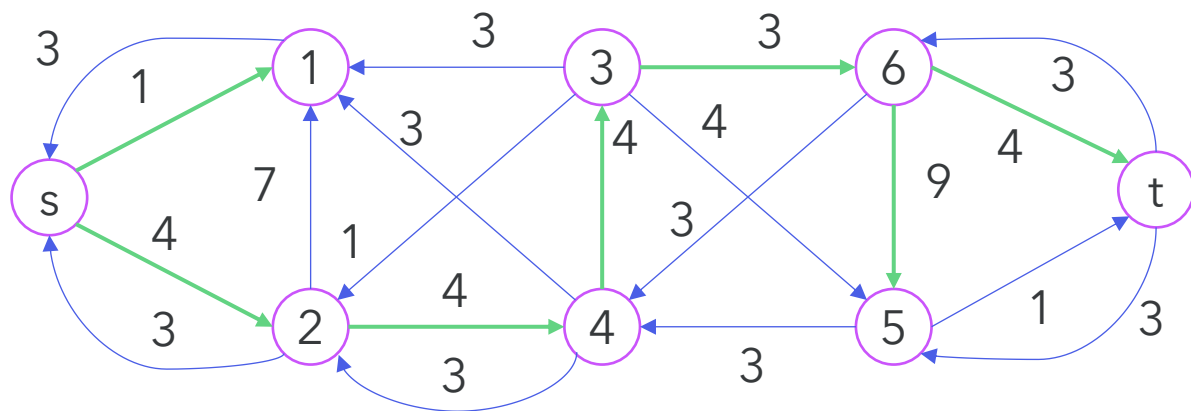
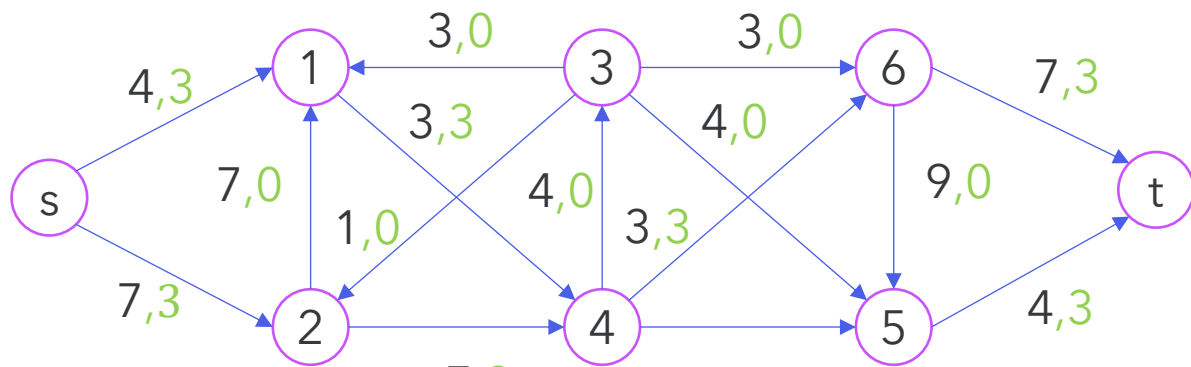
Scriviamo il grafo residuo corrispondente e cerchiamo un cammino orientato su tale grafo, che corrisponde ad un cammino aumentante sul grafo originale.



Trovo il cammino $(s,2), (2,4), (4,6), (6,t)$ con capacità 3.

Aggiorniamo il grafo con il nuovo cammino aumentante: otteniamo un flusso $v = 6$.

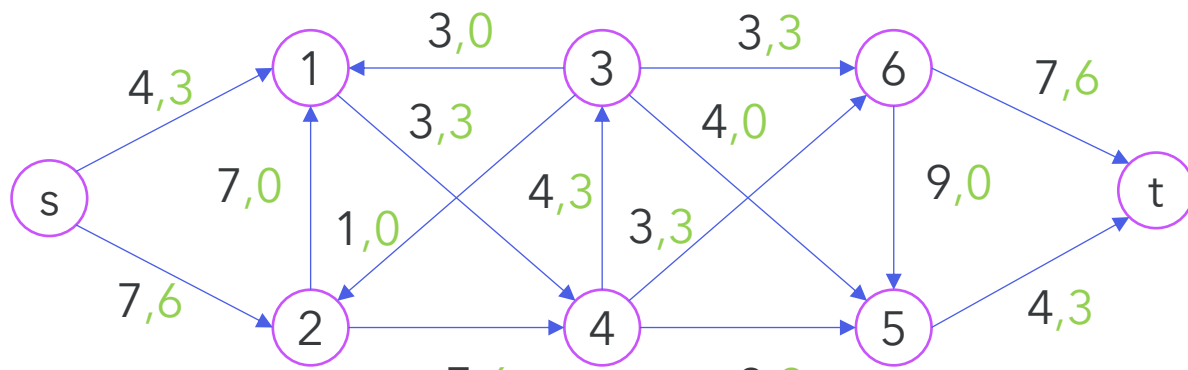




Scriviamo il grafo residuo corrispondente e cerchiamo un cammino orientato su tale grafo, che corrisponde ad un cammino aumentante sul grafo originale.

Trovo il cammino (s,2), (2,4), (4,3), (3,6), (6,t) con capacità 3.

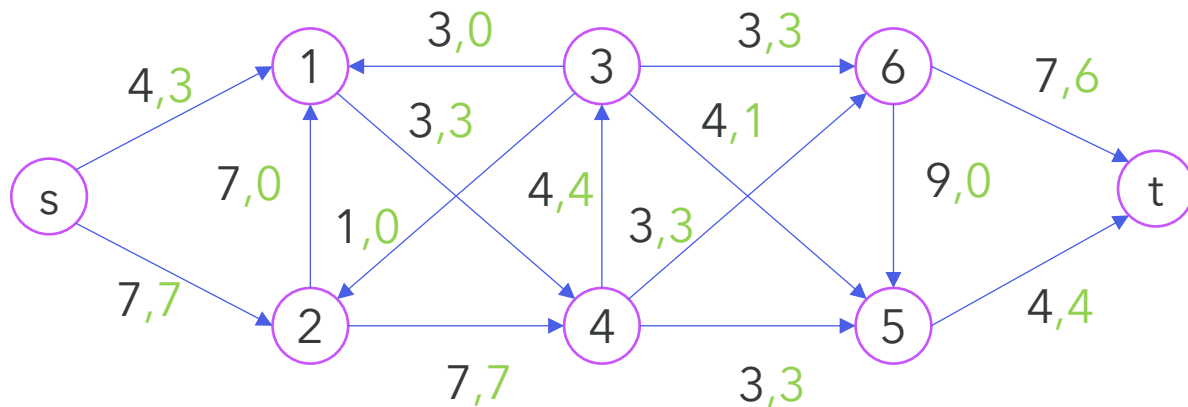
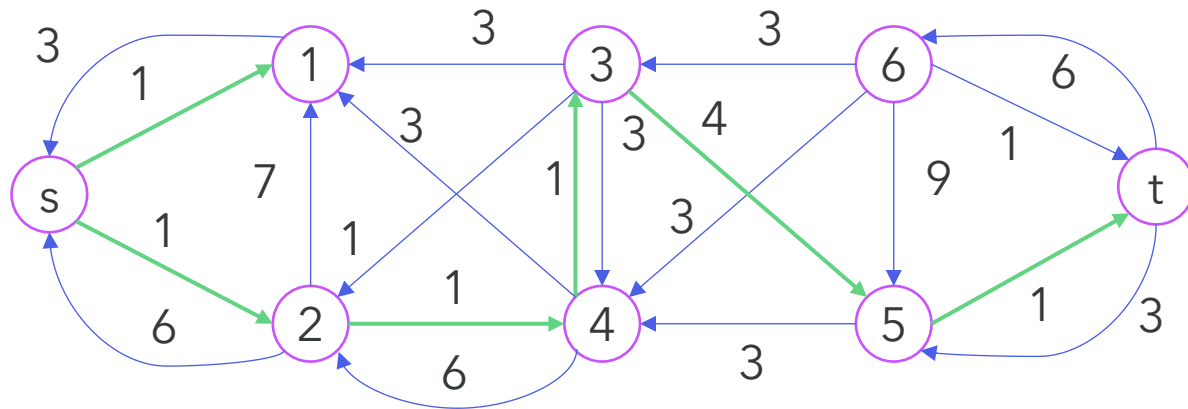
Aggiorniamo il grafo con il nuovo cammino aumentante: otteniamo un flusso $v = 9$.

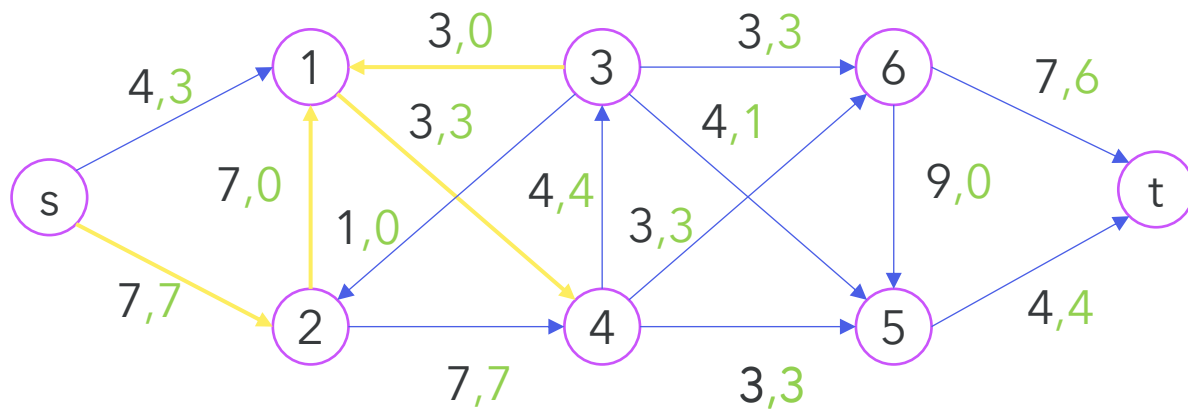
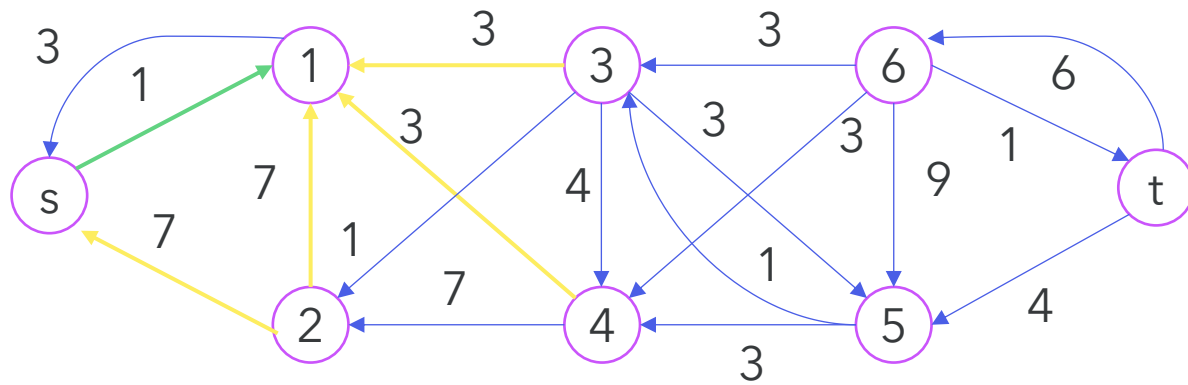
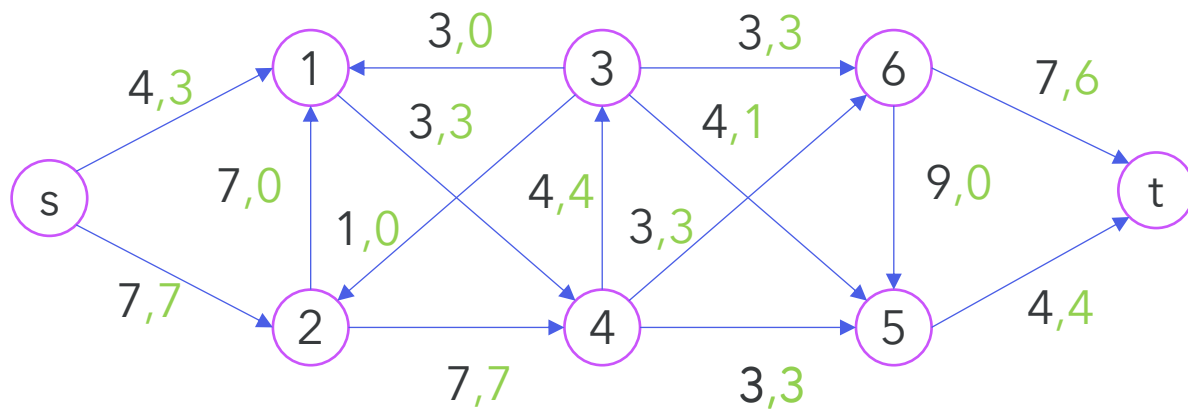


Scriviamo il grafo residuo corrispondente e cerchiamo un cammino orientato su tale grafo, che corrisponde ad un cammino aumentante sul grafo originale.

Trovo il cammino $(s, 2), (2, 4), (4, 3), (3, 5), (5, t)$ con capacità 1.

Aggiorniamo il grafo con il nuovo cammino aumentante: otteniamo un flusso $v = 10$.





Scriviamo il grafo residuo corrispondente e cerchiamo un cammino orientato su tale grafo, che corrisponde ad un cammino aumentante sul grafo originale.

Non troviamo un cammino. Troviamo un **taglio**, $N_s = \{s, 1\}$, con archi (3,1), (2,1) [vuoti], (1,4), (s,2) [saturi], con capacità pari a 10.

Aggiorniamo il grafo con il nuovo cammino aumentante: otteniamo un flusso $v = 10$.

Il problema di flusso di costo minimo non è ammissibile: $v < \sum_{i \in D} b_i$

Manca una unità di flusso per arrivare a 11. Avrei voluto trovare un taglio che includesse solo la supersorgente s.

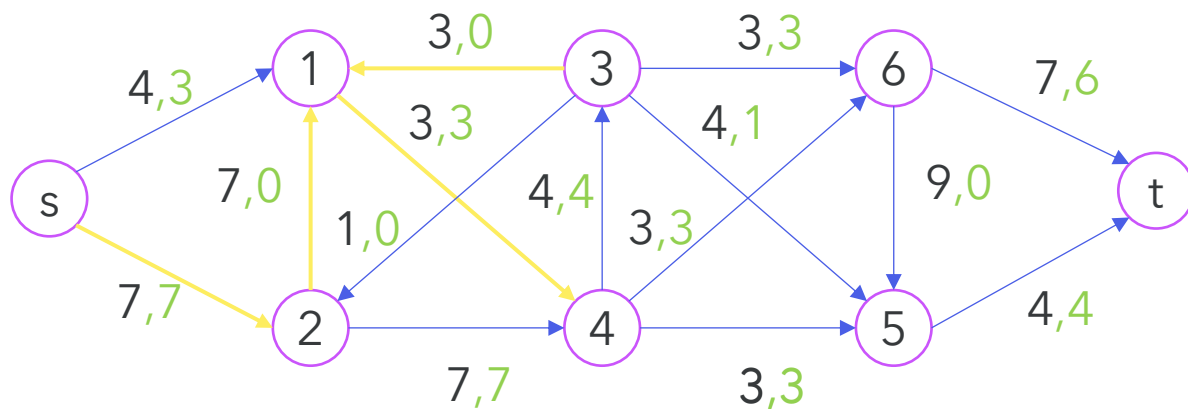
AMMISSIBILITÀ MCF COME PROBLEMA DI FLUSSO MASSIMO

Il **problema di ammissibilità** di un *problema di flusso di costo minimo* si riduce a un **problema di flusso massimo**:

- Super-sorgente s collegata a nodi con deficit negativo 0
- Super-pozzo t collegato a nodi con deficit positivo D
- Capacità archi: $|b_i|$ dei nodi (ricordiamo che dobbiamo avere $\sum_{i \in N} b_i = 0$)

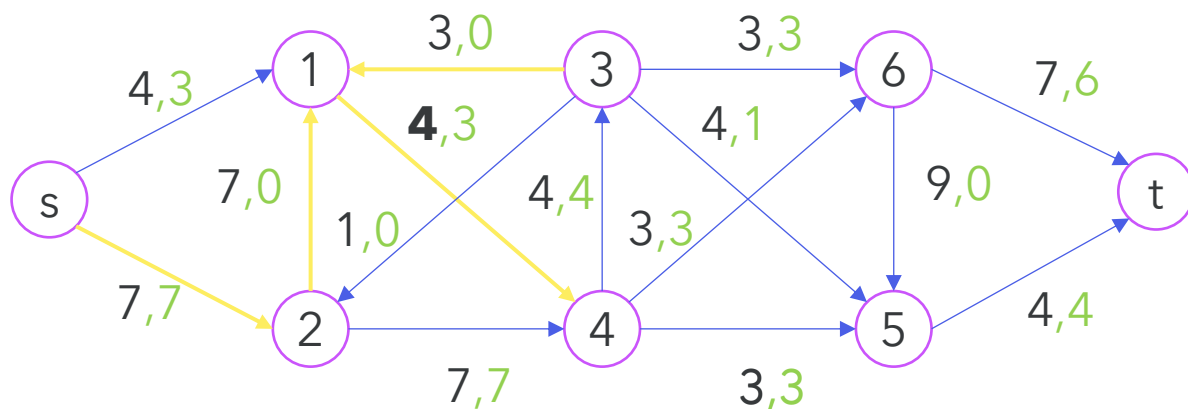
Se **esiste un flusso massimo** da s a t che **satura tutti gli archi uscenti da s e tutti quelli entranti in t** , allora **esiste un flusso ammissibile** per il problema di costo minimo.

Viceversa, se il **flusso massimo** ottenuto è **inferiore** a $\sum_{i \in D} b_i$, allora non esiste alcun flusso che possa soddisfare tutte le richieste: **il problema di flusso di costo minimo è inammissibile**.



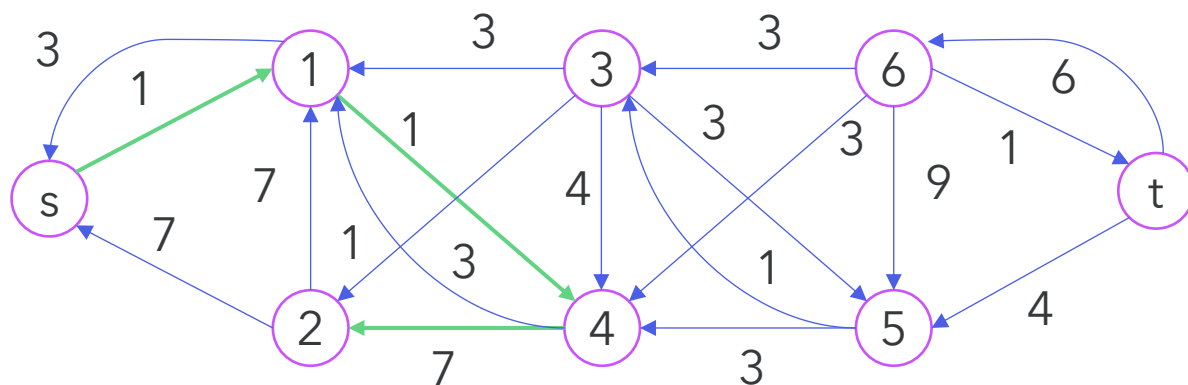
Se volessi mandare più flusso dovrei avere più capacità sugli archi. Su quali archi mi converrebbe «acquistare» più capacità?

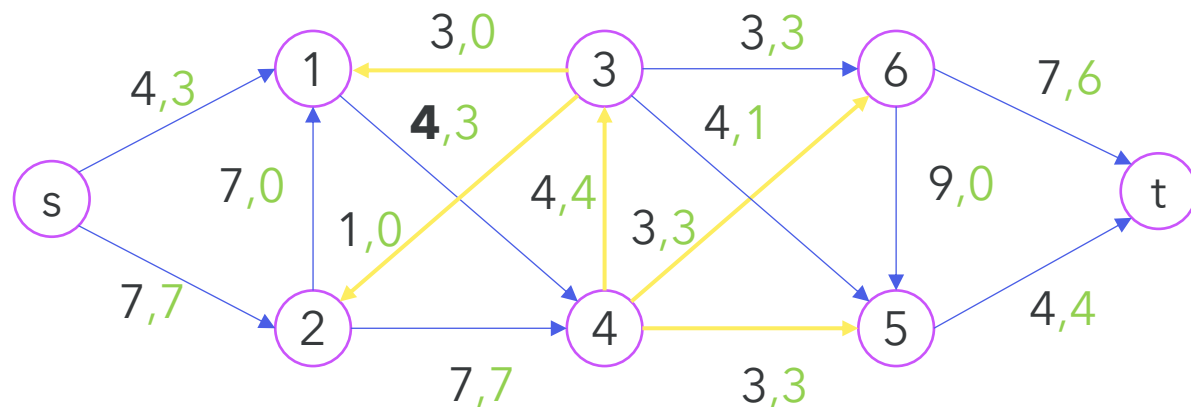
Converrebbe aumentare la capacità sugli archi del taglio di capacità minima.



Nel nostro caso dovrei aumentare la capacità dell'arco $(1,4)$ di una unità, perché $(s,2)$ non aiuterebbe a soddisfare la richiesta del nodo 1.

Allora scrivo il grafo residuo e cerco un cammino.



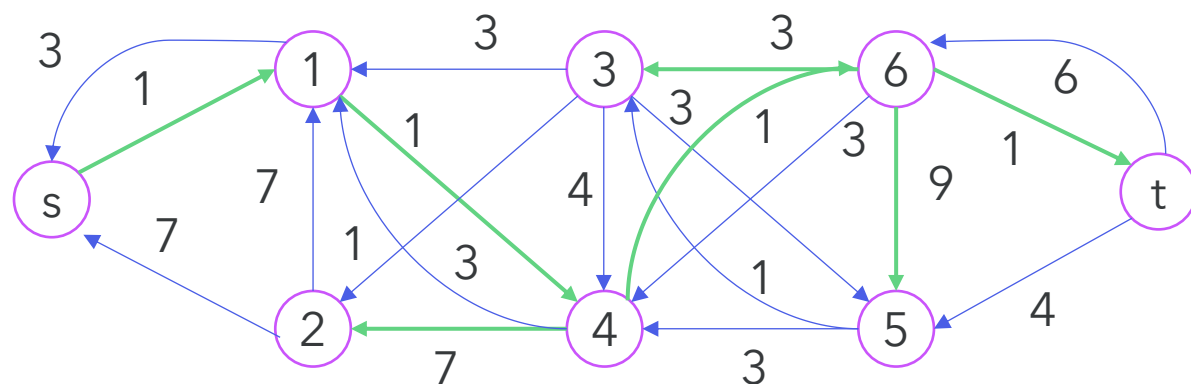
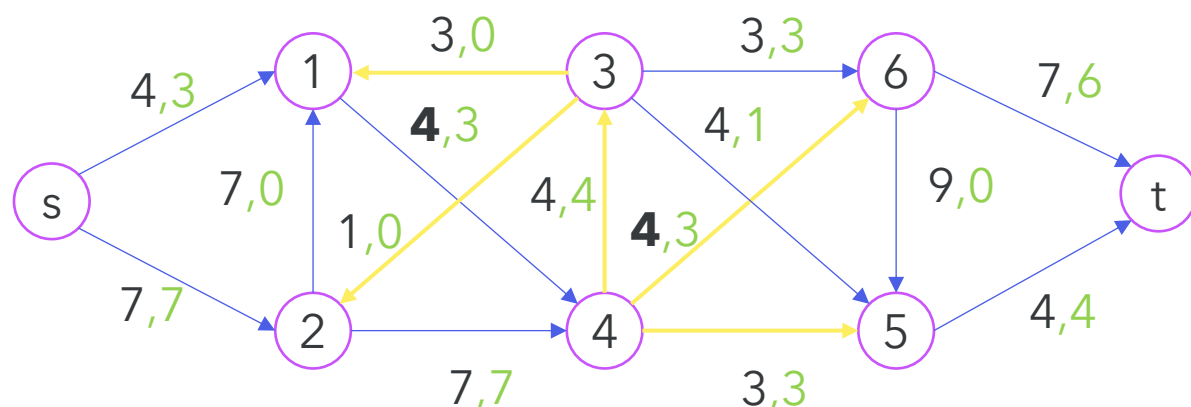


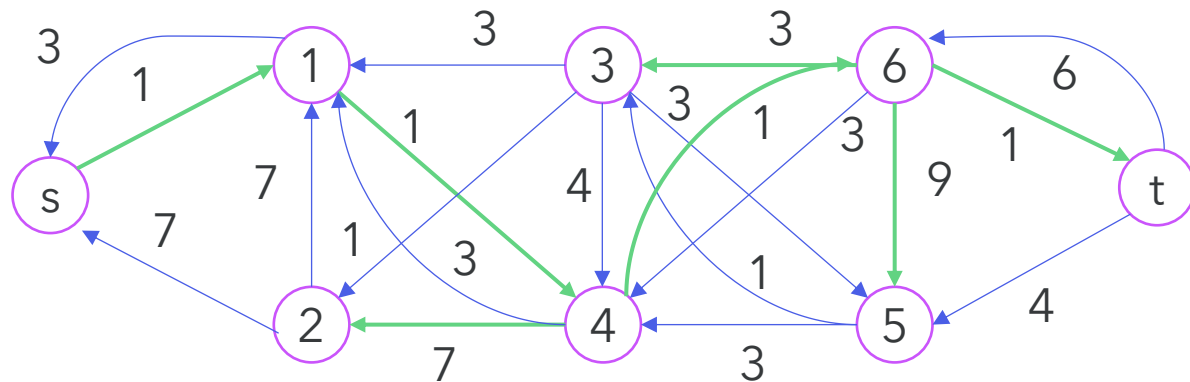
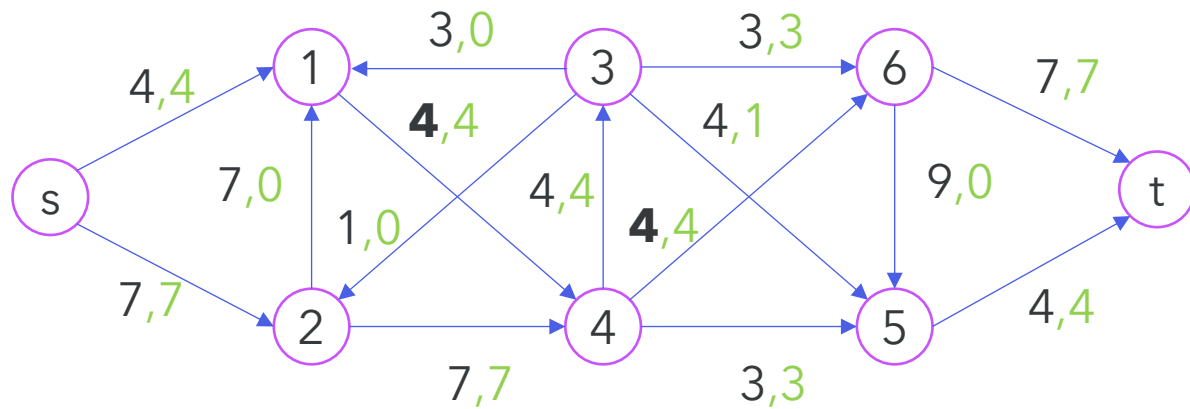
Abbiamo trovato un nuovo taglio $N_s = \{s, 1, 2, 4\}$ di capacità minima pari a 10.

Perciò non bastava aumentare la capacità dell'arco $(1,4)$, perché ci sono altri tagli di capacità minima.

Vedo che aumentare la capacità dell'arco $(4,6)$ di una unità potrebbe aiutare a portare il flusso restante.

Scrivo il nuovo grafo residuo e trovo il cammino $(s,1), (1,4), (4,6), (6,t)$ con capacità pari a 1.





Abbiamo trovato un nuovo taglio $N_s = \{s, 1, 2, 4\}$ di capacità minima pari a 10.

Perciò non bastava aumentare la capacità dell'arco $(1,4)$, perché ci sono altri tagli di capacità minima.

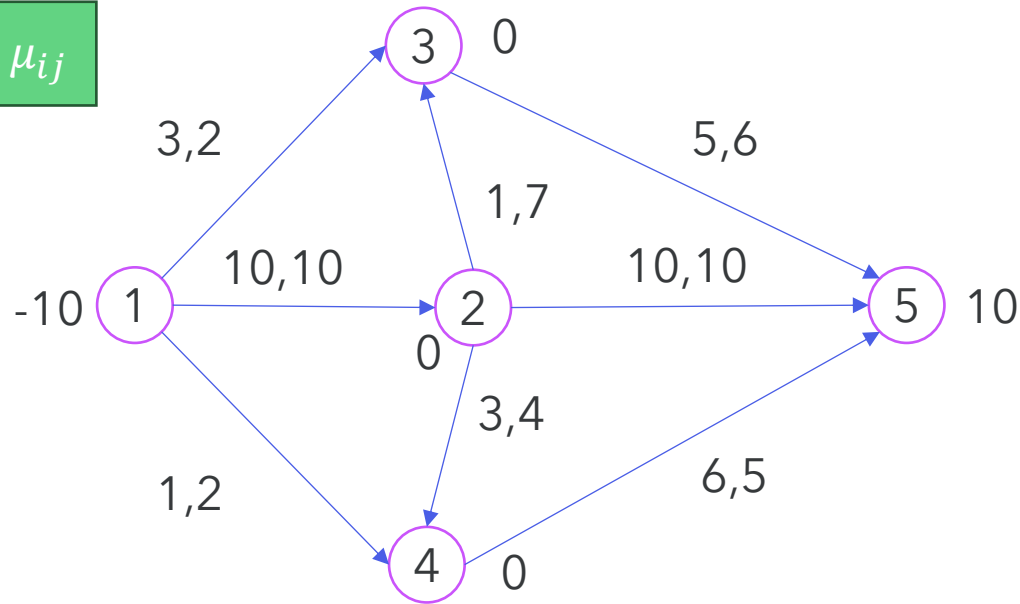
Vedo che aumentare la capacità dell'arco $(4,6)$ di una unità potrebbe aiutare a portare il flusso restante.

Scrivo il nuovo grafo residuo e trovo il cammino $(s,1), (1,4), (4,6), (6,t)$ con capacità pari a 1.

Se ci chiedessimo qual è il minimo numero di archi a cui aumentare la capacità per ottenere un flusso ammissibile: 2 archi, $(1,4)$ e $(4,6)$.

Se ci fossero stati due tagli con un arco coincidente: basterebbe aumentare la capacità di un solo arco: quello dell'intersezione tra i due tagli.

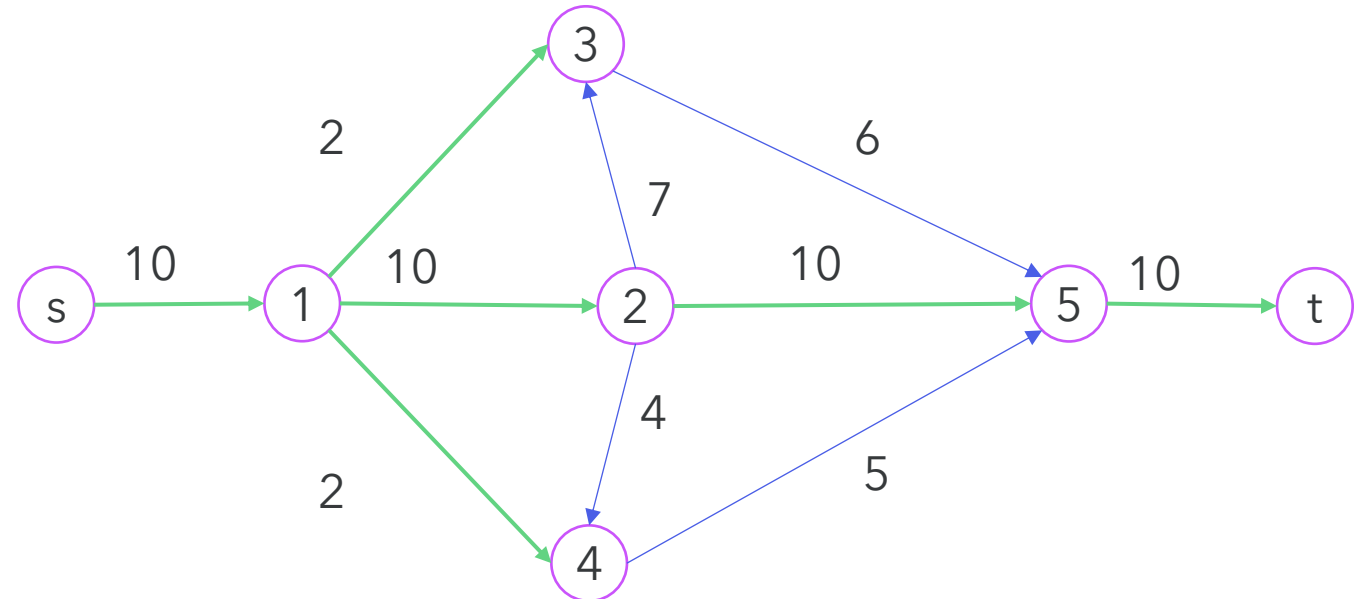
c_{ij}, μ_{ij}



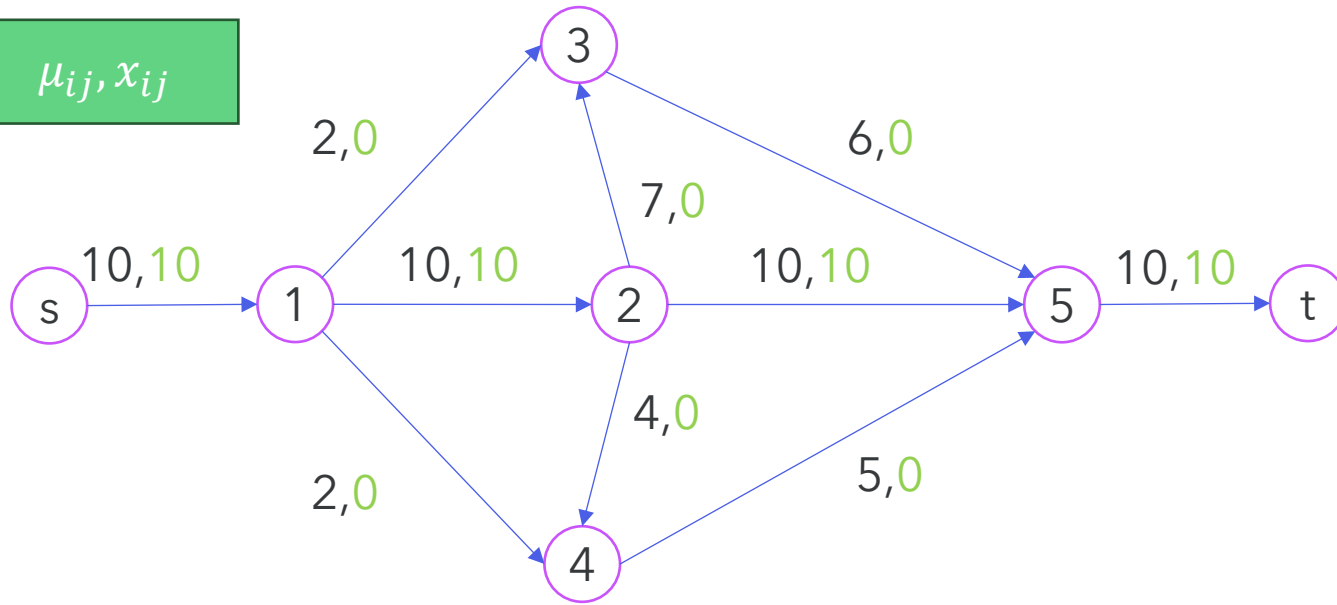
Dato un esempio in cui il problema di flusso di costo minimo ha una soluzione ammissibile come in figura.

Per capire se riesco a mandare flusso da 1 a 5 e quanto: risolvo un **problema di flusso massimo** sul grafo modificato con supersorgente e superpozzo.

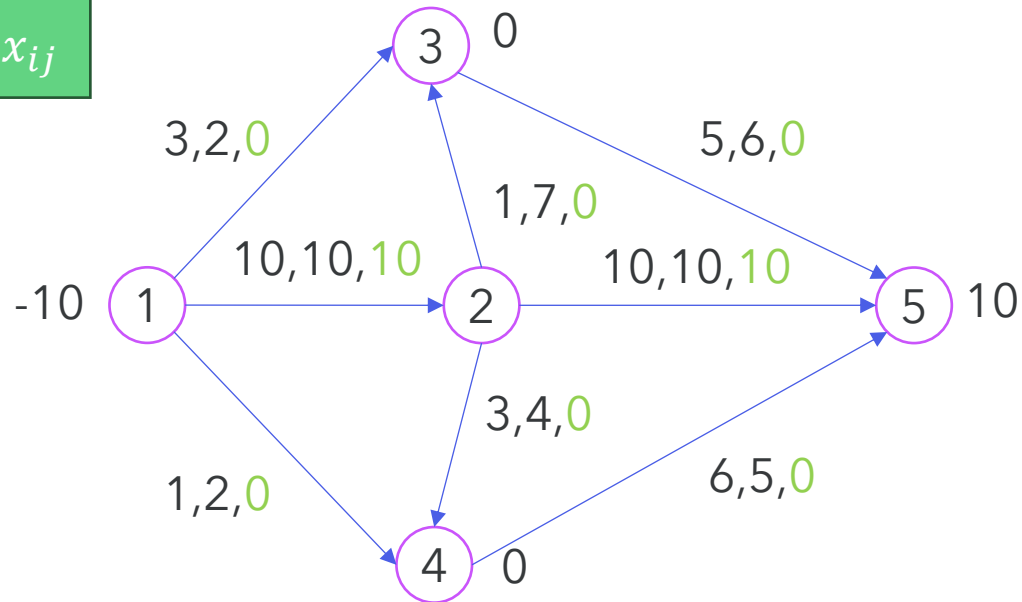
Cerco il un cammino da s a t sul grafo residuo: trovo $(s,1), (1,2), (2,5), (5,t)$ con capacità pari a 10.



μ_{ij}, x_{ij}



c_{ij}, μ_{ij}, x_{ij}



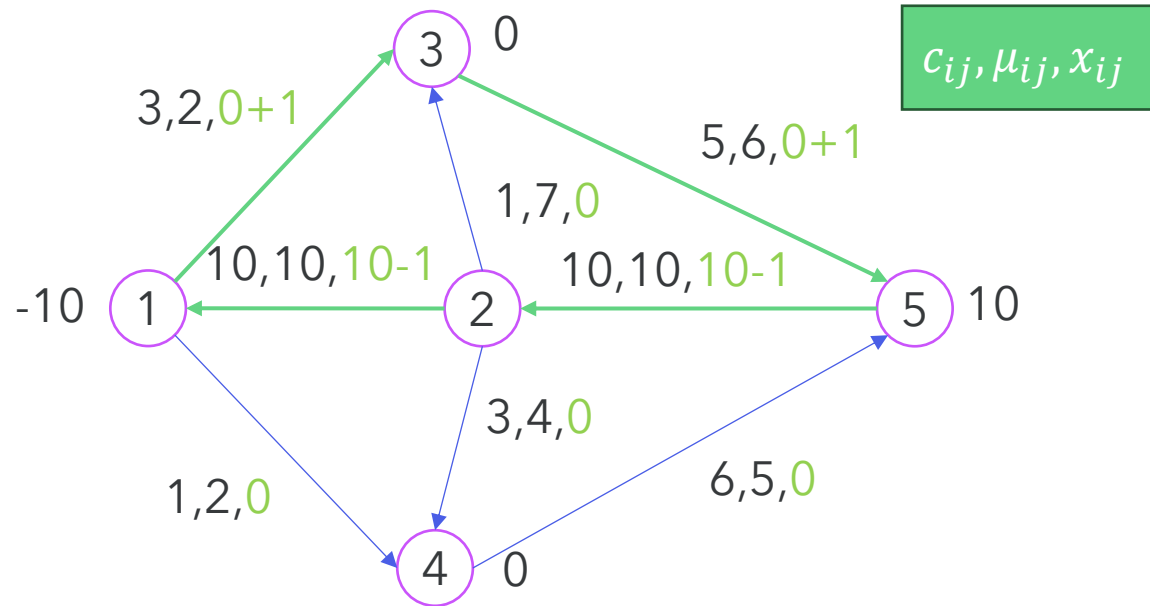
Ho trovato il flusso massimo. Ho trovato anche un taglio $N_s = \{s\}$ con capacità 10 (lo stesso valore dei deficit dei pozzi e delle sorgenti cambiato di segno).
Le capacità degli archi uscenti da s e entranti in t sono sature: flusso massimo.

Otengo un flusso ammissibile per il problema del flusso di costo minimo.

È il **flusso di costo minimo**?

Questo flusso ha un costo $c = 10 \cdot 10 + 10 \cdot 10 = 200$.

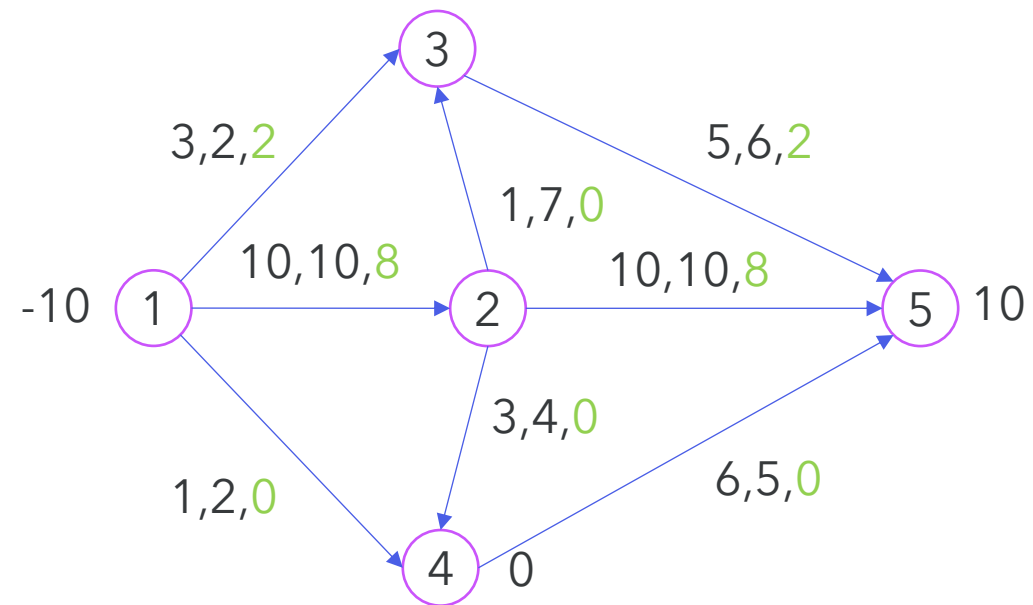
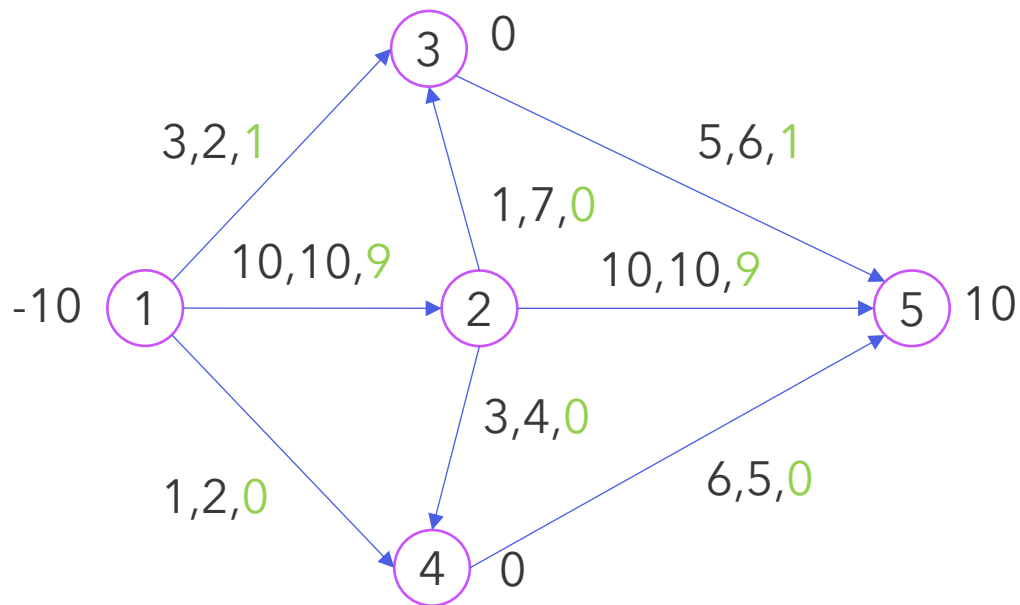
Posso fare di meglio?

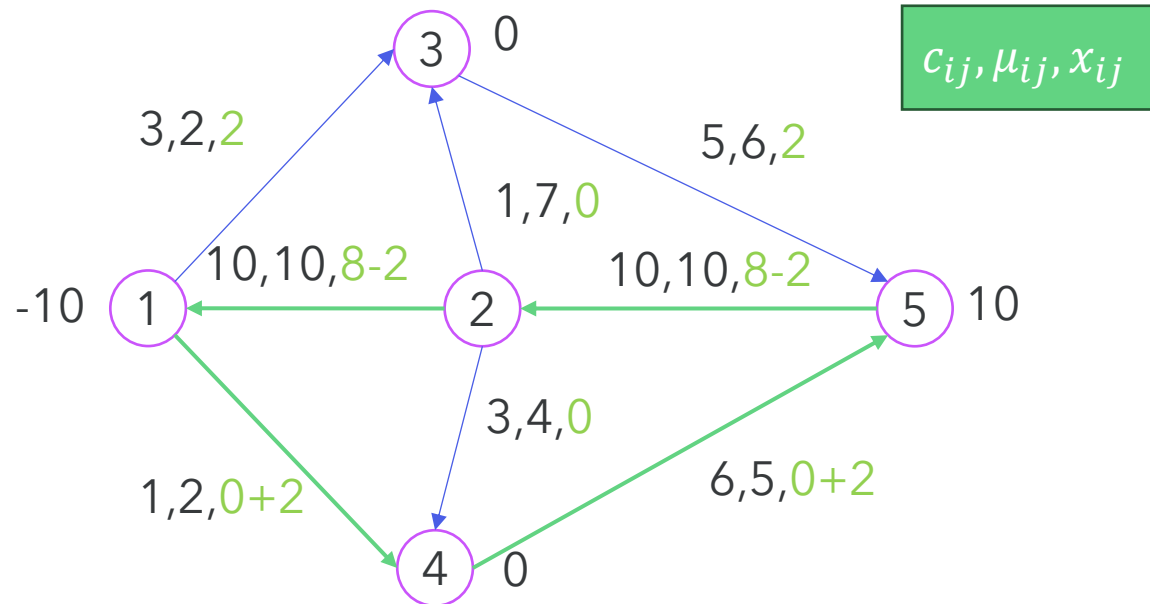


Notiamo che se mandassi una unità di flusso sul cammino $(1,3), (3,5)$ e la togliessi dal cammino $(5,2), (2,1)$ otterrei un guadagno: $200+2+6-10-10= 188$

Otteniamo un flusso ammissibile con costo minore: $c = 188$.

Se avessi portato 2 unità, invece di 1, avrei fatto ancora di meglio: $c = 200 + 16 - 40 = 176$.



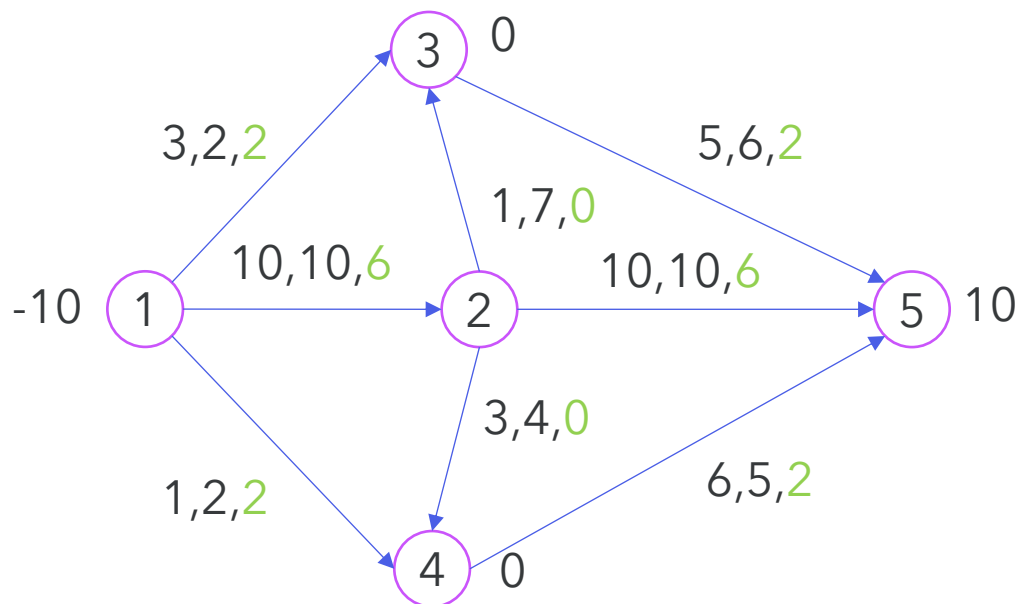


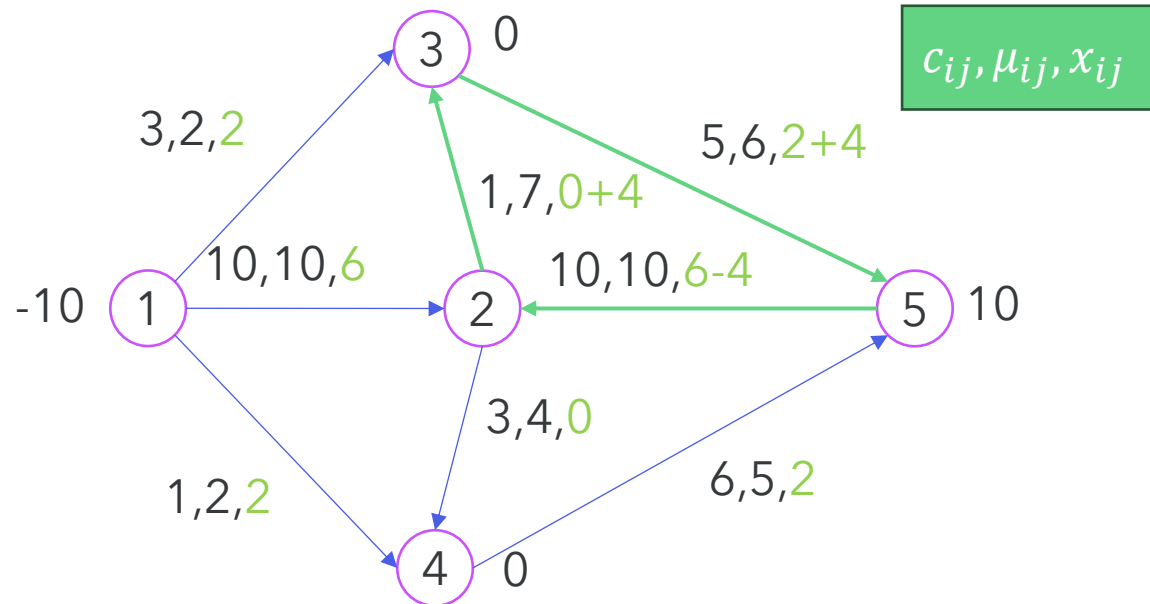
Il cammino (1,3), (3,5) è saturo. Quello ottenuto è il flusso ottimo?

Notiamo che portando una unità di flusso sul cammino (1,4), (4,5) e togliendola da (5,2), (2,1) possiamo migliorare il costo: per ogni unità risparmiamo: $20 - 7 = 13$.

Possiamo portare al massimo 2 unità per via della capacità dell'arco (1,4).

Il flusso resta ammissibile e il costo di questa soluzione è $c = 176 - 2 \cdot 13 = 150$



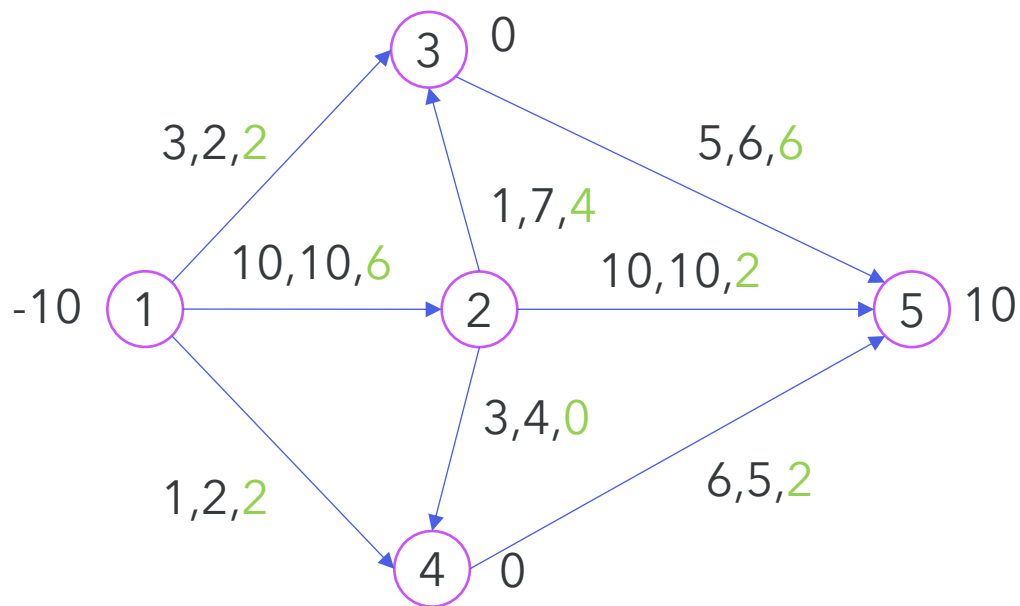


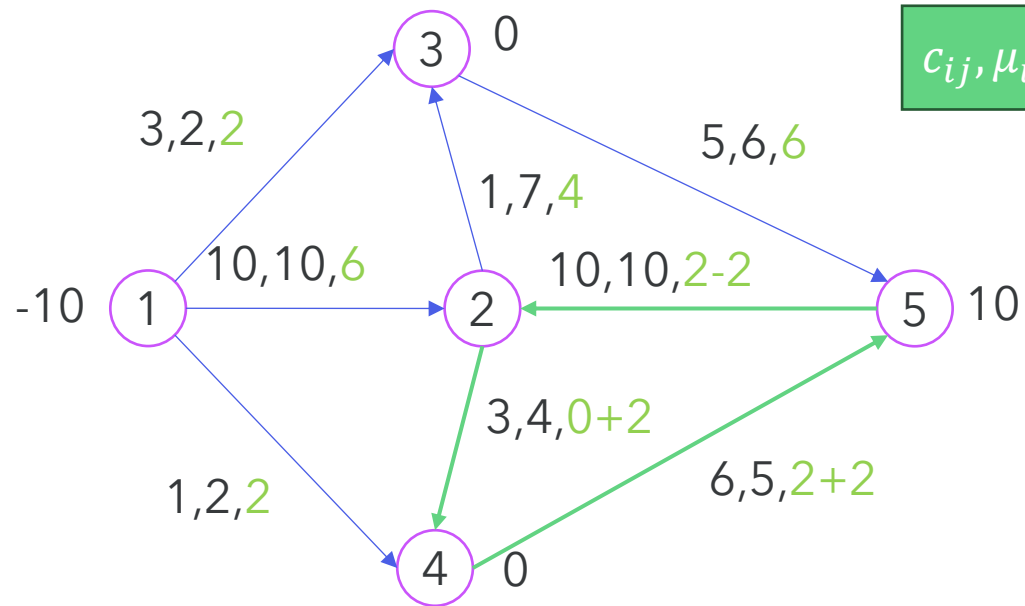
Il cammino (1,4), (4,5) è saturo. Quello ottenuto è il flusso ottimo?

Notiamo che portando una unità di flusso sul cammino (2,3), (3,5) e togliendola da (5,2) possiamo migliorare il costo: per ogni unità risparmiamo: $10 - 1 - 5 = 4$.

Possiamo portare al massimo 4 unità per via della capacità residua dell'arco (3,5).

Il flusso resta ammissibile e il costo di questa soluzione è $c = 150 - 4 \cdot 4 = 134$





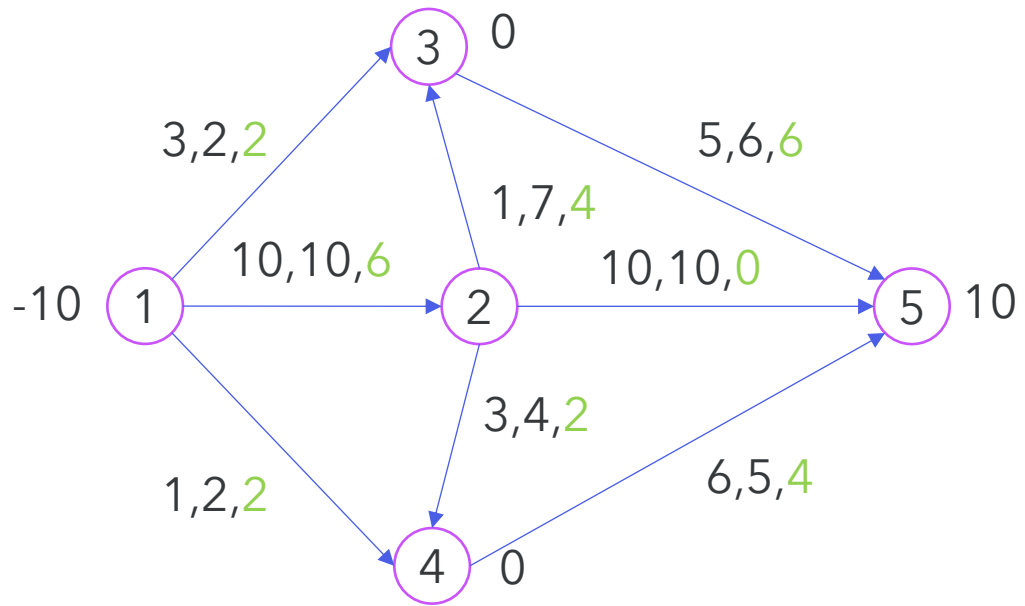
c_{ij}, μ_{ij}, x_{ij}

Il ciclo (2,3), (3,5), (5,2) è saturo. Quello ottenuto è il flusso ottimo?

Notiamo che portando una unità di flusso sul cammino (2,4), (4,5) e togliendola da (5,2) possiamo migliorare il costo: per ogni unità risparmiamo: $10 - 3 - 6 = 1$.

Possiamo portare al massimo 2 unità per via della flusso restante sull'arco (2,5).

Il flusso resta ammissibile e il costo di questa soluzione è $c = 134 - 1 \cdot 2 = 132$



Non vedo altri modi per migliorare il costo: è il flusso ottimo? **Formalizziamo.**

PSEUDOFIUSI

Dato un grafo $G = (N, A)$ con capacità μ :

Uno **pseudoflusso** è un vettore $x = [x_{ij}]_{(i,j) \in A}$ che rispetta solo i vincoli di capacità $0 \leq x_{ij} \leq \mu_{ij} \quad (i,j) \in A$

Non rispetta (necessariamente tutti) i vincoli di bilanciamento del flusso nei nodi.

SBILANCIAMENTO

Lo **sbilanciamento** rispetto allo pseudoflusso x di un nodo i :

$$e_x(i) = \sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} - b_i$$

$O_x = \{i: e_x(i) > 0\}$: eccedenza di flusso (come se fosse una sorgente)

$D_x = \{i: e_x(i) < 0\}$: difetto di flusso (come se fosse una pozzo)

Se $e_x(i) = 0$ allora il nodo è bilanciato

Funziona al
contrario dei
deficit

Dato uno pseudoflusso x , se $O_x = \emptyset$ allora $D_x = \emptyset$, ovvero $e_x(i) = 0 \forall i \in N$:
 x è un **flusso ammissibile**.

CAMMINI AUMENTANTI

Dati due nodi, s e t , non necessariamente la sorgente e il pozzo, chiamiamo P_{st} un **cammino** da s a t .

Tale cammino non deve essere necessariamente orientato rispetto agli archi del grafo, allora avremo $P_{st} = P_{st}^+ \cup P_{st}^-$ con:

- P_{st}^+ **archi concordi**
- P_{st}^- **archi discordi**

Posso scrivere un cammino aumentante tra due nodi sbilanciati in senso opposto.

Sia $\theta(P_{st}, x)$ la **capacità residua** del cammino rispetto alla **pseudoflusso** x .

Un **cammino aumentante** P_{st} è tale che la sua **capacità residua** $\theta(P_{st}, x) > 0$.

CAMMINI AUMENTANTI

Dati uno pseudoflusso x , un cammino P_{st} e uno scalare non negativo $\theta \leq \theta(P_{st}, x)$,
l'operazione di **composizione**:

$$x(\theta) = x \oplus \theta P_{st}$$

aggiorna solo gli sbilanciamenti dei nodi estremi s e t .

$$e'_x(i) = \begin{cases} e_{x(i)} - \theta & i = s \\ e_{x(i)} + \theta & i = t \\ e_{x(i)} & i \neq s, t \end{cases}$$

Preleva del flusso da s a t senza cambiare quello che succede nel mezzo (si mantengono gli sbilanciamenti).

Come selezionare s e t al fine di migliorare il costo di un flusso?

CICLI AUMENTANTI

Se prendiamo $s = t$, P_{st} è un **ciclo** (C)

Il ciclo non modifica alcuno sbilanciamento:

- Noi partivamo da un flusso ammissibile
- Componendo il flusso ammissibile con uno pseudoflusso (che rispetta le capacità) e non cambia gli sbilanciamenti: ottengo un nuovo flusso ammissibile.

Un ciclo C prende flussi ammissibili e li trasforma in flussi ammissibili.

Se la quantità θ del flusso inviato con un ciclo rispetta la capacità $\theta(C, x)$ ed è positiva:

$$\theta \leq \theta(C, x) > 0$$

Allora C è un **ciclo aumentante**.

COSTO DI UN CAMMINO O CICLO

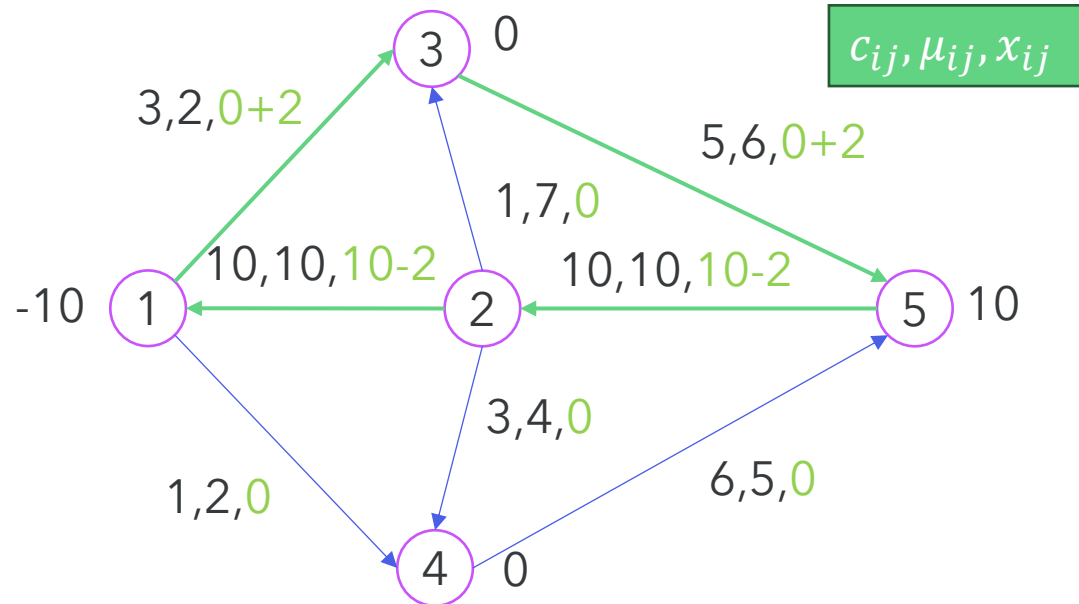
Costo di un cammino P :

$$c(P) = \sum_{(i,j) \in P^+} c_{ij} - \sum_{(i,j) \in P^-} c_{ij}$$

Effetto sul costo del flusso:

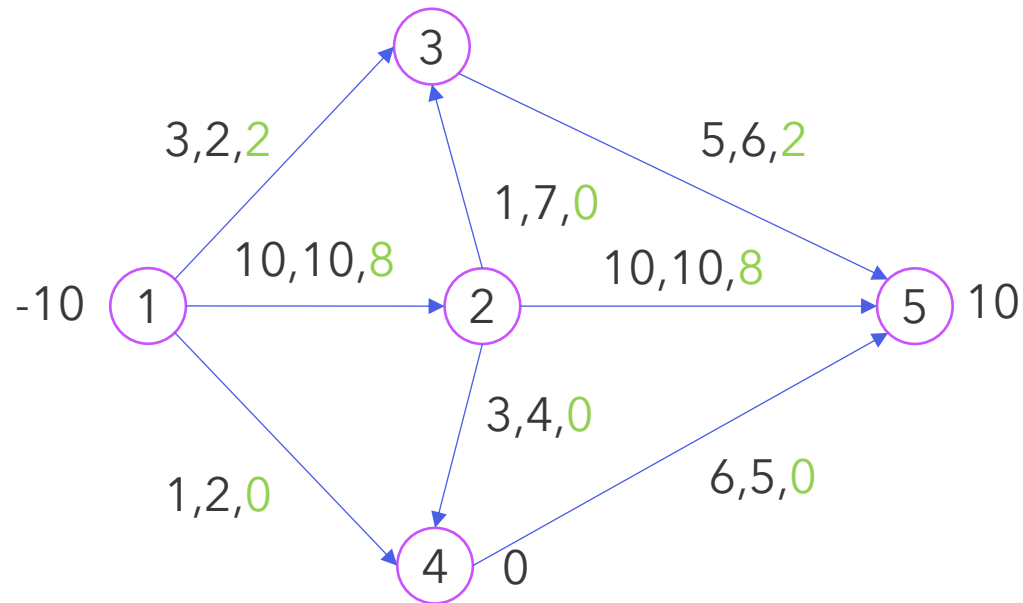
$$c(x \oplus \theta P) = c(x) + \theta c(P)$$

Cerchiamo cicli con costo negativo $c(P) < 0$: miglioramento del costo del flusso.



Tornando all'esempio, la prima cosa che abbiamo fatto, un volta ottenuto un flusso ammissibile, è stato determinare un **ciclo aumentante** $C: (1,3), (3,5), (5,2), (2,1)$ con $P^+ = \{(1,3), (3,5)\}$ e $P^- = \{(5,2), (2,1)\}$ capacità $\theta = 2$.

Il **costo del ciclo** è: $c(C) = 3 + 5 - 10 - 10 = -12$. È un ciclo aumentante di costo negativo: migliora la soluzione: $c(x') = c(c) + \theta c(C) = 200 - 24 = 176$.



Possiamo iniziare ad immaginare un algoritmo: iniziare cercando un flusso ammissibile usando un algoritmo per il flusso massimo sul grafo modificato. Se il flusso massimo non esiste o non rispetta le quantità che è necessario spostare nel problema originale: il problema non ha soluzione. Se ottengo un flusso ammissibile, cerco di migliorarlo. Cerco dei cicli aumentanti di costo negativo e li applico.

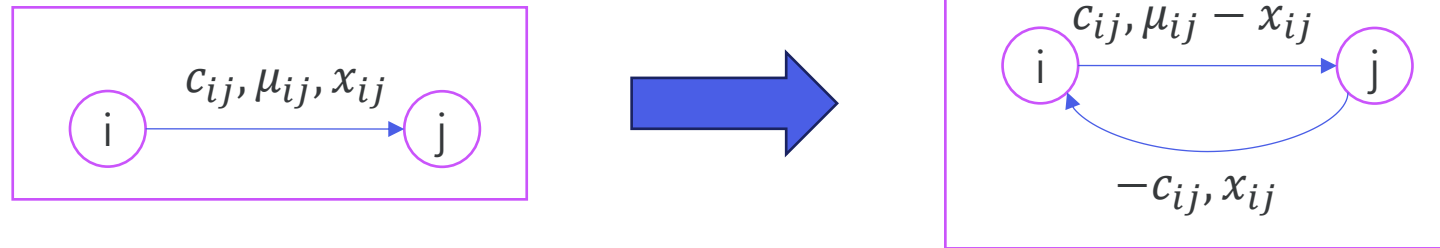
Come si trovano i cicli aumentanti di costo negativo?

GRAFO RESIDUO PESATO

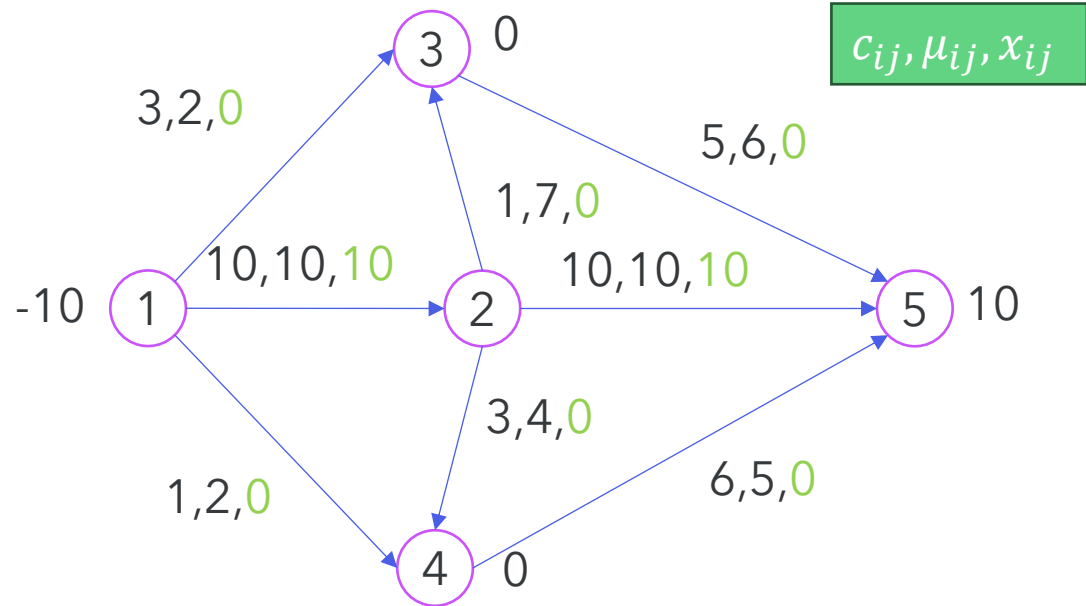
Per uno pseudoflusso x , si costruisce il **grafo residuo pesato** $G_x = (N, A_x)$:

Se $x_{ij} < \mu_{ij}$: arco (i, j) con costo $c'_{ij} = c_{ij}$

Se $x_{ij} > 0$: arco (j, i) con costo $c'_{ji} = -c_{ij}$

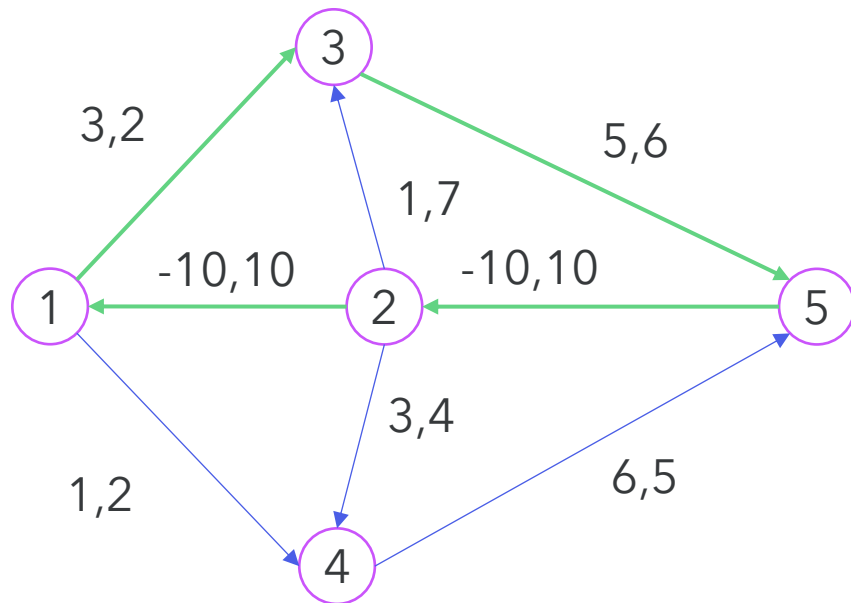


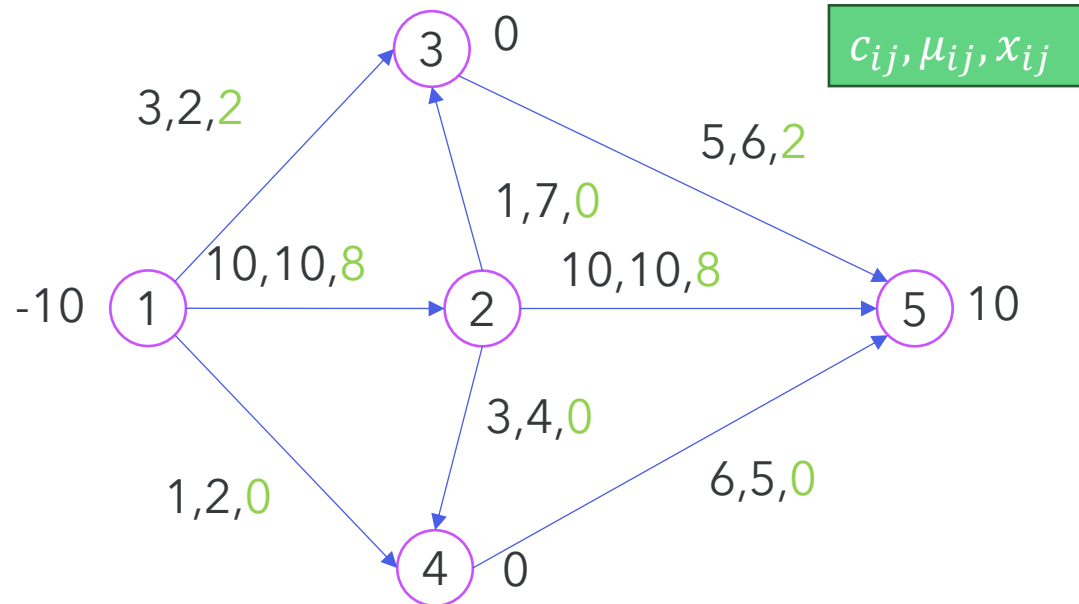
Lemma: ogni cammino aumentante sul grafo G corrisponde a un cammino orientato sul grafo residuo G_x **con stesso costo**.



Tornando all'esempio con il flusso ammissibile e scriviamo il **grafo residuo pesato**.

Troviamo un ciclo aumentante $(1,3), (3,5), (5,2), (2,1)$ di costo negativo $c(C) = -12$ con capacità: $\theta = \min\{2, 6, 10, 10\} = 2$.
Costo = $200 - 24 = 176$.

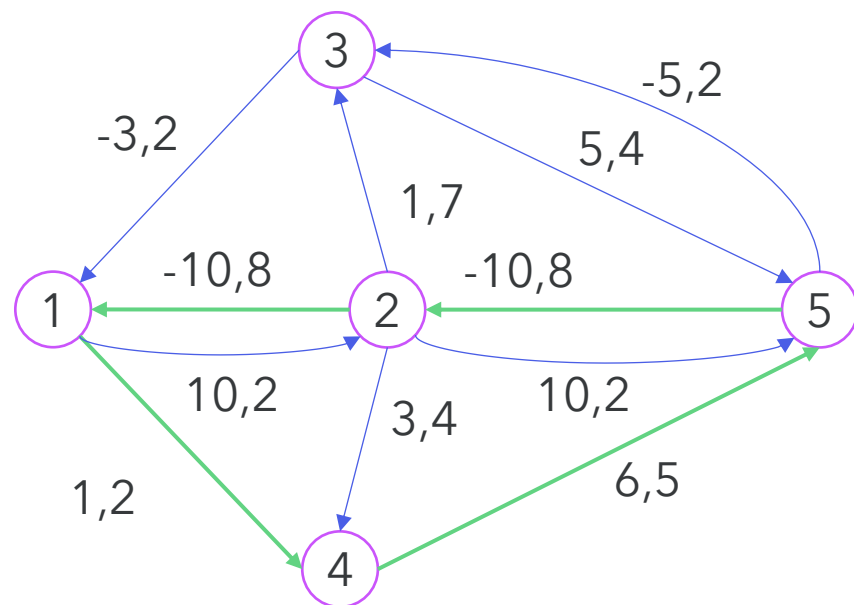




Scriviamo il grafo con il nuovo flusso e il **grafo residuo pesato**.

Troviamo un ciclo aumentante $(1,4), (4,5), (5,2), (2,1)$ di costo negativo $c(C) = -13$ con capacità: $\theta = \min\{2,5,8,8\} = 2$.

Costo = $176 - 26 = 150$



Il ciclo orientato di costo negativo sul grafo residuo, aumentante sul grafo originale, che abbiamo trovato in precedenza non esiste più: almeno un arco ora è saturo oppure almeno un arco è vuoto: cancelliamo il ciclo ad ogni iterazione. Per questo si parla di **Algoritmo di Cancellazione di Cicli**.

Come si trovano i cicli orientati di costo negativo?

TROVA-CICLI

Conosciamo già un algoritmo che permette di trovare dei cicli orientati di costo negativo su un grafo orientato: **SPT.L.Queue**.

Lanciamo l'SPT.L.QUEUE sul **grafo residuo pesato**.

Se ci sono dei nodi che escono dalla coda $n - 1$ volte: abbiamo trovato un **ciclo orientato di costo negativo**.

ALGORITMO BASATO SU CANCELLAZIONE DI CICLI

```
procedure (x, caso) = Cancella-Cicli(G, c, b, u)
{
  (x, caso) = Flusso-Ammissibile(G, b, u);
  if (caso == "vuoto") then return;

  for (;;) {
    (C,  $\theta$ ) = Trova-Ciclo(G, c, u, x);
    if ( $\theta$  == 0) then break;
    x = Cambia-Flusso(x, C,  $\theta$ );
  }
}
```

Input: un grafo con i costi e le capacità sugli archi e i deficit sui nodi.

Output: il flusso di costo minimo o che il problema è vuoto.

Iniziamo l'algoritmo con la procedura *Flusso-Ammissibile*, che lancia un flusso massimo sul grafo modificato e ci dà un flusso ammissibile o ci dice che il problema è vuoto.

Se il problema non è vuoto: iteriamo cercando di migliorare il flusso. Cerchiamo un ciclo orientato di costo negativo lanciando l'algoritmo *Trova-Ciclo*, per esempio usando l'SPT.L.Queue.

Se non si trova un ciclo aumentante allora si è ottenuto il flusso di costo minimo, altrimenti si aggiorna il flusso con una operazione di composizione dato il ciclo trovato (*Cambia-Flusso*).

ALGORITMO BASATO SU CANCELLAZIONE DI CICLI

```
procedure (x, caso) = Cancella-Cicli(G, c, b, u)
{
  (x, caso) = Flusso-Ammissibile(G, b, u);
  if (caso == "vuoto") then return;

  for (;;) {
    (C,  $\theta$ ) = Trova-Ciclo(G, c, u, x);
    if ( $\theta$  == 0) then break;
    x = Cambia-Flusso(x, C,  $\theta$ );
  }
}
```

Termina?

Quest'algoritmo generalizza l'algoritmo per cammini
aumentanti, perciò potrebbe non terminare.

Se non trovo cicli aumentanti: ottengo il flusso di costo
minimo? Ottengo il flusso ottimo?

DECOMPOSIZIONE DEGLI PSEUDOFLOSSI

Teorema: Decomposizione degli pseudoflussi

Dati due pseudoflussi x', x'' sullo stesso grafo $G = (N, A)$ con capacità, esistono al più $k = n + m$ cammini o cicli aumentanti $P_1, \dots, P_k$ (di cui al massimo m cicli) rispetto ad x' e k numeri θ_i , $0 \leq \theta_i \leq \theta(P_i, x')$, tali che:

$$x'' = x' \oplus \theta_1 P_1 \oplus \dots \oplus \theta_k P_k$$

Ovvero, dati due pseudoflussi arbitrari, esiste un numero finito di cammini e cicli aumentanti che possiamo comporre per passare da uno pseudoflusso all'altro.

Tutti i cammini e i cicli sono aumentanti rispetto al primo pseudoflusso, perciò possiamo mandare flusso sui cammini e cicli aumentanti senza un ordine. Se usiamo la massima capacità, nell'iterazione successiva saranno cancellati.

DECOMPOSIZIONE DEGLI PSEUDOFIUSSI

Teorema: continua

I P_i che sono dei cammini hanno origine s_i e destinazione t_i tale che

$$e_{x'}(s_i) \neq e_{x''}(s_i)$$

$$e_{x'}(t_i) \neq e_{x''}(t_i)$$

Se $e_{x'} = e_{x''}$ allora tutti i P_i sono cicli.

Se gli sbilanciamenti tra il primo flusso e il secondo flusso sono diversi: si usano dei cammini aumentanti per rettificare la differenza di sbilanciamento.

Una volta che non ci sono più nodi con sbilanciamenti diversi si usano i cicli aumentanti.

Ogni pseudoflusso può essere costruito da cammini e cicli elementari.

DECOMPOSIZIONE DEGLI PSEUDOFLOSSI

Corollario:

Se x' e x'' sono **flussi ammissibili** allora tutti i P_i sono cicli.

Dati due flussi ammissibili qualunque, possiamo trasformare l'uno nell'altro mandando flusso lungo al massimo m cicli aumentanti rispetto al primo.

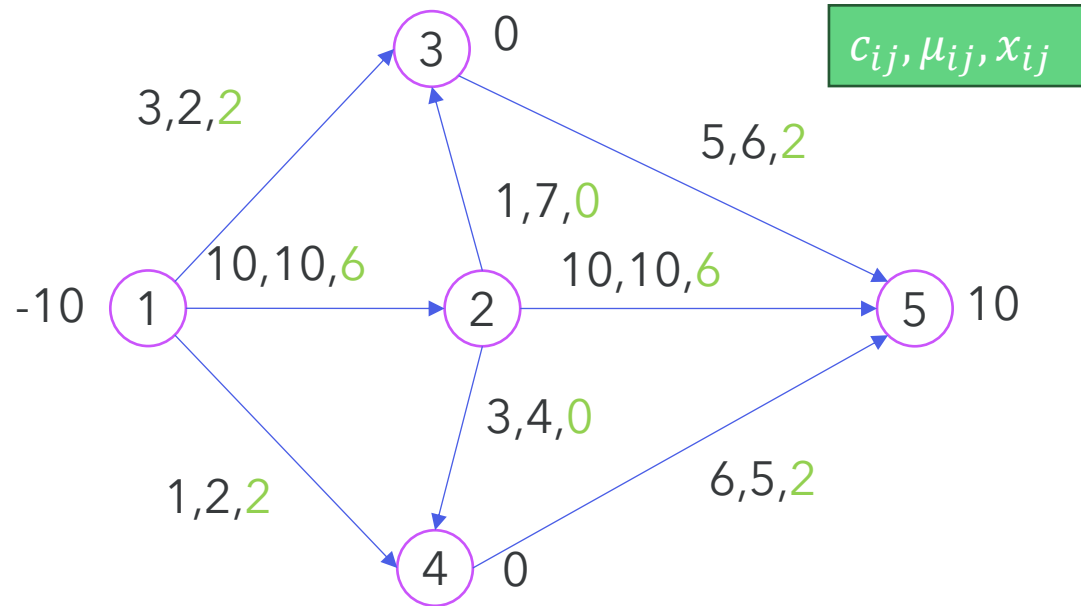
SINTESI:

Ogni pseudoflusso può essere "scomposto" in:

- **cammini** (che collegano nodi con sbilanciamento $\neq 0$), e
- **cicli** (che non cambiano lo sbilanciamento, ma fanno circolare flusso chiuso).

Quindi: un qualsiasi pseudoflusso può essere pensato come **somma di al più $m + n$ flussi su cammini e cicli**.

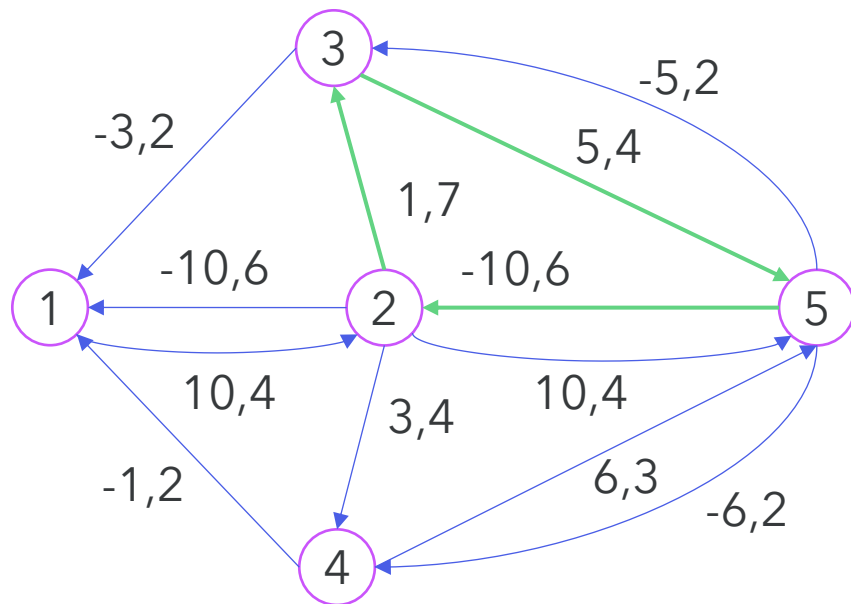
Se tutti i nodi hanno sbilanciamento nullo, servono solo cicli. Questi cicli possono essere al massimo m , perché posso saturare al più m archi (uno alla volta).

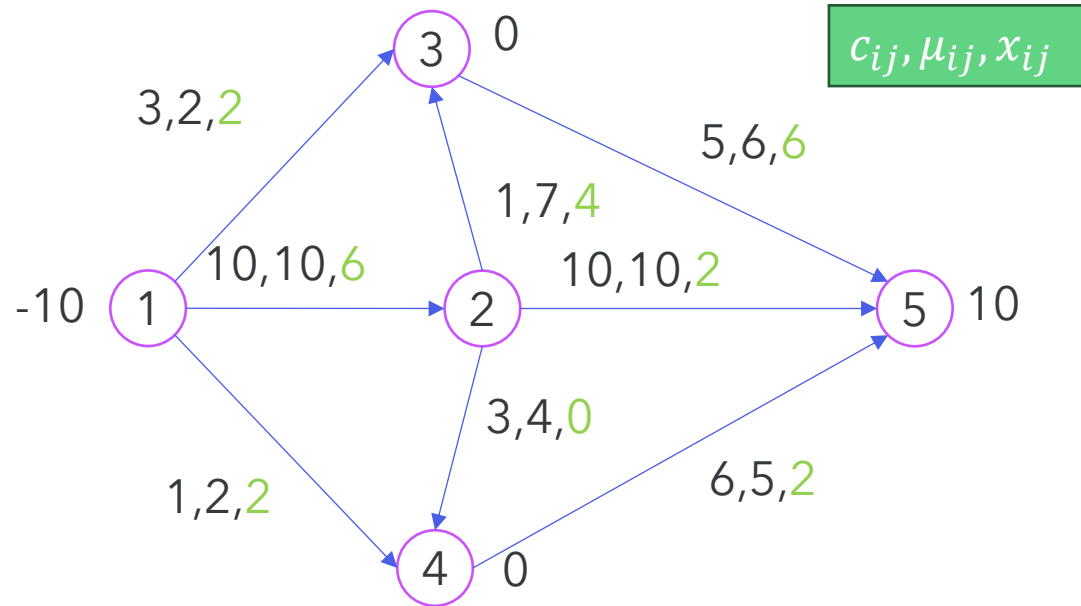


Scriviamo il grafo con il nuovo flusso e il **grafo residuo pesato**.

Troviamo un ciclo aumentante $(2,3), (3,5), (5,2)$ di costo negativo $c(C) = -4$ con capacità: $\theta = \min\{7,4,6\} = 4$.

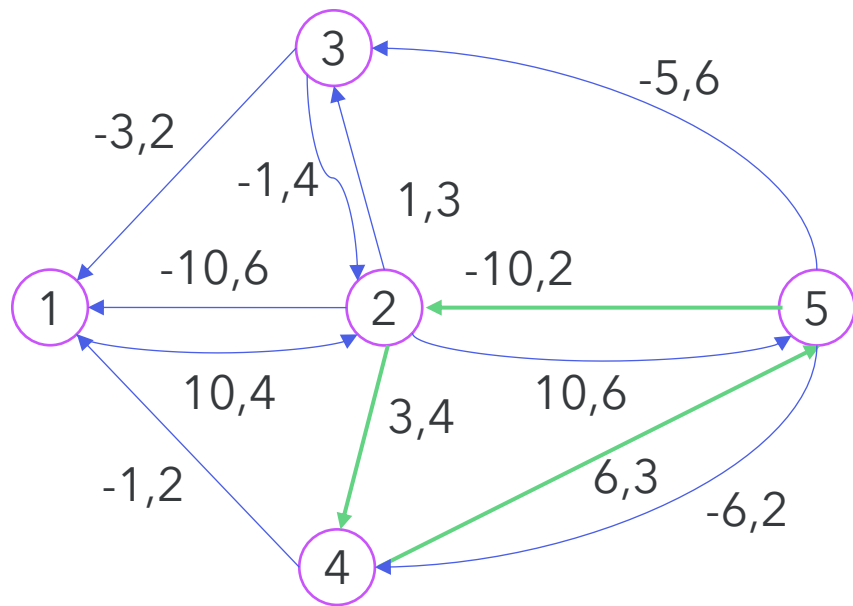
Costo = $150 - 16 = 134$.

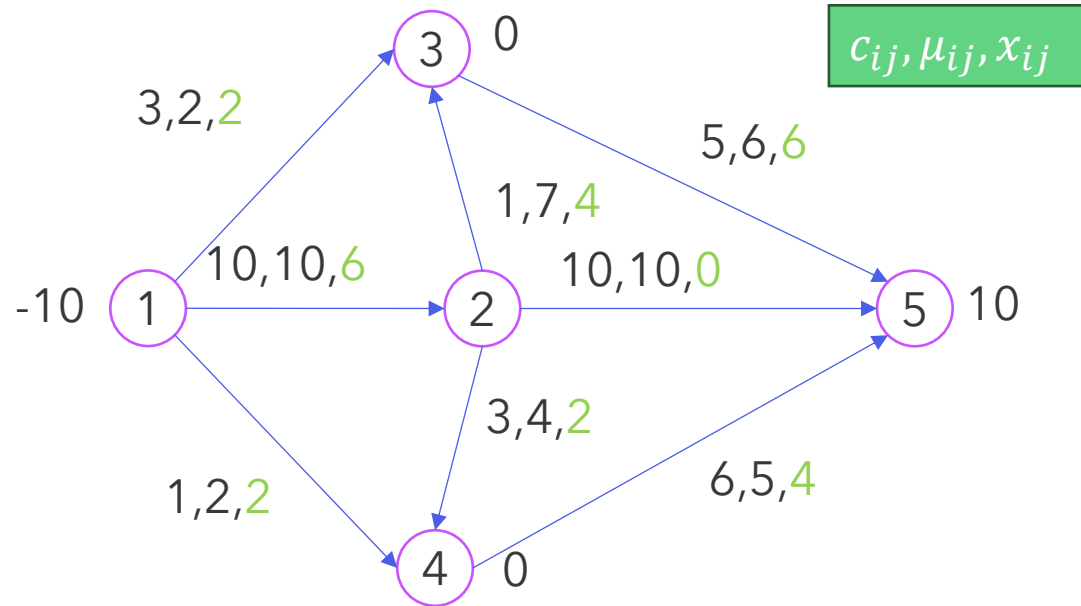




Scriviamo il grafo con il nuovo flusso e il **grafo residuo pesato**.

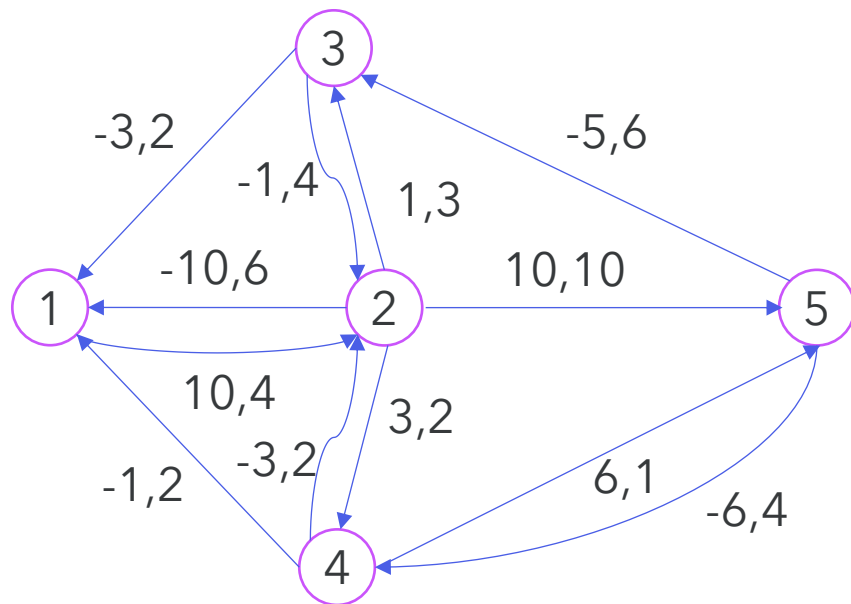
Troviamo un ciclo aumentante (2,4), (4,5), (5,2) di costo negativo $c(C) = -1$ con capacità: $\theta = \min\{4, 3, 2\} = 2$.
Costo = $134 - 2 = 132$.





Scriviamo il grafo con il nuovo flusso e il **grafo residuo pesato**.

Non ci sono più cicli orientati di costo negativo.

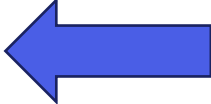


E' il flusso di costo minimo?

CONDIZIONI DI OTTIMALITÀ

Corollario - Criterio di ottimalità:

Un flusso ammissibile x è **ottimo** se e solo se **non esistono cicli aumentanti di costo negativo**.

Dimostrazione: 

1) Se esiste un ciclo C con $c(C) < 0$, si può ridurre il costo e quindi migliorare la soluzione: quindi la soluzione non era ottima.

CONDIZIONI DI OTTIMALITÀ

Dimostrazione:



2) *Se non ci sono cicli di flusso aumentante di costo negativo allora il flusso è ottimo.*

Supponiamo di avere un flusso x che non è ottimo e che tuttavia non esistano cicli aumentanti di costo negativo (procedo per contraddizione).

Se x non è ottimo, allora esiste un x^* tale che $c(x^*) < c(x)$.

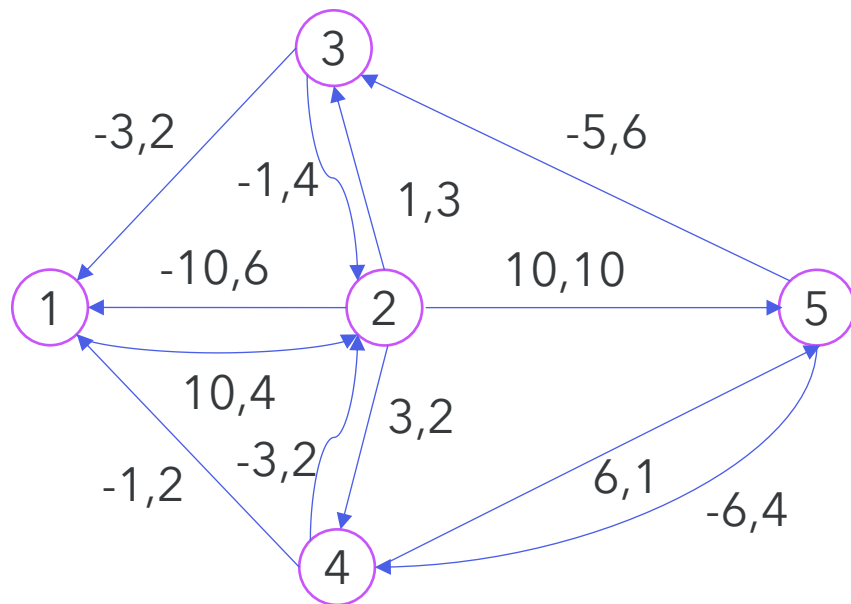
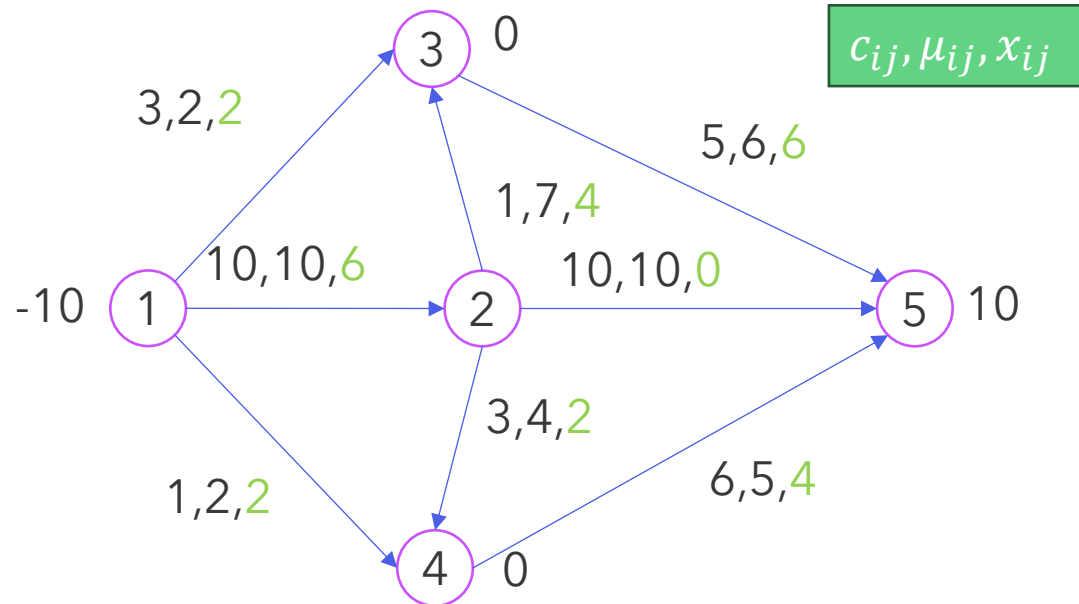
Uso il *Teorema di decomposizione degli pseudoflussi*: $x^* = x \oplus \theta_1 C_1 \oplus \dots \oplus \theta_k C_k$

Dove $C_1, \dots, C_k$ sono cicli (i flussi sono ammissibili).

Siccome sono tutti cicli aumentanti rispetto a x avrò: $c(x^*) = c(x) + \theta_1 c(C_1) + \dots + \theta_k c(C_k)$

Però $\theta_1, \dots, \theta_k > 0$ (altrimenti non sarebbero cicli aumentanti) e tutti i $c(C_1), \dots, c(C_k) \geq 0$ (non esistono cicli aumentanti di costo negativo).

Ciò implica che $c(x^*) \geq c(x)$, che contraddice $c(x^*) < c(x)$.



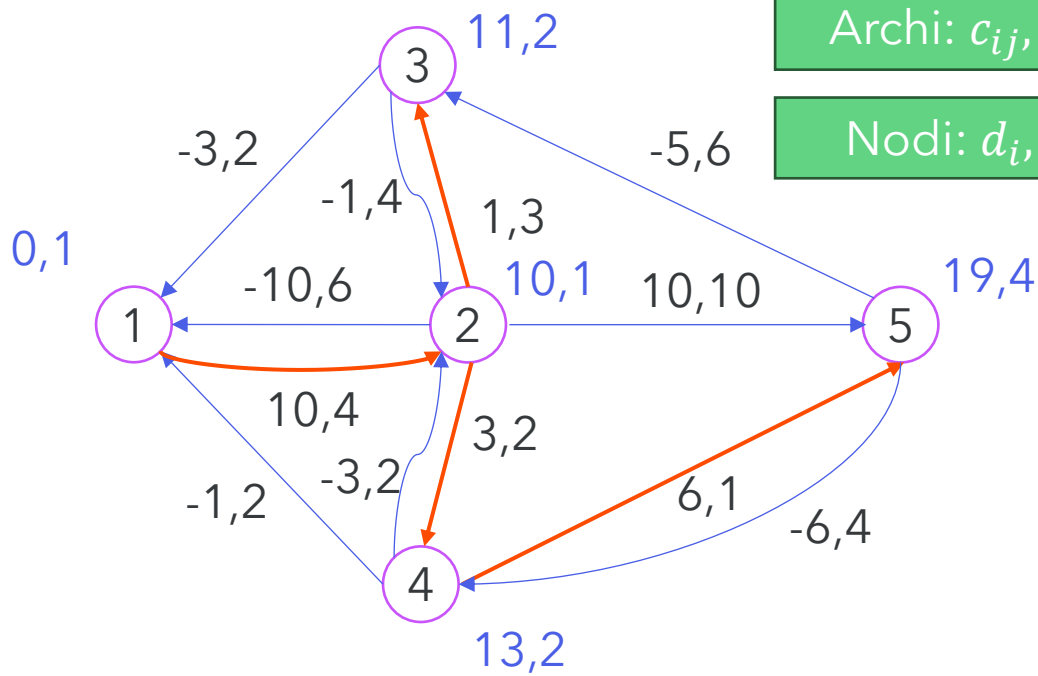
Data la soluzione ottima, per capire che è ottima costruiamo il **grafo residuo pesato**.

Per essere convinti che è il flusso di costo minimo possiamo vedere che esiste un albero dei cammini minimi sul grafo residuo.

O lo troviamo o, dato il vettore di etichette controlliamo che le **condizioni di Bellman** siano verificate.

Se esiste un albero dei cammini minimi allora non possono esistere dei cicli orientati di costo negativo e il flusso è di costo minimo.

Se volessimo costruire l'albero dei cammini minimi: che radice dovremmo usare?

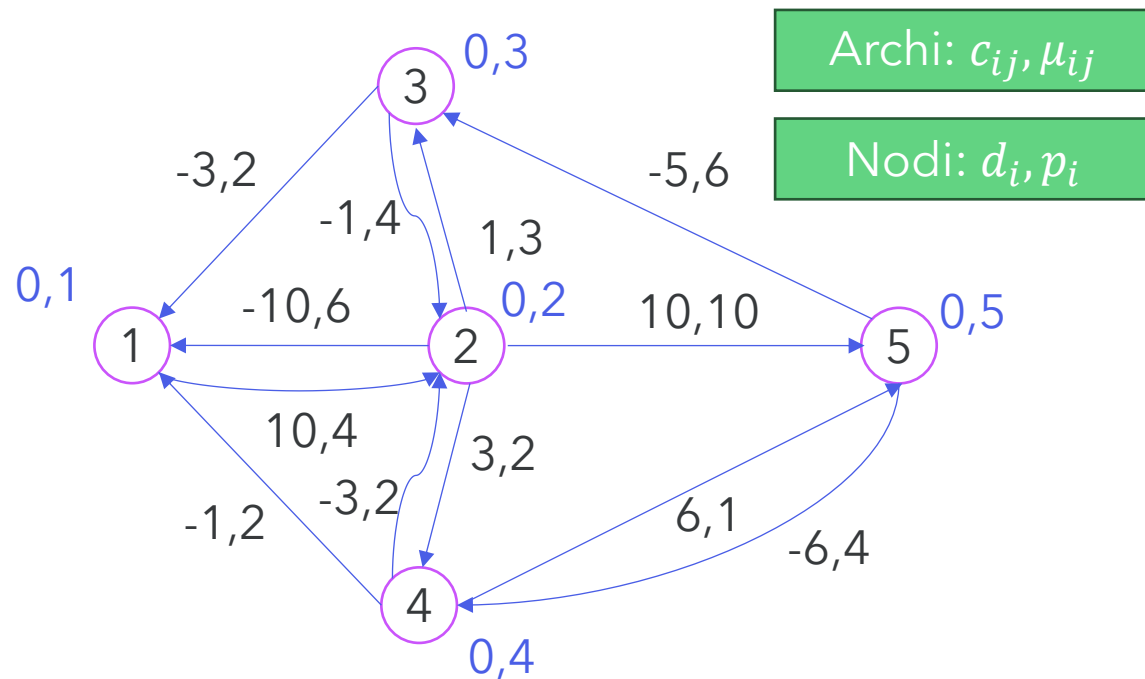


Archi: c_{ij}, μ_{ij}

Nodi: d_i, p_i

Potremmo scegliere una radice a caso e nel caso trovassimo un ciclo orientato di costo negativo lo elimineremmo.

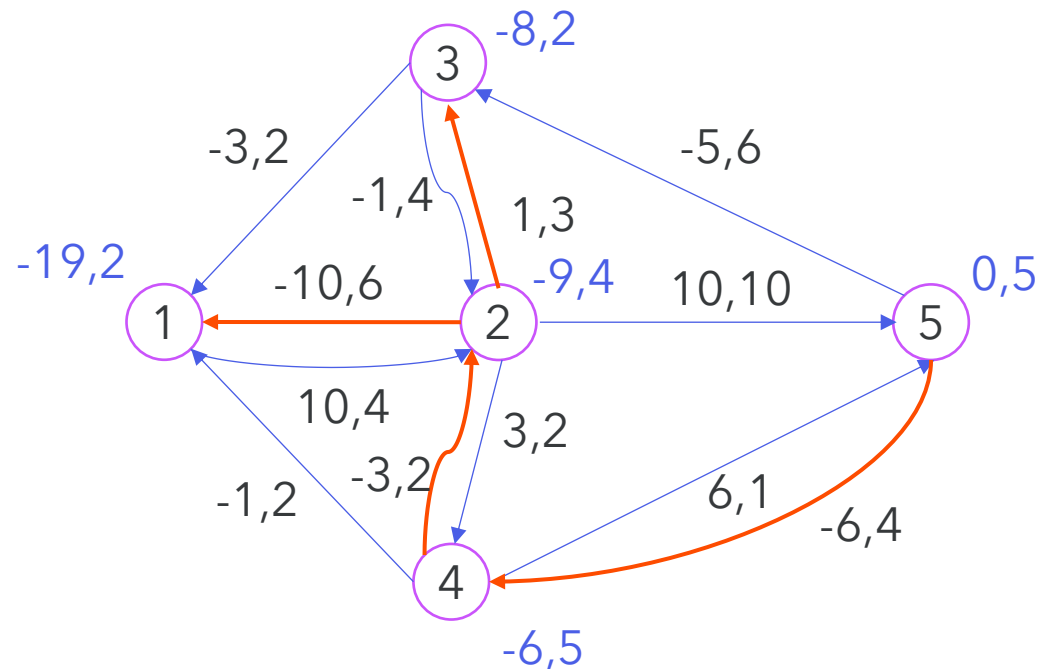
In caso non lo trovassimo potrebbe sempre darsi il caso che alcuni nodi non siano raggiungibili e che nella parte non raggiungibile ci sia un ciclo orientato di costo negativo. Potremmo scegliere un'altra radice nella parte del grafo non raggiungibile e così via fino ad essere sicuri che non ci siano cicli orientati di costo negativo.



Altrimenti potremmo scegliere tutti i nodi come radici: mettendo tutte le etichette a zero e tutti i nodi in coda.

Tutti gli archi rispettano le condizioni di Bellman, perciò non ci sono cicli orientati di costo negativo e dunque la soluzione è ottima.

L'albero dei cammini minimi e le etichette sono un certificato di **ottimalità**



TERMINAZIONE E COMPLESSITÀ

L'algoritmo è un'**estensione** di quello dei **cammini aumentanti**.
Se i **valori di capacità, deficit** fossero **irrazionali**, l'algoritmo potrebbe **non terminare**.

Condizioni sufficienti per la terminazione (e per l'integralità del flusso):

- Le **capacità** devono essere **interi**: $\mu \in N_+^m$
- I **deficit** devono essere **interi**: $b \in Z^n$

Allora:

- L'algoritmo parte da un **flusso nullo** (intero) e,
- a ogni iterazione, **aumenta** il flusso lungo **cicli** con capacità θ intera (perché dipende da capacità residue intere).
- Dopo ogni iterazione, il flusso rimane **intero** (invarianza dell'integralità).

TERMINAZIONE E COMPLESSITÀ

Potremmo prendere anche i **costi interi** ($c_{ij} \in \mathbb{Z}$) e definire i seguenti valori:

- Il costo più alto possibile: $\bar{c} = \max\{\max\{0, c_{ij}\} : (i, j) \in A\}$
- Il costo più basso possibile: $\underline{c} = \min\{\min\{0, c_{ij}\} : (i, j) \in A\}$
- E il massimo delle capacità: $U = \max\{\mu_{ij} : (i, j) \in A\}$

Allora possiamo limitare il *valore ottimo del problema del flusso di costo minimo* tra un limite inferiore e uno superiore:

$$\underline{c}mU \leq z(MCF) \leq \bar{c}mU$$

Ovvero:

- Nel caso peggiore, tutti gli archi hanno costo molto grande e li dobbiamo usare tutti
- Nel caso migliore, tutti gli archi hanno costo molto piccolo e li dobbiamo usare tutti
- La soluzione avrà un valore tra questi due valori.

TERMINAZIONE E COMPLESSITÀ

Ad ogni iterazione dell'algoritmo:

- Troviamo un ciclo C di costo negativo $c(C) < 0$ con costi interi:

$$c(C) \text{ intero e negativo: } c(C) \leq -1$$

- La capacità del ciclo è $\theta = \theta(C, x)$, che è positiva e intera.

$$\theta > 0 \text{ e intera, allora } \theta \geq 1.$$

Ad ogni iterazione il costo del nuovo flusso è:

$$c(x') = c(x) + \theta c(C) \leq c(x) - 1$$

Quindi, ad ogni iterazione il **costo del flusso diminuisce di almeno una unità** (esattamente una nel caso peggiore).

L'algoritmo termina in un numero finito di iterazioni

TERMINAZIONE E COMPLESSITÀ

Allora il caso peggiore è che:

- Il flusso iniziale costi $\bar{c}mU$
- E il flusso ottimo costi $\underline{c}mU$
- Facendo iterazioni che portano *una unità alla volta*.

Dunque il numero massimo di iterazioni è $O((\bar{c} - \underline{c})mU)$

Inoltre ogni singola iterazione costa un SPT.L.Queue, il cui numero di iterazioni è $O(nm)$

Perciò la complessità dell'algoritmo è $O((\bar{c} - \underline{c})mUnm)$

Pseudopolinomiale

MIGLIORAMENTO DELLA COMPLESSITÀ

Problemi principali:

- 1) La **soluzione iniziale** può essere molto lontana dall'ottimo.
- 2) I **cicli** trovati possono avere:
 - capacità piccola,
 - costo negativo poco significativo.
- 3) La **ricerca dei cicli** richiede molte iterazioni.

MIGLIORAMENTO DELLA COMPLESSITÀ

Tutte e tre queste cose possono essere migliorate:

- Si possono trovare cicli con capacità grande e costo molto negativo. Però non è facile. Per esempio trovare il ciclo più negativo è un problema NP-arduo.
- Usare **cicli a costo medio minimo** (costo/archi minimo), invece è un problema polinomiale: si ottiene una *complessità polinomiale*.
- Applicare **scalatura (scaling)** per ridurre gli intervalli di capacità e costi (si considerano prima le cifre più significative, normalmente in base 2, e poi si incrementa): si ottiene una *complessità polinomiale* in $\log U$ e $\log(\bar{c} - \underline{c})$.

Però il trucco principale è prendere l' $O(nm)$ e ridurlo con algoritmi più sofisticati.

MIGLIORAMENTO DELLA SOLUZIONE INIZIALE

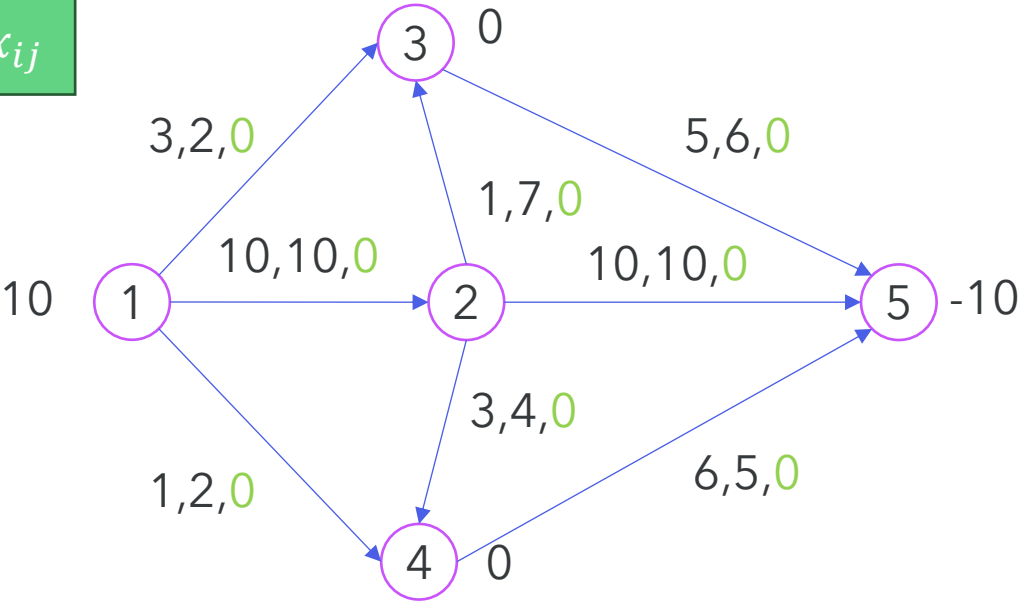
Il metodo più semplice per migliorare l'efficienza dell'algoritmo è **partire da una soluzione iniziale non troppo lontana dall'ottimo**.

Finora, per ottenere una soluzione ammissibile abbiamo usato l'**algoritmo del flusso massimo**, che:

- parte da un flusso nullo ($x_{ij} = 0$)
- cerca **cammini aumentanti** dai nodi sorgente a quelli destinazione;
- effettua la ricerca sul **grafo residuo** tramite una visita "a ventaglio".

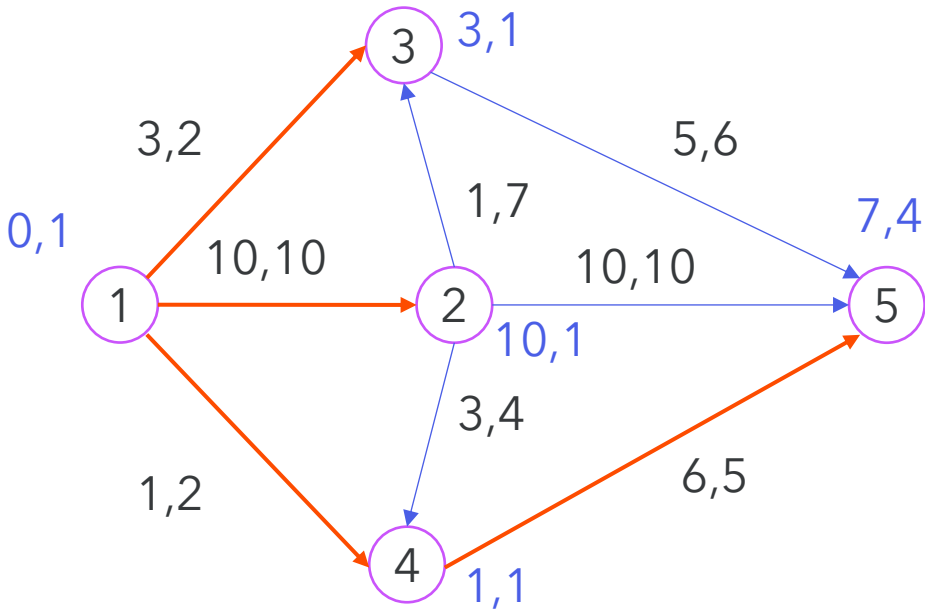
Dato che disponiamo anche dei **costi sugli archi**, possiamo modificare la procedura: invece di cercare cammini qualsiasi, cerchiamo **cammini aumentanti di costo minimo**, cioè i cammini più economici nel **grafo residuo pesato**.

c_{ij}, μ_{ij}, x_{ij}

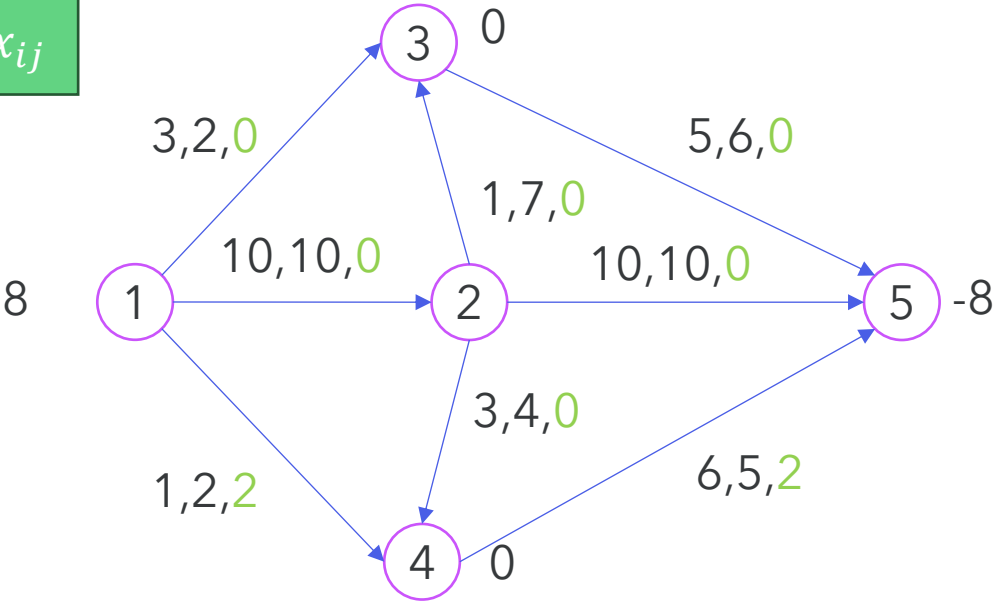


Riportiamo gli sbilanciamenti sui nodi (il contrario del deficit). Dobbiamo trovare dei cammini aumentanti che tengano conto dei costi. Ossia dei cammini orientati sul grafo residuo. Potremmo farlo cercando i **cammini minimi sul grafo residuo pesato**.

Otteniamo il cammino (1,4), (4,5) a costo 7 con capacità 2. Costo $c = 14$.



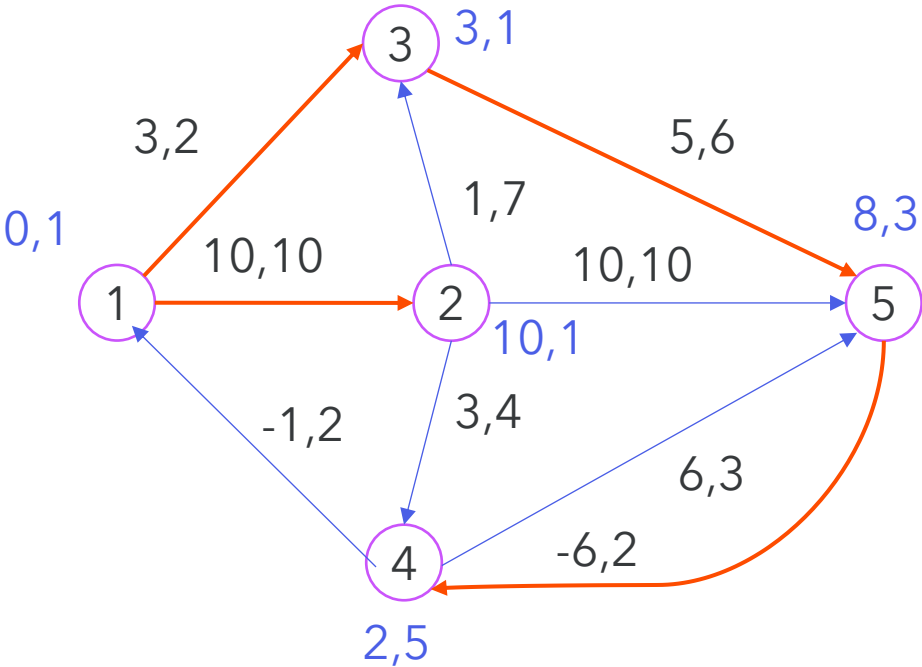
c_{ij}, μ_{ij}, x_{ij}



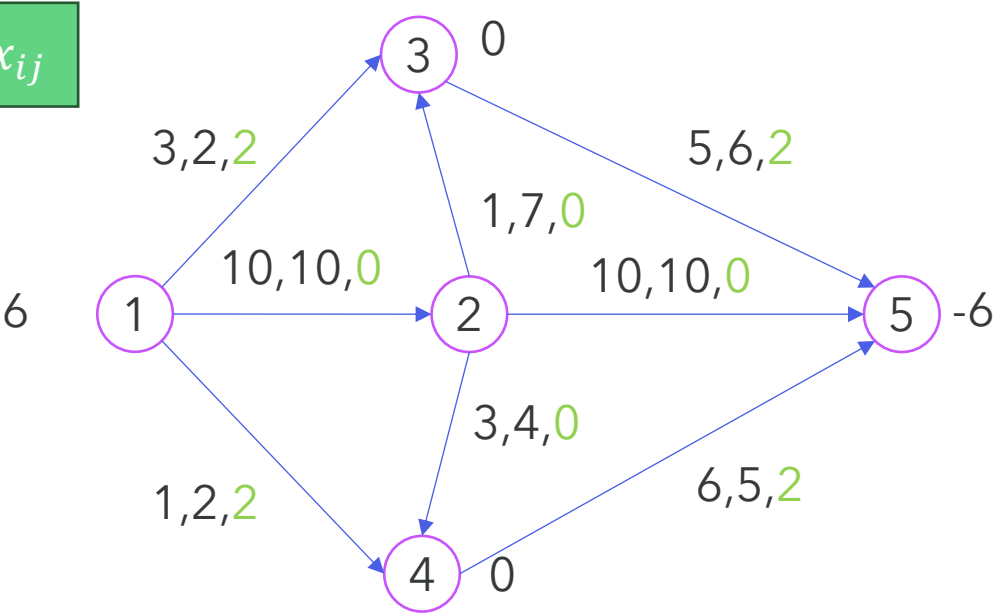
Aggiorno il grafo con il nuovo flusso. Gli sbilanciamenti cambiano.

Costruiamo il nuovo grafo residuo pesato e calcoliamo l'albero dei cammini minimi.

Otteniamo il cammino (1,3), (3,5) a costo 8 con capacità 2. Costo $c = 14 + 16 = 30$.



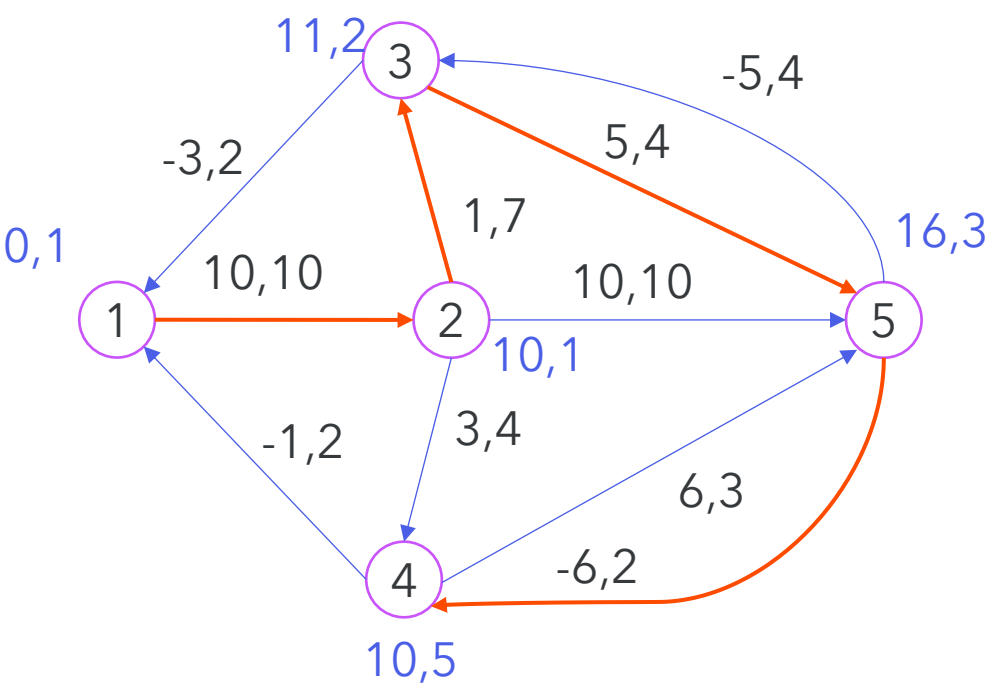
c_{ij}, μ_{ij}, x_{ij}



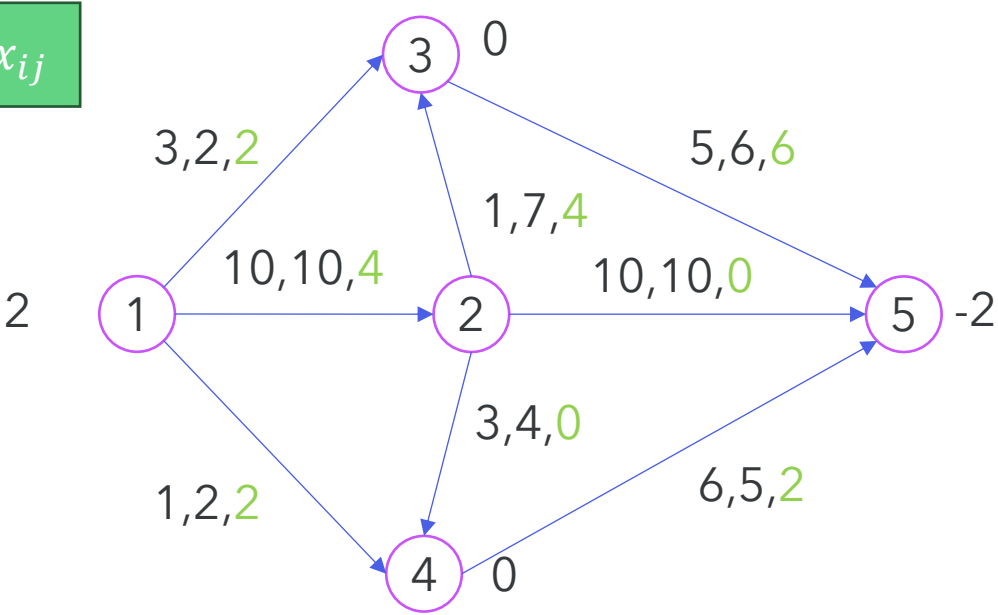
Aggiorno il grafo con il nuovo flusso. Gli sbilanciamenti cambiano.

Costruiamo il nuovo grafo residuo pesato e calcoliamo l'albero dei cammini minimi.

Otteniamo il cammino (1,2), (2,3), (3,5) a costo 16 con capacità 4. Costo = 30 + 64=94



c_{ij}, μ_{ij}, x_{ij}

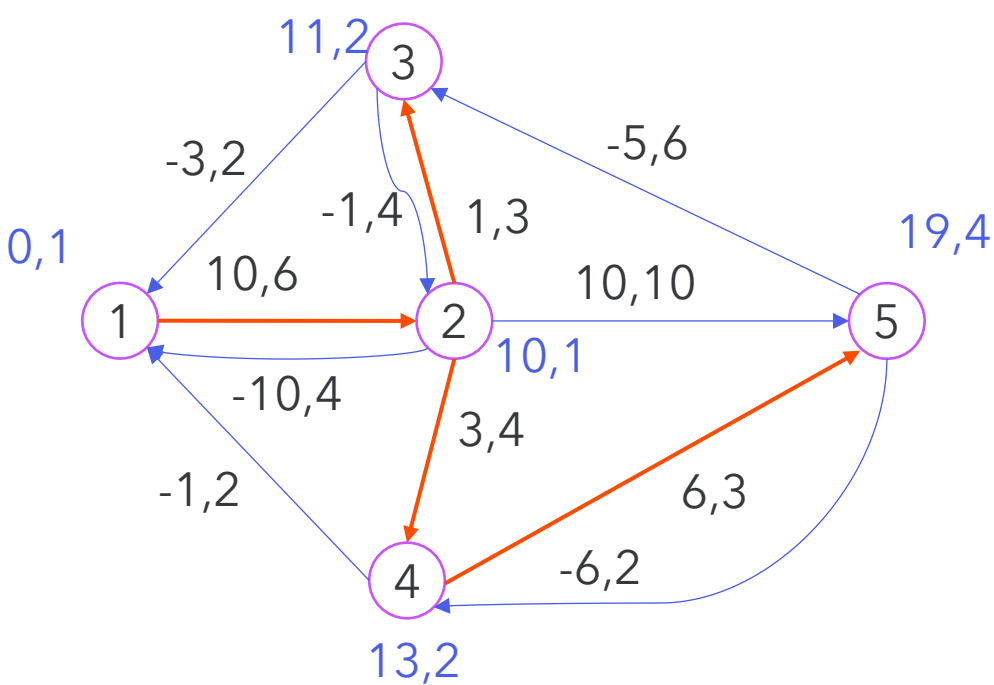


Aggiorno il grafo con il nuovo flusso. Gli sbilanciamenti cambiano.

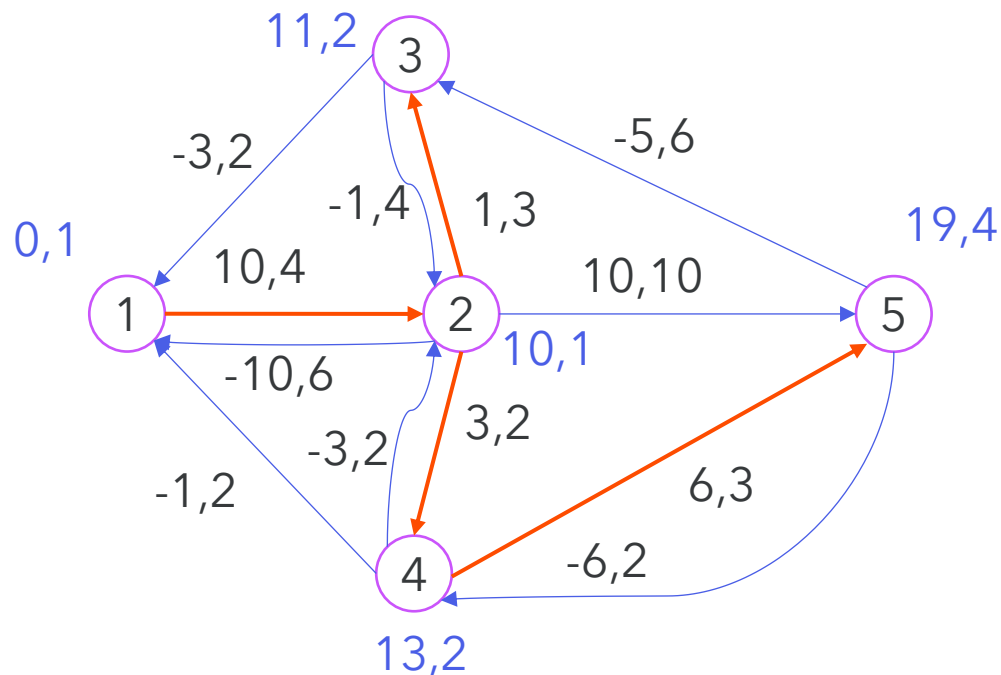
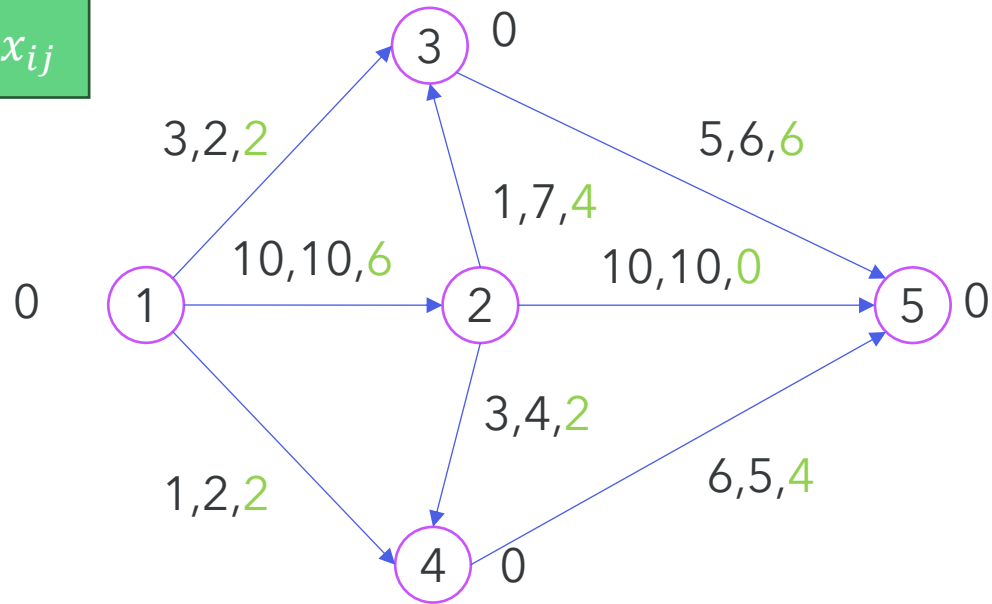
Costruiamo il nuovo grafo residuo pesato e calcoliamo l'albero dei cammini minimi.

Otteniamo il cammino (1,2), (2,4), (4,5) a costo 19 con capacità 2. *Potrei portare fino a 4, però creerei un nuovo sbilanciamento.*

Costo: $c = 94 + 38 = 132$.



c_{ij}, μ_{ij}, x_{ij}



Aggiorno il grafo con il nuovo flusso. Gli sbilanciamenti cambiano.

Costruiamo il nuovo grafo residuo pesato e calcoliamo l'albero dei cammini minimi.

Ora i nodi sono bilanciati. Abbiamo un flusso ammissibile e sperabilmente ha un costo minore rispetto alla soluzione iniziale fatta senza tenere conto dei costi.

In realtà abbiamo già ottenuto il flusso ottimo. Non ci sono cicli orientati di costo negativo sul grafo residuo pesato. Infatti lo stesso grafo presenta dei *cammini minimi che rispettano le condizioni di Bellman*.

Algoritmo dei cammini minimi successivi

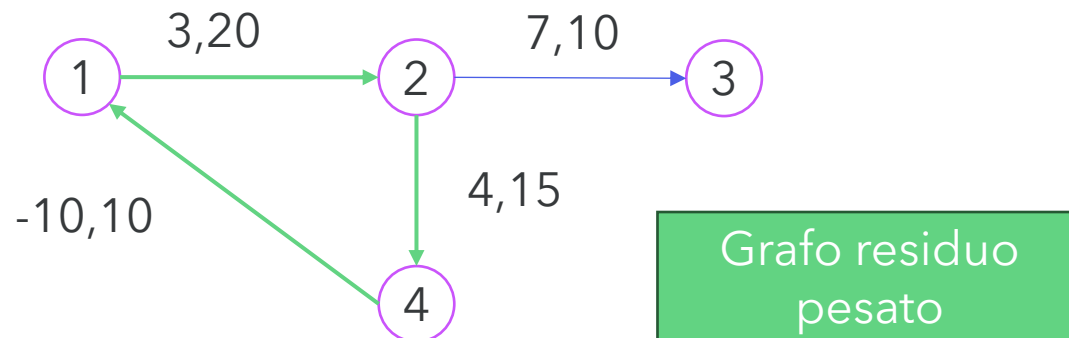
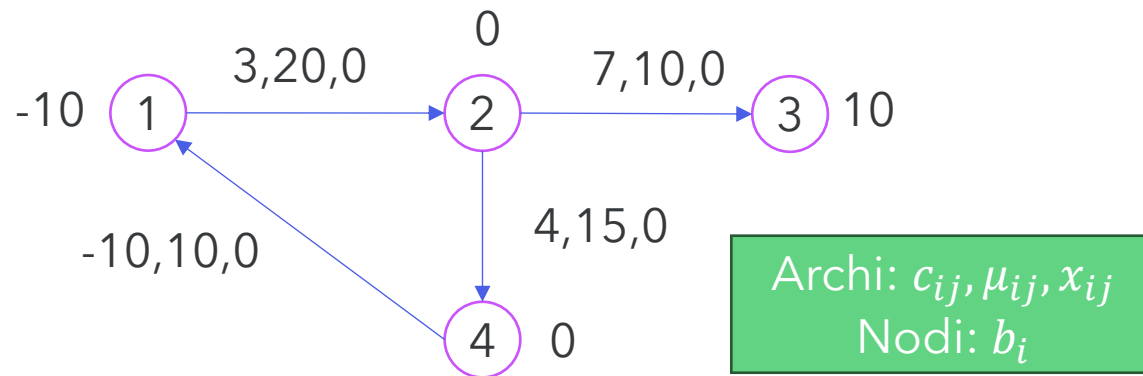
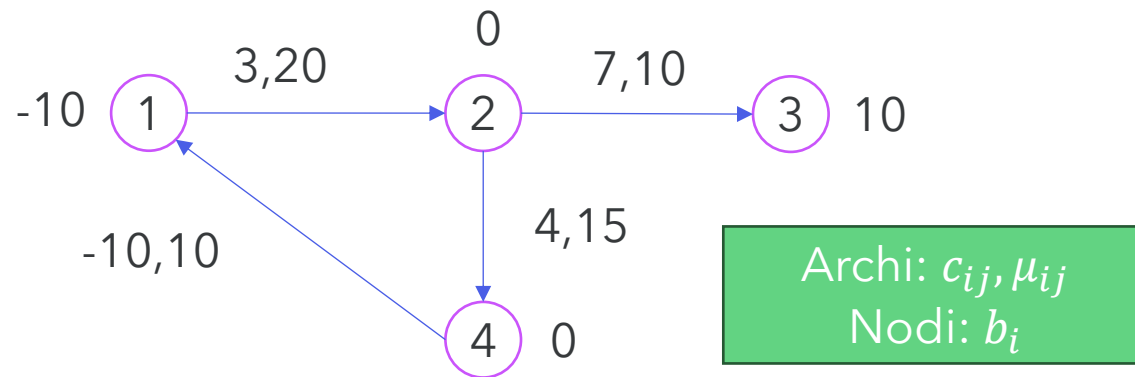
ALGORITMO DEI CAMMINI MINIMI SUCCESSIVI

L'algoritmo parte da una soluzione nulla e cerca di costruire una soluzione ammissibile inviando flusso lungo cammini aumentanti di costo minimo.

Se il processo termina, la soluzione ottenuta è ottima (a costo minimo).

Tuttavia, i **cammini minimi potrebbero non esistere**: ciò accade quando nel grafo residuo è presente un **ciclo orientato di costo negativo**, che può verificarsi anche fin dalla prima iterazione.

Ad esempio, basta che esista **un arco con costo negativo** per generare un ciclo di questo tipo, rendendo impossibile trovare un cammino di costo minimo ben definito.



Partiamo da un flusso nullo. Se provassimo ad eseguire **l'algoritmo dei cammini minimi successivi** dovremmo cercare un **cammino aumentante** da 1 a 3. Per farlo, dovremmo costruire il **grafo residuo pesato** che, visto che i flussi sono tutti nulli, è uguale al grafo originale. Poi dovremmo cercare un cammino orientato di costo negativo e, in particolare, vorremmo il cammino di costo minimo. Quindi risolveremmo il problema dei **cammini minimi sul grafo residuo pesato**.

Nel farlo otterremmo un **ciclo orientato di costo negativo**: (1,2), (2,4), (4,1) di costo -3.

Se l'abbiamo fatto con un algoritmo che non tiene conto dei cicli di costo negativo: non termina. In caso contrario non otteniamo comunque un cammino minimo, ma un ciclo orientato di costo negativo

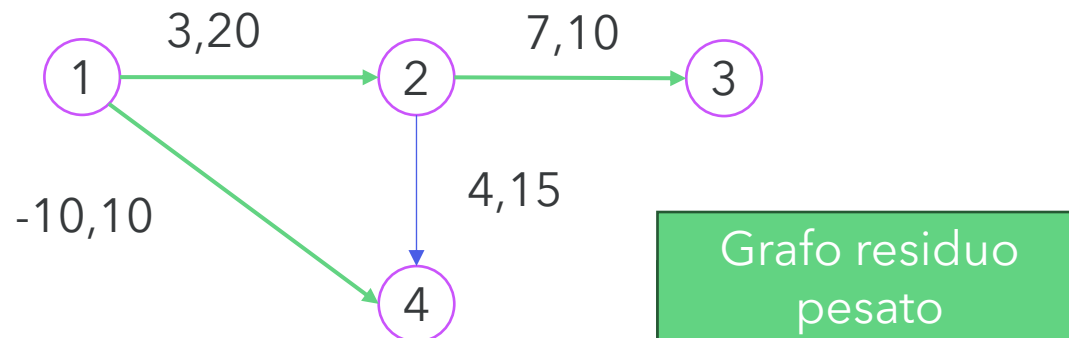
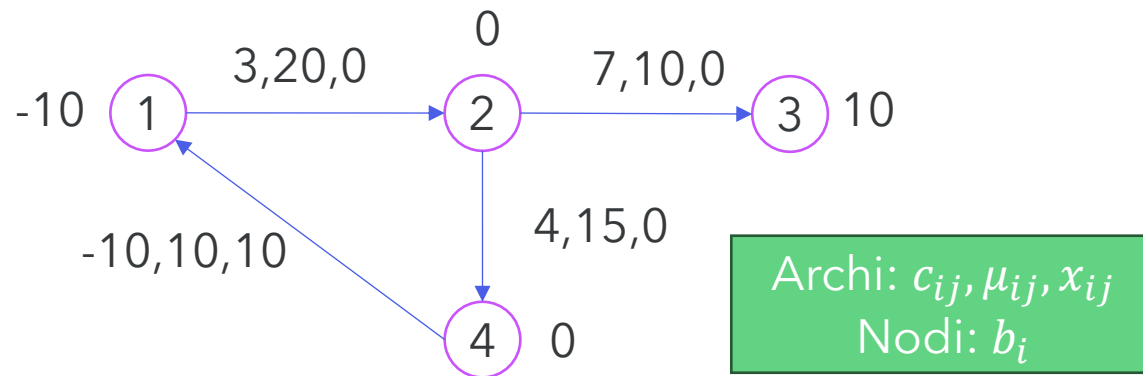
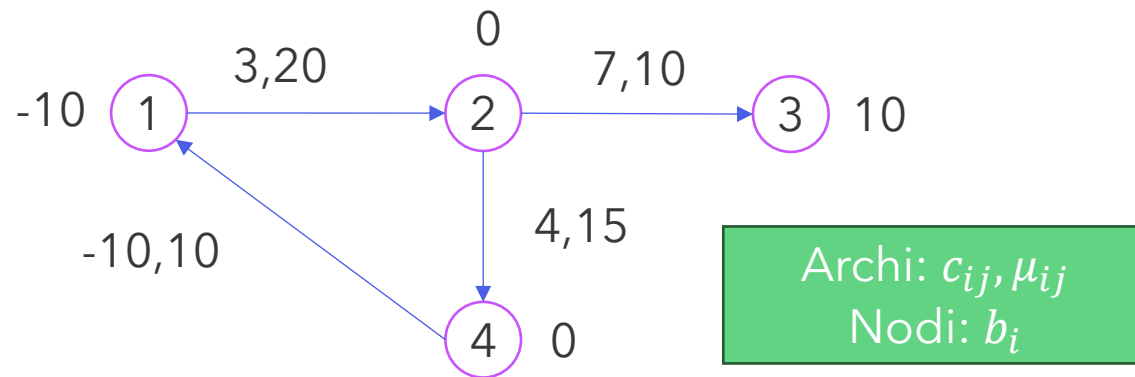
Come fare?

EVITARE CAMMINI ORIENTATI DI COSTO NEGATIVO

Potremmo sfruttare un **ciclo di costo negativo** per ridurre il costo del flusso.

Tuttavia, poiché vogliamo costruire un **algoritmo di cammini aumentanti successivi**, è necessario **evitare** la presenza di cicli di costo negativo nel **grafo residuo**.

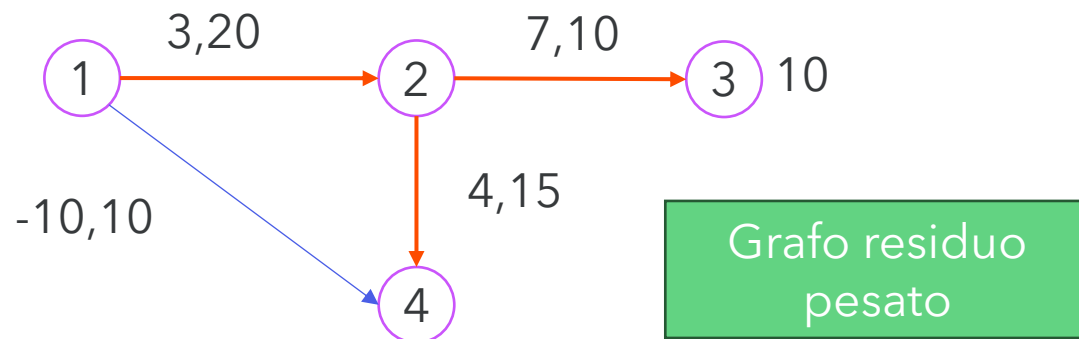
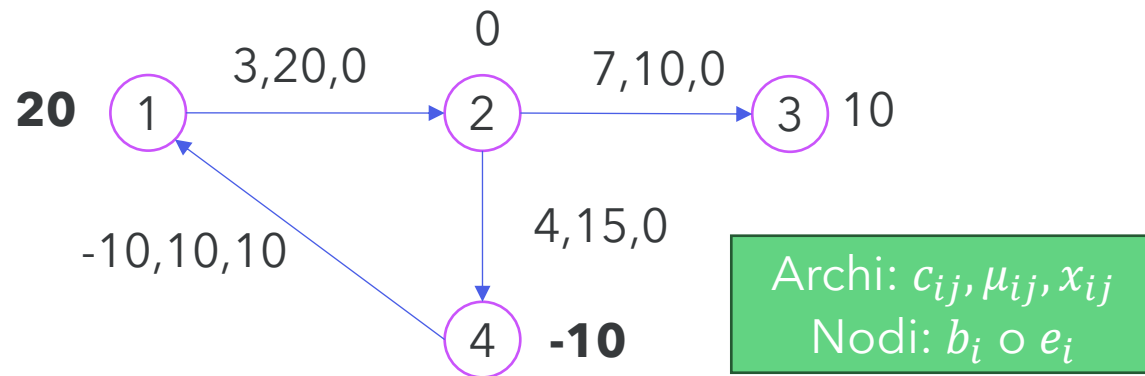
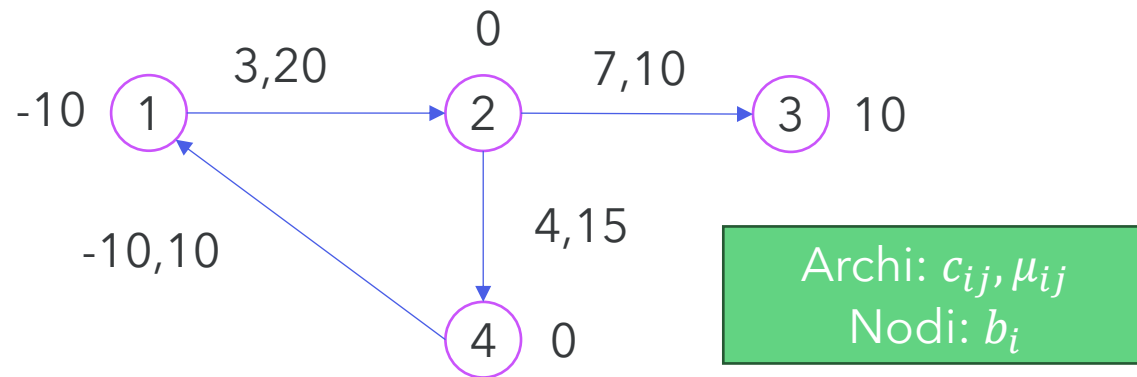
Come
farlo?



Se non ci sono costi negativi non ci possono essere cicli di costo negativo.

Allora: **saturiamo tutti gli archi di costo negativo.**

In questo modo sul grafo residuo non ci saranno più archi di costo negativo e perciò non ci potranno essere cicli orientati di costo negativo. E dunque l'albero dei cammini minimi avrà soluzione (se esiste).



Così facendo però, dobbiamo cambiare i deficit dei nodi alla testa e alla coda degli archi appena saturati: il nodo 1 riceve molto (diventa abbondante) e 4 diventa in difetto.

Aggiorniamo gli **sbilanciamenti** sui nodi su cui incidono gli archi appena saturati.

$$e_4 = 0 - 10 - 0 = -10$$

$$e_1 = 10 - 0 - (-10) = 20$$

Questo modifica le origini e/o le destinazioni:

- $O_x = \{1\}$ (nodi con eccedenza),
- $D_x = \{3,4\}$ (nodi con difetto).

Sul grafo residuo dovremo cercare dei cammini da O_x a D_x (possono cambiare le origini e le destinazioni), ma non avremo più cicli orientati di costo negativo.

INIZIALIZZAZIONE

Per ogni arco (i, j) :

$$x_{ij} = \begin{cases} 0, & \text{se } c_{ij} \geq 0 \\ \mu_{ij}, & \text{se } c_{ij} < 0 \end{cases}$$

cioè **si saturano tutti gli archi di costo negativo**.

Lo **sbilanciamento** di ciascun nodo i rispetto al flusso x è definito come:

$$e_x(i) = \sum_{(j,i) \in BS(i)} x_{ji} - \sum_{(i,j) \in FS(i)} x_{ij} - b_i$$

Dopo l'inizializzazione

- Se il nodo i è **origine** di un arco a **costo negativo**, invia flusso $\Rightarrow$ il suo sbilanciamento **diminuisce** (diventa più negativo).
- Se il nodo i è **destinazione** di un arco a **costo negativo**, riceve flusso $\Rightarrow$ il suo sbilanciamento **aumenta** (diventa più positivo).
- I nodi **non incidenti** ad archi di costo negativo restano con $e_x(i) = 0$.

ITERAZIONI SUCCESSIVE

Sul grafo residuo cercherò dei cammini da O_x a D_x , ma non avrò più cicli orientati di costo negativo. E poi potrei **iterare**.

Però se lo faccio come mi assicuro che non si creino altri cicli orientati di costo negativo sul grafo residuo?

In realtà quello ottenuto è uno pseudoflusso privo di cicli aumentanti di costo negativo.

Avevamo un teorema che diceva: « x è un flusso ottimo se e solo se non esistono cicli aumentanti di costo negativo».

Posso dire qualcosa di simile per gli pseudoflussi?

PSEUDOFUSSO MINIMALE

Uno pseudoflusso x , con il proprio vettore di sbilanciamenti e_x , è **minimale** se è di costo minimo tra tutti gli pseudoflussi con lo stesso vettore di sbilanciamenti.

$$c(x) \leq c(x') \quad \forall x', e_{x'} = e_x$$

Quello che abbiamo appena costruito nell'esempio è uno pseudoflusso minimale.
In particolare: è lo pseudoflusso che costa meno di tutti.

PSEUDOFUSSO MINIMALE

Anche per gli pseudoflussi minimali posso dire:

Teorema:

x è uno **pseudoflusso minimale** se e solo se **non esistono cicli aumentanti di costo negativo** (rispetto a x).

Nota: il flusso ottimo è uno pseudoflusso minimale i cui sbilanciamenti sono nulli.

PSEUDOFUSSO MINIMALE

Dimostrazione:

Sia x uno pseudoflusso per cui **non esistono cicli aumentanti di costo negativo**, ma supponiamo per assurdo che x **non sia minimale**.

Allora esiste uno pseudoflusso minimale x^* con lo stesso vettore di sbilanciamenti, cioè $e_{x^*} = e_x$.

Dal teorema sugli pseudoflussi, per trasformare x in x^* senza modificare gli sbilanciamenti si possono usare solo **cammini tra nodi con sbilanciamenti diversi** e **cicli**.

Poiché $e_{x^*} = e_x$, la differenza tra x e x^* consiste **solo in cicli**.

Ma x non ha cicli aumentanti di costo negativo per ipotesi, quindi non è possibile ottenere un pseudoflusso con costo minore. Contraddizione.

Quindi x deve essere minimale.

PSEUDOFUSSO MINIMALE

Con l'inizializzazione abbiamo costruito uno pseudoflusso minimale, perché non ha dei cicli aumentanti di costo negativo.

Allora possiamo costruire l'albero dei cammini minimi sul grafo residuo pesato e mandare, sul cammino di costo minimo, un flusso di quantità che rispetti la capacità del cammino.

Si ottiene così un altro pseudoflusso minimale, per il **Teorema della Minimalità**.

TEOREMA (MINIMALITÀ)

Teorema:

Se x è uno pseudoflusso minimale e P_{st} è il cammino aumentante di costo minimo da s a t (nodi qualsiasi), allora

$$0 \leq \theta \leq \theta(P_{st}, x)$$

e $x' = x \oplus \theta P$ è ancora **minimale**.

Otteniamo un nuovo pseudoflusso minimale con sbilanciamenti diversi. Infatti, se prendendo due nodi diversi e mandando flusso lungo il cammino aumentante che li unisce, gli sbilanciamenti cambiano:

- Quello del nodo di partenza diminuisce
- Quello del nodo di arrivo aumenta

x' può avere un costo maggiore di quello di x (come nel nostro caso), ma può essere sempre minimale.

DIMOSTRAZIONE: TEOREMA (MINIMALITÀ)

Partiamo con:

- Sia x uno pseudoflusso minimale.
- Sia P_{st} un cammino aumentante di **costo minimo** tra $s \in O_x$ e $t \in D_x$.
- Costruiamo $x' = x \oplus \theta P_{st}$.
- Consideriamo uno pseudoflusso qualunque x'' con gli **stessi sbilanciamenti di x'** .

DIMOSTRAZIONE: TEOREMA (MINIMALITÀ)

Decomposizione di x'' rispetto a x :

Dal teorema di decomposizione degli pseudoflussi:

$$x'' = x \oplus (\theta_1 P_1 \oplus \dots \oplus \theta_h P_h) \oplus (\theta_{h+1} C_1 \oplus \dots \oplus \theta_k C_{k-h})$$

dove:

$P_1, \dots, P_h$ sono cammini aumentanti da s a t ,

$C_1, \dots, C_{k-h}$ sono cicli aumentanti rispetto a x .

DIMOSTRAZIONE: TEOREMA (MINIMALITÀ)

Analisi dei costi:

- I cicli aumentanti non contribuiscono negativamente: $c(C_i) \geq 0$ perché x è minimale:

Se uno pseudoflusso è minimale non possono esserci cicli aumentanti di costo negativo, altrimenti non sarebbe minimale.

$$\sum_{j=1}^{k-h} \theta_{h+j} c(C_j) \geq 0$$

- I cammini hanno costo almeno quello del cammino minimo P_{st} : $c(P_i) \geq c(P_{st})$
- La quantità totale di flusso $\theta_1 + \dots + \theta_h = \theta$ (quanto porto sui cammini tra x e x'')

$$\sum_{i=1}^h \theta_i c(P_i) \geq \theta c(P_{st})$$

DIMOSTRAZIONE: TEOREMA (MINIMALITÀ)

Conclusione:

$$c(x'') = c(x) + \sum_{i=1}^h \theta_i c(P_i) + \sum_{j=1}^{k-h} \theta_{h+j} c(C_j) \geq c(x) + \theta c(P_{st}) = c(x')$$

Ricordando che $\theta_1 + \dots + \theta_h = \theta$ è anche quanto si porta dal flusso x al flusso x' perché x' e x'' hanno gli stessi sbilanciamenti.

Allora $c(x) + \theta c(P_{st}) = c(x')$, quindi:

$$c(x'') \geq c(x')$$

Quindi x' è **minimale tra tutti i pseudoflussi con gli stessi sbilanciamenti**.

ALGORITMO DEI CAMMINI-MINIMI SUCCESSIVI

Obiettivo:

Trovare un flusso di costo minimo costruendolo passo dopo passo, per **cammini aumentanti di costo minimo successivi**, mantenendo sempre uno **pseudoflusso minimale**.

procedure *Cammini-Minimi-Successivi*(G, c, b, μ):

$x = \text{Inizializza}(c, \mu)$

caso = "ottimo"

while $g(x) \neq 0$ **do**

$(p, d) = \text{Trova-Cammino-Minimo}(G_x, O_x, D_x)$

$t = \text{argmin} \{ d[i] : i \in D_x \}$

if $d[t] == \infty$ **then** { caso = "vuoto"; break }

$x = \text{Aumenta-Flusso}(x, p, t)$

end while

Input: grafo G , vettori dei deficit b , dei costi c , e delle capacità μ .

Output: il flusso di costo minimo o che il problema è vuoto.

Inizializziamo il flusso saturando tutti gli archi con costo negativo e ponendo a flusso nullo gli altri.

Finché lo sbilanciamento totale $g(x) = \sum_{i \in D_x} e_i(x)$ non è a zero: se trova un **cammino aumentante di costo minimo** da $s \in O_x$ (nodi con eccesso positivo) a $t \in D_x$ (nodi con deficit negativo) sul grafo residuo pesato G_x , invia lungo quel cammino una quantità di flusso θ : aggiorna $x \leftarrow x \oplus \theta P$. Lo sbilanciamento totale $g(x)$ diminuisce di θ finché non ho un flusso ammissibile, oppure dichiara il problema vuoto.

ALGORITMO DEI CAMMINI-MINIMI SUCCESSIVI

Inizializza(c,mu):

imposta $x_{ij} = 0$ se $c_{ij} \geq 0$,
e $x_{ij} = \mu_{ij}$ se $c_{ij} < 0$.

Trova-Cammino-Minimo:

Calcola un albero dei cammini minimi (SPT) da tutti i nodi in O_x verso D_x su G_x .
Se nessun cammino esiste, il problema è **inammissibile**.

Punto di partenza:

Produce uno pseudoflusso minimale iniziale perché è costruito in modo da non avere cicli di costo negativo.

Passo iterativo:

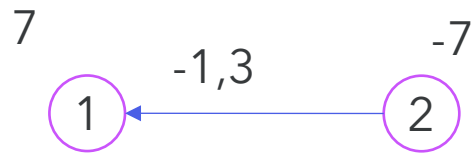
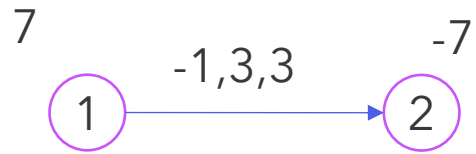
- Si trova un **cammino aumentante di costo minimo** tra due nodi scelti $s \in O_x$ e $t \in D_x$, cioè tra nodi con sbilanciamenti non nulli.
- Si invia flusso lungo questo cammino.

Quale cammino scegliere?

- Una delle cose ragionevoli da fare è scegliere un nodo con etichetta d con valore minima. L'etichetta è la lunghezza del cammino verso quel nodo: scegliamo il cammino più corto tra quelli trovati.
- Le etichette dei predecessori p ci permetteranno di trovare il nodo di partenza del cammino.
- Quindi mandiamo flusso sul **cammino minimo più corto dei cammini minimi**.
- Inoltre, scegliendo l'etichetta di valore minimo otteniamo un check di ammissibilità: se ha valore infinito, i nodi destinazione non sono raggiungibili da quelli sorgente: **soluzione inammissibile**.



Inizializzazione: arco di costo negativo, lo saturo.
 Aggiorno gli sbilanciamenti: ora $O_x = \{1\}$ e $D_x = \{2\}$.



Costruisco il grafo residuo pesato.
 Cerco un cammino minimo da un nodo di O_x a uno di D_x sul grafo residuo, però non esiste.
 In questo caso: $d_1 = 0$ e $d_2 = +\infty$.
 Allora l'etichetta minima tra tutti i nodi in D_x è $+\infty$, vuol dire che l'istanza è vuota (problema inammissibile).

ALGORITMO DEI CAMMINI-MINIMI SUCCESSIVI

Aumenta-Flusso:

invia θ unità di flusso lungo P e aggiorna il grafo residuo G_x .

Passo iterativo:

- Si trova un **cammino aumentante di costo minimo** tra due nodi scelti $s \in O_x$ e $t \in D_x$, cioè tra nodi con sbilanciamenti non nulli.
- Si invia flusso lungo questo cammino.

Quanto flusso invio?

$$\theta = \min\{\theta(P, x), e_x(s), -e_x(t)\}$$

Il minimo tra

- la capacità che satura il cammino $\theta(P, x)$,
- La quantità che s può inviare $e_x(s)$ senza passare ad avere uno sbilanciamento negativo (passando da O_x a D_x).
- La quantità che t può ricevere senza creare sbilanciamento positivo (passando da D_x a O_x).
- Vogliamo che gli sbilanciamenti si annullino.

CORRETTEZZA DELL'ALGORITMO DEI CAMMINI MINIMI SUCCESSIVI

Minimalità mantenuta:

- Dal teorema di minimalità, l'invio di flusso su un cammino di costo minimo **preserva la minimalità** dello pseudoflusso.

Albero dei cammini minimi:

Per costruire il cammino minimo serve risolvere un albero dei cammini minimi.

Poiché lo pseudoflusso iniziale non ha cicli orientati di costo negativo, e ogni passo preserva la minimalità, **nessuno pseudoflusso intermedio avrà cicli aumentanti negativi**.

Quindi l'albero dei cammini minimi può sempre essere costruito.

.

CORRETTEZZA DELL'ALGORITMO DEI CAMMINI MINIMI SUCCESSIVI

Terminazione e correttezza:

L'algoritmo termina quando tutti gli sbilanciamenti sono zero.

A quel punto lo pseudoflusso è **minimale e ammissibile**, quindi è un **flusso ottimo**.

Differenza con l'algoritmo di cancellazione dei cicli:

Nel metodo Cancella-Cicli si otteneva un flusso ammissibile, ma non si garantiva l'ottimalità.

Qui, l'ottimalità è garantita durante tutto il processo grazie alla scelta dei cammini di costo minimo, mentre l'ammissibilità è la condizione di terminazione.

TERMINAZIONE E COMPLESSITÀ

L'algoritmo è un'**estensione** di quello dei **cammini aumentanti**.
Se i **valori di capacità, deficit** fossero **irrazionali**, l'algoritmo potrebbe **non terminare**.

Condizioni sufficienti per la terminazione (e per l'integralità del flusso):

- Le **capacità** devono essere **interi**: $\mu \in N_+^m$
- I **deficit** devono essere **interi**: $b \in Z^n$

Allora:

- L'algoritmo parte da un **pseudoflusso** intero e,
- a ogni iterazione, **aumenta** il flusso lungo **cammini** con capacità θ intera (perché dipende da capacità residue intere).
- Dopo ogni iterazione, lo pseudoflusso e gli sbilanciamenti rimangono **interi** (invarianza dell'integralità).

TERMINAZIONE E COMPLESSITÀ

Assumiamo allora $\mu_{ij} \in \mathbb{N}_+^m$ e $b_i \in \mathbb{Z}^n$.

Definiamo lo sbilanciamento complessivo positivo:

$$g(x) = \sum_{i: e_x(i) > 0} e_x(i).$$

Ad ogni iterazione dell'algoritmo (si sceglie un cammino aumentante P da $s \in O_x$ a $t \in D_x$ e si invia flusso θ) pertanto:

$$\theta \geq 1$$

L'effetto è che $e_x(s)$ diminuisce di θ e $e_x(t)$ aumenta di θ , quindi:

$g(x)$ diminuisce di esattamente $\theta \geq 1$.

Quindi: **ad ogni iterazione $g(x)$ si riduce di almeno 1.**

L'algoritmo termina in un numero finito di iterazioni

TERMINAZIONE E COMPLESSITÀ

Limite sul numero massimo di iterazioni:

Un limite utile per lo sbilanciamento iniziale è:

$$g = \sum_{(i,j): c_{ij} < 0} \mu_{ij} + \sum_{i: b_i > 0} b_i$$

O, meno finemente: $mU + nB$,
con $U = \max\{\mu_{ij}: c_{ij} < 0, (i,j) \in A\}$
e $B = \max\{b_i: b_i > 0, i \in N\}$

Che rappresenta il massimo sbilanciamento iniziale prodotto dagli archi negativi saturati più le richieste positive. Poiché $g(x)$ diminuisce di almeno 1 ad ogni iterazione, il **numero massimo di iterazioni** è:

$$O(g)$$

TERMINAZIONE E COMPLESSITÀ

Il **numero massimo di iterazioni** è

$$O(g)$$

Trova-Cammino-Minimo con **SPT.L.Queue**:

$$O(mn)$$

Perché ho dei **costi negativi** e non posso usare altri algoritmi.

Complessità totale:

$$O(gmn)$$

Pseudopolinomiale

MIGLIORAMENTO COMPLESSITÀ

La complessità è $O(gmn)$ perché:

- Dobbiamo portare lo sbilanciamento di un numero molto grosso a zero
- E per farlo effettuiamo ogni volta dell'SPT.L.queue per trovare il cammino minimo.

$O(mn)$ è la migliore complessità che conosciamo per trovare l'albero dei cammini minimi i cui archi possono anche avere **costi negativi** (non ci sono cicli di costo negativo, però nel grafo residuo ci possono essere archi di costo negativo).

Se non ci fossero **archi di costo negativo** potrei usare implementazioni SPT.S con complessità $O(m \log n)$ o $O(n^2)$.

VARIANTE CON POTENZIALI – COSTI RIDOTTI

Però i cammini minimi li possiamo fare rispetto ai costi normali, che possono essere negativi, o rispetto a dei **costi ridotti**.

Dato un vettore di **potenziali** $\pi = [\pi_i]_{i \in N}$,
definiamo il **costo ridotto**:

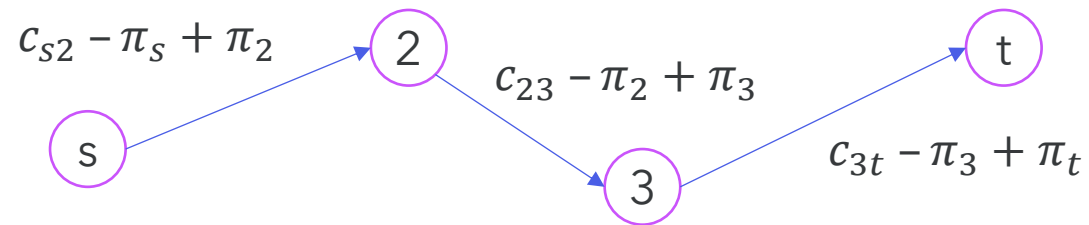
$$c_{ij}^{\pi} = c_{ij} + \pi_i - \pi_j$$

VARIANTE CON POTENZIALI – COSTI RIDOTTI

Proprietà:

I cammini minimi non cambiano se i costi vengono sostituiti con i **costi ridotti**.

Dato un cammino P :



Il costo del cammino usando i costi ridotti è:

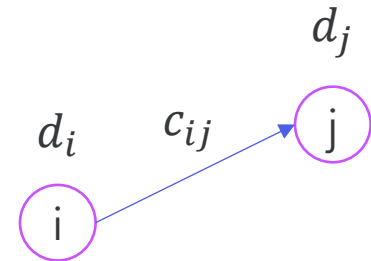
$$c_{s2} - \pi_s + \pi_2 + c_{23} - \pi_2 + \pi_3 + c_{3t} - \pi_3 + \pi_t = c(P) - \pi_s + \pi_t$$

Ma π_s e π_t sono costanti per tutti i cammini da s e t : sul grafo residuo con i costi ridotti otteniamo gli **stessi cammini minimi** del grafo con i costi originali.

VARIANTE CON POTENZIALI – COSTI RIDOTTI

Nella prima iterazione possiamo usare i costi originali perché non avremo archi di costo negativo utilizzabili sul grafo residuo pesato grazie all'inizializzazione.

Otteniamo dunque l'**albero dei cammini minimi** T e il **vettore di etichette** d .



Se T è l'albero dei cammini minimi sono rispettate le *condizioni di Bellman*:

$$d_j \leq c_{ij} + d_i \quad \Rightarrow \quad c_{ij} + d_i - d_j \geq 0$$

Allora prendiamo:

$$\pi_i = -d_i \quad \forall i \in N$$

e otteniamo

$$c_{ij}^{\pi} = c_{ij} - \pi_i + \pi_j \geq 0$$

Tutti i costi ridotti sono non negativi e ottengo gli stessi cammini.

VARIANTE CON POTENZIALI – VANTAGGI E CONNESSIONI

Algoritmo

Inizializza $\pi = 0$

Iterazioni:

- Calcola l'albero dei cammini minimi usando i **costi ridotti** c^π .
- Invia flusso lungo il cammino aumentante.
- Aggiorna $\pi = -d$ (etichette SPT).

Tutti i costi ridotti sono non negativi: si può usare **Dijkstra (SPT.S)** con complessità $O(m \log n)$ per trovare l'albero dei cammini minimi.

Se poi migliorassimo anche la complessità $O(g)$ con tecniche come lo **scaling**, l'algoritmo dei cammini minimi successivi può avere complessità polinomiale $O(\log U m \log n)$.

Connessione con gli algoritmi Primale-Duali

Alternano:

fase primale: aumenta il flusso (fissato π);

fase duale: aggiorna π per sbloccare nuovi archi a costo ridotto nullo.

Base teorica per gli algoritmi moderni di **flusso di costo minimo**.



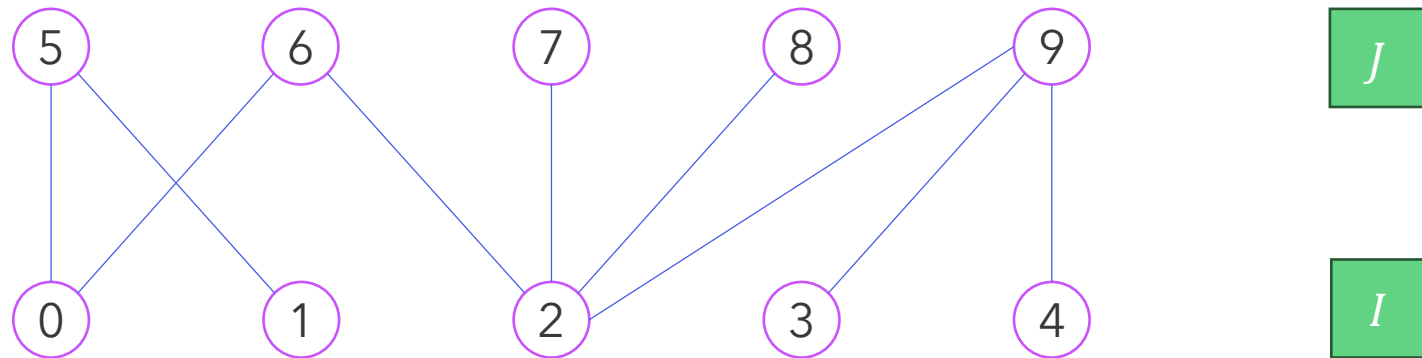
PROBLEMI DI ACCOPPIAMENTO

PAG 87 DISPENSE

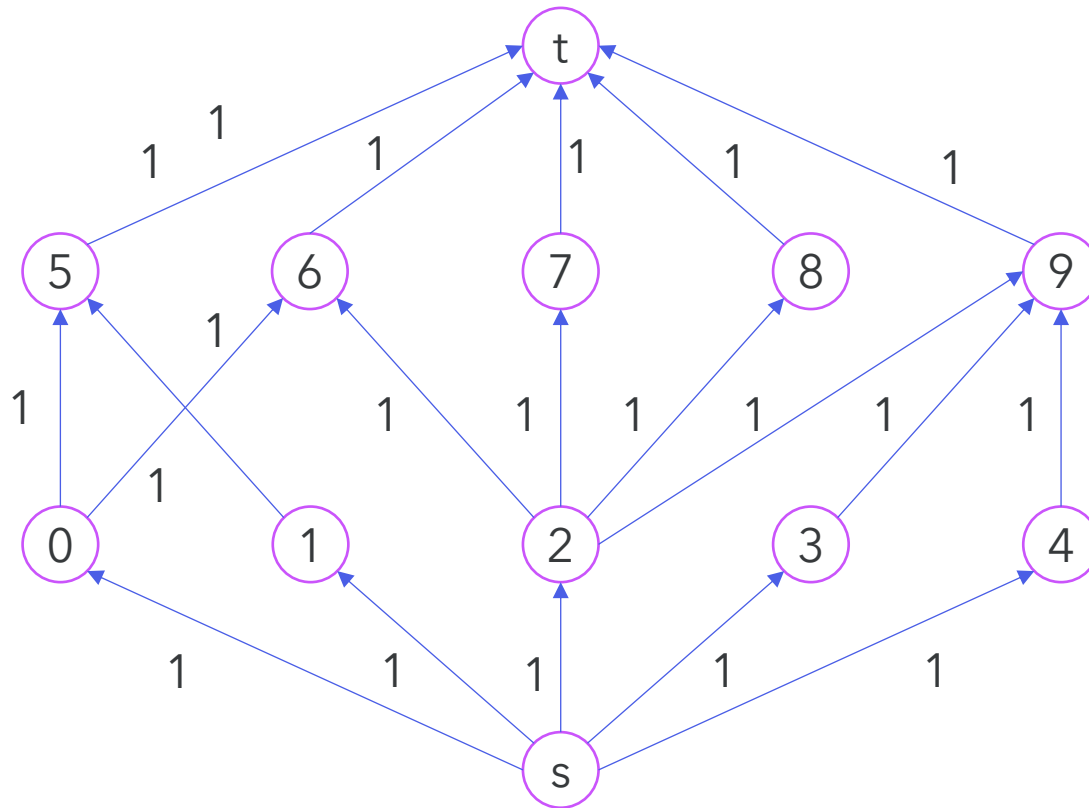
PROBLEMA DI ACCOPPIAMENTO DI MASSIMA CARDINALITÀ

Problema: Dato un grafo non orientato bipartito $G = (I \cup J, E)$ dove i lati rappresentano possibili accoppiamenti tra gli elementi di I e quelli di J . Ogni nodo di un sottoinsieme può essere accoppiato al più a un elemento dell'altro.

Obiettivo: trovare il massimo numero di accoppiamenti possibili.



PROBLEMA DI ACCOPPIAMENTO DI MASSIMA CARDINALITÀ



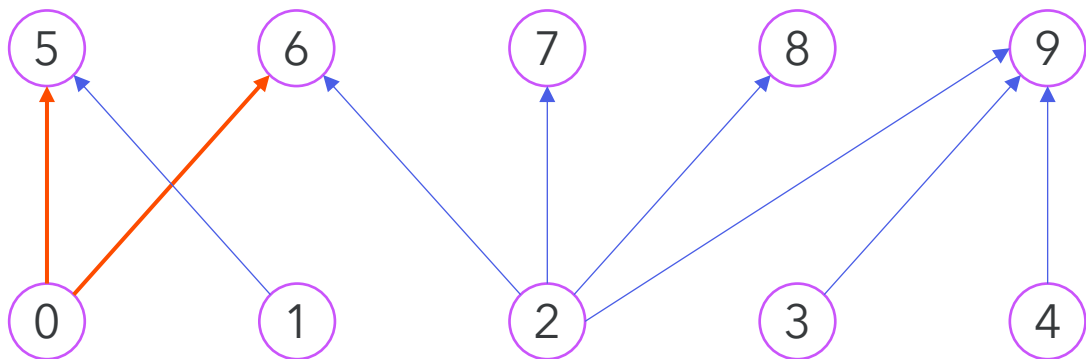
Anche questo è un **problema di flusso massimo**.

Aggiungiamo i nodi sorgente s e pozzo t :

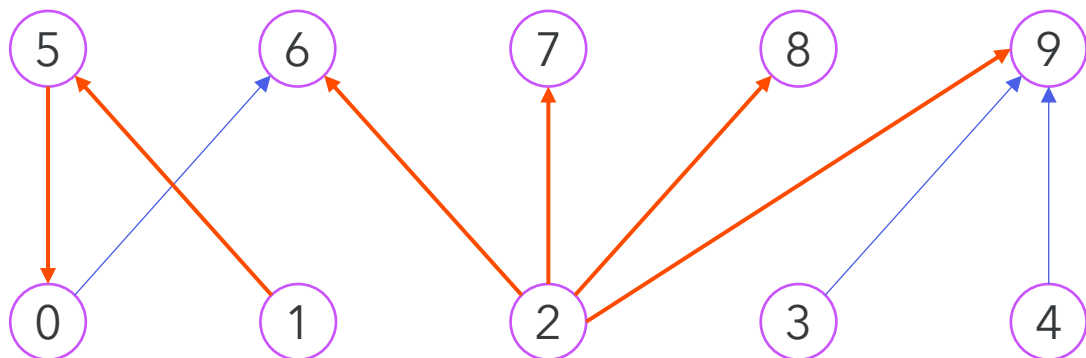
- Colleghiamo s con tutti i nodi in I con archi di capacità 1.
- Colleghiamo tutti i nodi in J con t con archi di capacità 1.

I lati tra I e J diventano archi dai nodi di I a quelli di J con capacità 1.

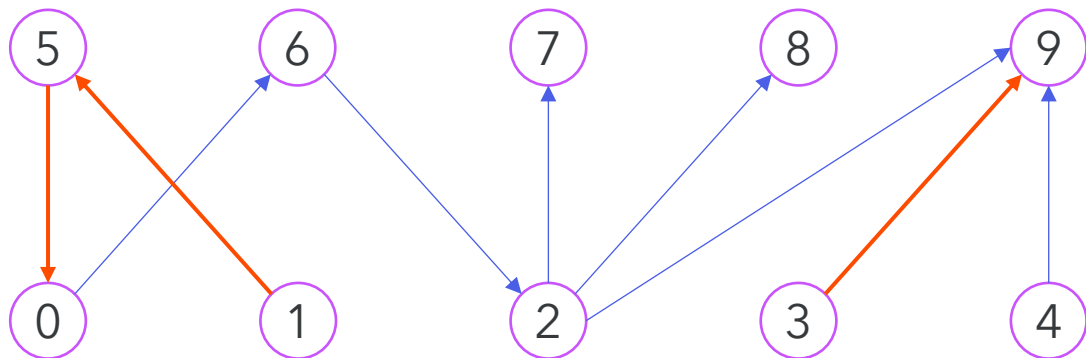
Risolvendo il flusso massimo da s a t , gli archi con flusso tra I e J determinano l'accoppiamento ottimo.



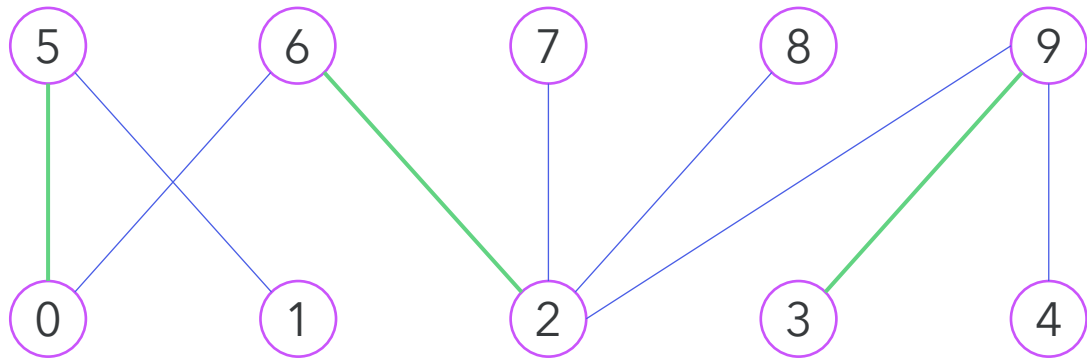
Applichiamo l'algoritmo di Edmonds-Karp. Nel grafo si riportano le visite del BFS (senza i nodi fittizi). Il primo cammino aumentante è $(s,0), (0,5), (5,t)$.
Associo 0 a 5.



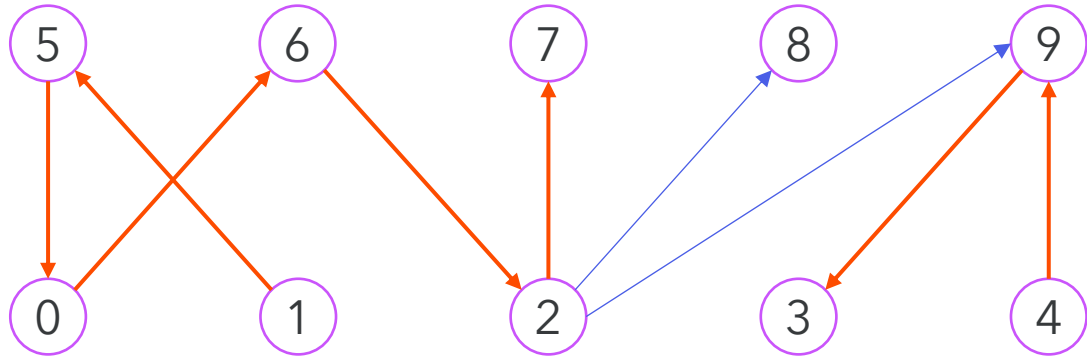
Nel grafo residuo si riportano le visite del BFS (senza i nodi fittizi). Il nuovo cammino aumentante è $(s,2), (2,6), (6,t)$.
Associo 2 a 6.



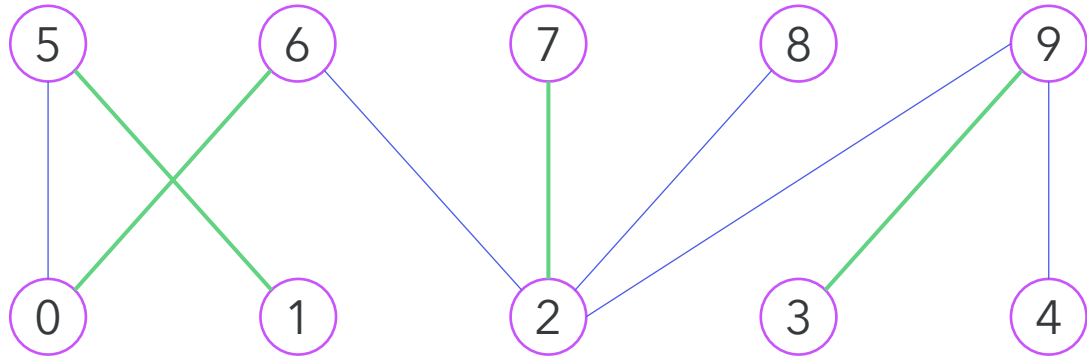
Nel grafo residuo si riportano le visite del BFS (senza i nodi fittizi). Il nuovo cammino aumentante è $(s,3), (3,9), (9,t)$.
Associo 3 a 9.

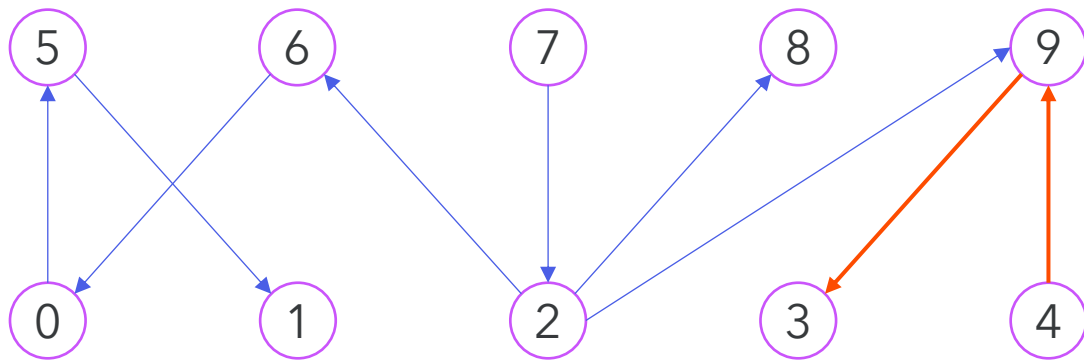


Dopo le prime 3 iterazioni ho assegnato
0 a 5, 2 a 6 e 3 a 9.



Nel grafo si riportano le visite del BFS
(senza i nodi fittizi). Il nuovo cammino
aumentante è $(s, 1), (1, 5), (5, 0), (0, 6),$
 $(2, 7), (7, t)$. C'è stato uno scambio di
assegnamenti alternato. Questo
cammino si chiama anche **cammino**
alternante.





Nel grafo residuo si riportano le visite del BFS (senza i nodi fittizi). Non riesco ad arrivare a t: ho ottenuto il flusso massimo e anche l'accoppiamento di cardinalità massima pari a 4: 0-6, 1-5, 2-7 e 3-9. Il nodo 4 rimane non assegnato e fa parte del taglio N_s .

I problemi di accoppiamento sono facili: si possono risolvere con un problema di flusso massimo con complessità $O(nmU) = O(nm1) = O(nm)$.

ASSEGNAMENTO DI COSTO MINIMO

Dato un **grafo bipartito pesato e non orientato** $G = (O \cup D, E)$:

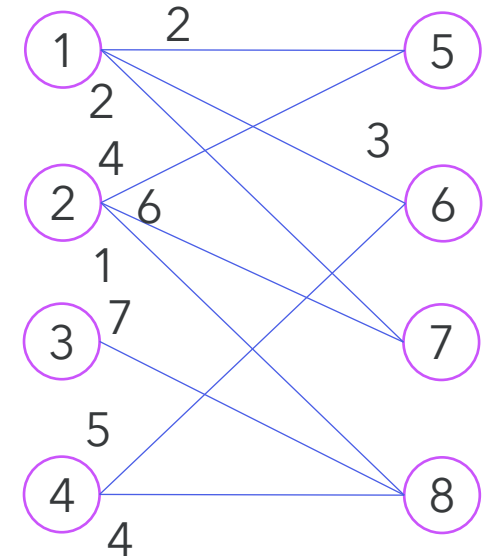
O : insieme delle origini (es. lavoratori)

D : insieme delle destinazioni (es. lavori)

Ogni lato $(i, j) \in E$ ha un **costo** c_{ij} .

Obiettivo: trovare un **accoppiamento perfetto** (ogni nodo assegnato a uno dell'altro insieme) **minimizzando il costo totale**.

Il **problema di assegnamento di costo minimo** è un caso particolare del **problema di flusso a costo minimo (MCF)**.



ASSEGNAMENTO DI COSTO MINIMO

Modellazione come problema di flusso

Attribuiamo un deficit ai nodi:

$b_i = -1$ per ogni nodo in O

$b_j = +1$ per ogni nodo in D

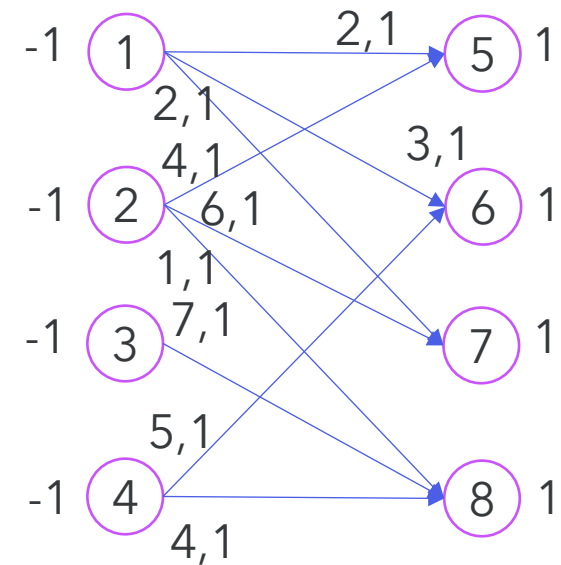
Convertiamo i lati $(i, j) \in E$ in archi orientati con:

capacità $\mu_{ij} = 1$

costo c_{ij} (invariato)

Ogni **flusso ammissibile** su questo grafo corrisponde a un **accoppiamento valido**.

Il **flusso di costo minimo** rappresenta l'**accoppiamento ottimo**.



ASSEGNAIMENTO DI COSTO MINIMO

Algoritmo di risoluzione

Si utilizza l'**algoritmo dei cammini minimi successivi**.

L'algoritmo:

- invia flussi unitari lungo cammini di **costo minimo residuo**,
- ripete l'operazione per un numero di iterazioni pari al numero di unità da inviare, cioè $n/2$ (cardinalità di O o D).

A ogni iterazione:

un nodo di O viene assegnato a un nodo di D , finché si ottiene un **accoppiamento perfetto**, oppure si scopre che **non esiste soluzione completa**.

COMPLESSITÀ

Dipende dall'implementazione dell'algoritmo dei cammini minimi:

Con SPT.L.Queue: $O(n^2m)$

con Dijkstra: $O(nm \log n)$

In pratica, l'algoritmo è **molto efficiente per grafi bipartiti di dimensioni moderate.**



GRAZIE

Stefano Novellani

stefano.novellani@unipi.it